

ML STUDIO | A VISUAL, HANDS-ON COURSE

MACHINE LEARNING FOR EVERYONE

From First Principles to Modern Models

Ensembles, kernels, clustering, and hands-on practice

● FREE ANDROID APP

● COMPANION TEACHER'S GUIDE



DR. ISMAEL KHORSHED ABDULRAHMAN

First Edition

Machine Learning for Everyone

From First Principles to Modern Models

Ensembles, Kernels, Clustering, and Hands-On Practice

STUDENT EDITION

Dr. Ismael Khorshed Abdulrahman

PhD in Electrical and Computer Engineering, Tennessee Technological University, USA

Free ML Studio App | Companion Teacher's Guide

Machine Learning for Everyone: From First Principles to Modern Models

Dr. Ismael Khorshed Abdulrahman PhD in Electrical and Computer Engineering, Tennessee Technological University, USA

Copyright © 2026 Ismael Khorshed Abdulrahman. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form without prior written permission from the copyright holder, except for brief quotations used for purposes such as review and scholarly citation.

AI tools were used to assist in preparing this book.

This book and its companion app, ML Studio, are provided to support learners everywhere, including students who are learning in a second or third language. Voluntary donations in support of their continued development are warmly welcomed and appreciated.

First edition.

Preface

This book teaches machine learning in three steps. You read an idea in plain words. You see it worked out with real numbers, one step at a time. Then you try it yourself, on paper and in ML Studio.

Machine learning can feel like two different subjects at once: a wall of unfamiliar words, and a wall of unfamiliar mathematics. This book tries to take down both walls at the same time, slowly and honestly. Every technical term is explained in plain English the first time it appears, so you always have a clear, plain-language definition close at hand. Later chapters use that same word freely, exactly the way a normal textbook trusts you to remember a word once you have learned it.

Every worked example in this book is built to be followed with nothing more than a pencil and a calculator. Every number is real, every step is shown, and every answer is checked. If an example in an earlier draft of this material was ever hard to follow, that is exactly the problem this edition was rewritten to fix: nothing is left for you to guess, and no step is skipped because it seemed "obvious."

This book is written for readers who are new to the subject, including readers whose first language is not English. Sentences are kept short. New ideas are built from ideas you already have. Case studies are drawn from real kinds of decisions — clinics, farms, shops, schools, banks, and factories — set in places around the world, including Iraq and the Kurdistan Region, so the examples feel close to real life rather than abstract.

The book follows the same teaching pattern, chapter after chapter: a short list of learning goals, a sequence of numbered lessons, WORD TO KNOW boxes for new vocabulary, EQUATION boxes that turn every formula into plain words and a table of symbols, WORKED EXAMPLE boxes with numbered steps, COMMON MISTAKE boxes, an ACTIVITY in the companion ML Studio app, a CASE STUDY, a chapter summary, and at least ten practice problems labeled Basic, Standard, or Challenge. That pattern repeats from the first chapter to the last, so you always know what to expect next.

The mathematics used in this book is real mathematics, not a simplified imitation of it — but it never goes beyond what a reader with ordinary arithmetic and simple algebra can follow, one small step at a time. Appendix A collects the small amount of extra mathematics this book leans on (means, distances, probability, gradients) in one place, with its own worked examples, for anyone who wants a refresher. Appendix B is a short, friendly field guide to Python, NumPy, and scikit-learn for anyone who wants to see these same ideas running as real code.

The book also comes with a working companion application, ML Studio, free for anyone who wants to learn. The same ideas you read on these pages live inside

the app: a Play tab where you pick a dataset and a model and watch a decision boundary or a cluster appear, Labs that animate ideas like gradient descent and k-means step by step, and a Practice tab with worked questions and a short quiz for every chapter.

This book covers machine learning from first principles through practical, everyday methods: how a model learns, how to measure it honestly, and the most useful algorithms for classification, regression, and finding structure in data, finishing with two complete, start-to-finish case studies. It ends exactly where deep neural networks begin. A companion volume, *Artificial Neural Networks*, continues from that point for readers who want to go further.

My hope is that by the last page, you will not only know what machine learning is. You will have fitted models by hand, checked their honesty, broken a few of them on purpose to see why they failed, and understood exactly why each one behaved the way it did.

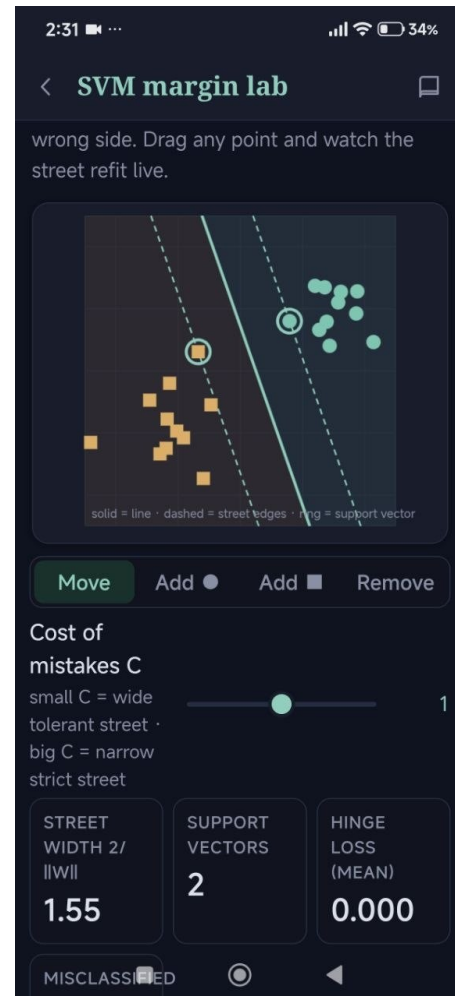
— Dr. Ismael Khorshed Abdulrahman, Tennessee Technological University, USA

Meet ML Studio

ML Studio is this book's free companion app. Every "Try It in ML Studio" note and every runnable-in-the-app code block in this book points to something you can actually touch: a real model, fitting on your own device, that you can watch, pause, and change.



The Play tab. Pick a dataset and an algorithm, and watch the decision boundary fit in real time — here, a *k*-nearest-neighbors model separating two spirals.



A Lab in motion. Drag the points and watch a support vector machine's margin refit live, so an idea like "cost of mistakes" stops being abstract.

The app runs the same NumPy code you will read in this book, fully offline, right on your phone or laptop. Nothing shown is a recording — every plot is computed while you watch. Install it once, and it is ready every time you open this book again.

Notation and Conventions

This book uses a small, consistent set of symbols from the first chapter to the last. You do not need to memorize this page — every symbol is also explained again, in plain words, the first time it is actually used. Keep this page as a quick reference.

Symbol	Meaning	Notes
x	one input feature, or a list of features for one example	plain letter
y	the true label or target value for one example	plain letter
$\hat{y}$	a model's prediction for one example	plain letter with a hat
n	the number of examples in a dataset	count
$\times$	multiply	operator
$\div$	divide	operator
$-$	subtract, or a negative sign	operator
$\sqrt{}$	square root	operator
2	squared (multiplied by itself)	operator
$\approx$	approximately equal to	operator
$\leq, \geq$	less than or equal to, greater than or equal to	operator
Σ	add up a list of numbers (the Greek letter "sigma")	operator

Bold, colored WORD TO KNOW boxes introduce new vocabulary. Shaded EQUATION boxes give a formula, a plain-English restatement, and a table of every symbol used. Green WORKED EXAMPLE boxes are fully solved, numbered, and checkable by hand. Numbers in worked examples and case studies are chosen to teach a method clearly; where a case study needs realistic figures, they are clearly presented as an illustrative example, not as verified statistics about any real organization.

Contents

Preface.....	4
Meet ML Studio.....	6
Notation and Conventions.....	7
Contents.....	8
Part I — Foundations of Machine Learning.....	13
What Is Machine Learning?.....	14
1.1 Learning from Examples.....	14
1.2 Data, Examples, Features, and Labels.....	16
1.3 Three Common Learning Tasks.....	17
1.4 Supervised and Unsupervised Learning.....	18
1.5 A Precise Learning Goal: Task, Experience, and Performance.....	19
1.6 Training, Testing, and Accuracy.....	20
1.7 Worked Example: Build a Classifier by Hand.....	22
1.8 Try It in ML Studio.....	25
Practice Problems.....	27
The Machine Learning Workflow.....	29
2.1 Begin with the Decision, Not the Algorithm.....	29
2.2 Name the Unit, Target, and Prediction Time.....	30
2.3 Inspect and Split the Data Early.....	32
2.4 Build a Baseline Before a Complex Model.....	34
2.5 Train, Validate, Test Once.....	36
2.6 Try It in ML Studio.....	38
Practice Problems.....	40
Data, Features, and Preprocessing.....	42
3.1 Read the Dataset as a Table of Examples.....	42
3.2 Find Missing, Incorrect, and Duplicate Records.....	44
3.3 Encode Categories Carefully.....	46
3.4 Put Numerical Features on Useful Scales.....	47
3.5 Use a Pipeline to Keep Steps Together.....	49
3.6 Try It in ML Studio.....	51
3.7 Case Study: Cleaning Building-Energy Meter Data at a University in Duhok.....	51
Practice Problems.....	53
Training, Validation, and Testing.....	55
4.1 A Metric Must Match the Real Goal.....	55
4.2 Read a Confusion Matrix.....	56
4.3 Precision, Recall, and F1.....	58
4.4 Regression Errors in Real Units.....	60
4.5 Cross-Validation Uses Data Efficiently.....	63
4.6 Try It in ML Studio.....	64
4.7 Case Study: Detecting Faulty Water Pumps from Sensor Data.....	65
Practice Problems.....	67
Overfitting, Underfitting, and Regularization.....	69
5.1 Two Ways a Model Can Fail.....	69

5.2 Bias and Variance.....	72
5.3 The Bias-Variance Trade-off.....	73
5.4 Hyperparameters and Model Capacity.....	74
5.5 Learning Curves: Reading Train vs. Validation Score.....	77
5.6 Regularization: Paying a Price for Simplicity.....	78
5.7 Try It in ML Studio.....	80
5.8 Case Study.....	81
Practice Problems.....	82
Part II — Supervised Learning.....	85
Linear Regression.....	86
6.1 A Line That Predicts a Number.....	86
6.2 Slope, Intercept, and Predictions by Hand.....	87
6.3 Residuals and the Loss Function.....	88
6.4 Finding the Best Line: Gradient Descent, Step by Step.....	91
6.5 Multiple Features: More Than One Input.....	95
6.6 Ridge and Lasso: Regularized Regression.....	97
6.7 Try It in ML Studio.....	99
6.8 Case Study.....	100
Practice Problems.....	101
Logistic Regression.....	104
7.1 From a Line to a Yes/No Answer.....	104
7.2 The Sigmoid Function.....	105
7.3 Log Loss: Scoring a Probability Honestly.....	107
7.4 The Decision Boundary.....	109
7.5 Fitting Logistic Regression with Gradient Descent.....	111
7.6 Try It in ML Studio.....	113
7.7 Case Study.....	113
Practice Problems.....	115
k-Nearest Neighbors.....	117
8.1 Predict Like Your Neighbors.....	117
8.2 Choosing k: Small vs. Large.....	119
8.3 Distance and Feature Scaling.....	120
8.4 Weighted Voting and High Dimensions.....	124
8.5 Try It in ML Studio and Case Study.....	125
Practice Problems.....	127
Decision Trees.....	129
9.1 A Tree Asks One Question at a Time.....	129
9.2 A Good Split Makes Children More Pure.....	131
9.3 Trees Can Easily Overfit.....	133
9.4 Regression Trees Predict Local Averages.....	136
9.5 Try It in ML Studio and Case Study.....	137
Practice Problems.....	140
Random Forests and Boosting.....	142
10.1 Many Trees Can Beat One.....	142
10.2 How a Random Forest Creates Variety.....	143
10.3 Out-of-Bag Examples Give a Free Check.....	146

10.4 Boosting Repairs Mistakes, Step by Step.....	148
10.5 Bagging vs. Boosting, Side by Side.....	151
10.6 Feature Importance Needs Care.....	152
10.7 Try It in ML Studio.....	154
Practice Problems.....	156
Neural Networks: A Short Bridge.....	158
11.1 From One Logistic Unit to a Network of Units.....	158
11.2 Hidden Layers and Learned Features.....	159
11.3 Activation Functions: Why Bending Matters.....	160
11.4 How Two Hidden Units Solve a Problem a Straight Line Cannot....	162
11.5 Backpropagation in Plain Words.....	164
11.6 When to Choose a Network, and Where This Book Ends.....	166
Practice Problems.....	169
Part III — Unsupervised Learning.....	171
Clustering with k-Means.....	172
12.1 Grouping Without Answers.....	172
12.2 The k-Means Algorithm, Step by Step.....	174
12.3 Choosing k: Inertia and the Elbow Method.....	179
12.4 Limitations: Shapes k-Means Cannot Find.....	181
12.5 Try It in ML Studio.....	183
12.6 Case Study.....	183
Practice Problems.....	185
Principal Component Analysis.....	187
13.1 Too Many Features, Too Little Insight.....	187
13.2 Finding the Direction of Greatest Spread.....	188
13.3 Principal Components, One at a Time.....	192
13.4 Explained Variance: How Much Did We Keep?.....	194
13.5 Try It in ML Studio.....	197
13.6 Case Study.....	197
Practice Problems.....	200
Anomaly Detection.....	202
14.1 What Makes an Example Unusual?.....	202
14.2 A Simple Method: Distance from the Center.....	204
14.3 z-Scores and Setting a Threshold.....	206
14.4 Try It in ML Studio.....	210
14.5 Case Study.....	210
Practice Problems.....	212
Part IV — Building Reliable Projects.....	214
Feature Engineering and Selection.....	215
15.1 A Good Feature Is Worth More Than a Fancy Model.....	215
15.2 Creating New Features from Old Ones.....	216
15.3 Encoding Text and High-Cardinality Categories, Briefly.....	219
15.4 Selecting Fewer, Better Features.....	221
15.5 Try It in ML Studio.....	223
15.6 Case Study.....	223
Practice Problems.....	225

Imbalanced Data and Useful Metrics.....	227
16.1 When 99% Accuracy Means Nothing.....	227
16.2 Resampling: Oversampling and Undersampling.....	230
16.3 Class Weights: Telling the Model What Matters.....	232
16.4 Moving the Decision Threshold.....	233
16.5 Calibration: Do the Probabilities Mean What They Say?.....	236
16.6 Try It in ML Studio.....	237
Practice Problems.....	240
Model Selection and Tuning.....	242
17.1 Every Model Has Dials to Turn.....	242
17.2 Grid Search: Trying Every Combination.....	243
17.3 Random Search: Trying Many, Not All.....	246
17.4 Avoiding a Common Trap: Tuning on the Test Set.....	248
17.5 Try It in ML Studio.....	250
17.6 Case Study.....	251
Practice Problems.....	253
Interpretability, Fairness, and Ethics.....	255
18.1 Why "It Works" Is Not Enough.....	255
18.2 Reading Coefficients and Feature Importance.....	257
18.3 Permutation Importance: Testing a Feature by Breaking It.....	258
18.4 Fairness: The Same Word, a Different Meaning.....	261
18.5 Protected Attributes and Proxy Variables.....	264
18.6 Try It in ML Studio.....	265
18.7 Case Study.....	266
Practice Problems.....	268
From Notebook to Production.....	270
19.1 From a Script to a Real System.....	270
19.2 Data Drift: The World Keeps Changing.....	272
19.3 Monitoring What Matters.....	274
19.4 Training-Serving Skew and Safe Rollback.....	275
19.5 Try It in ML Studio and Case Study.....	277
Practice Problems.....	280
Part V — Complete Case Studies.....	282
Case Study: Predicting House Prices.....	283
20.1 Frame the Decision: A Starting Price, Not an Appraisal.....	283
20.2 Audit the Data Before Trusting It.....	285
20.3 Split by Time, Not at Random.....	286
20.4 Build a Baseline First.....	287
20.5 Build One Safe Pipeline.....	288
20.6 Compare Models on the Validation Set.....	290
20.7 Final Test and Error Analysis.....	291
20.8 Explain the Model, Then Monitor It After Launch.....	293
Practice Problems.....	297
Case Study: Predicting Customer Churn.....	300
21.1 Frame the Prediction Around a Helpful Action.....	300
21.2 Prevent Customer Leakage and Time Leakage.....	302

21.3 Audit the Label and the Features.....	303
21.4 Build Two Baselines Before Any Real Model.....	305
21.5 Compare Models and Check Calibration.....	307
21.6 Choose a Threshold Using Capacity and Value.....	308
21.7 Train the Full Pipeline, Then Test Once.....	312
21.8 Fairness and the Human in the Loop.....	313
21.9 Monitor the Decision, Not Only the Model.....	314
Practice Problems.....	318
Reference.....	320
Mathematics When You Need It.....	321
A.1 Scalars, Vectors, and Matrices.....	321
A.2 Mean, Variance, and Standard Deviation.....	322
A.3 The Dot Product.....	324
A.4 Distance and Norm.....	325
A.5 Probability Basics.....	326
A.6 Derivatives and Gradients.....	328
A.7 Logarithms.....	330
Python, NumPy, and scikit-learn Field Guide.....	333
B.1 Variables, Lists, and Arrays.....	333
B.2 Rows, Columns, and Shapes.....	335
B.3 The Common scikit-learn Pattern.....	337
B.4 Reproducibility.....	339
B.5 A Safe Checklist for Every Code Example.....	341
Glossary.....	344

Part I — Foundations of Machine Learning

- Chapter 1** — What Is Machine Learning?
- Chapter 2** — The Machine Learning Workflow
- Chapter 3** — Data, Features, and Preprocessing
- Chapter 4** — Training, Validation, and Testing
- Chapter 5** — Overfitting, Underfitting, and Regularization

FOUNDATIONS OF MACHINE LEARNING

CHAPTER 1

What Is Machine Learning?

How computers learn useful rules from examples

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what machine learning is and how it differs from normal, rule-based programming
- Identify the features and the label inside a small dataset
- Tell classification, regression, and clustering apart, and choose the right one for a new project
- Describe the difference between supervised and unsupervised learning, and name reinforcement learning as a third path
- Write a project's task, experience, and performance measure (T, E, P) before choosing any model
- Split data into a training set and a test set, and compute a model's accuracy by hand

1.1 Learning from Examples

Think about buying watermelons at a market in Duhok. You want a sweet one, but you cannot taste it before you buy it. So you look at the color of the skin. You feel the weight in your hand. You knock on it with one knuckle and listen to the sound it makes. After buying and tasting many melons over many visits, you slowly get better at guessing which ones will be sweet. Nobody taught you an exact rule. You built your own rule from experience, one melon at a time.

This is the basic idea behind machine learning too. Let's build the idea slowly, starting with how computers normally work.

A normal computer program starts with rules that a person writes. A programmer thinks carefully about a problem, writes exact steps, and the computer follows those steps every single time. Such a set of exact steps has its own name.

WORD TO KNOW

Algorithm — A clear, exact set of steps that solves a problem or finishes a task. A learning algorithm is the exact set of steps a computer follows to search for a useful pattern inside data.

In normal programming, the program receives data as input, follows the programmer's rules exactly, and produces an answer as output. If the rules are correct, the answer is correct. If the rules are wrong or incomplete, the answer is wrong, and only a person can go back and fix the rules by hand.

Machine learning changes one important part of this picture. Instead of a person writing every rule, we give the computer many examples, along with the correct answer for each example. A learning algorithm studies these examples and searches for a useful rule by itself, without a person writing that rule directly.

WORD TO KNOW

Machine learning — A way to build computer programs that find useful rules from data, instead of a person writing every rule by hand. The computer studies many examples and works out a pattern on its own.

The rule that comes out of this process also has its own name, and it is important not to confuse it with the algorithm that produced it.

WORD TO KNOW

Model — The rule that a learning algorithm produces after studying data. A model takes a new input and returns an output, such as a prediction.

The algorithm and the model are not the same thing, even though beginners often mix them up. Think about a cooking teacher and a recipe card. The teacher's method is how the recipe card gets written in the first place. The recipe card is what you actually use in your kitchen, again and again, afterward. In the same way, a learning algorithm is the method that studies data and builds a model. The model is the finished rule you actually use afterward to make new predictions. You run the learning algorithm only once, or once in a while, to train a model. You then use that trained model again and again, sometimes thousands or millions of times, without ever running the learning algorithm again.

The output a trained model gives for one new example also needs its own name, because you will see this word constantly through the rest of this book.

WORD TO KNOW

Prediction — The output a model gives for one new example. A prediction can be a category, such as "sweet," or a number, such as a weight in kilograms.

So the full loop looks like this, in plain words. A learning algorithm studies training examples, together with their known answers, and produces a model. The finished model then looks at new inputs it has never seen before, and returns a prediction for each one. This one loop — examples in, model out, then new input in, prediction out — is the heart of every idea in this book. Every chapter after this one is really just a different way to build a better model, check it more honestly, or use it more responsibly.

COMMON MISTAKE

x Wrong: Calling every computer program that makes automatic decisions "machine learning," even a program that only follows fixed rules a person typed in by hand, such as "if the temperature is above 40, send a warning."

✓ **Right:** Ask where the rule came from. If a person wrote the exact rule, it is normal programming. If a learning algorithm found the rule by studying labeled examples, it is machine learning.

1.2 Data, Examples, Features, and Labels

Machine learning always begins with data. In this book, data almost always means a table, very much like a spreadsheet. Each row in the table is one example. Each column in the table is one measured property of that example.

WORD TO KNOW

Dataset — An organized collection of examples used for training or testing a model. In a table, one row is usually one example, and one column is usually one measured property.

Let's go back to the melon market and turn it into a small dataset. Imagine a market inspector who tastes four melons and writes down what she notices about each one. Melon A weighs 4.2 kilograms, made a hollow sound when tapped, and tasted sweet. Melon B weighs 2.8 kilograms, made a dull sound with no hollow ring, and did not taste sweet. Melon C weighs 3.9 kilograms, made a hollow sound, and tasted sweet. Melon D weighs 3.1 kilograms, made a dull sound, and did not taste sweet.

This small collection of four melons, each with three pieces of information, is already a dataset. Weight and sound are the columns we could measure before tasting a melon. Sweetness is the column we are actually trying to predict.

WORD TO KNOW

Feature — One measured property of an example, used as an input to a model. Weight, sound, age, and price can all be features.

WORD TO KNOW

Label — The known, correct answer attached to one training example. A label can be a category, such as "sweet," or a number, such as a price.

In the melon dataset, weight and sound are features. Sweetness is the label. A model trained on this data studies the features of each melon and tries to predict the label. Later, when a new melon arrives without a known sweetness result yet, the model can look at its weight and sound and guess whether it will be sweet, before anyone has to cut it open and taste it.

One more small piece of vocabulary is useful here, even though this book will not turn it into its own boxed term. The row itself, one single melon with all of its features and its label together, is usually called an example. You will see the word "example" used this way constantly through this book: always meaning one complete row of the table, features and label together.

COMMON MISTAKE

✗ **Wrong:** Letting the label leak into the list of features by accident, for example allowing a model to see the sweetness result while it is also trying to predict sweetness.

✓ **Right:** Keep the label completely separate from the features you feed into a model. The model should only ever see columns that would genuinely be available before the true answer is known.

1.3 Three Common Learning Tasks

Not every machine learning project wants the same kind of answer. Some projects want a category. Some want a number. Some do not even have a known answer to learn from, only a pile of examples to sort into groups. This book begins with three of the most common learning tasks, and you will meet all three again, in far more detail, in Parts II and III.

WORD TO KNOW

Classification — A learning task where the model predicts a category, chosen from a fixed, small list of possible answers. "Spam or not spam" and "sweet or not sweet" are both classification tasks.

WORD TO KNOW

Regression — A learning task where the model predicts a number that can take almost any value on a scale. Tomorrow's temperature and a house's selling price are both regression targets.

WORD TO KNOW

Clustering — A learning task where the model groups similar examples together, even though nobody gave the groups a name in advance. The computer finds the groups; a person usually studies each group afterward and decides what it seems to represent.

Classification and regression feel similar at first, because both of them predict something about a new example using its features. The real difference is the shape of the answer. Classification answers always come from a short, fixed list, such as "spam" or "not spam," or "sweet," "not sweet," and "overripe."

Regression answers can be almost any number on a continuous scale, such as 3.7 kilograms, or 21.5 degrees Celsius, or \$184,000. Here is a simple rule of thumb. If you can count the possible answers on your fingers, the task is probably classification. If the answer could be almost any number, with many possible decimal values in between, the task is probably regression.

Clustering is a different kind of task altogether, because it does not use labels at all. Imagine an online store that wants to understand its customers better, but has never asked any customer to describe their own shopping "type."

Clustering can still group customers who buy similar products, at similar times, in similar amounts, into the same cluster, purely by noticing which customers look alike in the data. A person can then look inside each cluster and give it a

sensible name, such as "weekend bargain shoppers," but the clustering method itself never invents that name. It only finds who belongs together.

WORKED EXAMPLE — IDENTIFY THE LEARNING TASK FOR FOUR REAL PROJECTS

A data team has four different projects this month. Project 1: sort incoming emails into "spam" or "not spam." Project 2: predict the price a house will sell for, in dollars. Project 3: group website visitors into similar groups, with no existing group names to learn from. Project 4: predict whether a bank loan will be repaid on time or not repaid on time. Identify the learning task for each project.

1. Project 1 asks for one of two fixed categories, spam or not spam, so Project 1 is classification.
2. Project 2 asks for a price in dollars, which could be almost any number, so Project 2 is regression.
3. Project 3 has no known groups and no labels to learn from, only a request to find similar examples, so Project 3 is clustering.
4. Project 4 asks for one of two fixed categories, repaid or not repaid, so Project 4 is classification.

Answer: Projects 1 and 4 are classification, Project 2 is regression, and Project 3 is clustering. Two projects can use the exact same learning task even when their subjects, emails and bank loans, have nothing else in common.

COMMON MISTAKE

✗ Wrong: Assuming that any label written as a number must mean regression, even when the number is really just a category code, such as "1" for spam and "0" for not spam.

✓ Right: Look at what the number actually means, not just whether it looks like a number. If the values come from a small, fixed set of categories, the task is still classification, even when those categories happen to be written as digits.

1.4 Supervised and Unsupervised Learning

Classification and regression share one important feature: both of them learn from examples that already carry a known, correct answer. Clustering is different, because it learns from examples that have no known answer at all. This difference is important enough that machine learning gives it its own pair of names.

WORD TO KNOW

Supervised learning — Learning from examples that already have a correct label attached. Classification and regression are both supervised learning, because every training example already carries its own answer.

WORD TO KNOW

Unsupervised learning — Learning from examples that have no label at all. Clustering is unsupervised learning, because the model only ever sees the features, never a correct answer.

The word "supervised" is a helpful picture to hold in your mind. Imagine a teacher supervising a student while the student practices a new skill, correcting each mistake as it happens. Supervised learning works in a similar way: every training example already carries the correct answer, so the learning algorithm can check its own guesses against a known truth while it learns, mistake by mistake. Unsupervised learning has no such supervisor watching over it. The learning algorithm only ever sees the raw features, with no answer key at all, and it must find structure using the features alone. This is exactly why clustering can group similar customers together, as in the example above, but can never tell you on its own what makes that group special. Only a person, looking at the finished groups, can add that meaning.

Most of this book focuses on these two families, because together they cover the large majority of real machine learning projects. But it is worth naming one more family here honestly, even though this book will not teach it in depth.

WORD TO KNOW

Reinforcement learning — A third kind of learning, less common in this book, where a computer program called an agent takes actions inside an environment and learns from rewards and penalties, instead of from labeled examples.

Reinforcement learning shows up in projects such as game-playing programs and robots learning to walk, where there is no ready-made table of correct answers, only a score that goes up or down depending on what the agent does. Training a computer to play a board game well is a classic reinforcement learning problem. Nobody hands the program a table of "correct moves" for every possible situation, but the program can play millions of practice games against itself and slowly learn which actions tend to lead to a win. This book stays with supervised and unsupervised learning, which cover the great majority of everyday business, science, and engineering projects, but it is useful to recognize the name "reinforcement learning" when you meet it again elsewhere.

1.5 A Precise Learning Goal: Task, Experience, and Performance

Before choosing any model, a careful machine learning project first answers three simple questions, in plain words. What exactly must the model do? What data will it learn from? How will we know whether it worked? These three questions have a short name in this book: T, E, and P, for Task, Experience, and Performance.

Task, or T, is a plain sentence describing what the model must predict, and for what kind of example. Experience, or E, describes the exact data the model will

learn from: how many examples, and where they came from. Performance, or P, is how success will be measured, once a trained model actually exists. Writing all three down, in order, before touching any data or any model, keeps a project honest and focused from the very first day.

WORKED EXAMPLE — DEFINE A PROJECT USING TASK, EXPERIENCE, AND PERFORMANCE

An online store in Erbil ships thousands of orders every week. A small number of orders turn out to be fraudulent, paid for with a stolen card. The store manager wants to use machine learning to flag risky orders before they ship. Define this project using Task, Experience, and Performance.

1. Task (T): for each new order, decide whether it is likely fraudulent or likely genuine, before the order ships.
2. Experience (E): 5,000 past orders, each one already labeled by the store's fraud team as "fraudulent" or "genuine."
3. Performance (P): measure how many of the truly fraudulent orders the model catches, and separately measure how many genuine orders it wrongly flags, because these two kinds of mistakes cost the store very differently.
4. Only after T, E, and P are written down in plain sentences should the team choose which model to train.

Answer: This project is a classification task: predict "fraudulent" or "genuine" for each order, learn from 5,000 labeled past orders, and judge success by how many real fraud cases are caught and how many honest customers are wrongly bothered, not by accuracy alone. Writing T, E, and P first stops the team from jumping straight to a model before the goal is even clear.

IMPORTANT

A high score on the wrong performance measure can hide a serious problem. For a health project, missing one sick patient is usually far more costly than sending one healthy patient for an extra check. Always choose the performance measure P before choosing a model, and make sure it truly matches what matters most in the real project.

Notice that none of the three questions mention any specific algorithm. That is intentional. A common beginner habit is to start a project by picking a famous algorithm, then forcing the data to fit it. A far safer habit, and the one this whole book follows, is to start with T, E, and P, and only choose an algorithm once those three things are completely clear.

1.6 Training, Testing, and Accuracy

Once T, E, and P are clear, and a dataset is ready, the actual learning can begin. But there is an important trap waiting here, and this book returns to it again and again: a model that looks perfect on the data it studied can still fail badly on new data it has never seen.

WORD TO KNOW

Training — The process of using examples to fit, or adjust, a model. During training, the model changes its internal rule so that it matches the training examples more closely.

To catch this trap, we never judge a model using only the data it trained on. Instead, we set aside a separate group of examples before training even begins, and we do not let the model see this group until training is completely finished.

WORD TO KNOW

Test set — A group of examples kept completely separate from training, then used once to check how well the finished model performs on data it has never seen before.

WORKED EXAMPLE — SPLIT 240 MELONS INTO A TRAINING SET AND A TEST SET

A market inspector collects careful measurements on 240 melons across one harvest season. The team decides to keep 25% of the melons aside as a test set, and use the rest to train the model. How many melons go into each set?

1. Start with all 240 melons.
2. Find 25% of 240, because that fraction becomes the test set: $0.25 \times 240 = 60$.
3. The test set therefore contains 60 melons.
4. The training set gets the rest of the melons: $240 - 60 = 180$.
5. Check the work by adding the two groups back together: $180 + 60 = 240$, which matches the original total.

Answer: 180 melons go into the training set, and 60 melons go into the test set. The model will learn its rule only from the 180 training melons, and the 60 test melons stay hidden until the very end.

Once a model is trained, we can finally check it honestly, by letting it make predictions on the test set and comparing each prediction to the true, known label. One simple, common performance measure for classification counts how many of those predictions were exactly correct.

WORD TO KNOW

Accuracy — The fraction of predictions a model gets exactly right, usually written as a percentage. Accuracy is one common performance measure, though later chapters show it is not always the right one.

EQUATION — ACCURACY

$$\text{Accuracy} = \frac{c}{n}$$

In words: Accuracy equals the number of correct predictions, divided by the total number of predictions made.

c number of correct predictions on the test set (count)

n total number of predictions made on the test set (count)

Note: Multiply the result by 100 to write accuracy as a percentage, so 0.85 becomes 85%.

WORKED EXAMPLE — COMPUTE THE TEST ACCURACY OF THE MELON CLASSIFIER

The trained melon classifier from the previous example is used on all 60 test melons. It correctly predicts sweetness or non-sweetness for 51 of them, and gets the other 9 wrong. Find its test accuracy.

1. Identify the two numbers needed: correct predictions $c = 51$, and total predictions $n = 60$.
2. Divide correct predictions by total predictions: $51 \div 60$.
3. Carry out the division: $51 \div 60 = 0.85$.
4. Turn the result into a percentage by multiplying by 100: $0.85 \times 100 = 85$.

Answer: The model's test accuracy is 85%. Out of every 100 melons like the ones in the test set, the model is expected to correctly judge sweetness for about 85 of them.

COMMON MISTAKE

✗ Wrong: Reporting the accuracy a model achieves on the very same rows it was trained on, and presenting that number as proof the model is ready to use.

✓ Right: Always compute accuracy on a separate test set that the model never saw during training. Training accuracy is almost always too optimistic, sometimes wildly so, and Chapter 5 explains exactly why.

1.7 Worked Example: Build a Classifier by Hand

This lesson builds a complete, simple classifier by hand, from start to finish, using only arithmetic you already know: addition, subtraction, division, and squaring a number. The method is called nearest-centroid classification. Despite its long name, the idea is simple: find the "average melon" for each class, then classify a new melon by seeing which average melon it resembles most closely.

WORD TO KNOW

Centroid — The average position of a group of points. To find a centroid, average each feature separately across every point in the group.

EQUATION — CENTROID OF A GROUP, ONE FEATURE AT A TIME

$$\text{centroid} = \frac{\sum v}{n}$$

In words: A centroid's value for one feature is the sum of that feature's values across every point in the group, divided by the number of points in the group. Repeat this once for each feature, completely separately.

centroid the average value of one feature, across the whole group (same unit as that feature)

$\sum v$ the sum of that one feature's value, added across all n points (same unit as that feature)

n the number of points in the group (count)

Note: A centroid with two features, such as weight and sound score, is really just two of these one-feature averages, computed side by side and never mixed together.

Suppose a market inspector measures two features for four melons she has already tasted: weight, in kilograms, and a sound score from 0 to 10, given by a handheld ripeness meter, where a higher score means a hollower, riper-sounding thump. Two of the melons were ripe. Two of the melons were not yet ripe. We will find the centroid, the "average melon," for each of these two classes, then use both centroids to judge a brand-new melon.

To decide which centroid a new melon is closer to, we need one more small formula: a way to measure distance between two points that each have two features.

EQUATION — SQUARED DISTANCE BETWEEN TWO POINTS (TWO FEATURES)

$$\text{squared distance} = (a_1 - a_2)^2 + (b_1 - b_2)^2$$

In words: To find the squared distance between two points, subtract their first-feature values and square the result, subtract their second-feature values and square that result, then add the two squared results together.

a₁, a₂ the first feature of point 1 and of point 2 (same unit as that feature)

b₁, b₂ the second feature of point 1 and of point 2 (same unit as that feature)

Note: This chapter only compares squared distances to see which one is smaller, so there is no need to take a square root of either one.

WORKED EXAMPLE — CLASSIFY A NEW MELON USING NEAREST CENTROID

Ripe melon 1 weighs 4 kg with a sound score of 8. Ripe melon 2 weighs 6 kg with a sound score of 10. Unripe melon 1 weighs 2 kg with a sound score of 2. Unripe melon 2 weighs 2 kg with a sound score of 4. A new melon, melon Q, weighs 5 kg and has a sound score of 7. Use the nearest-centroid method to classify melon Q as Ripe or Unripe.

1. Find the Ripe centroid's weight by averaging the two Ripe weights: $(4 + 6) \div 2 = 5$.
2. Find the Ripe centroid's sound score by averaging the two Ripe sound scores: $(8 + 10) \div 2 = 9$. The Ripe centroid is (5, 9).
3. Find the Unripe centroid's weight by averaging the two Unripe weights: $(2 + 2) \div 2 = 2$.
4. Find the Unripe centroid's sound score by averaging the two Unripe sound scores: $(2 + 4) \div 2 = 3$. The Unripe centroid is (2, 3).
5. Find the squared distance from melon Q (5, 7) to the Ripe centroid (5, 9). Weight difference: $5 - 5 = 0$, then $0^2 = 0$. Sound difference: $7 - 9 = -2$, then $(-2)^2 = 4$. Add the two squared differences: $0 + 4 = 4$.
6. Find the squared distance from melon Q (5, 7) to the Unripe centroid (2, 3). Weight difference: $5 - 2 = 3$, then $3^2 = 9$. Sound difference: $7 - 3 = 4$, then $4^2 = 16$. Add the two squared differences: $9 + 16 = 25$.

7. Compare the two squared distances. 4 is smaller than 25, so melon Q sits closer to the Ripe centroid than to the Unripe centroid.

Answer: Melon Q is classified as Ripe. Its weight and sound score, taken together, sit much closer to the typical ripe melon than to the typical unripe melon.

Notice that steps 5 and 6 never take a square root. We only need to know which distance is smaller, and squared distances put the two candidates in the same order that true distances would, so the square-root step can simply be skipped to save work.

WORKED EXAMPLE — CLASSIFY A SECOND MELON NEAR THE BOUNDARY BETWEEN CLASSES

Using the same two centroids as before — Ripe at (5, 9) and Unripe at (2, 3) — classify a second new melon, melon R, which weighs 3.5 kg and has a sound score of 5.5.

1. Find the squared distance from melon R (3.5, 5.5) to the Ripe centroid (5, 9). Weight difference: $3.5 - 5 = -1.5$, then $(-1.5)^2 = 2.25$. Sound difference: $5.5 - 9 = -3.5$, then $(-3.5)^2 = 12.25$. Add the two squared differences: $2.25 + 12.25 = 14.5$.
2. Find the squared distance from melon R (3.5, 5.5) to the Unripe centroid (2, 3). Weight difference: $3.5 - 2 = 1.5$, then $1.5^2 = 2.25$. Sound difference: $5.5 - 3 = 2.5$, then $2.5^2 = 6.25$. Add the two squared differences: $2.25 + 6.25 = 8.5$.
3. Compare the two squared distances. 8.5 is smaller than 14.5, so melon R sits closer to the Unripe centroid, even though its numbers sit between the two groups.

Answer: Melon R is classified as Unripe. This example shows why nearest-centroid still gives a clear answer even for a melon that looks "in between": the method always picks whichever centroid is closer, and here Unripe wins clearly, 8.5 against 14.5.

COMMON MISTAKE

✗ Wrong: Comparing raw distances when one feature uses much bigger numbers than another, for example weight measured in grams sitting next to a sound score from 0 to 10 — the large-number feature then quietly dominates every distance.

✓ Right: Put features on similar scales before comparing distances between them. Chapter 3 explains exactly how to do this, using a method called standardization, and Chapter 8 puts it to work on distance calculations just like this one.

The following short program repeats the exact same arithmetic as the worked example above, using the Python programming language and a library called NumPy, which is built for working with lists of numbers.

Python · NumPy

 runnable in the app

```
import numpy as np

ripe = np.array([[4, 8], [6, 10]])
unripe = np.array([[2, 2], [2, 4]])
q = np.array([5, 7])

centroid_ripe = ripe.mean(axis=0)
centroid_unripe = unripe.mean(axis=0)

dist_ripe = ((q - centroid_ripe) ** 2).sum()
dist_unripe = ((q - centroid_unripe) ** 2).sum()

prediction = "Ripe" if dist_ripe < dist_unripe else "Unripe"
print(centroid_ripe, centroid_unripe, dist_ripe, dist_unripe, prediction)
```

The first two lines store the two ripe melons and the two unripe melons as small tables of numbers, and the line after them stores melon Q the same way. The command `mean(axis=0)` averages each column separately, which finds the two centroids exactly like steps 1 through 4 above. The two `dist` lines subtract, square, and add, exactly like steps 5 and 6 above. The last line simply picks whichever class has the smaller distance, exactly like step 7. Running this code prints the same two centroids and the same two distances, 4 and 25, that we already found by hand.

1.8 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Blobs" dataset, which contains two groups of colored points, and choose k-Nearest Neighbors as the model.
3. Press Fit, and look at the colored regions that appear behind the points.
4. Change k from 1 to 25, then press Fit again. Write down what changes about the shape of the colored regions.
5. Now choose Logistic Regression instead, keep the same data, and press Fit once more. Which model drew a straighter boundary between the colors?

You need: Computer, tablet, or phone with the ML Studio app

A model is never simply "good" on its own. It is good, or not good, for a particular dataset and a particular goal. A flexible boundary can follow curved, complicated data well, but it can also follow small accidents in the data too closely. Later chapters, especially Chapter 5, explain exactly how to tell these two situations apart.

CASE STUDY — FINDING STUDENTS WHO MAY NEED EXTRA SUPPORT AT A SECONDARY SCHOOL IN ERBIL

A secondary school in Erbil wants to find students who could benefit from extra academic support, early enough in the term for that support to actually help. Right now, teachers notice struggling students mostly by accident, often not until after a student has already failed a major exam. The school's counseling office wonders whether machine learning could help them notice earlier, so a counselor can step in while there is still time to change the outcome.

The counseling team starts exactly the way this chapter recommends, with Task, Experience, and Performance, instead of picking a model first. The task is to flag, by the end of the fourth week of term, which students may benefit from a short supportive check-in with a counselor. The experience is five years of past student records: attendance rate, number of completed homework assignments, and quiz results from the first four weeks of each past term, together with a label the school already knows for past students, whether that student later needed real academic support that term. The performance measure is not simple accuracy. The counseling team cares much more about not missing a student who truly needed help than about one extra, unnecessary check-in, so they agree to track both kinds of mistakes separately, not just one combined score.

With T, E, and P written down clearly, the features and the label become obvious. The features are attendance rate, completed assignments, and early quiz scores, all measured only through week four. The label is whether that student, looking back, turned out to need real support that term. This is a classification task, because the answer is one of two categories, needs support or does not currently need support, not a number on an open scale.

The team is also careful about what the data cannot tell them. A low attendance rate can come from many different causes: a part-time job, a family responsibility, a long walk to school, or an illness. The data does not know which cause applies to any one student, only the number itself. For this reason, the school agrees that the model's output will only ever create a private list for a trained counselor to review, never an automatic mark on a student's official record, and never a decision made without a caring adult looking at the whole picture first.

IMPORTANT

A prediction is not a fact about a person, and it is not proof of what will happen in the future. A model trained on old records can also learn old, unfair patterns without anyone intending it. Always use a sensitive prediction to offer help and to trigger a caring human review, never to remove a person's opportunity automatically.

COMMON MISTAKES TO AVOID

- Calling every automatic decision "machine learning," even one that only follows fixed rules a person wrote by hand.
- Letting the label leak into the features, so the model can see the very answer it is supposed to predict.

- Assuming a number-shaped label always means regression, even when the number is really a small set of category codes.
- Reporting accuracy measured on the training data itself, instead of on a separate, untouched test set.
- Comparing raw distances between features that sit on very different scales, letting one large-number feature dominate.
- Choosing a model before writing down the task, the experience, and the performance measure in plain sentences.

CHAPTER SUMMARY

- Machine learning builds a model by studying labeled examples; the learning algorithm produces the model, and the model then produces predictions on new, unseen inputs.
- A dataset is a table of examples; features are the input columns, and the label is the known-answer column.
- Classification predicts a category, regression predicts a number, and clustering finds groups with no given labels.
- Supervised learning uses labeled examples; unsupervised learning does not; reinforcement learning learns from rewards instead of labels.
- Every project should state its task (T), experience (E), and performance measure (P) before any model is chosen.
- Training fits a model to data; a separate test set gives an honest check on data the model has never seen.
- Accuracy is the fraction of test predictions that were exactly correct, often written as a percentage.
- A nearest-centroid classifier finds the average point of each class, then predicts whichever class's centroid is closest to a new point.
- A prediction is a helpful guide for a human decision, never proof of the future and never a fact about a person.
- You can now describe the whole loop from raw examples to a fairly tested model — Chapter 2 turns this loop into a complete, repeatable project workflow.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] In your own words, explain the difference between a learning algorithm and a model.

Problem 2 [Basic] A dataset has columns for a car's age, mileage, and selling price. Which column is most likely the label, if the goal is to predict resale value?

Problem 3 [Basic] State whether each task is classification or regression: (a) predicting whether a tumor is dangerous or harmless, (b) predicting a runner's finishing time in minutes.

Problem 4 [Basic] A dataset has 800 rows. The team keeps 20% for testing. How many rows are in the training set, and how many are in the test set?

Problem 5 [Basic] Define supervised learning and unsupervised learning in one sentence each.

Problem 6 [Standard] A classifier gets 72 correct predictions out of 80 test examples. Calculate its test accuracy as a percentage, showing every step.

Problem 7 [Standard] Class A contains the points (2, 2) and (4, 6). Find its centroid, showing the average of each feature separately.

Problem 8 [Standard] Class B has centroid (7, 5). A new point $q = (5, 4)$. Find the squared distance from q to Class B's centroid, showing every step.

Problem 9 [Standard] A shop groups its customers into similar buying patterns without using any pre-existing group names. Name the learning task, and explain in one sentence why it is not classification.

Problem 10 [Standard] Write the Task, Experience, and Performance for a project that predicts whether a delivery will arrive late.

Problem 11 [Challenge] A model reaches 99% accuracy on its training data but only 71% accuracy on its test data. (a) What warning sign does this gap give? (b) Which of the two numbers should you trust more when deciding if the model is ready to use, and why?

Problem 12 [Challenge] Design a small machine learning idea for your own school, workplace, or town. State the task, the experience, the performance measure, and one possible risk if the prediction were used carelessly.

Problem 13 [Challenge] Open ML Studio and fit k-Nearest Neighbors on the Moons dataset, first with $k = 1$ and then with $k = 25$. Describe two visible differences between the two fitted boundaries, and suggest one reason for each difference.

FOUNDATIONS OF MACHINE LEARNING

CHAPTER 2

The Machine Learning Workflow

A safe order for every project

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a project should begin with a decision, not an algorithm
- Name the unit of observation, the target, and the prediction time for a new project
- Explain data leakage, and inspect and split data before any learning happens
- Build a simple baseline and explain why every project needs one
- Use a training set, a validation set, and a test set correctly, in the right order
- Choose between a random split and a time-based split, depending on the project

2.1 Begin with the Decision, Not the Algorithm

Chapter 1 gave you the full loop of machine learning: examples go in, a **model** comes out, and the model then makes **predictions** on new data. This chapter gives you something just as important: a safe order of steps for turning a real-world question into a working project, without falling into the traps that catch so many beginners.

WORD TO KNOW

Workflow — An ordered set of steps used to complete a project in a safe, repeatable way. Following the same workflow every time helps a whole team avoid easy-to-make mistakes.

A machine learning project should always begin with a decision that some person needs to make. It should not begin with the name of an algorithm. "Let's use a neural network" is not a project goal, no matter how impressive it sounds. "Predict how long a patient will wait, so the clinic can tell them at check-in" is a real project goal, because it names a decision, a person who needs it, and a moment when the answer becomes useful.

One simple habit keeps this straight from day one. Before touching any data, write one plain sentence: "At prediction time, the model will use ___ to predict ___, so that ___." The first blank names the information available right now. The second blank names the answer you actually want. The third blank names why anyone cares. This one sentence, written before any data is collected or any code is written, catches an enormous number of problems early, while they are still cheap and easy to fix.

This mistake is easy to fall into, because algorithm names sound exciting and specific, while decisions can feel vague at first. A team that starts with "let's try deep learning" often spends weeks building something technically impressive that nobody asked for and nobody actually uses. A team that starts with "the triage nurse needs to know, within the first two minutes, which patients should be seen soonest" already knows what data to gather, what a correct answer looks like, and how to tell whether the finished project truly helped. The algorithm comes later, almost as a small implementation detail.

WORKED EXAMPLE — WRITE THE DECISION SENTENCE FOR A CLINIC PROJECT

A health clinic in Sulaymaniyah wants patients to understand how long they will wait after checking in. The clinic manager wants to write one decision sentence before any data work begins: "At prediction time, the model will use ___ to predict ___, so that ___." Fill in the three blanks.

1. Decide the moment of prediction: the instant a patient checks in at the front desk, before they take a seat in the waiting room.
2. Decide the information available at that exact moment: the day of the week, the check-in hour, the number of patients already waiting, and the number of clinicians currently on duty.
3. Decide the target, the answer the model must produce: the number of minutes that patient will actually wait before being seen.
4. Decide the purpose, why the prediction matters: so the clinic can show the patient an honest estimate at check-in, instead of no information at all.

Answer: "At prediction time, the model will use the day, hour, number of patients waiting, and number of clinicians on duty to predict how many minutes this patient will wait, so that the clinic can show the patient an honest estimate at check-in." One clear sentence, written first, keeps the whole project pointed at a real decision.

COMMON MISTAKE

✗ Wrong: Choosing a specific algorithm or an impressive-sounding tool first, then trying to force the data to fit it, instead of starting from the real decision a person needs to make.

✓ Right: Write the plain decision sentence first: what the model will use, what it will predict, and why. Choose an algorithm only after that sentence is completely clear.

2.2 Name the Unit, Target, and Prediction Time

The decision sentence from Lesson 2.1 hides three separate ideas inside it, and each one deserves its own careful attention before any data is collected.

WORD TO KNOW

Unit of observation — The real-world thing that one row of a dataset represents. A unit of observation might be one person, one house, one email, one machine, or, as in this chapter's clinic example, one patient visit.

A **dataset** can look perfectly correct while secretly mixing two different units of observation. One row might describe a single patient visit, while another row secretly summarizes an entire day at the clinic. One row's number would describe a single patient's wait, while another row's identical-looking number would actually describe a whole day's busy total, and the model would have no way to tell the two meanings apart. A model trained on such a mixed table receives confusing, inconsistent examples, even though the table looks tidy at first glance. Always state the unit of observation in one plain sentence before collecting a single row.

WORD TO KNOW

Target — The answer a model is trained to predict for each unit of observation. In classification the target is a category; in regression the target is a number.

The target and the **label** you already know from Chapter 1 are closely related, and it is worth being precise about the difference between them. Target is the project-level idea of what you want to predict, decided during planning, before any data even exists. Label is that same answer, once it is actually attached to real training rows inside your dataset. In practice, most people use the two words interchangeably once a project is running, and this book does too, from this point on.

WORKED EXAMPLE — NAME THE UNIT, TARGET, AND PREDICTION TIME FOR THE CLINIC PROJECT

Using the clinic project from Lesson 2.1, name the unit of observation, the target, and the prediction time in one short phrase each.

1. Identify the unit of observation: one patient visit, meaning one single check-in at the clinic on one particular day.
2. Identify the target: the number of minutes that visit's patient waits before being seen by a clinician.
3. Identify the prediction time: the moment of check-in, before any of that waiting has actually happened yet.
4. Confirm that every planned input, such as the current queue length, is genuinely known at that exact moment, and that the target itself is not yet known then.

Answer: Unit of observation: one patient visit. Target: waiting time in minutes. Prediction time: the moment of check-in. Naming these three things clearly stops a common, costly error: accidentally using information from later in the visit, such as the true consultation length, to predict something that must be guessed before the visit even begins.

COMMON MISTAKE

✗ Wrong: Mixing two different units of observation inside one dataset, for example letting some rows represent a single patient visit while other rows secretly represent a whole day at the clinic.

✓ **Right:** Choose exactly one unit of observation for a dataset, and check that every row matches it. If a project truly needs both kinds of rows, build two separate datasets instead of blending them together.

2.3 Inspect and Split the Data Early

With the decision, the unit, the target, and the prediction time all written down clearly, it is finally time to look at real data. But there is a strict order to follow here too, and skipping it is one of the most common mistakes in all of machine learning.

First, inspect the data: check column names, units, missing values, unusual values, and how many examples fall into each category. In the clinic data, an unusual value might be a recorded wait of -5 minutes, which is impossible, or a wait of 400 minutes, which is far more likely to be a typing mistake than a real visit. Catching problems like these early, before any splitting or training, saves the team from teaching a model a lesson learned from broken data. Do not clean, scale, or otherwise learn anything from the complete dataset before splitting it. Some cleaning steps quietly learn something from the data, such as an average or a typical range, and if that learning happens before the split, information from data meant to stay hidden can slip into training without anyone noticing.

WORD TO KNOW

Data leakage — Information entering training that would not actually be available at the real moment of prediction. Leakage makes a model's test score look better than the model will truly perform in real use.

WORKED EXAMPLE — CHECK THREE CANDIDATE FEATURES FOR LEAKAGE

The clinic team is choosing which columns to use as features. Three candidates are suggested: (1) the number of patients already waiting when this patient checks in, (2) the exact length of this patient's consultation with the clinician, and (3) the average waiting time for this same weekday and hour, calculated only from past visits. For each candidate, decide whether it is safe to use or whether it would cause data leakage.

1. Check candidate 1 against the prediction time, the moment of check-in: the number of patients already waiting is fully known at that exact moment, so it is safe to use.
2. Check candidate 2 against the prediction time: the consultation length only becomes known after the visit is completely finished, long after the prediction must be made, so using it would be data leakage.
3. Check candidate 3 against the prediction time: an average built only from past visits, before today, is fully known at check-in, so it is safe to use.

Answer: Candidates 1 and 3 are safe features. Candidate 2 must be thrown out, because it would let the model quietly peek at information from the future, making test scores look wonderful while the real,

deployed system could never actually produce that number in time to use it.

To avoid leakage, split the data into groups before any learning step touches it, and keep those groups strictly separate afterward.

WORD TO KNOW

Validation set — A group of examples, separate from both training and test data, used to compare candidate models and choose settings before ever opening the test set. A model's settings may be adjusted based on validation results, but the test set stays completely untouched until the very end.

The **training** set fits the model's internal rule. The validation set helps a team compare candidate models and choose settings fairly. The **test set** you already know from Chapter 1 gives one final, honest check, opened only once, after every other decision has already been made.

WORKED EXAMPLE — SPLIT 1,200 CLINIC VISITS INTO TRAINING, VALIDATION, AND TEST SETS

The clinic has collected 1,200 past patient visits. The team decides on a 70% training, 15% validation, and 15% test split. How many visits go into each group?

1. Find the training visits: $0.70 \times 1,200 = 840$.
2. Find the validation visits: $0.15 \times 1,200 = 180$.
3. Find the test visits: $0.15 \times 1,200 = 180$.
4. Check the total: $840 + 180 + 180 = 1,200$, which matches the original number of visits.

Answer: The training set gets 840 visits, the validation set gets 180 visits, and the test set gets 180 visits. Every one of the 1,200 visits belongs to exactly one group, never more than one.

A random split like this one works well when every row is roughly independent of every other row. But the clinic's visits happened in a real order, month after month, and waiting times can change with the seasons, with staffing changes, and with local events. When time matters this way, a random split can quietly leak the future into training, simply because a few visits from next month might land in the training set purely by chance.

WORD TO KNOW

Time-based split — A way of splitting data by date instead of by random chance, so that every training example happens before every validation or test example. A time-based split protects a project from a model accidentally learning from the future.

WORKED EXAMPLE — CHOOSE A TIME-BASED SPLIT INSTEAD OF A RANDOM SPLIT

The clinic's 1,200 visits were recorded over the last 12 months, spread evenly at 100 visits per month. Instead of splitting randomly, the team chooses a time-based split: the most recent 2 months become the test set, the 2 months before

that become the validation set, and the remaining older months become the training set. How many visits are in each group?

1. Find how many months belong to training: 12 total months, minus 2 test months, minus 2 validation months, leaves $12 - 2 - 2 = 8$ months.
2. Find the training visits: 8 months $\times$ 100 visits per month = 800 visits.
3. Find the validation visits: 2 months $\times$ 100 visits per month = 200 visits.
4. Find the test visits: 2 months $\times$ 100 visits per month = 200 visits.
5. Check the total: $800 + 200 + 200 = 1,200$, which matches the original number of visits.

Answer: The training set has 800 visits from the oldest 8 months, the validation set has 200 visits from the next 2 months, and the test set has 200 visits from the most recent 2 months. Every training visit happens before every validation visit, and every validation visit happens before every test visit, so the model can never learn from the future by accident.

Notice that this time-based split gives slightly different group sizes than the random 70/15/15 split in the previous example, 800/200/200 instead of 840/180/180, even though both start from the same 1,200 visits. Neither split is "wrong." A random split is usually the right default choice. A time-based split becomes the right choice whenever the order of events could matter to the target, which is true for a great many real projects: demand forecasting, equipment failure, weather, and prices. A simple rule of thumb helps here: ask whether tomorrow's example could ever be influenced by something that only became true today. If the answer is yes, reach for a time-based split instead of a random one.

COMMON MISTAKE

✗ Wrong: Cleaning, scaling, or otherwise learning anything at all from the complete dataset before splitting it into training, validation, and test groups.

✓ Right: Split the data first. Learn any cleaning rule, such as a typical value or a typical range, only from the training set, then apply that exact same rule to the validation and test sets afterward.

2.4 Build a Baseline Before a Complex Model

WORD TO KNOW

Baseline — A simple method used as a minimum comparison for a new model. A useful model should outperform the baseline on the chosen performance measure.

A baseline is deliberately simple, almost boring, on purpose. For a regression project, a common baseline always predicts the average training target, no matter what the **features** say. For a classification project, a common baseline always predicts the most common class in the training data. A baseline takes only minutes to build, and it honestly answers an important question: is a

complicated model actually worth its extra cost and complexity? Skipping this step is tempting, because a baseline feels too simple to bother building. But without one, a team has no honest way to know whether weeks of extra modeling work were actually worth the effort, or whether a two-line rule would have done almost as well.

EQUATION — ABSOLUTE ERROR FOR ONE PREDICTION

$$\text{absolute error} = |y - \hat{y}|$$

In words: The absolute error for one example is the distance between the true value and the predicted value, ignoring whether the prediction was too high or too low.

y the true value for one example (same unit as the target)

$\hat{y}$ the predicted value for one example (same unit as the target)

Note: The vertical bars mean "ignore the sign." An error of -3 and an error of $+3$ both have an absolute error of 3.

WORKED EXAMPLE — BUILD AND TEST A REGRESSION BASELINE FOR WAITING TIME

Before building any complex model, the clinic team builds a baseline: always predict the average waiting time from the training data. Five sample training visits had waiting times of 18, 25, 30, 12, and 20 minutes. Find the baseline's prediction, then find its absolute error on a new visit that actually waited 24 minutes.

1. Add the five training waiting times together: $18 + 25 + 30 + 12 + 20 = 105$.
2. Divide by the number of visits, which is 5, to find the average: $105 \div 5 = 21$.
3. The baseline predicts 21 minutes for every new visit, no matter what the day, hour, or crowd looks like.
4. Find the absolute error on the new visit: the true wait was 24 minutes, so the error is $|24 - 21| = 3$ minutes.

Answer: The baseline predicts 21 minutes for every visit, and on this particular new visit it was off by 3 minutes. Any real model the clinic builds later must beat this simple baseline, across many visits, to be worth using at all.

A baseline can also expose a hidden weakness that a single performance number might otherwise hide, especially for classification.

WORKED EXAMPLE — EVALUATE A CLASSIFICATION BASELINE THAT ALWAYS PREDICTS THE MAJORITY CLASS

Recall the online store's fraud-detection project from Chapter 1: 5,000 past orders, of which 4,900 are genuine and 100 are fraudulent. A simple classification baseline always predicts "genuine" for every single order, no matter what. Find this baseline's accuracy, then explain its hidden weakness.

1. Count the correct predictions: the baseline is correct on every genuine order, which is 4,900 orders, and wrong on every fraudulent order, which is 100 orders.

2. Divide correct predictions by total predictions to find accuracy: $4,900 \div 5,000$.
3. Carry out the division: $4,900 \div 5,000 = 0.98$.
4. Convert the result to a percentage: $0.98 \times 100 = 98$.

Answer: The baseline reaches 98% accuracy, which sounds excellent at first glance, but it catches exactly zero of the 100 fraudulent orders, the very orders the whole project exists to catch. A high accuracy number can hide a completely useless model; Chapter 16 returns to this exact trap in far more detail.

COMMON MISTAKE

✗ Wrong: Reporting a new model's score by itself, with no baseline shown next to it, which can make an ordinary, unimpressive result look like real progress.

✓ Right: Always report a baseline's score next to a new model's score, measured on the exact same data, so anyone reading the result can judge how much real improvement the model actually provides.

2.5 Train, Validate, Test Once

With a baseline in hand, the team can finally train real candidate models and compare them fairly. Training fits each candidate to the training set. The validation set is then used to compare those candidates and choose between them, changing one important setting at a time so the team can understand what actually helped. Only after one final method has been chosen does the team touch the test set, and only once.

Changing only one thing at a time matters more than it might first seem. If a team changes the model type, the input features, and a setting all at once, and the validation score improves, nobody can say afterward which change actually deserves the credit. Patient, one-change-at-a-time comparisons cost a little more time up front, but they build real understanding of what helps and what does not, and that understanding pays off in every future project, not only this one.

IMPORTANT

If you check the test result repeatedly, and keep changing the model afterward based on what you see, the test set quietly turns into another validation set. Its score then stops being an honest final estimate, no matter how carefully it was collected in the first place.

WORKED EXAMPLE — COMPARE TWO CANDIDATE MODELS, THEN TEST ONCE

The clinic team trains two candidate models on the training set. On the validation set, Model 1 has an average error of 6.4 minutes, and Model 2 has an average error of 5.1 minutes. The team selects the better validation model, then checks it, one time only, on the untouched test set, where it scores an average error of 5.6 minutes. Which model does the team choose, and what final error do they report?

1. Compare the two validation errors: 6.4 minutes for Model 1, and 5.1 minutes for Model 2.
2. A smaller error means more accurate predictions, and 5.1 is smaller than 6.4, so Model 2 performs better on the validation set.
3. The team selects Model 2 as the final candidate, based only on this validation comparison.
4. The team opens the test set for the first and only time, and evaluates Model 2 there, finding an average error of 5.6 minutes.
5. This test error, 5.6 minutes, becomes the final, honestly reported number, even though it is slightly worse than the validation error of 5.1 minutes.

Answer: The team chooses Model 2, and reports its final test error as 5.6 minutes. A small rise from the validation score to the test score is normal and expected; the test score is trusted precisely because the team looked at it exactly once.

COMMON MISTAKE

✗ **Wrong:** Stopping after the validation score looks good, and never checking a genuine, untouched test set at all before calling the project finished.

✓ **Right:** Always finish with one honest test-set evaluation, even when the validation score already looks strong. The test set is the closest available preview of how the model will perform in real use.

The following short program shows the same random-splitting idea from Lesson 2.3, using a popular machine learning library called scikit-learn.

Python · NumPy

 runnable in the app

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)
```

The first line takes the full feature table X and target list y , and pulls out 30% of the rows as a temporary holdout, leaving 70% for training, matching Lesson 2.3's numbers. The second line splits that 30% temporary holdout exactly in half, 50/50, which gives 15% for validation and 15% for test, out of the original whole. The `random_state` number simply makes the random split repeatable, so a colleague using the same data and the same number gets the exact same rows in each group. This two-step pattern, splitting once and then splitting the leftover piece again, is the standard way to create three groups using a tool that is only built to split data into two groups at a time.

2.6 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose a regression dataset, such as "Clinic Wait Times," and choose any model you like.
3. Press Fit, and note the baseline score that ML Studio shows automatically next to your model's score.
4. Open the settings panel, choose a 70/15/15 training, validation, and test split, and press Fit again. Compare the training score with the validation score.
5. Switch the split type from Random to Time-based, if the dataset offers a date column, and press Fit once more. Write down whether the validation score changes, and guess why.

You need: Computer, tablet, or phone with the ML Studio app

CASE STUDY — PREDICTING PATIENT WAITING TIME AT A CLINIC IN SULAYMANIYAH

A busy public clinic in Sulaymaniyah receives dozens of walk-in patients every morning, and almost every one of them asks the same question at the front desk: how long will I wait? For years, the honest answer was simply "we do not know." Patients grew frustrated, some left without being seen at all, and staff spent time fielding the same question again and again instead of helping patients directly. The clinic director decides this is exactly the kind of decision machine learning can support, and asks her small data team to follow the safe workflow from this chapter, step by step.

The team starts with the decision sentence from Lesson 2.1: at prediction time, at check-in, the model will use the day, the hour, the number of patients already waiting, and the number of clinicians on duty, to predict how many minutes this patient will wait, so the front desk can share an honest estimate immediately.

The unit of observation is one patient visit. The target is waiting time in minutes. The team is careful to exclude any column that would not truly be known at check-in, such as the eventual length of the consultation itself; including it would quietly leak the future into a prediction that must be made before the future has happened.

Because clinic life changes with the seasons, with flu outbreaks, and with staffing schedules, the team chooses a time-based split rather than a random one, training on the oldest eight months of visits, validating on the next two months, and testing on the most recent two months, exactly as in Lesson 2.3's worked example. Before training anything fancy, they build the simple baseline from Lesson 2.4: always predict the average wait time for that hour of that weekday. This baseline alone already beats "we do not know," and it sets a clear bar for anything more complex to clear. The team then trains two candidate models, compares them fairly on the validation set, and opens the test set exactly once, at the very end, exactly as Lesson 2.5 describes.

The finished system does not print a single, falsely precise number like "17 minutes." Instead, it shows a friendly range, such as "about 15 to 20 minutes," together with a short note that urgent cases are always seen first, regardless of any prediction. The clinic director also asks her team to check the model's numbers every month after launch, because a clinic is a living place: new clinicians join, patient flow shifts, and a model trained on last year's pattern can slowly become outdated if nobody keeps watching it.

Three months after launch, the clinic director reviews the results with her team. Patient complaints about not knowing how long they will wait have dropped noticeably, and the front desk reports calmer mornings, because patients can now decide whether to wait, run a quick errand, or come back later. The team also learns something humbling: the model's estimate is noticeably less accurate on the first Saturday of each month, when a nearby vaccination campaign brings in an unusual wave of walk-ins that the training data barely captured. Rather than quietly ignoring this, the team adds a simple flag for campaign days and plans to retrain once enough campaign-day data has built up. This is exactly the spirit of a safe workflow: not a one-time project, but a habit of checking, learning, and improving.

IMPORTANT

A predicted waiting time is a helpful estimate, not a promise and not a medical judgment. Staff must always keep watching each patient's real condition, and a genuinely urgent case must always be seen immediately, no matter what any prediction says.

COMMON MISTAKES TO AVOID

- Starting a project with an algorithm's name instead of a plain decision sentence.
- Mixing two different units of observation inside a single dataset.
- Cleaning, scaling, or otherwise learning from the complete dataset before splitting it.
- Using a feature that would not actually be known yet at the true moment of prediction.
- Reporting a model's score with no baseline shown next to it for comparison.
- Checking the test set repeatedly and adjusting the model afterward based on what it shows.
- Using a random split when the data's time order could matter to the target.

CHAPTER SUMMARY

- A machine learning project should begin with a plain decision sentence, not with the name of an algorithm.
- The unit of observation is the real-world thing one row represents; keep it consistent across an entire dataset.

- The target is the project-level answer to be predicted; once attached to real rows, it becomes the label.
- Data leakage happens when information that would not be available at prediction time enters training by accident.
- Inspect data first, then split it, before any cleaning or scaling step ever touches the full dataset.
- A training set fits the model, a validation set compares candidates, and a test set is opened only once, at the end.
- A baseline is a deliberately simple method that any real model must beat to prove it is worth using.
- A random split works well when rows are independent; a time-based split protects projects where order matters.
- Checking a test score repeatedly and adjusting the model afterward quietly destroys its honesty as a final estimate.
- With this safe order in hand, you are ready for Chapter 3, which turns raw, messy data into trustworthy model inputs.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] In your own words, explain why a project should start with a decision sentence instead of an algorithm.

Problem 2 [Basic] Define unit of observation, using an example that is not from this chapter.

Problem 3 [Basic] Explain the difference between a target and a label, in one or two sentences.

Problem 4 [Basic] A regression baseline always predicts the average training target. A classification baseline always predicts the most common training class. Explain, in your own words, why both are useful even though neither one is "smart."

Problem 5 [Standard] Split 2,000 rows into 70% training, 15% validation, and 15% test, showing every step.

Problem 6 [Standard] Explain why scaling an entire dataset before splitting it counts as data leakage.

Problem 7 [Standard] A regression baseline predicts 50 for every example. The true values for three new examples are 44, 52, and 61. Find each absolute error, and find their average.

Problem 8 [Standard] A dataset has 9,000 negative examples and 1,000 positive examples. Give a sensible classification baseline, and find its accuracy.

Problem 9 [Standard] Explain, in your own words, why the test set should be opened only once.

Problem 10 [Challenge] A ride-hailing company wants to predict how long a driver will take to reach a passenger. (a) Write the decision sentence. (b) Name the unit of observation, the target, and the prediction time. (c) Explain whether a random split or a time-based split fits this project better, and why.

Problem 11 [Challenge] A dataset has 3,650 days of records, spread evenly across ten years. Design a time-based split that keeps the most recent year for testing, the year before that for validation, and everything older for training. State how many days fall into each group.

Problem 12 [Challenge] Open ML Studio, fit a model on a regression dataset with a random split, then refit it with a time-based split. Compare the two validation scores, and explain in a few sentences which split you trust more for this dataset, and why.

FOUNDATIONS OF MACHINE LEARNING

CHAPTER 3

Data, Features, and Preprocessing

Turning raw records into trustworthy model inputs

IN THIS CHAPTER YOU WILL LEARN TO

- Identify numerical, categorical, ordinal, and text features inside a raw dataset
- Find missing values, likely outliers, and duplicate records before training a model
- Encode categories safely using one-hot encoding, without inventing false order
- Standardize numerical features so different scales become comparable
- Build a pipeline that keeps every preprocessing step and the model together, in order
- Explain why every preprocessing rule must be learned from training data only

3.1 Read the Dataset as a Table of Examples

Imagine a small rental website that lists apartments for rent across Erbil, Duhok, and Sulaymaniyah. Before any of the ideas from Chapter 1 or Chapter 2 can be used, someone has to look closely at the raw table of listings itself. A **dataset** always arrives a little messy. Numbers are sometimes typed wrong. Words are sometimes spelled two different ways. Some cells are simply left blank. This chapter is about cleaning and preparing a table like this one, so that whatever model a later chapter builds is actually trustworthy.

It helps to think of this chapter as the kitchen work that happens before any cooking begins. A skilled cook washes the vegetables, checks the meat is fresh, and measures every ingredient carefully, long before turning on the stove. None of that preparation is the finished dish, but skipping it almost always ruins the meal later, in a way that is hard to fix once cooking has already started. Preprocessing plays exactly this role for machine learning: unglamorous, easy to rush past, and absolutely essential to a trustworthy result.

WORD TO KNOW

Preprocessing — All the steps that prepare raw data for a model, done before training begins. Preprocessing can mean cleaning mistakes, filling in missing values, turning categories into numbers, or putting numbers on a shared scale.

Eventually, a future chapter could train a model on data like this to predict a fair monthly rent for a brand-new listing. Every column such a model would use as an input, size, city, and condition, is a **feature**. The column holding the true

answer the model would try to predict, monthly rent, is the **label**. This chapter focuses entirely on getting the features ready, so whatever model a later chapter builds can actually trust them.

Every column of the rental table plays a different role, and every column also has a different type of value inside it. Learning to name these types is the first step of preprocessing, because the right cleaning method depends completely on which type a column is.

A **numerical** column holds a number where the actual size of that number carries real meaning, and where almost any value on a scale is possible. An apartment's size in square meters is numerical: 85 square meters is genuinely bigger than 60 square meters, and a size of 71.5 square meters is a perfectly normal value too.

A **categorical** column holds a name chosen from a set of options, where the options have no natural order. The city of a listing, such as Erbil, Duhok, or Sulaymaniyah, is categorical. Duhok is not "more" or "less" than Erbil in any numerical sense; the three cities are simply different.

An **ordinal** column is a special kind of categorical column, where the options DO have a natural order, even though they are still names rather than numbers. A listing's condition, rated as Needs Repair, Fair, Good, or Excellent, is ordinal. Excellent is clearly better than Fair, even though neither word is a number by itself.

A **text** column holds free-form sentences that a person typed, such as a listing's description: "Bright two-bedroom apartment near the market, recently painted." Text needs very different handling from the other three types, because a model cannot use raw sentences directly. Chapter 15 returns to text columns in more detail; for now, it is enough to recognize one when you see it.

WORKED EXAMPLE — CLASSIFY THE FEATURE TYPES IN ONE RENTAL LISTING

One row of the rental website's table looks like this: Size = 85 square meters. City = "Erbil". Condition = "Good". Description = "Bright two-bedroom apartment near the market, recently painted." Classify each of these four columns as numerical, categorical, ordinal, or text.

1. Look at Size: its value is a number, 85, and any nearby number, such as 84 or 86.5, would also be a valid, meaningful size. This matches the definition of a numerical feature.
2. Look at City: its value, "Erbil," is one name chosen from a small set of city names, and none of those city names is naturally "greater than" another. This matches the definition of a categorical feature.
3. Look at Condition: its value, "Good," is also chosen from a small set of names, but this time the names have a clear order: Needs Repair, then Fair, then Good, then Excellent. This matches the definition of an ordinal feature.

4. Look at Description: its value is a full sentence written by a person, not a single number or a single category name. This matches the definition of a text feature.

Answer: Size is numerical, City is categorical, Condition is ordinal, and Description is text. Four columns from the very same row can need four completely different kinds of preprocessing, so this classification step always comes first.

COMMON MISTAKE

✗ Wrong: Treating every column that contains digits as a numerical feature, such as a listing's ID number or its building's postal code.

✓ Right: Ask whether the size of the number is actually meaningful. A size of 100 square meters is genuinely twice as large as 50 square meters, so size is numerical. Listing ID 100 is not "twice" listing ID 50 in any real sense, so an ID number should be treated as categorical, not numerical, even though it is written using digits.

3.2 Find Missing, Incorrect, and Duplicate Records

Once you know what type each column is, the next step is to look for problems hiding inside the values themselves. Three problems show up constantly in real data: missing values, values that are likely mistakes, and rows that are accidental duplicates.

WORD TO KNOW

Missing value — A feature value that was not recorded, and is simply blank or empty in the table. A missing value is not automatically zero, and it is not automatically an error; it just means the true value is unknown.

A missing value can happen for many honest reasons. A landlord may not have measured the size of a storage room. A description field may have been left empty because the listing was posted quickly. Before deciding what to do about a missing value, it helps to ask why it might be missing at all, because the reason can matter as much as the value itself.

IMPORTANT

A free-text field, such as a listing's description, can accidentally contain private information: a landlord's personal phone number, a full name, or a home address typed directly into the sentence. Always check text columns for this kind of private detail, and remove or mask it, before that data is stored more widely or used to train a model.

Besides missing values, a column can also contain values that are technically present but clearly wrong: a negative size, a rent of exactly \$0, or a size so different from every other listing that it looks like a typing mistake. This last kind of problem has its own name.

WORD TO KNOW

Outlier — A value that sits far away from most of the other values in its column. An outlier might be a genuine, unusual example, or it might be a mistake, such as an extra digit typed by accident. Only careful checking can tell the two apart.

WORKED EXAMPLE — SPOT A LIKELY DATA-ENTRY ERROR AMONG SIX APARTMENT SIZES

Six training apartments have the following sizes, in square meters: 58, 62, 65, 70, 74, and 640. Decide which value looks like an outlier, and explain whether it is more likely to be a genuine apartment or a data-entry mistake.

1. Look at five of the six values: 58, 62, 65, 70, and 74. All five sit fairly close together, within a range of just 16 square meters of each other.
2. Compare the sixth value, 640, against that same group. It is far larger than any of the other five sizes.
3. Measure how many times larger 640 is than the next-largest size, 74: $640 \div 74 \approx 8.6$. So this one listing claims to be about 8.6 times bigger than the next-largest apartment in the whole group.
4. Consider a likely real-world explanation: someone may have typed an extra digit, and the true size was probably 64.0 square meters, not 640 square meters, which would fit neatly among the other five sizes.

Answer: The value 640 is a likely outlier and a probable data-entry error, most likely a missing decimal point that should have made it 64.0 square meters. The correct next step is not to delete this row silently, but to flag it for a person to confirm, since the true answer is not yet certain.

IMPORTANT

An unusual value is not automatically wrong. A genuinely large, luxury apartment can be real, and quietly deleting every unusual value can throw away true, valuable examples along with real mistakes. Always check before deleting or changing a value, and keep a written note of every change you make, so a teammate could retrace your decision later.

Suppose the rental website's team contacts the landlord of that unusual listing, and confirms the size really was a typing mistake: the true size was 64 square meters. The team corrects this one value, and the six training sizes become 58, 62, 64, 65, 70, and 74. Now imagine a seventh apartment, still waiting to be listed, whose size was never recorded at all. The next worked example fills in that missing value using the training data alone.

WORKED EXAMPLE — FILL A MISSING SIZE USING THE TRAINING MEDIAN

After correcting the typing mistake above, six training apartment sizes, in square meters, are: 58, 62, 64, 65, 70, and 74. A seventh apartment's size is missing. Fill the missing size using the median of the training sizes.

1. Sort the six training sizes from smallest to largest: 58, 62, 64, 65, 70, 74. They are already in order.

2. Count how many training sizes there are: 6 values, which is an even number, so the median will be the average of the two middle values, not a single middle value.
3. Find the two middle values. With 6 sorted values, the middle two are the 3rd and 4th values: 64 and 65.
4. Average the two middle values: $(64 + 65) \div 2 = 129 \div 2 = 64.5$.

Answer: Fill the missing size with 64.5 square meters. This number was learned only from the six training sizes; if it had been learned from validation or test rows too, that quiet peek would count as data leakage, exactly the trap Chapter 2 warned about.

COMMON MISTAKE

✗ **Wrong:** Assuming every blank cell means zero, such as treating a missing rent value as \$0 instead of as "unknown."

✓ **Right:** Treat a missing value as unknown, not as zero. Choose a sensible fill value, such as the median, or use a clear marker meaning "unknown," and learn that rule from the training set only.

A duplicate record causes a quieter kind of damage. If the very same listing is accidentally saved twice, a model treats it as two separate pieces of evidence instead of one, and it can start to overweight whatever that one listing happens to say. Suppose a landlord clicks the "submit" button twice by accident while posting a listing, and the website's system saves it as two identical rows: the same size, the same city, the same condition, the same rent. If nobody catches this, a model sees that one combination of features twice, and quietly treats it as two separate, independent opinions about the rental market, even though only one real apartment exists. A dataset with many such accidental repeats can end up leaning on a handful of over-counted listings far more than it should. Before training anything, it is worth checking a dataset for rows that are exact copies of each other, or copies in every column except an automatically generated ID, and this check should happen early, right after collecting the data and before splitting it into training, **validation**, and **test sets**.

3.3 Encode Categories Carefully

A model, underneath everything, only works with numbers. A categorical column such as City, holding words like "Erbil" or "Duhok," must somehow become numbers before any model can use it. This step needs its own name and its own care.

WORD TO KNOW

Encoding — Turning a category into numbers that a model can use. Different encoding methods are needed for unordered categories and for ordered, ordinal categories.

A tempting shortcut is to simply number the categories: Erbil = 1, Duhok = 2, Sulaymaniyah = 3. This shortcut causes a real problem. Many models pay attention to the size and order of a number, and a model given these codes has

no way to know that 1, 2, and 3 were only arbitrary labels. It may quietly learn that Sulaymaniyah, coded as 3, is somehow "three times" Erbil, coded as 1, even though that idea makes no sense at all for a city name. The safer method for an unordered category is called one-hot encoding.

WORD TO KNOW

One-hot encoding — A way to encode an unordered category by creating one new 0-or-1 column for every possible category value. Exactly one of those new columns is set to 1 for each row, and every other new column is set to 0.

WORKED EXAMPLE — ONE-HOT ENCODE THE CITY COLUMN FOR FOUR LISTINGS

Four rental listings have the following City values, in order: Erbil, Duhok, Erbil, and Sulaymaniyah. One-hot encode the City column, using three new columns: City_Erbil, City_Duhok, and City_Sulaymaniyah.

1. Count how many different city names appear across the training data: Erbil, Duhok, and Sulaymaniyah, which is 3 categories, so one-hot encoding creates exactly 3 new columns.
2. Listing 1's city is Erbil, so its row becomes City_Erbil = 1, City_Duhok = 0, City_Sulaymaniyah = 0.
3. Listing 2's city is Duhok, so its row becomes City_Erbil = 0, City_Duhok = 1, City_Sulaymaniyah = 0.
4. Listing 3's city is Erbil, exactly like Listing 1, so its row becomes City_Erbil = 1, City_Duhok = 0, City_Sulaymaniyah = 0 as well.
5. Listing 4's city is Sulaymaniyah, so its row becomes City_Erbil = 0, City_Duhok = 0, City_Sulaymaniyah = 1.

Answer: Every listing gets exactly one "1" among the three new city columns, always in the column matching its own city, and "0" everywhere else. A model can now use city information without ever being tricked into thinking one city outranks another.

Condition, unlike City, is ordinal, so it does not need one-hot encoding at all. Because Needs Repair, Fair, Good, and Excellent already have a genuine, meaningful order, they can safely become the ordered numbers 1, 2, 3, and 4. This time, a model treating 4 as "more" than 1 is exactly correct, because Excellent genuinely is a better condition than Needs Repair. The rule to remember is simple: keep ordered integers for a column whose order is real, and switch to one-hot encoding the moment that order disappears.

COMMON MISTAKE

✗ Wrong: Numbering an unordered category, such as City = 1, 2, 3 for Erbil, Duhok, Sulaymaniyah, and feeding those numbers straight into a model.

✓ Right: Use one-hot encoding for any category with no real order. Only use ordered integers, like Condition = 1, 2, 3, 4, when the categories genuinely have a meaningful order from low to high.

3.4 Put Numerical Features on Useful Scales

Numerical features rarely arrive on matching scales. An apartment's size might range from about 40 to 200 square meters, while its monthly rent might range from about 200 to 2,000 dollars. A model that measures distance, or that multiplies each feature by a learned weight, can end up quietly dominated by whichever feature simply happens to use bigger numbers. Chapter 1's melon example showed this same trap with weight and sound score. The fix is to put every numerical feature on a shared, fair scale.

WORD TO KNOW

Standardization — Rescaling a numerical feature by subtracting its training mean and dividing by its training standard deviation, so the feature ends up centered around 0, with most values falling between about -2 and 2 .

Standardization leans on two quantities from ordinary statistics: the mean, which is simply the everyday average, and the standard deviation, a single number that describes how spread out a group of values typically is. A small standard deviation means most values sit close to the mean; a large standard deviation means values are spread out widely. Appendix A shows exactly how to compute a standard deviation from scratch, by hand, if you would like to see every one of its steps. In this chapter, the training mean and training standard deviation are simply given, so the lesson can focus on the new idea: the standardization formula itself.

EQUATION — STANDARDIZATION

$$z = \frac{x - \text{mean}}{\text{std}}$$

In words: A standardized value equals the original value, minus the training mean, divided by the training standard deviation.

- z** the standardized value (no unit (a plain number))
- x** the original feature value (same unit as the feature)
- mean** the training data's average value for this feature (same unit as the feature)
- std** the training data's standard deviation for this feature (same unit as the feature)

Note: A standardized value of 0 means "exactly average." A positive value means "above average," and a negative value means "below average," measured in standard deviations.

WORKED EXAMPLE — STANDARDIZE A LARGE APARTMENT'S SIZE

Across the training data, the average apartment size is 80 square meters, and the standard deviation of size is 15 square meters. A new apartment measures 110 square meters. Find its standardized size.

1. Identify the three needed numbers: $x = 110$, $\text{mean} = 80$, $\text{std} = 15$.
2. Subtract the mean from x : $110 - 80 = 30$.
3. Divide that result by the standard deviation: $30 \div 15 = 2$.

Answer: The standardized size is $z = 2$. This apartment's size sits 2 standard deviations above the training average, which marks it as a notably large apartment compared to the rest of the training data.

WORKED EXAMPLE — STANDARDIZE A SMALL APARTMENT'S SIZE

Using the same training mean of 80 square meters and the same training standard deviation of 15 square meters, find the standardized size of a second new apartment that measures only 50 square meters.

1. Identify the three needed numbers: $x = 50$, $\text{mean} = 80$, $\text{std} = 15$.
2. Subtract the mean from x : $50 - 80 = -30$.
3. Divide that result by the standard deviation: $-30 \div 15 = -2$.

Answer: The standardized size is $z = -2$. This apartment's size sits 2 standard deviations below the training average, marking it as a notably small apartment. Comparing the two worked examples together, a size of 110 and a size of 50 sit exactly the same distance from average, just in opposite directions, and their matching z -values of 2 and -2 show that clearly.

IMPORTANT

Chapter 2 warned that cleaning or scaling using the complete dataset causes data leakage. Standardization is a perfect example of this same trap. Always compute the training mean and the training standard deviation from the training set alone, then reuse those exact same two numbers to standardize the validation set, the test set, and every future new listing. Never recompute the mean or standard deviation later using data that is supposed to stay hidden.

3.5 Use a Pipeline to Keep Steps Together

By now, the rental website's team has several separate preprocessing rules: fill a missing size with 64.5, one-hot encode City into three columns, and standardize size using a training mean of 80 and a training standard deviation of 15. Keeping track of several separate rules by hand is risky. A teammate could easily apply them in the wrong order, or use a slightly different mean by accident the second time. The fix is to bundle every rule together into one single, repeatable object.

WORD TO KNOW

Pipeline — An ordered chain of preprocessing steps and a model, bundled together into one single object. A pipeline always applies its steps in the exact same order, using rules learned only once, from the training data.

WORKED EXAMPLE — RUN A TRAINED PIPELINE ON ONE NEW APARTMENT

A pipeline has already learned two rules from training data: standardize size using $\text{mean} = 80$ and $\text{std} = 15$, and one-hot encode City into City_Erbil, City_Duhok, and City_Sulaymaniyah. A brand-new apartment arrives with size =

110 square meters and city = "Duhok". Use the pipeline to turn this raw listing into numbers ready for a model.

1. Start from the new apartment's two raw values: size = 110, city = "Duhok".
2. Apply the pipeline's learned standardization rule to size: $(110 - 80) \div 15 = 30 \div 15 = 2$, exactly the same arithmetic used earlier in this lesson.
3. Apply the pipeline's learned one-hot encoding rule to city: since the city is "Duhok," set City_Erbil = 0, City_Duhok = 1, City_Sulaymaniyah = 0.
4. Combine the results into one row of numbers ready for a model:
standardized size = 2, City_Erbil = 0, City_Duhok = 1, City_Sulaymaniyah = 0.

Answer: The pipeline turns the raw listing "110 square meters, Duhok" into the numbers 2, 0, 1, 0, using the exact same rules learned during training, with no extra effort from anyone. This is exactly why a pipeline matters: it cannot forget a step, and it cannot accidentally use a different mean the second time around.

The following short program builds a pipeline like the one in the worked example, using the scikit-learn library.

Python · NumPy

 runnable in the app

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LinearRegression

preprocess = ColumnTransformer([
    ("scale_size", StandardScaler(), ["size_m2"]),
    ("encode_city", OneHotEncoder(), ["city"]),
])

pipeline = Pipeline([
    ("preprocess", preprocess),
    ("model", LinearRegression()),
])

pipeline.fit(X_train, y_train)
```

The `ColumnTransformer` groups the two preprocessing rules together, telling `StandardScaler` to work only on the size column and `OneHotEncoder` to work only on the city column. The `Pipeline` then chains that whole `preprocess` step directly into a model, here `LinearRegression`, so the two pieces always travel together. Calling `pipeline.fit(X_train, y_train)` does everything at once: it learns the mean and standard deviation for size, learns the list of cities to one-hot encode, and trains the model, all from the training data alone. Later, calling `pipeline.predict(...)` on brand-new listings

automatically repeats the exact same standardization and encoding rules learned here, exactly like the worked example above.

COMMON MISTAKE

✗ Wrong: Manually scaling and encoding data by hand during training, then forgetting to repeat one of those exact same steps later, when the model is actually used on new listings.

✓ Right: Wrap every preprocessing step inside one pipeline object together with the model. The pipeline then repeats the exact same steps, in the exact same order, using the exact same learned numbers, automatically, every single time.

3.6 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Home Listings" dataset, which includes size and price on very different scales, and choose k-Nearest Neighbors as the model.
3. Make sure scaling is turned off in the settings panel, then press Fit, and look at the decision regions.
4. Turn on standardization in the same settings panel, and press Fit again. Write down what changes about the shape of the regions.
5. Guess which single feature was quietly controlling most of the distance calculation before scaling was turned on, and explain, in one sentence, why.

You need: Computer, tablet, or phone with the ML Studio app

3.7 Case Study: Cleaning Building-Energy Meter Data at a University in Duhok

CASE STUDY — CLEANING BUILDING-ENERGY METER DATA AT A UNIVERSITY IN DUHOK

A university in Duhok installed smart electricity meters on its main classroom building two years ago, partly to track ordinary demand, and partly to see how much of that demand its new rooftop solar panels can cover on a sunny day. The facilities office now wants to prepare this meter data for a future forecasting model, one that could predict tomorrow's electricity demand and help the team decide how much power to draw from the grid versus the building's own solar and battery system. Before anyone can build that forecasting model, someone has to make the raw meter data trustworthy, using exactly the steps from this chapter.

The team starts by naming every column's feature type. Hour of day and outdoor temperature are numerical. Day type, recorded as a name such as "Class Day," "Weekend," or "Holiday," is categorical. A short building-occupancy rating, recorded as Low, Medium, or High, is ordinal. A free-text maintenance log, written by hand by the building engineer, is text, and mostly set aside for now. Looking closely at the numbers, the team finds several honest problems. A handful of hourly readings show exactly 0 kilowatt-hours in the middle of a

normal class day, which is almost certainly a meter communication failure, not real zero demand, so these rows are flagged as missing rather than trusted as true zeros. An older sensor recorded temperature in Fahrenheit for six months before being replaced with one that records Celsius, so every Fahrenheit reading first has to be converted using a simple formula before the two halves of the dataset can be compared fairly. The Day Type column also has three different spellings of the university's spring holiday, "Newroz," "Nawroz," and "New Roz," which the team merges into one single, consistent category before any encoding happens.

With the data cleaned, the team applies the rest of this chapter's toolkit. Day Type becomes several one-hot encoded columns, because there is no meaningful order between "Class Day" and "Holiday." Occupancy stays as ordered integers, Low = 1, Medium = 2, High = 3, because that order is real and worth keeping. Hour of day and outdoor temperature are standardized, using a mean and standard deviation learned only from the training months, exactly as this chapter recommends, so that a later distance-based or gradient-based model treats both features fairly. Every one of these steps is wrapped inside a single pipeline, and the team splits the data by time, training on the oldest months and validating on the most recent ones, echoing Chapter 2's warning about order-sensitive data.

Once the cleaned data is ready, the facilities director notices something interesting even before any forecasting model is built: midday demand on sunny class days is visibly lower than midday demand on cloudy class days at the exact same hour and occupancy level, a first hint that the rooftop solar panels really are offsetting a meaningful share of the building's own electricity use. Every correction the team makes along the way, fixing a unit, merging a spelling variant, flagging a suspicious zero, is written down in a short data-quality log, so that if a future result ever looks strange, anyone on the team can trace exactly what was changed, and why, instead of wondering whether some silent, undocumented fix might be the real cause.

The facilities director also insists on one more habit before the project moves any further: rerunning the entire cleaning pipeline on a fresh export of the meter data every few months, rather than treating the first cleaned version as finished forever. New sensors get installed, old ones get replaced, and a spelling variant nobody has seen yet can always appear in next winter's holiday schedule. Building this habit early, while the stakes are still low and no forecasting model depends on it yet, costs the team almost nothing today, and it saves them from repeating the same detective work under pressure later, once a real forecast is quietly relying on this same data every single morning.

IMPORTANT

Careful preprocessing is not a detour on the way to "the real work" of modeling; it usually decides whether a project succeeds at all. A brilliant model trained on mislabeled units, unmerged spelling variants, or silently wrong zeros will confidently produce a wrong answer, and it will do so without ever announcing that anything is wrong.

COMMON MISTAKES TO AVOID

- Treating an ID-like number, such as a listing ID or a postal code, as a true numerical feature.
- Assuming every blank cell means zero, instead of treating it as unknown.
- Deleting or changing an unusual value without checking whether it might be a genuine, real example.
- Using arbitrary integer codes, such as 1, 2, 3, for categories that have no real order.
- Learning a scaling or fill-in rule from the complete dataset instead of from the training set alone.
- Applying preprocessing steps by hand, inconsistently, instead of wrapping them inside one shared pipeline.
- Letting private information hide inside a free-text field without checking it first.

CHAPTER SUMMARY

- A dataset's columns can be numerical, categorical, ordinal, or text; the correct preprocessing method depends on which type a column is.
- A missing value should be treated as unknown, not zero, and any fill-in rule should be learned from training data only.
- An outlier is a value far from the rest of its column; check it carefully before deciding whether it is a mistake or a real, unusual case.
- One-hot encoding turns an unordered category into several 0-or-1 columns, so a model never mistakes category order for meaning that does not exist.
- An ordinal feature's real, natural order can safely become ordered integers, unlike an unordered category.
- Standardization subtracts the training mean and divides by the training standard deviation, putting differently scaled features on fair, comparable ground.
- A pipeline bundles every preprocessing step together with the model, so the exact same steps run automatically, in the exact same order, every time.
- Every preprocessing rule must be learned from the training set alone; applying it to validation, test, or new data instead would cause data leakage.
- With trustworthy features finally in hand, Chapter 4 shows how to measure a trained model's performance honestly.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Explain, in your own words, what preprocessing means and why it happens before training.

Problem 2 [Basic] A hospital dataset has these four columns: patient age, blood type, pain level from 1 to 10, and a doctor's written notes. Classify each column as numerical, categorical, ordinal, or text.

Problem 3 [Basic] Explain, in one or two sentences, why a missing value should not automatically be treated as zero.

Problem 4 [Basic] Find the median of these five delivery times, in minutes: 22, 18, 35, 20, 26.

Problem 5 [Standard] Four fruit-stand records have the Fruit values: apple, banana, apple, cherry. One-hot encode this column, listing the resulting 0-or-1 columns for each of the four records.

Problem 6 [Standard] A feature has a training mean of 50 and a training standard deviation of 5. Standardize a new value of 62, showing every step.

Problem 7 [Standard] A feature has a training mean of 200 and a training standard deviation of 25. Standardize a new value of 150, showing every step, and explain in one sentence whether this value is above or below average.

Problem 8 [Standard] Explain, in your own words, why fitting a scaler on the complete dataset before splitting it counts as data leakage.

Problem 9 [Standard] Give an example of a categorical feature and an example of an ordinal feature, neither one taken from this chapter, and explain the difference between them.

Problem 10 [Standard] Explain, in your own words, why a pipeline is safer than applying preprocessing steps by hand each time.

Problem 11 [Challenge] Seven recorded temperatures, in °C, are: 19, 21, 20, 22, 18, 20, and 95. (a) Identify the likely outlier, and explain why it looks unusual. (b) Find the median of the other six temperatures. (c) Describe what the team should do with the unusual value, instead of simply deleting it.

Problem 12 [Challenge] Describe a dataset from your own school, workplace, or town that includes at least one numerical, one categorical, one ordinal, and one text feature. For each feature, name one preprocessing step it would need.

Problem 13 [Challenge] Open ML Studio, and fit k-Nearest Neighbors on a dataset with two numerical features on very different scales, first without standardization and then with it. Describe two visible differences between the two fitted boundaries, and explain what caused each difference.

FOUNDATIONS OF MACHINE LEARNING

CHAPTER 4

Training, Validation, and Testing

Measuring the model honestly

IN THIS CHAPTER YOU WILL LEARN TO

- Choose a performance metric that matches a project's real goal, before comparing any models
- Build and read a confusion matrix, and identify true positives, false positives, false negatives, and true negatives
- Calculate precision, recall, and F1 score by hand from a confusion matrix
- Calculate MAE, MSE, and RMSE by hand for a regression model's predictions
- Use cross-validation to get a more reliable estimate of a model's performance
- Explain why a single score is almost never the whole story

4.1 A Metric Must Match the Real Goal

A regional water utility keeps hundreds of hand pumps and small motorized pumps running across farming villages in Punjab. A pump that fails without warning can leave an entire village without safe drinking water for days, until a technician can travel out and repair it. The utility cannot afford to send a technician to inspect every single pump every single week, so it installs simple sensors, measuring vibration, flow rate, and motor temperature, on each pump, hoping a model can flag which pumps are likely to fail soon, so a technician can reach the right pump first.

This project already uses several ideas from earlier chapters, even before any new one is introduced. The **unit of observation** is one pump, checked at one point in time. The **features** are the sensor readings: vibration, flow rate, and motor temperature. The **target** is whether that pump will fail within the next two weeks. What this chapter adds is a careful, honest way to grade the finished model, once it exists, instead of just trusting a single, comfortable-looking number.

Once such a model is trained, the team needs an honest way to say how good it actually is. **Accuracy**, from Chapter 1, is one obvious choice, but it is not the only one, and it is not always the right one.

WORD TO KNOW

Metric — A number used to measure how well a model is performing. Different metrics pay attention to different kinds of mistakes, so choosing the right metric for a project matters as much as choosing the right model.

A metric is never just a mathematical detail; it is a statement about which mistakes matter most. This water utility's model can make two very different kinds of mistakes. It can miss a pump that is truly about to fail, leaving a village without water and without warning. Or it can raise a false alarm on a pump that was actually fine, costing a technician a wasted trip. These two mistakes do not cost the same amount. A missed failure can mean days without clean water for an entire village; a false alarm costs one technician one unnecessary visit. Any metric this team chooses should reflect that real difference, not treat every mistake as equally small.

IMPORTANT

Choose a performance measure that reflects what a mistake actually costs in the real world, not just whichever number is easiest to compute. For this water utility, missing a genuine failure can leave a village without safe drinking water for days; a false alarm costs one technician one unnecessary trip. These are not equally serious mistakes, and a fair metric should not pretend that they are.

COMMON MISTAKE

✗ **Wrong:** Training several candidate models first, then picking whichever metric happens to make the best-looking model look even better.

✓ **Right:** Decide which metric matters most before training or comparing any models, based on what each kind of mistake would truly cost in the real project.

Most days, the great majority of pumps are working perfectly fine. This matters, because a **baseline** that always predicts "Normal" could reach a very high accuracy score without ever catching a single true failure, exactly the way Chapter 2's fraud-detection baseline reached 98% accuracy while catching zero fraud. The next lesson introduces a tool built exactly for seeing past this trap: the confusion matrix.

4.2 Read a Confusion Matrix

For the rest of this lesson, call a pump the model flags as "Will Fail Soon" a positive prediction, and a pump it leaves as "Normal" a negative prediction. Every single prediction a classifier makes falls into exactly one of four groups, depending on what was predicted and what was actually true.

WORD TO KNOW

Confusion matrix — A small table that sorts every prediction a classifier makes into one of four groups, by comparing what the model predicted against what was actually true. The four groups are usually shortened to four letters. table: TP (true positive) | the true class was positive | the model predicted positive too, so the prediction was correct FN (false negative) | the true class was positive | the model predicted negative, so the prediction was wrong — a miss FP (false positive) | the true class was negative | the model predicted positive, so the prediction was wrong — a false alarm TN (true negative) | the

true class was negative | the model predicted negative too, so the prediction was correct

Notice that two of these four groups, TP and TN, are the model's correct predictions, and the other two, FN and FP, are its two different kinds of mistakes. For this water utility, "positive" means "Will Fail Soon," so a false negative is a genuine failure the model missed, and a false positive is a false alarm on a pump that was actually fine. The name "confusion matrix" simply describes what the table shows: exactly where the model's predictions and the true answers agree, and exactly where they become "confused" with one another.

WORKED EXAMPLE — BUILD A CONFUSION MATRIX FOR 50 PUMP INSPECTIONS

Over one month, technicians follow up on 50 pumps that the model made a prediction for. The model predicted "Will Fail Soon" for 12 of these pumps, and "Normal" for the other 38. Looking back at what genuinely happened, 10 of the 50 pumps truly failed within two weeks, and 40 did not. Of the 10 pumps that truly failed, the model had correctly flagged 9 of them in advance. Find TP, FN, FP, and TN.

1. Identify true positives (TP): pumps that truly failed AND were correctly flagged by the model in advance. The problem states this directly: $TP = 9$.
2. Identify false negatives (FN): pumps that truly failed but the model did NOT flag. Since 10 pumps truly failed in total, and the model caught 9 of them: $FN = 10 - 9 = 1$.
3. Identify false positives (FP): pumps the model flagged as "Will Fail Soon" that did NOT actually fail. Since the model flagged 12 pumps in total, and 9 of those flags were correct: $FP = 12 - 9 = 3$.
4. Identify true negatives (TN): pumps that did not fail and were correctly left unflagged. Since 40 pumps did not fail in total, and 3 of those were wrongly flagged: $TN = 40 - 3 = 37$.
5. Check the total: $TP + FN + FP + TN = 9 + 1 + 3 + 37 = 50$, which matches the 50 pumps in the study.

Answer: $TP = 9$, $FN = 1$, $FP = 3$, $TN = 37$. The model caught 9 of the 10 pumps that truly needed urgent attention, missed just 1 of them, and sent technicians on 3 unnecessary trips.

WORKED EXAMPLE — FIND THE ACCURACY OF THE PUMP MODEL

Using the confusion matrix from the previous example, $TP = 9$, $FN = 1$, $FP = 3$, $TN = 37$, find the model's overall accuracy.

1. Find the number of correct predictions: correct predictions = $TP + TN = 9 + 37 = 46$.
2. Find the total number of predictions: total = $TP + FN + FP + TN = 9 + 1 + 3 + 37 = 50$.
3. Divide correct predictions by total predictions: $46 \div 50 = 0.92$.
4. Convert the result to a percentage: $0.92 \times 100 = 92$.

Answer: The model's accuracy is 92%. That single number sounds strong, but it hides one important detail: 1 of the 10 pumps that truly needed urgent repair was missed completely, exactly the kind of detail Lesson 4.3 looks at next.

COMMON MISTAKE

✗ Wrong: Looking only at the correct-prediction cells of a confusion matrix, TP and TN, and ignoring what the false negatives and false positives actually represent in the real project.

✓ Right: Always read all four numbers. In this water pump project, the single false negative is arguably the most important pump of the entire month, because it is the one that failed with no warning at all.

4.3 Precision, Recall, and F1

Accuracy treats every correct prediction the same, and it treats every mistake the same, too. Precision and recall split accuracy's single number into two more specific questions, each one paying attention to a different half of the confusion matrix. Together, they let a team see past a single misleading number, straight to the two separate things that number was quietly hiding.

WORD TO KNOW

Precision — Among all the examples a model predicted as positive, the fraction that were genuinely positive. Precision answers the question, "when this model raises an alarm, how often is it right?"

EQUATION — PRECISION

$$\text{Precision} = \frac{TP}{TP + FP}$$

In words: Precision equals true positives, divided by every example the model predicted as positive, whether that prediction was right or wrong.

TP true positives: correctly predicted as positive (count)

FP false positives: wrongly predicted as positive (count)

Note: Precision says nothing about positive cases the model missed entirely; that is recall's job, covered next.

WORKED EXAMPLE — CALCULATE THE PUMP MODEL'S PRECISION

Using the confusion matrix from Lesson 4.2, TP = 9 and FP = 3, find the model's precision.

1. Identify the two needed numbers: TP = 9, FP = 3.
2. Add them together to find the total number of positive predictions: $9 + 3 = 12$.
3. Divide TP by that total: $9 \div 12 = 0.75$.
4. Convert the result to a percentage: $0.75 \times 100 = 75$.

Answer: Precision is 0.75, or 75%. Of the 12 pumps the model flagged as "Will Fail Soon," 9 of them, or 75%, truly were about to fail.

WORD TO KNOW

Recall — Among all the examples that were genuinely positive, the fraction the model actually found. Recall answers the question, "of all the true cases out there, how many did this model catch?"

EQUATION — RECALL

$$\text{Recall} = \frac{TP}{TP + FN}$$

In words: Recall equals true positives, divided by every example that was genuinely positive, whether the model caught it or missed it.

TP true positives: correctly predicted as positive (count)

FN false negatives: genuinely positive examples the model missed (count)

Note: Recall says nothing about false alarms; that is precision's job, covered above.

WORKED EXAMPLE — CALCULATE THE PUMP MODEL'S RECALL

Using the same confusion matrix, $TP = 9$ and $FN = 1$, find the model's recall.

1. Identify the two needed numbers: $TP = 9$, $FN = 1$.
2. Add them together to find the total number of genuinely positive pumps: $9 + 1 = 10$.
3. Divide TP by that total: $9 \div 10 = 0.9$.
4. Convert the result to a percentage: $0.9 \times 100 = 90$.

Answer: Recall is 0.9, or 90%. Of the 10 pumps that truly failed, the model caught 9 of them, or 90%, before they broke down completely.

Precision and recall often pull in opposite directions. A model can raise its recall by flagging more pumps, but doing so usually lowers its precision, because some of those extra flags will be wrong. A single score that balances both numbers together is useful, and it has its own name.

WORD TO KNOW

F1 score — A single score that combines precision and recall, using a kind of average that only stays high when both precision and recall are high. F1 drops sharply if either precision or recall is low, even if the other one is excellent.

EQUATION — F1 SCORE

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

In words: F1 equals 2, times precision, times recall, divided by the sum of precision and recall.

Precision fraction of positive predictions that were correct (value from 0 to 1)

Recall fraction of true positives the model found (value from 0 to 1)

Note: A shortcut gives the same answer straight from the confusion matrix: $F1 = (2 \times TP) \div (2 \times TP + FP + FN)$.

WORKED EXAMPLE — COMBINE PRECISION AND RECALL INTO THE PUMP MODEL'S F1 SCORE

Using precision = 0.75 and recall = 0.9, found in the two worked examples above, calculate the model's F1 score.

1. Multiply precision and recall: $0.75 \times 0.9 = 0.675$.
2. Multiply that result by 2: $2 \times 0.675 = 1.35$.
3. Add precision and recall together: $0.75 + 0.9 = 1.65$.
4. Divide the result of step 2 by the result of step 3: $1.35 \div 1.65 \approx 0.818$.
5. Convert the result to a percentage: $0.818 \times 100 \approx 81.8$.

Answer: $F1 \approx 0.818$, or about **81.8%**. This single score sits between precision (75%) and recall (90%), pulled down toward whichever of the two numbers was weaker, which is exactly what makes F1 useful for comparing models at a glance.

IMPORTANT

No single score, including F1, tells the whole story by itself. Always ask what a model's specific false negatives and false positives would cost in the real project, not only which model has the highest score on a leaderboard.

COMMON MISTAKE

✗ Wrong: Assuming precision and recall always rise and fall together, or using the two words interchangeably in conversation.

✓ Right: Keep the two questions separate. Precision asks, "of the positive flags, how many were right?" Recall asks, "of the true positives, how many did we find?" A model can have high precision and low recall, or the reverse.

4.4 Regression Errors in Real Units

The water utility later builds a second, different kind of model. Instead of a plain yes-or-no flag, this second model predicts a number: roughly how many days remain before a given pump is likely to need maintenance. This is a **regression** task, from Chapter 1, and it needs its own kind of metric, one that describes error using the target's own real-world unit, days, instead of TP, FP, FN, and TN.

WORD TO KNOW

Mean absolute error (MAE) — The average size of a regression model's errors, after ignoring whether each one was too high or too low. MAE stays in the same real-world unit as the target itself.

EQUATION — MEAN ABSOLUTE ERROR (MAE)

$$\text{MAE} = \frac{\sum |y - \hat{y}|}{n}$$

In words: Mean absolute error equals the sum of every example's absolute error, divided by the number of examples.

- y** the true value for one example (same unit as the target)
- $\hat{y}$** the predicted value for one example (same unit as the target)
- n** number of examples (count)

Note: The vertical bars mean "ignore the sign," so an error of -3 and an error of $+3$ both count as an absolute error of 3.

WORKED EXAMPLE — CALCULATE MAE FOR FOUR PUMP MAINTENANCE PREDICTIONS

For four pumps, the true number of days until maintenance was needed were 10, 25, 15, and 30. The model's predicted days were 12, 20, 18, and 26. Find the MAE.

1. Find each error, true minus predicted: $10 - 12 = -2$. $25 - 20 = 5$. $15 - 18 = -3$. $30 - 26 = 4$.
2. Take the absolute value of each error, dropping its sign: $|-2| = 2$. $|5| = 5$. $|-3| = 3$. $|4| = 4$.
3. Add the four absolute errors together: $2 + 5 + 3 + 4 = 14$.
4. Divide by the number of examples, which is 4: $14 \div 4 = 3.5$.

Answer: MAE = 3.5 days. On average, this model's prediction is about 3.5 days away from the true number of days until a pump needs maintenance.

MAE treats every error the same size, no matter how large it is. Sometimes a project cares more about avoiding a few very large errors than about the average error size. That is exactly what the next metric is built for.

WORD TO KNOW

Mean squared error (MSE) — The average of a regression model's squared errors. Squaring each error removes its sign automatically, and it also makes large errors count for much more than small ones.

EQUATION — MEAN SQUARED ERROR (MSE)

$$\text{MSE} = \frac{\sum (y - \hat{y})^2}{n}$$

In words: Mean squared error equals the sum of every example's squared error, divided by the number of examples.

- y** the true value for one example (same unit as the target)
- $\hat{y}$** the predicted value for one example (same unit as the target)
- n** number of examples (count)

Note: MSE's unit is the target's unit, squared, such as "days squared," which is hard to picture in the real world. This is exactly why RMSE, below, exists.

WORKED EXAMPLE — CALCULATE MSE FOR THE SAME FOUR PREDICTIONS

Using the same four errors from the MAE worked example above, -2 , 5 , -3 , and 4 , find the MSE.

1. Square each error: $(-2)^2 = 4$. $(5)^2 = 25$. $(-3)^2 = 9$. $(4)^2 = 16$.
2. Add the four squared errors together: $4 + 25 + 9 + 16 = 54$.
3. Divide by the number of examples, which is 4: $54 \div 4 = 13.5$.

Answer: $\text{MSE} = 13.5$ "days squared." Notice the one large error, 5 days on the second pump, contributes 25 of the total 54, almost half, far more than its fair one-quarter share, exactly because squaring punishes bigger errors more.

WORD TO KNOW

Root mean squared error (RMSE) — The square root of MSE. Taking the square root undoes the squaring step, bringing the error back into the target's original, real-world unit.

EQUATION — ROOT MEAN SQUARED ERROR (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

In words: Root mean squared error equals the square root of the mean squared error.

MSE mean squared error, already computed (target's unit, squared)

Note: RMSE is always at least as large as MAE on the same set of errors, and the gap between them grows wider whenever a few errors are much larger than the rest.

WORKED EXAMPLE — CALCULATE RMSE FROM THE MSE ALREADY FOUND

Using $\text{MSE} = 13.5$ "days squared," found in the worked example above, find RMSE.

1. Start from $\text{MSE} = 13.5$.
2. Take the square root of 13.5: $\sqrt{13.5} \approx 3.67$.

Answer: $\text{RMSE} \approx 3.67$ days. Compare this to MAE, which was 3.5 days on the exact same four predictions. RMSE came out a little higher than MAE, because the one unusually large error, 5 days, pulled it upward more than it pulled MAE.

The following short program checks these three regression metrics using the scikit-learn library, on the exact same four numbers used by hand above.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error

y_true = [10, 25, 15, 30]
y_pred = [12, 20, 18, 26]

mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)

print(mae, mse, rmse)
```

The lists `y_true` and `y_pred` store the same four true values and four predicted values used in the worked examples above. The function `mean_absolute_error` repeats steps 1 through 4 of the MAE worked example in one line, and `mean_squared_error` repeats the MSE worked example the same way. Since scikit-learn has no separate RMSE function, the last line simply takes the square root of `mse` using NumPy, matching the RMSE worked example exactly. Running this code prints 3.5, 13.5, and about 3.674, the same three numbers already found by hand.

COMMON MISTAKE

✗ Wrong: Comparing an MAE of 3.5 days from one model to an RMSE of 500 dollars from a completely different model, as if a smaller number always means a better model.

✓ Right: Only compare error metrics that share the same unit and the same target. MAE and RMSE are meaningful next to a baseline, or next to another model trained on the exact same target, never on their own.

4.5 Cross-Validation Uses Data Efficiently

The **validation set** from Chapter 2 gives one honest score for comparing candidate models. But that one score depends partly on which rows happened to land in the validation set. A different, equally fair split might have given a slightly different score, just by chance. Cross-validation is a way to reduce this luck factor, by reusing the training data more than once, in a careful, structured way.

WORD TO KNOW

Cross-validation — A method that repeats training and validation across several different splits of the training data, then combines the resulting scores into one more reliable estimate. The test set stays completely untouched during every round.

In five-fold cross-validation, the training data is divided into five roughly equal groups, called folds. The model trains on four of the five folds, and validates on

the one fold left out. This is repeated five times in total, leaving out a different fold to validate on each time, so that every single row gets used for validation exactly once, and used for training four times. The five resulting scores are then combined, usually by averaging them, into one final estimate. Because every row eventually gets a turn as validation data, cross-validation squeezes more honest use out of a limited training set than a single, one-time split ever could.

WORKED EXAMPLE — COMBINE FIVE CROSS-VALIDATION SCORES FOR THE PUMP MODEL

The pump classifier is evaluated using five-fold cross-validation, giving these five F1 scores: 0.75, 0.80, 0.78, 0.73, and 0.79. Find the average cross-validation score, and find the range between the highest and lowest fold.

1. Add all five scores together: $0.75 + 0.80 + 0.78 + 0.73 + 0.79 = 3.85$.
2. Divide by the number of folds, which is 5: $3.85 \div 5 = 0.77$.
3. Find the highest score among the five folds: 0.80.
4. Find the lowest score among the five folds: 0.73.
5. Subtract the lowest from the highest to find the range: $0.80 - 0.73 = 0.07$.

Answer: The average cross-validation F1 score is 0.77, with a range of 0.07 between the best and worst fold. That small range is reassuring: the model performs fairly consistently no matter which rows happen to sit in the validation fold.

Cross-validation is especially useful with a smaller dataset, where setting aside one large validation set would leave too little data for training. It does not replace the **test set**; the test set is still opened only once, at the very end, exactly as Chapter 2 describes, after cross-validation has already helped choose the final model and its settings.

Five folds is a common choice, but it is not the only one. Using more folds, such as ten, means each model trains on more data per fold and the final average tends to be even more reliable, at the cost of training the model that many more times. Using fewer folds trains faster but leans on less data each round, and can leave a noisier estimate. Five folds is usually a comfortable, practical balance between these two costs, which is exactly why it is such a common default choice.

COMMON MISTAKE

✗ Wrong: Using ordinary random cross-validation on data where the order matters, such as pump sensor readings collected over time, letting some folds accidentally train on data from after the validation fold.

✓ Right: When time order matters, use a **time-based split**, as Chapter 2 introduced, or a time-respecting version of cross-validation, instead of ordinary random folds.

4.6 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.

2. Choose the "Pump Failures" dataset, an imbalanced two-class dataset, and fit any classifier.
3. Open the confusion matrix panel, and write down TP, FP, FN, and TN.
4. Find the decision threshold slider, move it lower so the model flags more pumps as "Will Fail Soon," and press Fit again. Watch how precision and recall change.
5. Keep adjusting the threshold until recall reaches at least 90%. Write down what happened to precision at that same threshold, and explain, in one sentence, why.

You need: Computer, tablet, or phone with the ML Studio app

4.7 Case Study: Detecting Faulty Water Pumps from Sensor Data

CASE STUDY — DETECTING FAULTY WATER PUMPS FROM SENSOR DATA IN RURAL PUNJAB

The water utility from this chapter's running example serves more than three hundred villages across rural Punjab, most of them relying on a single shared pump for drinking water, irrigation, or both. A sudden pump failure is never just an inconvenience; it can mean a full day's walk to the next working water source until a technician arrives. With only a small maintenance team and a large territory to cover, the utility's engineers decide to build the sensor-based model this chapter has followed from the start, and to evaluate it exactly as this chapter recommends, using more than one honest metric.

The team begins by writing down what each kind of mistake would cost, before training a single model. A missed failure, a false negative, could leave a village without water for days. A false alarm, a false positive, costs one technician one wasted visit, an annoyance but not a crisis. Given this real difference in cost, the team decides recall matters more than precision for this project: catching genuine failures matters more than avoiding every unnecessary trip, though precision still cannot be ignored completely, since a technician who receives constant false alarms may eventually start ignoring the system altogether. The team settles on tracking recall, precision, and F1 together, rather than leaning on accuracy alone, exactly because most pumps are fine on any given day and accuracy alone would hide the very failures the project exists to catch.

Using five-fold cross-validation on two full years of sensor readings, the team compares three candidate models and settles on the one with the strongest, most consistent recall across folds. On the untouched test set, opened only once, the final model reaches a recall of 90% and a precision of 75%, numbers that echo this chapter's own worked examples. In plain language, the model finds 9 out of every 10 pumps that are truly about to fail, at the cost of about 1 unnecessary technician visit for every 3 real warnings. The engineering team judges this trade acceptable, but only after showing the actual numbers, in plain language, to the field technicians who will act on the model's flags every day.

Three months after launch, the team reviews real results instead of only test-set numbers. Technicians report that the flagged pumps do fail noticeably more often than pumps chosen at random for routine inspection, and villages served by flagged pumps see fewer surprise multi-day outages. The team also learns something humbling: recall drops during the monsoon season, when water pressure naturally becomes more unpredictable across the whole network, and a few genuine failures slip through unflagged. Rather than hiding this seasonal weakness, the team documents it clearly and plans to retrain the model with a full monsoon season of new data, once enough of it has been collected.

The engineers also learn to explain their metrics in village meetings, not only in technical reports. Telling a village council "our model finds about 9 out of every 10 pumps that are truly about to fail" means far more to them than any single decimal number would, and it also sets honest expectations: the model will occasionally miss a failure, and it will occasionally send a technician who finds nothing wrong. Framing both kinds of mistakes openly, before they happen, turns a future misstep into an expected, already-understood part of the system, rather than a broken promise.

IMPORTANT

A model's flag is a reason to send a technician for a closer look, never a replacement for that technician's own judgment once they arrive. A pump can fail in a way the sensors never learned to recognize, and a technician's own inspection remains the final, trusted check.

IMPORTANT

Report a model's honest mistakes, such as a seasonal drop in recall, clearly and early, to the people who depend on its flags every day. A team that hides a known weakness, hoping nobody notices, risks losing the trust that makes the whole system worth using in the first place.

COMMON MISTAKES TO AVOID

- Judging a model by accuracy alone, especially when the positive class is rare.
- Reading only the correct-prediction cells of a confusion matrix, and ignoring what the false negatives and false positives actually mean.
- Assuming precision and recall always rise and fall together.
- Comparing error metrics like MAE or RMSE across two different targets or units, as if a smaller number always means a better model.
- Choosing a metric only after already seeing which model it would favor.
- Using ordinary random cross-validation when the data's time order matters.
- Reporting a single score with no baseline, and no discussion of what its specific mistakes would cost.

CHAPTER SUMMARY

- A metric must match the real cost of a project's mistakes; choose it before comparing any models, not after.

- A confusion matrix sorts every classification prediction into true positive, false positive, false negative, or true negative.
- Accuracy counts every correct prediction, but it can hide a model that misses almost every rare, important positive case.
- Precision measures how many positive predictions were actually correct; recall measures how many true positives were actually found.
- F1 combines precision and recall into one score that stays low whenever either one of them is low.
- MAE measures the average size of a regression model's errors, expressed in the target's own real-world unit.
- MSE squares each error before averaging, giving extra weight to large mistakes; RMSE takes the square root of MSE to return to the original unit.
- Cross-validation repeats training and validation across several different splits of the training data, then combines the results into one more reliable score.
- A single score is never the whole story; always ask what a model's specific mistakes would cost in the real world.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define metric in your own words, and explain why the metric should be chosen before comparing models.

Problem 2 [Basic] Without looking back at the chapter, define TP, FP, FN, and TN in your own words.

Problem 3 [Basic] A classifier has $TP = 30$, $TN = 50$, $FP = 10$, and $FN = 10$. Find its accuracy, showing every step.

Problem 4 [Basic] Using the same numbers as problem 3, explain in one or two sentences what the 10 false negatives would mean in a real project of your choice.

Problem 5 [Standard] Using $TP = 30$ and $FP = 10$ from problem 3, calculate precision, showing every step.

Problem 6 [Standard] Using $TP = 30$ and $FN = 10$ from problem 3, calculate recall, showing every step.

Problem 7 [Standard] Using your precision and recall from problems 5 and 6, calculate F1, showing every step.

Problem 8 [Standard] True values are 20, 30, 50, and 40. Predicted values are 18, 34, 45, and 44. Find the MAE, showing every step.

Problem 9 [Standard] Using the same true and predicted values from problem 8, find the MSE and the RMSE, showing every step.

Problem 10 [Standard] Five-fold cross-validation gives these scores: 0.68, 0.72, 0.75, 0.70, and 0.80. Find the average score.

Problem 11 [Challenge] (a) Explain, in your own words, why 98% accuracy can still describe a useless model. (b) Invent a confusion matrix for 200 examples where accuracy is above 95%, but recall for a rare positive class is below 50%. Show your TP, FP, FN, TN, and both calculations.

Problem 12 [Challenge] A hospital wants to screen patients for a rare, serious disease. (a) Explain which mistake is more costly, a false positive or a false negative, and why. (b) Explain how that answer should shape the team's choice of metric.

Problem 13 [Challenge] Open ML Studio, fit a classifier on an imbalanced dataset, and move the decision threshold while watching the confusion matrix update. Describe how precision and recall change as the threshold moves, and describe one real situation where you would accept lower precision in exchange for higher recall.

FOUNDATIONS OF MACHINE LEARNING

CHAPTER 5

Overfitting, Underfitting, and Regularization

Why a model can be too simple, too complex, or just right

IN THIS CHAPTER YOU WILL LEARN TO

- Tell underfitting and overfitting apart, and recognize the "too simple, too complex, just right" pattern in a fitted model
- Explain bias and variance in plain language, and describe how the two trade off against each other
- Compute training error and validation error at several model-capacity levels, and use the pattern to catch overfitting in numbers
- Explain what a hyperparameter is, and name examples that control how much capacity a model has
- Read a learning curve to decide whether more data, a different model, or regularization is the right next step
- Explain what regularization does, and use a validation set to choose a regularization strength that avoids both underfitting and overfitting

5.1 Two Ways a Model Can Fail

Chapter 4 gave you careful ways to measure a **model's** performance, including several honest **metrics** for both classification and regression. But a good score, by itself, can still hide a serious problem. A model can score beautifully on the data it already knows, then fail badly on the very first new example it meets. This chapter is about that gap. It is also the most important chapter in this book, because almost every later chapter leans on the ideas introduced here.

Every model in this book is built the same way. A learning algorithm studies a **training set** and produces a model. But the real test is not how well that model explains the training set it already saw. The real test is how well it handles brand-new examples: a new patient, a new customer, a new field of wheat. This idea is important enough to deserve its own name.

WORD TO KNOW

Generalization — A model's ability to perform well on new examples it has never seen before, not only on the examples it trained on. Generalization, not training performance, is the true goal of every machine learning project.

Generalization is the goal. But a model can fail to reach that goal in two opposite ways. It can be too simple to capture the real pattern in the data. Or it can cling too closely to the exact training data it happened to see. Both failures

produce a model nobody should trust, even though they look almost opposite on the page.

WORD TO KNOW

Underfitting — Failing to learn enough of the real pattern in the data, so the model performs poorly even on the training data itself. An underfit model is usually too simple for the problem it is trying to solve.

WORD TO KNOW

Overfitting — Learning the noise and small accidents of the training data so well that the model performs worse on new data it has not seen before. An overfit model has often memorized its training data instead of learning a general rule.

Picture these two failures side by side, using one running example for the whole chapter. An agricultural researcher near Duhok studies several wheat fields. For each field, she records the total rainfall during the growing season, in millimeters, and that field's final yield, in tons per hectare. Plotted on a graph, with rainfall along the bottom and yield up the side, each field becomes one dot. More rainfall generally means a higher dot. But the dots do not sit in one perfectly straight line, because real fields have real differences in soil, timing, and luck.

Now imagine three different curves drawn through these dots, each one a different model.

The first curve is a flat, horizontal line. It ignores rainfall completely. It predicts the exact same yield for a dry field and a wet field. This line is too simple. It underfits, because it misses even the obvious upward trend the dots already show.

The second curve is a gently rising, mostly straight line. It follows the dots' general upward direction. It does not touch any single dot exactly. It treats the small ups and downs between neighboring fields as ordinary noise, not as a pattern worth chasing. This line generalizes well.

The third curve is a wild, wiggling line. It swoops up and down so sharply that it passes exactly through every single dot. It looks perfect on the fields it has already seen. But between those fields, its wiggles are really just guesses. Those guesses are shaped by accidents in this one particular batch of data, not by any real rule about rainfall and wheat. This curve overfits.

Hold that picture in your mind: too straight, too wiggly, and just right. The rest of this chapter turns that picture into numbers you can compute yourself.

EQUATION — AVERAGE SIZE OF A MODEL'S MISTAKES

$$\text{average error} = \frac{\sum |\text{true} - \text{predicted}|}{n}$$

In words: To find a model's average error, find the distance between the true value and the predicted value for every example, ignoring whether the prediction was too high or too low, add all of those distances together, then divide by the number of examples.

true the real, correct value for one example (same unit as the target)

predicted the model's guess for that same example (same unit as the target)

n the number of examples (count)

Note: A smaller average error means the model's predictions sit closer to the true values. An average error of 0 means every single prediction was exactly correct.

WORKED EXAMPLE — COMPARE THREE MODELS USING ONLY TRAINING DATA

An agricultural researcher near Duhok records rainfall, in millimeters, and wheat yield, in tons per hectare, for 5 training fields: Field 1 (300 mm, 2.0 t/ha), Field 2 (350 mm, 2.6 t/ha), Field 3 (400 mm, 3.0 t/ha), Field 4 (450 mm, 3.4 t/ha), Field 5 (500 mm, 4.0 t/ha). Three candidate models are fit to these 5 fields. The Flat model predicts 3.0 for every field, no matter what the rainfall is. The Line model predicts 2.1, 2.5, 2.9, 3.3, and 3.7 for Fields 1 through 5, in order. The Wiggly model predicts 2.0, 2.6, 3.0, 3.4, and 4.0 for Fields 1 through 5, in order. Find the average training error for each of the three models.

1. Find the Flat model's errors: $|2.0 - 3.0| = 1.0$. $|2.6 - 3.0| = 0.4$. $|3.0 - 3.0| = 0.0$. $|3.4 - 3.0| = 0.4$. $|4.0 - 3.0| = 1.0$.
2. Add the Flat model's five errors, then divide by 5 fields: $1.0 + 0.4 + 0.0 + 0.4 + 1.0 = 2.8$, and $2.8 \div 5 = 0.56$.
3. Find the Line model's errors: $|2.0 - 2.1| = 0.1$. $|2.6 - 2.5| = 0.1$. $|3.0 - 2.9| = 0.1$. $|3.4 - 3.3| = 0.1$. $|4.0 - 3.7| = 0.3$.
4. Add the Line model's five errors, then divide by 5 fields: $0.1 + 0.1 + 0.1 + 0.1 + 0.3 = 0.7$, and $0.7 \div 5 = 0.14$.
5. Find the Wiggly model's errors: $|2.0 - 2.0| = 0.0$. $|2.6 - 2.6| = 0.0$. $|3.0 - 3.0| = 0.0$. $|3.4 - 3.4| = 0.0$. $|4.0 - 4.0| = 0.0$.
6. Add the Wiggly model's five errors, then divide by 5 fields: $0.0 + 0.0 + 0.0 + 0.0 + 0.0 = 0.0$, and $0.0 \div 5 = 0.0$.

Answer: The Flat model's average training error is 0.56 t/ha. The Line model's is 0.14 t/ha. The Wiggly model's is 0.00 t/ha, a perfect score. Judged only by training error, the Wiggly model looks like the clear winner. Lesson 5.4 checks all three models again, on fields none of them has ever seen, and the ranking will not stay the same.

COMMON MISTAKE

✗ Wrong: Deciding a model is the best choice because it has the lowest error, or the highest **accuracy**, on the training data it already learned from.

✓ Right: A perfect, or near-perfect, training score is not proof of a good model. It can just as easily be a warning sign of overfitting. Always check a separate **validation set** before trusting a training score.

The next two lessons give this whole picture a more precise vocabulary: bias and variance.

5.2 Bias and Variance

Underfitting and overfitting describe what a model's mistakes look like from the outside: too simple, or too tied to one specific training set. Bias and variance describe why those mistakes happen. These two words are the vocabulary the rest of this book uses to talk about a model's weaknesses, so it is worth spending real time on them here.

WORD TO KNOW

Bias — A model's tendency to make the same kind of mistake, in the same direction, again and again, because its assumptions are too simple for the real pattern. High bias shows up as underfitting.

WORD TO KNOW

Variance — A model's tendency to change a great deal when it is trained on a different sample of the same kind of data. High variance shows up as overfitting.

A simple target game makes both ideas easy to picture. Imagine a circle drawn on a wall, with a small bullseye at its center. Four different players each throw five small stones at the target, aiming for the bullseye every time. The first player's stones land in a tight little cluster, but that cluster sits far from the bullseye, always toward the upper left. This player has high bias. The mistakes are consistent, and consistently wrong in the same direction. The second player's stones land all over the target, scattered in every direction, with almost no pattern. This player has high variance. The result depends heavily on tiny differences from one throw to the next. The third player manages both problems at once, with stones scattered widely and also far from the bullseye on average. The fourth player's stones land in a tight cluster right around the bullseye itself. Every machine learning model behaves a little like one of these four players, and the goal of this whole chapter is to become more like the fourth.

Look back at the Flat model from Lesson 5.1. It predicted 3.0 for every field, no matter what the rainfall was. Its errors were not random. Fields with low rainfall were consistently over-predicted, and fields with high rainfall were consistently under-predicted, every single time. That is exactly what high bias looks like: a consistent, systematic mistake, caused by a model that is too simple to represent the real, rising pattern in the data.

Variance shows up differently. Imagine two different research teams, each sampling a different set of wheat fields near Duhok in the same season. The two teams' data are similar, but not identical, simply because they happened to visit different fields. Both teams fit a Flat model and a Wiggly model to their own data, then use each model to predict the yield of one test field with 400 mm of rainfall.

WORKED EXAMPLE — COMPARE HOW MUCH TWO MODELS' PREDICTIONS CHANGE ACROSS SAMPLES

Research Team 1 fits a Flat model and a Wiggly model to its own sample of fields. For a test field with 400 mm of rainfall, Team 1's Flat model predicts 3.0

t/ha, and Team 1's Wiggly model predicts 2.6 t/ha. Research Team 2 samples a different set of fields, and fits the same two kinds of models. For the same test field, Team 2's Flat model predicts 3.1 t/ha, and Team 2's Wiggly model predicts 4.1 t/ha. Find how much each model type's prediction changed between the two teams.

1. Find the Flat model's change between teams: $|3.1 - 3.0| = 0.1$ t/ha.
2. Find the Wiggly model's change between teams: $|4.1 - 2.6| = 1.5$ t/ha.
3. Compare the two changes: 0.1 t/ha is much smaller than 1.5 t/ha.

Answer: The Flat model barely moved at all, changing by only 0.1 t/ha, even though it was trained on a different sample of fields. The Wiggly model swung by 1.5 t/ha, a huge change for a yield that is only a few tons per hectare in total. The Flat model has low variance. The Wiggly model has high variance.

Put these two ideas side by side. The Flat model has high bias and low variance. It is consistently wrong, but consistently wrong in the same, predictable way. The Wiggly model has low bias and high variance. On the exact training data it sees, it is barely wrong at all, but its predictions swing wildly depending on which fields happened to be sampled. Neither extreme is the fourth player from the target game. Lesson 5.3 explains why a model almost always has to trade a little bit of one problem for a little bit of the other.

5.3 The Bias-Variance Trade-off

Lesson 5.2 sorted two models into two extremes: high bias and low variance on one side, low bias and high variance on the other. Real projects are rarely so extreme. Most of the time, choosing a model means choosing how much of each problem you are willing to accept, because you almost never get to erase both at once.

WORD TO KNOW

Bias-variance trade-off — The pattern where lowering a model's bias, usually by giving it more capacity, tends to raise its variance, and lowering its variance tends to raise its bias. Because of this trade-off, the best model is usually not the simplest one and not the most flexible one, but somewhere in between.

EQUATION — TOTAL ERROR HAS THREE PARTS

$$\text{expected error} \approx \text{bias}^2 + \text{variance} + \text{irreducible noise}$$

In words: A model's expected error on new data comes from three separate sources added together: how wrong its assumptions are on average, squared, plus how much its predictions swing from one training sample to another, plus a small amount of randomness that no model could ever remove.

bias² the systematic part of the error, squared, so it can never cancel out to a negative number (same unit as the target, squared)

variance how much the model's predictions swing across different training samples (*same unit as the target, squared*)

irreducible noise randomness in the real world that no model, however good, can ever predict away (*same unit as the target, squared*)

Note: This formula is a way of thinking, more than a number you will calculate by hand in this book. Its lesson is simple: pushing bias toward zero tends to push variance up, and pushing variance toward zero tends to push bias up. A good model finds the smallest total, not the smallest of any one piece alone.

Picture total error as a U-shaped curve. Its bottom axis runs from a very simple model on the left to a very flexible model on the right. On the far left, bias is high and variance is low, so total error is high. On the far right, bias is low and variance is high, so total error is high again, for the opposite reason. Somewhere in the middle, the curve dips down to its lowest point. That dip is not the model with zero bias. It is not the model with zero variance either. It is the model where the sum of the two is smallest.

WORKED EXAMPLE — RANK THE THREE FARM MODELS BY BIAS AND BY VARIANCE

Using what Lessons 5.1 and 5.2 already showed, rank the Flat model, the Line model, and the Wiggly model from highest to lowest bias, and separately from highest to lowest variance.

1. Recall the Flat model's training error, 0.56 t/ha, the worst of the three, caused by a consistent, one-direction mistake. This is high bias.
2. Recall the Wiggly model's training error, 0.00 t/ha, a perfect fit with no consistent direction to its mistakes at all. This is low bias.
3. Place the Line model's training error, 0.14 t/ha, between the other two, so its bias is also between the other two: moderate bias.
4. Recall Lesson 5.2's comparison: the Flat model barely changed across two different samples, 0.1 t/ha, while the Wiggly model swung by 1.5 t/ha. This makes the Flat model low variance and the Wiggly model high variance.
5. The Line model changes only gently across different samples of similar fields, because a straight line is only nudged slightly by a few different points. Its variance is also moderate.

Answer: From highest to lowest bias: Flat, Line, Wiggly. From highest to lowest variance: Wiggly, Line, Flat. The Line model is not the best on either measure alone, but it is the only model that keeps both problems moderate at the same time, which is exactly why Lesson 5.4 shows it winning on new data.

Finding the bottom of the U-shaped curve by eye is not realistic for a real project. Lesson 5.4 gives you the tool that finds it in practice: a systematic way to change how much capacity a model is allowed to have, and to watch what happens to both training and validation error as you do.

5.4 Hyperparameters and Model Capacity

Every model in this book has at least one setting that controls how flexible it is allowed to be. In Lesson 5.1, that setting was simply which of three model types we chose: Flat, Line, or Wiggly. In a real project, that setting usually has a name, and a number attached to it.

WORD TO KNOW

Hyperparameter — A setting chosen before training begins, rather than a value the learning algorithm discovers from data. Changing a hyperparameter changes how a model learns, including how much freedom it has to bend and twist.

A hyperparameter is different from the numbers a model learns during training, such as the position of a centroid from Chapter 1. Those learned numbers are chosen automatically, by the learning algorithm, once training data is available. A hyperparameter is chosen beforehand, by a person, or by a careful search process covered in Chapter 17.

WORD TO KNOW

Model capacity — How wide a range of patterns a model is able to represent, from simple to wildly complicated. A low-capacity model can only represent simple patterns, such as a straight line. A high-capacity model can represent complicated, wiggly patterns.

Many familiar hyperparameters are really just capacity dials in disguise. The maximum depth of a decision tree, covered in Chapter 9, controls how many questions the tree is allowed to ask before making a prediction, and a deeper tree has higher capacity. The number of neighbors a k-Nearest Neighbors model consults, covered in Chapter 8, works the opposite way: a smaller number gives higher capacity, because the model can react to every small local detail, while a larger number gives lower capacity, because each prediction is smoothed across more neighbors. The strength of a regularization penalty, covered later in this very chapter, is a capacity dial too. Every one of these settings answers the same underlying question: how much freedom should this model have to bend toward the training data?

COMMON MISTAKE

✗ Wrong: Believing that training automatically finds the best tree depth, the best number of neighbors, or the best regularization strength, the same way it finds the best split point inside a tree that already has a fixed shape.

✓ Right: Training only adjusts a model's learned parameters. A person, or a systematic search using a validation set, must choose the hyperparameters first. Chapter 17 covers systematic hyperparameter search in full.

WORKED EXAMPLE — REVEAL THE VALIDATION ERROR FOR ALL THREE CAPACITY LEVELS

The same three models from Lesson 5.1 — Flat (capacity level 1), Line (capacity level 2), and Wiggly (capacity level 3) — are now checked on 3 validation fields they have never seen during training: Field 6 (320 mm, true yield 2.3 t/ha), Field 7 (420 mm, true yield 3.0 t/ha), and Field 8 (480 mm, true yield 3.7 t/ha).

The Flat model predicts 3.0, 3.0, and 3.0 for Fields 6 through 8. The Line model predicts 2.3, 3.1, and 3.5. The Wiggly model predicts 2.9, 2.6, and 4.2. Find each model's average validation error, then compare it with its training error from Lesson 5.1.

1. Find the Flat model's validation errors: $|2.3 - 3.0| = 0.7$. $|3.0 - 3.0| = 0.0$. $|3.7 - 3.0| = 0.7$.
2. Add the Flat model's three errors, then divide by 3 fields: $0.7 + 0.0 + 0.7 = 1.4$, and $1.4 \div 3 \approx 0.47$.
3. Find the Line model's validation errors: $|2.3 - 2.3| = 0.0$. $|3.0 - 3.1| = 0.1$. $|3.7 - 3.5| = 0.2$.
4. Add the Line model's three errors, then divide by 3 fields: $0.0 + 0.1 + 0.2 = 0.3$, and $0.3 \div 3 = 0.10$.
5. Find the Wiggly model's validation errors: $|2.3 - 2.9| = 0.6$. $|3.0 - 2.6| = 0.4$. $|3.7 - 4.2| = 0.5$.
6. Add the Wiggly model's three errors, then divide by 3 fields: $0.6 + 0.4 + 0.5 = 1.5$, and $1.5 \div 3 = 0.50$.
7. Line up training error against validation error for all three capacity levels.
Capacity level 1, Flat model: training error 0.56, validation error 0.47.
Capacity level 2, Line model: training error 0.14, validation error 0.10.
Capacity level 3, Wiggly model: training error 0.00, validation error 0.50.

Answer: Training error keeps falling as capacity increases: 0.56, then 0.14, then 0.00. Validation error does not. It falls from 0.47 to 0.10, then rises back up to 0.50. The Wiggly model fit its training fields perfectly, yet it is now the worst model of the three on fields it has never seen, almost as bad as the overly simple Flat model. The Line model, at capacity level 2, wins on the number that actually matters: performance on new data.

This is the U-shaped curve from Lesson 5.3, now built entirely from numbers you calculated yourself. Capacity level 1 sits on the high-bias side of the curve. Capacity level 3 sits on the high-variance side. Capacity level 2 sits closest to the bottom of the dip. A real project almost never knows the right capacity in advance. It finds it exactly the way this example did: by trying several levels and comparing validation error, never training error alone.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.metrics import mean_absolute_error

y_train = np.array([2.0, 2.6, 3.0, 3.4, 4.0])
y_val = np.array([2.3, 3.0, 3.7])

flat_train, flat_val = [3.0, 3.0, 3.0, 3.0, 3.0], [3.0, 3.0, 3.0]
line_train, line_val = [2.1, 2.5, 2.9, 3.3, 3.7], [2.3, 3.1, 3.5]

print("Flat:", mean_absolute_error(y_train, flat_train),
      mean_absolute_error(y_val, flat_val))
print("Line:", mean_absolute_error(y_train, line_train),
      mean_absolute_error(y_val, line_val))
```

The first two lines store the true training and validation yields as NumPy arrays, the same numbers used in the worked example above. The next two lines store each model's predictions as plain lists, one list for training fields and one for validation fields. The `mean\_absolute\_error` function then repeats the exact same subtract-and-average arithmetic from the steps above, once for training and once for validation. Running this code prints 0.56 and 0.47 for the Flat model, and 0.14 and 0.10 for the Line model, matching the hand calculations exactly.

Trying capacity levels one at a time, the way this lesson just did, works well with only three candidates. Lesson 5.5 introduces a tool built for a more realistic question: not just which capacity is best, but whether the real problem is too little data, too little capacity, or too much.

5.5 Learning Curves: Reading Train vs. Validation Score

Lesson 5.4 changed model capacity while holding the amount of data fixed. This lesson does the opposite. It holds capacity fixed and changes the amount of training data, to see what that reveals.

WORD TO KNOW

Learning curve — A plot of training performance and validation performance as the size of the training set grows. Reading the shape of the two lines together helps diagnose whether a model is underfitting, overfitting, or generalizing well.

Three patterns come up again and again. If both the training score and the validation score stay poor, and stay close together, even as more data is added, the model is likely underfitting. More data rarely fixes this on its own, because the real problem is a lack of capacity or useful features, not a lack of examples. If the training score stays strong while the validation score is noticeably weaker, but the gap between them keeps narrowing as more data is added, the model is likely overfitting, and more representative data may genuinely help. If

both scores are strong and sit close together, and stay that way as more data is added, the model is generalizing well, and collecting more data is unlikely to change much.

WORKED EXAMPLE — DIAGNOSE TWO LEARNING CURVES FOR A WHEAT-YIELD MODEL

The Duhok research team tries a flexible decision-tree model. With 100 training fields, its training error is 0.05 t/ha and its validation error is 0.60 t/ha. After collecting data for more seasons, with 1,000 training fields, its training error rises slightly to 0.15 t/ha and its validation error falls to 0.30 t/ha. Separately, the team also tries a flat model. With 100 training fields, its training error is 0.50 t/ha and its validation error is 0.52 t/ha. With 1,000 training fields, its training error is 0.49 t/ha and its validation error is 0.50 t/ha. Diagnose each model.

1. Find the tree model's gap at 100 fields: $0.60 - 0.05 = 0.55$ t/ha.
2. Find the tree model's gap at 1,000 fields: $0.30 - 0.15 = 0.15$ t/ha.
3. Compare the tree model's two gaps: the gap shrank from 0.55 to 0.15, and the validation error itself fell from 0.60 to 0.30, as more data was added.
4. Find the flat model's gap at 100 fields: $0.52 - 0.50 = 0.02$ t/ha.
5. Find the flat model's gap at 1,000 fields: $0.50 - 0.49 = 0.01$ t/ha.
6. Compare the flat model's two gaps: the gap was already tiny at 0.02, and both scores stayed poor, close to 0.50, even after adding 900 more fields.

Answer: The tree model was overfitting, and more data genuinely helped, narrowing the gap and lowering validation error. The flat model was underfitting the whole time. Its small gap looks reassuring, but both scores stayed poor, which means the model lacks the capacity to learn the pattern at all, no matter how many more fields the team collects.

COMMON MISTAKE

✗ Wrong: Collecting more training data as a first response to any weak validation score, without first checking whether the training score is also weak.

✓ Right: Check the training score first. If training performance is already poor, the model is underfitting, and more data is unlikely to help much. Consider a higher-capacity model, better features, or less regularization instead.

A learning curve tells you whether to reach for more data. Lesson 5.6 gives you a different tool for the overfitting side of the curve: a way to keep a flexible model's high capacity, while still discouraging it from chasing every accident in the training data.

5.6 Regularization: Paying a Price for Simplicity

Lessons 5.1 through 5.5 mostly chose between separate models: Flat, Line, or Wiggly. Real projects often prefer a different approach. Keep one flexible model, capable of wiggling if it truly needs to, but add a cost for every unnecessary bend. This is regularization.

WORD TO KNOW

Regularization — Adding a penalty, inside training itself, that discourages a model from becoming more complex than the data truly justifies. A model can still bend where the data strongly supports bending, but it must "earn" every bend.

Think of regularization as a toll booth. A flexible model may add a new twist or bend to its curve, but only by paying a toll first. If a twist would improve the fit by only a tiny amount, it is not worth the toll, and the model leaves that part of the curve smooth. If a twist would improve the fit by a large amount, because the data truly demands it, the model happily pays the toll. A strong toll, a high regularization strength, keeps the curve almost flat everywhere. A weak toll, a low regularization strength, lets the curve wiggle almost freely.

EQUATION — REGULARIZED LOSS

$$\text{regularized loss} = \text{fit error} + \lambda \times \text{complexity}$$

In words: The score a training algorithm tries to minimize is no longer just the fit error on the training data. It becomes the fit error plus a penalty, where the penalty grows with both the model's complexity and the strength of a chosen setting, λ .

fit error how far the model's predictions are from the true training values, before any penalty (*same unit as the target*)

λ the regularization strength, a hyperparameter chosen before training (*no unit, just a number*)

complexity a measure of how large or how wiggly the model's internal numbers are (*depends on the model*)

Note: When $\lambda = 0$, there is no penalty at all, and training only cares about fitting the training data. As λ grows, training cares more and more about staying simple, even if that means fitting the training data a little less closely.

WORKED EXAMPLE — TURN ONE FLEXIBLE MODEL'S REGULARIZATION DIAL

Instead of switching between three separate model types, the research team now takes one flexible model and turns its regularization strength, λ , up and down. At $\lambda = 5$, the model's training error is 0.56 t/ha and its validation error is 0.47 t/ha. At $\lambda = 0.2$, its training error is 0.14 t/ha and its validation error is 0.10 t/ha. At $\lambda = 0$, its training error is 0.00 t/ha and its validation error is 0.50 t/ha. Compare the three settings and choose the best one.

1. At $\lambda = 5$, the strong penalty forces the model to stay almost flat, matching the Flat model's numbers from Lesson 5.4 exactly: training error 0.56, validation error 0.47.
2. At $\lambda = 0$, no penalty at all, the model wiggles freely and fits the training data perfectly, matching the Wiggly model's numbers exactly: training error 0.00, validation error 0.50.
3. At $\lambda = 0.2$, a moderate penalty, the model bends only where the data strongly supports it, matching the Line model's numbers exactly: training error 0.14, validation error 0.10.

4. Compare the three validation errors: 0.47, 0.10, and 0.50. The smallest belongs to $\lambda = 0.2$.

Answer: $\lambda = 0.2$ gives the lowest validation error, 0.10 t/ha, so it is the best regularization strength of the three tried here. Notice that turning one dial, λ , reproduced the exact same three outcomes as choosing among three separate model types in Lesson 5.4. Regularization and model capacity are two different knobs that control the very same trade-off.

COMMON MISTAKE

✗ Wrong: Turning up regularization strength to try to fix a poor validation score that is actually caused by data leakage or by mixing up which fields belong in training versus validation.

✓ Right: Regularization only helps with overfitting caused by too much capacity. Check for data leakage and a clean split first. No amount of regularization repairs a validation set that was never truly kept separate.

IMPORTANT

A model that fits its training data almost perfectly can still sound completely confident while being badly wrong on a new field, a new season, or a new farmer's land. Before letting any prediction guide a real decision, such as how much to invest in irrigation, check its validation performance honestly, and say so plainly if that performance is weak.

You now have a full toolkit: recognize underfitting and overfitting, understand bias and variance, control capacity with hyperparameters, read a learning curve, and tame a flexible model with regularization. Lesson 5.7 lets you watch all five ideas happen live, on screen, in ML Studio.

5.7 Try It in ML Studio

Numbers on a page are useful, but watching a curve bend in front of you, live, makes the U-shape from Lesson 5.3 impossible to forget.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Wheat Yield" dataset, and choose Decision Tree as the model.
3. Set the maximum depth to 1, press Fit, and write down the training score and the validation score shown on screen.
4. Increase the maximum depth one step at a time, trying 2, 4, 8, and "unlimited," pressing Fit and recording both scores at each depth.
5. Find the depth with the best validation score. Then open the Labs tab, turn on the regularization slider for a linear model on the same data, and write one sentence comparing what the slider does to what changing tree depth did.

You need: Computer, tablet, or phone with the ML Studio app

5.8 Case Study

The case study below follows one team through this entire chapter's ideas, from a first, overconfident model to a careful, validated one.

CASE STUDY — CHOOSING MODEL CAPACITY FOR A WHEAT-YIELD MODEL NEAR DUHOK

Anwar manages a small agricultural extension office near Duhok. Every spring, farmers ask her office the same question. Given this season's rainfall forecast, how much wheat should they expect, and is extra irrigation worth paying for? For years, the office gave only a rough, experience-based guess. Anwar hires a young data analyst, Sara, to see whether the office's own records, gathered from dozens of fields over several seasons, can produce a more careful estimate.

Sara is eager to impress. She builds a highly flexible model, one with many branches, studying rainfall alongside soil type, elevation, and planting date. On the historical records, it looks remarkable. Its training error is almost zero. She is ready to present it to Anwar as the office's new tool. Before the meeting, a senior colleague asks one simple question: what did it score on data it never trained on? Sara has not checked. When she finally does, the validation error is far worse than the training error, just like the Wiggly model earlier in this chapter. Her impressive-looking model has memorized old seasons instead of learning a rule that transfers to new ones.

Sara starts over, this time following the chapter's full method. She compares several capacity levels using validation error, not training error, and finds that a plain, gently curving relationship performs best. She checks a learning curve too, training her leading candidate on progressively more seasons of data, and finds that the training-validation gap barely narrows. That tells her the problem was too much capacity, not too little data. Finally, she adds regularization to her flexible model, so it can still respond a little to soil type and elevation, but only where the data truly supports it. The regularized model's validation error lands close to her simplest good candidate, with a little more useful nuance built in.

The finished tool does not print a single confident number, such as "3.4 tons per hectare." It prints a range, with a short note that the estimate is guidance, not a guarantee. Anwar's office pairs every printed estimate with the judgment of a local agronomist before advising any farmer to spend money on extra irrigation. Sara also sets a reminder to retrain the model at the end of every season, because a rule learned from five years of rainfall is not guaranteed to hold in a sixth year that behaves differently. The lesson she takes from her first, overconfident attempt stays with her for the rest of her career: a training score alone never proves a model is ready.

COMMON MISTAKES TO AVOID

- Judging a model's quality only by its training score, without checking a separate validation set.
- Believing training automatically searches for the best hyperparameter, such as tree depth or regularization strength, on its own.

- Assuming more training data always fixes a weak validation score, even when the training score is already weak too.
- Trying to repair data leakage, or a broken training-validation split, by increasing regularization strength.
- Choosing a model's capacity, or its regularization strength, by repeatedly checking the test set instead of the validation set.
- Treating the most complex, most flexible model as automatically the best model.
- Treating a small gap between training and validation score as success, even when both scores are poor.

CHAPTER SUMMARY

- Generalization, performing well on new data, is the true goal of a model; strong training performance alone can never prove it.
- Underfitting means a model is too simple and performs poorly even on training data; overfitting means a model has memorized training data and performs worse on new data.
- Bias is a consistent, systematic mistake caused by an overly simple model; variance is how much a model's predictions swing across different training samples.
- The bias-variance trade-off means lowering one usually raises the other; total error is smallest somewhere in the middle, not at either extreme.
- A hyperparameter is a setting chosen before training, such as tree depth or regularization strength, that controls a model's capacity: how wide a range of patterns it can represent.
- Comparing training error and validation error at several capacity levels reveals a classic pattern: training error keeps falling, while validation error falls, then rises again.
- A learning curve plots training and validation score against the size of the training set, showing whether more data, rather than a capacity change, is the right next step.
- Regularization adds a penalty for unnecessary complexity, letting one flexible model behave like a simple model or a complex model, depending on its strength, λ .
- Regularization only fixes overfitting caused by too much capacity; it cannot repair data leakage or a broken training-validation split.
- Chapter 6 puts all of these ideas to work immediately, fitting and regularizing your first real model: linear regression.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define underfitting and overfitting in your own words, one sentence each.

Problem 2 [Basic] A model scores 55% accuracy on training data and 54% on validation data. Name the more likely problem, underfitting or overfitting, and explain your choice in one sentence.

Problem 3 [Basic] A model scores 99% accuracy on training data and 61% on validation data. Name the more likely problem, underfitting or overfitting, and explain your choice in one sentence.

Problem 4 [Basic] Explain, in your own words, the difference between a parameter that training learns and a hyperparameter.

Problem 5 [Standard] A fourth candidate model, Model D, predicts 2.3, 2.7, 2.8, 3.6, and 3.8 for the five training fields from Lesson 5.1 (true yields 2.0, 2.6, 3.0, 3.4, and 4.0 t/ha). Find Model D's average training error, showing every step.

Problem 6 [Standard] A team predicting exam scores from study hours tries three capacity levels. Capacity 1: training error 8 points, validation error 9 points. Capacity 2: training error 4 points, validation error 5 points. Capacity 3: training error 1 point, validation error 7 points. Identify which capacity level is underfitting, which is overfitting, and which is the best choice.

Problem 7 [Standard] Explain bias and variance using a real-life example of your own, different from the target game or the wheat farm used in this chapter.

Problem 8 [Standard] A spam-detection model has training accuracy 96% and validation accuracy 70% when trained on 200 emails. Trained on 2,000 emails, its training accuracy is 93% and its validation accuracy is 89%. Find the training-validation gap at each size, and diagnose the model.

Problem 9 [Standard] At $\lambda = 1$, training error is 0.30 and validation error is 0.55. At $\lambda = 0.05$, training error is 0.05 and validation error is 0.20. At $\lambda = 20$, training error is 0.60 and validation error is 0.65. Choose the best λ , and explain your choice in one sentence.

Problem 10 [Standard] Explain, in your own words, why increasing regularization strength cannot fix a validation score that is low because of data leakage.

Problem 11 [Challenge] A team tests a decision tree at three depths on a customer-churn dataset: depth 2 (training accuracy 74%, validation accuracy 73%), depth 8 (training accuracy 92%, validation accuracy 84%), depth 20 (training accuracy 99%, validation accuracy 78%). (a) Which depth would you choose, and why? (b) Which depth shows the clearest sign of overfitting, and what evidence shows it? (c) If even the best of the three still has a noticeable training-validation gap, name two things the team could try next, besides simply choosing a different depth.

Problem 12 [Challenge] Open ML Studio, choose a regression dataset, and fit a k-Nearest Neighbors model. Try at least four different values of k, recording the

training and validation score each time. Identify which values of k underfit, which overfit, and which value you would finally choose. Explain your reasoning in a few sentences.

Problem 13 [Challenge] Pick a prediction problem from your own field of study or interest. Name one hyperparameter of a model you could use that controls its capacity. Explain, in a few sentences, how you would use a validation set, not the test set, to choose a good value for that hyperparameter, and what a learning curve might additionally tell you.

Part II — Supervised Learning

- Chapter 6** — Linear Regression
- Chapter 7** — Logistic Regression
- Chapter 8** — k-Nearest Neighbors
- Chapter 9** — Decision Trees
- Chapter 10** — Random Forests and Boosting
- Chapter 11** — Neural Networks: A Short Bridge

SUPERVISED LEARNING

CHAPTER 6

Linear Regression

Fitting the best straight line through data

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what linear regression predicts, and write its equation for one feature
- Compute a prediction by hand from a coefficient, an intercept, and a feature value
- Calculate a residual, and use least squares to find the loss for a candidate line
- Walk through several rounds of gradient descent by hand, updating the coefficient and the intercept one small step at a time
- Extend a line to several features at once, and interpret one coefficient while holding the others fixed
- Explain what ridge regression and lasso regression add to plain linear regression, and why regularization needs standardized features

6.1 A Line That Predicts a Number

Chapters 1 through 5 built the whole foundation this book needs: what a **model** is, how to run a safe **workflow**, how to prepare **features**, how to measure a model honestly, and how a model can go wrong by being too simple or too complex. Now it is time to build a real model, from the ground up, with numbers you can check by hand at every step.

Meet Rania. She works for the electricity company in Erbil. Every month, her office wants to give homeowners an early estimate of their electricity bill, before the official meter reading arrives. Rania has records for many homes: the size of each home, in square meters, and that home's actual monthly bill, in dollars. She wants a simple rule that turns a home's size into a predicted bill.

The simplest useful rule is a straight line. Picture a graph with home size along the bottom and monthly bill up the side. Each home Rania already has data for becomes one dot on this graph. A straight line drawn through these dots can be described by just two numbers: how steeply the line rises, and where the line starts. Once Rania has those two numbers, she can predict a bill for any home size at all, even a home she has never seen before, just by reading a point off that line.

WORD TO KNOW

Linear regression — A method that predicts a number by multiplying each feature by its own number, adding the results together, and then adding one

more fixed number at the end. With a single feature, this is exactly a straight line drawn through the data.

Linear regression is a **regression** method, so its output is always a number, such as a bill in dollars, not a category such as "sweet" or "not sweet." Chapter 7 uses a close cousin of this same idea to answer yes-or-no questions instead.

EQUATION — THE ONE-FEATURE REGRESSION LINE

$$\hat{y} = w \times x + b$$

In words: The predicted value equals a number we multiply by the feature, plus a fixed number we add at the end.

$\hat{y}$ the predicted value (same unit as the target)

x the feature value for one example (depends on the feature)

w a number that says how much the prediction changes for each extra unit of x (target unit per unit of x)

b a fixed number, the predicted value when $x = 0$ (same unit as the target)

Note: Lesson 6.2 gives w and b their own names. For now, just notice the shape of the recipe: multiply by w , then add b .

WORKED EXAMPLE — PREDICT ONE HOUSE'S ELECTRICITY BILL

Rania's colleague suggests a starting rule for predicting a home's monthly electricity bill in Erbil: predicted bill = $0.7 \times \text{home size} + 15$, where home size is measured in square meters and the bill is measured in dollars. Use this rule to predict the bill for a 90 m² home.

1. Write the rule with the home size filled in: predicted bill = $0.7 \times 90 + 15$.
2. Multiply first: $0.7 \times 90 = 63$.
3. Add the fixed number: $63 + 15 = 78$.

Answer: The rule predicts a monthly bill of \$78 for a 90 m² home.

Lesson 6.2 checks this same rule against real homes, to see how close its predictions really are.

Notice what this rule does not claim. It does not claim that every 90 m² home in Erbil pays exactly \$78. Real homes differ in insulation, in how many people live there, and in how often the air conditioner runs. A regression line gives the best single guess it can, using only the information it was given. The rest of this chapter is about finding a good line, checking how good it really is, and being honest about what it can and cannot promise.

6.2 Slope, Intercept, and Predictions by Hand

The two numbers inside a regression line each deserve their own name, because you will read and interpret them constantly, in this chapter and beyond.

WORD TO KNOW

Coefficient — The number a feature is multiplied by inside a linear regression equation. A coefficient of 0.7 means the prediction rises by 0.7 for every extra unit of that feature. A coefficient is sometimes called a weight, because it controls how heavily that feature counts toward the prediction.

WORD TO KNOW

Intercept — The fixed number added at the end of a linear regression equation, after every coefficient has already been multiplied by its feature. The intercept is the model's prediction when every feature equals zero.

In Rania's rule, predicted bill = $0.7 \times \text{home size} + 15$, the coefficient is 0.7 and the intercept is 15. The coefficient tells Rania that each extra square meter adds 0.7 dollars to the predicted bill. The intercept, \$15, is the rule's mathematical starting point before size counts for anything. No home in Erbil truly has 0 m² of floor space, so this intercept is best read as a mathematical anchor, not a real prediction — completely normal, since an intercept only needs a sensible real-world meaning when $x = 0$ is a value the data could actually reach.

WORKED EXAMPLE — PREDICT ALL FIVE OF RANIA'S HOUSES

Using the rule from Lesson 6.1, predicted bill = $0.7 \times \text{home size} + 15$, predict the monthly bill for five homes with sizes 50, 70, 90, 110, and 130 square meters.

1. Multiply each home size by the coefficient, 0.7: $50 \times 0.7 = 35$. $70 \times 0.7 = 49$. $90 \times 0.7 = 63$. $110 \times 0.7 = 77$. $130 \times 0.7 = 91$.
2. Add the intercept, 15, to each result: $35 + 15 = 50$. $49 + 15 = 64$. $63 + 15 = 78$. $77 + 15 = 92$. $91 + 15 = 106$.

Answer: The rule predicts bills of \$50, \$64, \$78, \$92, and \$106 for the five homes, in that order. Lesson 6.3 compares these predictions with the homes' real, already-known bills.

COMMON MISTAKE

✗ Wrong: Computing a prediction by multiplying the coefficient by the feature, then stopping, and forgetting to add the intercept.

✓ Right: Always complete both steps of the recipe: multiply first, then add the intercept. Skipping the intercept quietly shifts every single prediction by the same fixed amount, and that shift can be large.

IMPORTANT

A coefficient describes a pattern found in past data. It is not proof of a cause. A positive coefficient on home size does not prove that extra floor space, by itself, causes a higher bill. Bigger homes may also have more rooms to cool, more appliances, or more people living in them. Treat a coefficient as a useful clue about a pattern, never as proof of what would happen if only one thing in the real world changed.

6.3 Residuals and the Loss Function

Lesson 6.2 predicted five bills. Rania already knows the real bills for these same five homes, because they are past customers. Comparing a prediction with the truth, one home at a time, gives every single prediction its own small report card.

WORD TO KNOW

Residual — The true value minus the predicted value for one example: $\text{residual} = y - \hat{y}$. A positive residual means the model predicted too low. A negative residual means the model predicted too high. A residual of zero means the prediction was exactly right.

WORKED EXAMPLE — FIND THE RESIDUAL FOR EACH OF RANIA'S FIVE HOMES

Rania's five homes have true monthly bills of \$50, \$66, \$82, \$94, and \$110. Lesson 6.2's rule predicted \$50, \$64, \$78, \$92, and \$106 for these same five homes, in the same order. Find the residual for each home.

1. Subtract the first home's predicted bill from its true bill: $50 - 50 = 0$.
2. Subtract the second home's predicted bill from its true bill: $66 - 64 = 2$.
3. Subtract the third home's predicted bill from its true bill: $82 - 78 = 4$.
4. Subtract the fourth home's predicted bill from its true bill: $94 - 92 = 2$.
5. Subtract the fifth home's predicted bill from its true bill: $110 - 106 = 4$.

Answer: The five residuals are \$0, \$2, \$4, \$2, and \$4. Every single residual here is zero or positive, so this rule never once predicted too high; it only ever matched exactly or predicted a little too low. A pattern like this, where every residual points in the same direction, is exactly the sign of the consistent, high bias described in Chapter 5.

A list of five separate residuals is useful for checking one home at a time, but Rania also needs one single number that grades the whole line at once, so she can compare it fairly against other candidate lines. That single number has its own name.

WORD TO KNOW

Loss function — A formula that turns a model's mistakes into a single number, where a smaller number always means a better fit to the data. Training a model almost always means searching for the coefficients and the intercept that make the loss function as small as possible.

WORD TO KNOW

Least squares — The most common loss function for linear regression. It squares every residual, so that positive and negative residuals never cancel each other out, then adds all the squares together. The best line, by least squares, is whichever line makes this sum as small as possible.

EQUATION — SUM OF SQUARED ERRORS AND MEAN SQUARED ERROR

$$SSE = \sum (y - \hat{y})^2, \quad \text{and} \quad MSE = \frac{SSE}{n}$$

*In words: The sum of squared errors adds up the square of every residual. The ****mean squared error****, already familiar from Chapter 4, divides that sum by the number of examples, so two lines can be compared fairly even when they were checked on different numbers of homes.*

SSE sum of squared errors, the raw total that least squares tries to shrink (same unit as the target, squared)

MSE mean squared error (same unit as the target, squared)

y the true value for one example (same unit as the target)

$\hat{y}$ the predicted value for one example (same unit as the target)

n the number of examples (count)

Note: Squaring does two jobs at once. It removes the sign, so a residual of -4 counts exactly the same as a residual of 4 , and it punishes large residuals far more heavily than small ones.

WORKED EXAMPLE — FIND THE LOSS OF LINE A

Line A's five residuals, found above, are \$0, \$2, \$4, \$2, and \$4. Find the sum of squared errors and the mean squared error for Line A.

1. Square each residual: $0^2 = 0$. $2^2 = 4$. $4^2 = 16$. $2^2 = 4$. $4^2 = 16$.
2. Add the five squares together to find the sum of squared errors: $0 + 4 + 16 + 4 + 16 = 40$.
3. Divide by the number of homes, 5, to find the mean squared error: $40 \div 5 = 8$.

Answer: Line A's sum of squared errors is 40, and its mean squared error is 8 dollars-squared. On its own, 8 is hard to picture; it is only truly useful for comparing this line against another candidate, which is exactly what the next worked example does.

WORKED EXAMPLE — COMPARE LINE A WITH A SECOND CANDIDATE, LINE B

A second rule, Line B, predicts $\text{bill} = 0.8 \times \text{home size} + 10$. For the same five homes (sizes 50, 70, 90, 110, and 130 m²; true bills \$50, \$66, \$82, \$94, and \$110), find Line B's predictions, residuals, sum of squared errors, and mean squared error. Then decide which line fits better, A or B.

1. Compute Line B's predictions: $0.8 \times 50 + 10 = 50$. $0.8 \times 70 + 10 = 66$. $0.8 \times 90 + 10 = 82$. $0.8 \times 110 + 10 = 98$. $0.8 \times 130 + 10 = 114$.
2. Find each residual, true minus predicted: $50 - 50 = 0$. $66 - 66 = 0$. $82 - 82 = 0$. $94 - 98 = -4$. $110 - 114 = -4$.
3. Square each residual: $0^2 = 0$. $0^2 = 0$. $0^2 = 0$. $(-4)^2 = 16$. $(-4)^2 = 16$.
4. Add the squares to find the sum of squared errors: $0 + 0 + 0 + 16 + 16 = 32$.
5. Divide by 5 to find the mean squared error: $32 \div 5 = 6.4$.
6. Compare the two mean squared errors: Line A scored 8, and Line B scored 6.4.

Answer: Line B fits better, with a lower mean squared error of 6.4 dollars-squared, against Line A's 8. Line B's first three predictions are

perfect, and it only misses on the two largest homes, predicting a little too low both times. Least squares has no idea these are "the two largest homes"; it simply adds up squared misses wherever they happen to fall.

COMMON MISTAKE

✗ Wrong: Comparing two candidate lines using only the raw sum of squared errors, especially when the two lines were checked on different numbers of homes.

✓ Right: Compare lines using mean squared error, the sum divided by the number of examples, exactly as Chapter 4 recommended. A larger dataset almost always produces a larger raw sum, even for an equally good line.

Line B beat Line A, but is Line B truly the best line possible? Guessing candidate lines one at a time, the way this lesson just did, is not a serious strategy once real projects have thousands of homes and several features at once. Lesson 6.4 replaces guessing with an automatic method that finds a good line for you, one small, careful step at a time.

6.4 Finding the Best Line: Gradient Descent, Step by Step

Imagine hiking down a hillside at night, with no flashlight, trying to reach the lowest point of the valley below. You cannot see the whole landscape, but you can always feel the ground right under your feet. So you feel which direction slopes downward, take one small step that way, feel the ground again, and repeat, until, step by step, you reach the bottom.

This lesson's method follows exactly that plan. Picture a landscape where the ground's height at any point represents the loss, the mean squared error from Lesson 6.3, for one particular choice of coefficient and intercept. A low, flat valley floor means a small loss: a good line. A high peak means a large loss: a bad line. This method starts somewhere, usually the simple guess coefficient = 0 and intercept = 0, then repeatedly takes one small step toward lower ground.

WORD TO KNOW

Gradient descent — A method that finds good values for a model's coefficients and intercept by taking many small steps, each step moving a little further in the direction that makes the loss function smaller. "Descent" means moving downward, exactly like walking down a hill.

WORD TO KNOW

Learning rate — A small number, chosen before training begins, that controls how big each gradient descent step is. A learning rate that is too large can overshoot the best line and never settle down; a learning rate that is too small wastes time taking tiny, timid steps.

The learning rate is a **hyperparameter**, exactly the kind of setting Chapter 5 taught you to choose with care, usually by trying a few values and watching what happens, rather than a number training discovers by itself.

Every single round of gradient descent repeats the same four actions, in the same order. First, use the current coefficient and intercept to predict every training example. Second, find each example's residual, exactly the way Lesson 6.3 already taught you. Third, use those residuals to work out which direction reduces the loss, and by roughly how much. Fourth, take one small step of the coefficient and one small step of the intercept in that direction, before starting the next round.

EQUATION — ONE GRADIENT DESCENT STEP

$$w_{\text{new}} = w + \text{rate} \times \sum (x \times \text{residual}), \quad \text{and} \quad b_{\text{new}} = b + \text{rate} \times \sum (\text{residual})$$

In words: To update the coefficient, multiply each example's feature value by that same example's residual, add all of those products together across every example, multiply the total by the learning rate, then add the result to the current coefficient. To update the intercept, follow the same recipe using the plain residuals, without multiplying by x first.

w_{new} the coefficient after this step (same unit as w)

b_{new} the intercept after this step (same unit as b)

rate the learning rate, a small hyperparameter chosen before training begins (no unit, just a number)

residual $y - \hat{y}$ for one example, using the CURRENT coefficient and intercept, before this step's update (same unit as the target)

$\Sigma(x \times \text{residual})$ multiply each example's feature by that same example's residual, then add every one of those products together (depends on the feature and the target)

Note: This recipe comes from the mathematics of least squares. You do not need to derive it yourself here, only to apply it carefully, the same way you can use the formula for a circle's area without deriving it first.

Notice why this makes sense. If every residual is positive, the model is predicting too low, so $\Sigma(\text{residual})$ is positive, and adding a positive amount to the intercept raises every future prediction — exactly the right fix. If examples with a larger feature value also tend to have larger residuals, $\Sigma(x \times \text{residual})$ is positive too, so the coefficient increases, tilting the line up right where it was falling short.

COMMON MISTAKE

✗ Wrong: Choosing a learning rate that is far too large, hoping to reach the best line in fewer rounds.

✓ Right: A learning rate that is too large can overshoot the best line so badly that the loss bounces around or even grows larger with each round, instead of shrinking. Start with a small learning rate, confirm that the loss shrinks steadily round after round, and only then consider raising it.

Real gradient descent runs on many examples at once and takes hundreds or thousands of rounds, far too many to trace by hand. To watch its gears turn, let's shrink the whole problem down to a tiny practice dataset: just 3 homes, and a simpler number to predict, each home's daily electricity cost in dollars, instead of a full monthly bill. Home 1 runs 1 air conditioner and has a daily cost of \$3. Home 2 runs 2 air conditioners and has a daily cost of \$5. Home 3 runs 3 air conditioners and has a daily cost of \$7.

We start gradient descent with the simplest possible guess: coefficient $w = 0$ and intercept $b = 0$. This guess predicts \$0 for every home, obviously wrong, but gradient descent does not need a clever starting guess, only patience and enough rounds. We will use a learning rate of 0.04 for every round that follows.

WORKED EXAMPLE — TAKE THE FIRST ROUND OF GRADIENT DESCENT

Using the tiny 3-home dataset (1 air conditioner → \$3 per day; 2 air conditioners → \$5 per day; 3 air conditioners → \$7 per day), a starting guess of $w = 0$ and $b = 0$, and a learning rate of 0.04, find the updated coefficient and intercept after one round of gradient descent.

1. Predict each home's cost with the current guess, $w = 0$ and $b = 0$: $0 \times 1 + 0 = 0$. $0 \times 2 + 0 = 0$. $0 \times 3 + 0 = 0$.
2. Find each residual, true cost minus predicted cost: $3 - 0 = 3$. $5 - 0 = 5$. $7 - 0 = 7$.
3. Add the three residuals together: $3 + 5 + 7 = 15$.
4. Multiply each residual by that home's own feature value, then add the three products together: $(1 \times 3) + (2 \times 5) + (3 \times 7) = 3 + 10 + 21 = 34$.
5. Update the coefficient: $w_{\text{new}} = 0 + 0.04 \times 34 = 0 + 1.36 = 1.36$.
6. Update the intercept: $b_{\text{new}} = 0 + 0.04 \times 15 = 0 + 0.6 = 0.6$.

Answer: After one round, the coefficient is 1.36 and the intercept is 0.6, so the line is now predicted cost = $1.36 \times \text{air conditioners} + 0.6$. One single round already moved the line a long way from its useless starting guess of \$0 for every home.

WORKED EXAMPLE — TAKE THE SECOND ROUND OF GRADIENT DESCENT

Continue from the end of the first round, where $w = 1.36$ and $b = 0.6$. Using the same tiny dataset and the same learning rate of 0.04, find the updated coefficient and intercept after the second round.

1. Predict each home's cost with the current line, $w = 1.36$ and $b = 0.6$: $1.36 \times 1 + 0.6 = 1.96$. $1.36 \times 2 + 0.6 = 3.32$. $1.36 \times 3 + 0.6 = 4.68$.
2. Find each residual, true cost minus predicted cost: $3 - 1.96 = 1.04$. $5 - 3.32 = 1.68$. $7 - 4.68 = 2.32$.
3. Add the three residuals together: $1.04 + 1.68 + 2.32 = 5.04$.
4. Multiply each residual by that home's own feature value, then add the three products: $(1 \times 1.04) + (2 \times 1.68) + (3 \times 2.32) = 1.04 + 3.36 + 6.96 = 11.36$.

5. Update the coefficient: $w_{\text{new}} = 1.36 + 0.04 \times 11.36 = 1.36 + 0.4544 = 1.81$, rounded to two decimal places (we will round every coefficient and intercept this way from here on).
6. Update the intercept: $b_{\text{new}} = 0.6 + 0.04 \times 5.04 = 0.6 + 0.2016 = 0.80$, rounded to two decimal places.

Answer: After two rounds, the coefficient is about 1.81 and the intercept is about 0.80, so the line is now predicted cost $\approx 1.81 \times \text{air conditioners} + 0.80$. Compare this round's three residuals, 1.04, 1.68, and 2.32, with the very first round's residuals, 3, 5, and 7: every residual has already shrunk considerably, which is a direct sign that the loss itself is shrinking too.

WORKED EXAMPLE — TAKE THE THIRD ROUND OF GRADIENT DESCENT

Continue from the end of the second round, where $w = 1.81$ and $b = 0.80$. Using the same tiny dataset and the same learning rate of 0.04, find the updated coefficient and intercept after the third round.

1. Predict each home's cost with the current line, $w = 1.81$ and $b = 0.80$: $1.81 \times 1 + 0.80 = 2.61$. $1.81 \times 2 + 0.80 = 4.42$. $1.81 \times 3 + 0.80 = 6.23$.
2. Find each residual, true cost minus predicted cost: $3 - 2.61 = 0.39$. $5 - 4.42 = 0.58$. $7 - 6.23 = 0.77$.
3. Add the three residuals together: $0.39 + 0.58 + 0.77 = 1.74$.
4. Multiply each residual by that home's own feature value, then add the three products: $(1 \times 0.39) + (2 \times 0.58) + (3 \times 0.77) = 0.39 + 1.16 + 2.31 = 3.86$.
5. Update the coefficient: $w_{\text{new}} = 1.81 + 0.04 \times 3.86 = 1.81 + 0.1544 = 1.96$, rounded to two decimal places.
6. Update the intercept: $b_{\text{new}} = 0.80 + 0.04 \times 1.74 = 0.80 + 0.0696 = 0.87$, rounded to two decimal places.

Answer: After three rounds, the coefficient is about 1.96 and the intercept is about 0.87, so the line is now predicted cost $\approx 1.96 \times \text{air conditioners} + 0.87$. The residuals have shrunk again, to 0.39, 0.58, and 0.77, far smaller than the first round's residuals of 3, 5, and 7.

Look at how quickly the fit improved. The very first guess, $w = 0$ and $b = 0$, had a loss of $3^2 + 5^2 + 7^2 = 9 + 25 + 49 = 83$. After just three rounds, the loss is only $0.39^2 + 0.58^2 + 0.77^2 = 0.15 + 0.34 + 0.59 = 1.08$. Three small, patient steps shrank the loss from 83 down to about 1.08.

This tiny dataset was built so that a perfect line exists for it: predicted cost $= 2 \times \text{air conditioners} + 1$ fits all three homes exactly, since $2 \times 1 + 1 = 3$, $2 \times 2 + 1 = 5$, and $2 \times 3 + 1 = 7$. Gradient descent was never told this shortcut. It only felt its way downhill, one step at a time, yet after three rounds its guess, $w \approx 1.96$ and $b \approx 0.87$, already sits close to the true $w = 2$ and $b = 1$. A few more rounds would close the rest of that gap. This is the whole idea behind training almost every model in this book: guess, measure the mistakes, take a small step that shrinks them, and repeat until the steps stop mattering.

Python · NumPy

 runnable in the app

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([3, 5, 7])
w, b, rate = 0, 0, 0.04

for round_number in range(1, 4):
    predicted = w * x + b
    residual = y - predicted
    w = w + rate * np.sum(x * residual)
    b = b + rate * np.sum(residual)
    print("Round", round_number, "-> w:", round(w, 4), " b:", round(b, 4))
```

The first two lines store the tiny dataset as NumPy arrays; the third line sets the starting coefficient, intercept, and learning rate. Each loop pass repeats the same four actions as the worked examples: predict, find the residual, then update w and b with the same sum-and-multiply recipe. Running this prints $w \approx 1.36$, 1.8144 , and 1.9644 after rounds 1 through 3, matching the hand calculations before rounding.

IMPORTANT

A model trained on only 3 examples, like the tiny practice dataset above, is only useful for learning how gradient descent works. Never trust a model trained on so little data to make real predictions. Chapter 5's lessons on overfitting and generalization apply here just as strongly as anywhere else in this book: more examples, and a proper validation set, are always needed before trusting a model with a real decision.

6.5 Multiple Features: More Than One Input

Home size alone is a useful clue about an electricity bill, but it is far from the only one. Two homes of the exact same size can have very different bills, if one runs three air conditioners around the clock and the other runs only one. Rania's real dataset actually includes a second column: the number of major appliances running in each home, counting things such as air conditioners, refrigerators, and electric water heaters. A regression line that uses both features at once can capture more of the real story than either feature could alone.

EQUATION — A REGRESSION LINE WITH SEVERAL FEATURES

$$\hat{y} = b + w_1x_1 + w_2x_2 + \cdots + w_kx_k$$

In words: The predicted value equals the intercept, plus every feature multiplied by its own coefficient, with all of those products added together, one term for each feature the model uses.

$\hat{y}$ the predicted value (same unit as the target)

b the intercept (same unit as the target)

$x_1, x_2, \dots, x_k$ the feature values for one example, one value for each of the k features used (each feature's own unit)

$w_1, w_2, \dots, w_k$ the coefficient that matches each feature (target unit per unit of that feature)

Note: With one feature, this is exactly Lesson 6.2's line. With several features at once, this is sometimes called a hyperplane instead of a line, but the recipe never changes: multiply each feature by its own coefficient, then add everything together, including the intercept.

WORKED EXAMPLE — PREDICT A BILL USING TWO FEATURES AT ONCE

Rania fits a new rule using two features: predicted bill = $5 + 0.4 \times \text{home size} + 9 \times \text{number of major appliances}$, where home size is in square meters. Use this rule to predict the bill for a 120 m² home with 4 major appliances.

1. Multiply the size coefficient by the home's size: $0.4 \times 120 = 48$.
2. Multiply the appliances coefficient by the number of appliances: $9 \times 4 = 36$.
3. Add both results to the intercept: $5 + 48 + 36 = 89$.

Answer: The rule predicts a monthly bill of \$89 for this home. Notice that adding a second feature did not change the recipe at all, only the number of terms to multiply and add.

Every coefficient in a multiple-feature line has a precise meaning: it is the change in the prediction for one extra unit of that one feature, while every other feature stays exactly the same. The next worked example makes this concrete by comparing two homes that differ in only one feature.

WORKED EXAMPLE — INTERPRET THE APPLIANCES COEFFICIENT BY COMPARING TWO HOMES

Home X is 120 m² with 4 major appliances. Home Y is also 120 m², but has 5 major appliances instead of 4. Using the same rule, predicted bill = $5 + 0.4 \times \text{size} + 9 \times \text{appliances}$, find both predictions and compare them.

1. Predict Home X's bill, exactly as found above: $5 + 0.4 \times 120 + 9 \times 4 = 5 + 48 + 36 = 89$.
2. Predict Home Y's bill, using 5 appliances instead of 4: $5 + 0.4 \times 120 + 9 \times 5 = 5 + 48 + 45 = 98$.
3. Find the difference between the two predictions: $98 - 89 = 9$.

Answer: Home Y's predicted bill is \$9 higher than Home X's. That \$9 gap exactly matches the appliances coefficient, 9, because size was held fixed at 120 m² for both homes. This is exactly what a coefficient measures: the effect of one feature, on its own, with every other feature held perfectly still.

COMMON MISTAKE

✗ Wrong: Concluding that appliances "matter more" than home size just because its coefficient, 9, is a much bigger number than size's coefficient, 0.4.

✓ **Right:** A coefficient's size depends partly on the units of its feature, not only on how much that feature truly matters. Home size is measured in whole square meters and often ranges over 100 or more, while appliance count only ranges from about 1 to 6. Comparing coefficients fairly first requires putting every feature on a similar scale, using **standardization** from Chapter 3.

Gradient descent, from Lesson 6.4, extends to several features with no new ideas at all. Every coefficient simply gets its own small step, computed the very same way, once per round, alongside the intercept's own step. A model with 2 features updates 3 numbers each round instead of 2; a model with 20 features updates 21 numbers each round instead of 2. The recipe never changes, only the count of numbers being updated together.

IMPORTANT

Adding more features is not automatically an improvement. Chapter 5 already warned that a model with too much capacity can overfit, and every new feature gives a linear regression model a little more capacity to chase noise in the training data. A useful feature should have a real, sensible reason to matter, not just be added because it happened to be available in a spreadsheet column.

6.6 Ridge and Lasso: Regularized Regression

Lesson 6.5 warned that extra features can let a model chase noise instead of a real pattern, which Chapter 5 named overfitting. **Regularization**, Chapter 5's fix, adds a penalty for complexity directly into training, so the model must earn every large coefficient rather than take one for free. Chapter 5 gave this idea in general form: regularized loss = fit error + $\lambda \times$ complexity. Ridge regression and lasso regression are simply two precise ways of measuring what "complexity" means for a regression line's coefficients.

WORD TO KNOW

Ridge regression — Linear regression with an L2 penalty added to the loss: λ times the sum of every coefficient squared. Ridge shrinks every coefficient toward zero, shrinking large coefficients the most, but it almost never pushes a coefficient all the way to exactly zero.

WORD TO KNOW

Lasso regression — Linear regression with an L1 penalty added to the loss: λ times the sum of the absolute value of every coefficient. Lasso can shrink a weak coefficient all the way to exactly zero, which removes that feature from the model entirely.

EQUATION — RIDGE PENALTY AND LASSO PENALTY

$$\text{ridge penalty} = \lambda \times \sum w^2, \quad \text{and} \quad \text{lasso penalty} = \lambda \times \sum |w|$$

In words: The ridge penalty multiplies λ by the sum of every coefficient squared. The lasso penalty multiplies λ by the sum of every coefficient's absolute value, ignoring its sign. Neither penalty ever includes the intercept.

λ the regularization strength, a hyperparameter chosen before training (no unit, just a number)

$\sum w^2$ square every coefficient in the model, then add all of the squares together (depends on the feature, squared)

$\sum w$

Note: Total regularized loss is still fit error plus this penalty, exactly as Chapter 5 showed. Only the coefficients are penalized; the intercept is always left alone.

WORKED EXAMPLE — COMPARE THE RIDGE PENALTY AND THE LASSO PENALTY FOR ONE MODEL

A fuller version of Rania's model, trained on more homes and more features, finds a coefficient of 0.5 for home size and 6 for major appliances, before any regularization, with a fit error (SSE) of 40. Using $\lambda = 0.2$, find the ridge penalty, the lasso penalty, and the total regularized loss under each method.

1. Square each coefficient, for the ridge penalty: $0.5^2 = 0.25$. $6^2 = 36$.
2. Add the two squared coefficients, then multiply by λ : $(0.25 + 36) \times 0.2 = 36.25 \times 0.2 = 7.25$. This is the ridge penalty.
3. Take the absolute value of each coefficient, for the lasso penalty: $|0.5| = 0.5$. $|6| = 6$.
4. Add the two absolute values, then multiply by λ : $(0.5 + 6) \times 0.2 = 6.5 \times 0.2 = 1.3$. This is the lasso penalty.
5. Add the ridge penalty to the fit error: $40 + 7.25 = 47.25$. This is the total regularized loss under ridge.
6. Add the lasso penalty to the fit error: $40 + 1.3 = 41.3$. This is the total regularized loss under lasso.

Answer: The ridge penalty is 7.25, and the lasso penalty is only 1.3, even though both started from the exact same two coefficients. Squaring, which ridge uses, punishes the one large coefficient, 6, far more heavily than lasso's plain absolute value does. Notice, too, how much of each penalty came from the appliances coefficient alone, 36 out of 36.25 for ridge: this is exactly why comparing raw coefficients across differently scaled features, without standardizing first, can quietly punish one feature far more than it truly deserves.

Ridge and lasso differ in one more important way. As λ grows larger, ridge shrinks every coefficient smoothly toward zero, a little more at a time, but almost never lands exactly on zero. Lasso can push a genuinely weak coefficient all the way to exactly 0.0, and that feature then disappears from the model completely, as if it had never been included. A model that starts with 30 candidate features and a strong lasso penalty might keep only 6 or 7 of them, with the rest set to exactly zero. This is why lasso doubles as a quick way to spot which features matter least, not only as a defense against overfitting.

COMMON MISTAKE

✗ Wrong: Applying the ridge or lasso penalty to the intercept, the same way it is applied to every coefficient.

✓ Right: The penalty applies only to the coefficients, never to the intercept. The intercept simply sets the line's baseline height; it says nothing about how complex the model is, so penalizing it would only push every prediction toward zero for no good reason.

Python · NumPy

 runnable in the app

```
from sklearn.linear_model import Ridge, Lasso
import numpy as np

X = np.array([[50, 1], [90, 3], [130, 5]]) # columns: size, appliances
y = np.array([50, 82, 110])

ridge = Ridge(alpha=0.2).fit(X, y)
lasso = Lasso(alpha=0.2).fit(X, y)
print("Ridge coefficients:", ridge.coef_, "intercept:", ridge.intercept_)
print("Lasso coefficients:", lasso.coef_, "intercept:", lasso.intercept_)
```

The X array stores two features side by side, home size and appliance count, one row per home. Ridge and Lasso work like other scikit-learn models: create the model with a chosen strength, call fit, then read coef_ and intercept_. Note that scikit-learn calls the regularization strength alpha, a different use of a name close to Lesson 6.4's learning rate, worth watching for once you read real code. Running this finds every coefficient automatically, the same job gradient descent did by hand, now scaled up with a penalty built in.

You have now met every major idea behind linear regression: predicting with a line, measuring mistakes with residuals and a loss function, finding the best line automatically with gradient descent, extending to several features at once, and taming an overly flexible model with ridge or lasso. Lesson 6.7 lets you explore all of these ideas hands-on, watching a line and its loss change together, live on screen.

6.7 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Home Bills" dataset, and choose Linear Regression as the model.
3. Press Fit, and watch the line settle into place while a small loss counter ticks downward beside it.

4. Open the Labs tab, choose the Gradient Descent Lab, and press Step several times in a row. Watch the line move a little with each press, exactly like the 3 rounds you worked out by hand in Lesson 6.4.
5. Back in the Play tab, switch the model from Linear Regression to Ridge, and slide λ from 0 up toward its largest value. Write down what happens to the line and to the size of the coefficients shown on screen.

You need: Computer, tablet, or phone with the ML Studio app

6.8 Case Study

CASE STUDY — ESTIMATING ELECTRICITY BILLS BEFORE THE METER READING, IN ERBIL

Rania's rules so far used just a handful of homes and one or two features: useful for learning the method, far too small to trust with real customers. Her office's real records cover 4,000 homes across two years, with home size, appliance count, occupant count, and building age recorded alongside each month's true bill.

She starts the way Chapter 2 taught her, with a clear decision first: give a representative an instant estimated bill range while a customer is still on the phone, before the meter reading arrives. She splits her 4,000 homes into a training set, a validation set, and a test set, exactly as Chapters 1 and 2 recommend.

A plain linear regression on all four features fits the training data closely, but the coefficient for building age swings to a huge, oddly specific number, far larger than any sensible relationship between age and electricity use would suggest. A handful of very old buildings, with unusually high bills for unrelated reasons, are pulling that coefficient off course — exactly the overfitting risk Lesson 6.5 warned about.

Rania tries ridge regression instead, testing several values of λ and comparing validation error at each one, the same disciplined search Chapter 5 taught her. A moderate λ tames the building-age coefficient without hurting accuracy on the validation set. Out of curiosity, she checks lasso too, and finds it drops the occupant-count feature to exactly zero, suggesting it added little once size and appliances were already known.

The finished tool never shows a representative one falsely confident number. It shows a range, built from the model's typical validation error, with a reminder that unusual homes, new construction, or a recent renovation can make the estimate less reliable. Rania's office retrains the model every year, since appliance habits and prices change over time, and a rule learned from old bills is never guaranteed to fit new ones.

COMMON MISTAKES TO AVOID

- Multiplying the coefficient by the feature and then forgetting to add the intercept.
- Comparing candidate lines using the raw sum of squared errors instead of the mean squared error.

- Choosing a learning rate so large that gradient descent overshoots, and the loss bounces around or grows instead of shrinking.
- Trusting a model trained on a tiny practice dataset, like the 3-home example in Lesson 6.4, as if it were ready to make real predictions.
- Comparing coefficients across features with very different scales, without standardizing first.
- Applying the ridge or lasso penalty to the intercept, instead of only to the coefficients.
- Adding a new feature to a model simply because it happened to be available, without a sensible reason to expect it matters.

CHAPTER SUMMARY

- Linear regression predicts a number by multiplying each feature by its own coefficient, then adding one fixed intercept.
- A coefficient measures how much the prediction changes for one extra unit of its feature; the intercept is the prediction when every feature equals zero.
- A residual is the true value minus the predicted value; a loss function, usually least squares, turns every residual into one single number that grades a whole line.
- Gradient descent finds a good coefficient and intercept automatically, taking many small steps, each one shrinking the loss a little, using the residuals to decide which direction to move.
- The learning rate controls how big each gradient descent step is; too large a rate can overshoot the best line, and too small a rate wastes time on tiny steps.
- A regression line extends naturally to several features at once; each coefficient still measures its own feature's effect while every other feature is held fixed.
- Ridge regression (an L2 penalty) shrinks every coefficient toward zero smoothly; lasso regression (an L1 penalty) can shrink a weak coefficient all the way to exactly zero, dropping that feature from the model.
- The regularization strength, λ , is a hyperparameter, and like every hyperparameter, it should be chosen using a validation set, never the test set.
- A regression prediction is a best estimate, not a guaranteed fact; an honest tool shows a range and stays humble about unusual cases.
- Chapter 7 adapts this same coefficient-and-intercept idea to a new kind of question: a yes-or-no answer, using a close cousin of linear regression called logistic regression.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define coefficient and intercept in your own words, one sentence each.

Problem 2 [Basic] A regression line is predicted bill = $12 + 0.6 \times \text{size}$. What does this rule predict for a 0 m² home, and why is that particular prediction not meaningful in the real world?

Problem 3 [Basic] A model predicts \$75 for a home whose true bill is \$81. Find the residual, and state whether the model predicted too high or too low.

Problem 4 [Basic] Explain, in your own words, the main difference between ridge regression and lasso regression.

Problem 5 [Standard] Using predicted bill = $8 + 0.5 \times \text{size} + 7 \times \text{appliances}$, find the predicted bill for a 100 m² home with 3 major appliances, showing every step.

Problem 6 [Standard] A line's five residuals, in dollars, are 3, -2, 1, -4, and 2. Find the sum of squared errors and the mean squared error, showing every step.

Problem 7 [Standard] A tiny dataset has $x = [2, 4]$ and $y = [6, 10]$. Starting from $w = 0$ and $b = 0$, with a learning rate of 0.05, find w and b after one round of gradient descent, showing every step.

Problem 8 [Standard] A model has a coefficient of 3 for one feature and 1 for another. Using $\lambda = 0.5$, find the ridge penalty and the lasso penalty, showing every step.

Problem 9 [Standard] Explain, in your own words, why one coefficient should not be compared directly against another coefficient measured in very different units.

Problem 10 [Standard] A model's loss shrinks steadily for the first 10 rounds of gradient descent, then bounces up and down without settling for the next 10 rounds. Suggest one likely cause, and one fix.

Problem 11 [Challenge] A model with 5 features and no regularization has a training MSE of 4 and a validation MSE of 19. (a) Which of Chapter 5's problems does this pattern suggest? (b) Name two specific fixes from this chapter that could help, and explain briefly how each one works. (c) If a strong lasso penalty later sets 3 of the 5 coefficients to exactly zero, what does that suggest about those 3 features?

Problem 12 [Challenge] Open ML Studio, fit Ridge regression on the "Home Bills" dataset, and try at least four different values of λ , recording the training score and the validation score at each one. Identify which λ underfits, which overfits, and which you would finally choose, explaining your reasoning in a few sentences.

Problem 13 [Challenge] Design your own two-feature linear regression project for a setting you know well, such as a shop, a farm, a school, or a clinic. Name your two features and your target, write a plausible regression equation with

made-up coefficients, and explain in a few sentences why you would consider ridge or lasso regularization before trusting the model.

SUPERVISED LEARNING

CHAPTER 7

Logistic Regression

Turning a score into a probability

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why plain linear regression is a poor fit for yes/no questions, and what logistic regression does differently
- Apply the sigmoid function to a score by hand, and read the result as a probability between 0 and 1
- Calculate log loss for a single prediction, and explain why it punishes confident, wrong predictions so heavily
- Find the decision boundary of a logistic regression model, and use it to turn a probability into a Pass/Fail prediction
- Explain how gradient descent trains a logistic regression model, and connect it to Chapter 6's method
- Name two other common classifiers, and describe in one sentence what each one does differently

7.1 From a Line to a Yes/No Answer

Chapter 6 predicted a number: an electricity bill in dollars. Many real questions are not about a number at all, but about a plain yes or no. Will this loan be repaid? Will this transaction turn out to be fraud? Will this student pass the final exam? This chapter answers exactly this second kind of question, using a close cousin of Chapter 6's method.

Meet Soran. He teaches a first-year statistics course at a university in Sulaymaniyah. Every term, a few students struggle and fail the final exam, often discovering the trouble too late to fix it: by the time a low midterm grade appears, only a few weeks remain before the final exam. Soran wants to predict, early in the term, which students are at risk of failing, so he can offer extra help while there is still time to act, in the same spirit as Chapter 1's school counseling example. He starts with the simplest clue available: how many hours a student reports studying each week.

Soran has records from 6 past students, showing hours studied per week and the final result: 1 hour, Fail. 2 hours, Fail. 3 hours, Fail. 6 hours, Pass. 8 hours, Pass. 9 hours, Pass. To work with this label mathematically, Soran writes Fail as 0 and Pass as 1. This is simply a labeling convention, not a claim that failing is worth "zero" of anything.

The most natural first idea is to reuse Chapter 6's method directly: fit a straight line to these 0s and 1s, using hours studied as the feature and the 0/1 outcome

as the target. Suppose training this way produces the line: predicted score = $-0.1 + 0.15 \times \text{hours}$. Let's see what this line predicts at two extremes.

WORKED EXAMPLE — SEE WHAT GOES WRONG WHEN LINEAR REGRESSION PREDICTS A YES/NO ANSWER

A line fit directly to 0/1 pass-fail data is predicted score = $-0.1 + 0.15 \times \text{hours}$ studied. Find the predicted score for a student who studied 0 hours, and for a student who studied 12 hours.

1. Substitute 0 hours into the line: $-0.1 + 0.15 \times 0 = -0.1 + 0 = -0.1$.
2. Substitute 12 hours into the line: $-0.1 + 0.15 \times 12 = -0.1 + 1.8 = 1.7$.

Answer: The line predicts **-0.1** for the first student and **1.7** for the second. Neither number can be a real probability, because a probability must always sit between 0 and 1. A "probability" of **-0.1** or **1.7** is meaningless, and this is exactly the problem logistic regression is built to fix.

COMMON MISTAKE

✗ Wrong: Reading a plain linear regression score, such as 1.7 or -0.1, as if it were already a probability.

✓ Right: A linear regression score has no upper or lower limit, so it can never be trusted as a probability on its own. Logistic regression fixes this by passing the score through one more step, the sigmoid function, before treating it as a probability.

Soran needs a method built specifically for this shape of problem: a method that still starts with a familiar score, built the same way as Chapter 6's, but never lets that score wander outside the range a true probability is allowed to occupy.

WORD TO KNOW

Logistic regression — A method that predicts the probability of a yes-or-no answer. It starts with the same recipe as linear regression, a coefficient for each feature plus an intercept, but then squeezes that score through the sigmoid function, so the result always lands between 0 and 1.

Despite its name, logistic regression is used for classification, not regression: it predicts a probability, which is then turned into a category, Pass or Fail. Lesson 7.2 introduces the squeezing step that makes this possible.

7.2 The Sigmoid Function

Logistic regression needs a function that takes any score at all, however large, however small, however negative, and always returns a number between 0 and 1. That function is the sigmoid function, and its output traces a smooth S-shaped curve: very low for very negative scores, very high for very positive scores, rising smoothly through the middle. Picture the curve on a graph: it hugs the bottom near 0 on the left, climbs steeply through the center, then flattens out near 1 on the right, never quite touching either boundary.

WORD TO KNOW

Sigmoid function — A function that squeezes any real number into a value between 0 and 1, tracing a smooth S-shaped curve. Logistic regression uses it to turn a plain score into a probability.

EQUATION — THE SIGMOID FUNCTION

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In words: The predicted probability equals 1, divided by 1 plus e raised to the power of negative z. As z grows very large and positive, the probability approaches 1. As z grows very large and negative, the probability approaches 0. When z is exactly 0, the probability is exactly 0.5.

$\sigma(z)$ the predicted probability (probability, between 0 and 1)

z any score at all, such as a coefficient times a feature, plus an intercept (no fixed unit)

e a fixed number, approximately 2.71828, that appears naturally in many growth formulas (no unit, just a number)

Note: You do not need to know where e comes from to use this formula, only its approximate value, the same way you use π in a circle's area without deriving it first.

In logistic regression, z is exactly the same kind of score Chapter 6 already taught you to compute: an **intercept**, plus a **coefficient** times each feature. Suppose Soran's logistic regression finds the line $\text{score} = -5 + 1 \times \text{hours studied}$. This score, z , then passes through the sigmoid function to become a predicted probability of passing.

IMPORTANT

A logistic regression coefficient describes a pattern in past data, not proof of cause. A positive coefficient on hours studied does not prove that studying more, by itself, causes a student to pass. Students who study more may also attend more, sleep more, or simply feel more prepared going in. Treat a coefficient as a useful clue, never as proof of what would happen if only one thing changed.

WORKED EXAMPLE — APPLY THE SIGMOID FUNCTION TO A SCORE OF ZERO

Student A studied exactly 5 hours, giving a score of $z = -5 + 1 \times 5 = 0$ under Soran's line. Find the predicted probability that Student A passes.

1. Substitute $z = 0$ into the sigmoid formula: $\sigma(0) = 1 \div (1 + e^0)$.
2. Any number raised to the power 0 equals 1, so $e^0 = 1$.
3. Add 1 and 1: $1 + 1 = 2$.
4. Divide: $1 \div 2 = 0.5$.

Answer: The predicted probability of passing is exactly 0.5, or 50%. A score of exactly 0 always lands exactly halfway between 0 and 1, leaning neither toward Pass nor toward Fail.

WORKED EXAMPLE — APPLY THE SIGMOID FUNCTION TO A HIGHER AND A LOWER SCORE

Student B studied 7 hours, giving a score of $z = -5 + 1 \times 7 = 2$. Student C studied 3 hours, giving a score of $z = -5 + 1 \times 3 = -2$. Find the predicted probability of passing for each student.

1. Write Student B's formula: $\sigma(2) = 1 \div (1 + e^{-2})$.
2. Using a calculator, $e^{-2} \approx 0.1353$. Add 1: $1 + 0.1353 = 1.1353$.
3. Divide: $1 \div 1.1353 \approx 0.881$. Student B's predicted probability of passing is about 88.1%.
4. Write Student C's formula: $\sigma(-2) = 1 \div (1 + e^2)$.
5. Using a calculator, $e^2 \approx 7.389$. Add 1: $1 + 7.389 = 8.389$.
6. Divide: $1 \div 8.389 \approx 0.119$. Student C's predicted probability of passing is about 11.9%.

Answer: Student B, who studied more, gets a high predicted probability, 88.1%. Student C, who studied less, gets a low predicted probability, 11.9%. Notice the two probabilities are equally far from 50%, in opposite directions: scores equally far above and below zero always produce probabilities equally far above and below 0.5.

COMMON MISTAKE

✗ Wrong: Assuming a negative score, z , must produce a negative probability.

✓ Right: Sigmoid never outputs a number below 0 or above 1, no matter how negative or positive z is. A very negative score simply produces a probability very close to 0, never below it.

A probability alone does not yet grade whether Soran's model is doing a good job. Lesson 7.3 introduces the loss function built specifically for probabilities: log loss.

7.3 Log Loss: Scoring a Probability Honestly

Chapter 6 graded a regression line with mean squared error, the average of each squared residual. Grading a probability calls for a different kind of loss, one that reacts sharply to how confident, and how wrong, a prediction turns out to be. That loss is log loss.

WORD TO KNOW

Log loss — A loss function for probabilities, also called cross-entropy loss, that heavily punishes a confident prediction that turns out to be wrong. A perfect, fully confident, correct prediction scores a log loss of 0; a confident, wrong prediction can score a very large loss.

EQUATION — LOG LOSS FOR ONE EXAMPLE

$$L = -[y \times \ln(p) + (1 - y) \times \ln(1 - p)]$$

In words: Log loss compares the true label, always 0 or 1, with the predicted probability that the label is 1. When the true label is 1, only the first term survives, and log loss is

simply $-\ln(p)$. When the true label is 0, only the second term survives, and log loss is simply $-\ln(1 - p)$.

L the log loss for one example (no unit, just a number)

y the true label, always 0 or 1 (count)

p the predicted probability, from the sigmoid function, that the label is 1 (probability, between 0 and 1)

ln the natural logarithm, a fixed mathematical function found on any calculator, the reverse of raising e to a power (no unit)

Note: A small p paired with a true label of 1, or a large p paired with a true label of 0, always produces a large log loss. A confident, correct prediction produces a log loss close to 0.

Natural logarithms behave in a useful way for this purpose: $\ln(1)$ is exactly 0, and $\ln$ of a number just below 1 is a small negative number, while $\ln$ of a number close to 0 is a large negative number. Flipping the sign with the minus in front of the formula turns these into a small loss for a confident, correct prediction, and a large loss for a confident, wrong one. The three worked examples below make this concrete with real numbers.

WORKED EXAMPLE — FIND THE LOG LOSS WHEN STUDENT B PASSES, AS PREDICTED

Student B's predicted probability of passing, from Lesson 7.2, was $p = 0.881$. Student B's true result arrives: Student B passed ($y = 1$). Find the log loss for this one prediction.

1. Since the true label $y = 1$, only the first part of the formula matters: log loss $= -\ln(p)$.
2. Substitute $p = 0.881$: log loss $= -\ln(0.881)$.
3. Using a calculator, $\ln(0.881) \approx -0.127$.
4. Apply the negative sign from the formula: $-(-0.127) = 0.127$.

Answer: The log loss is about 0.127, a small number. Soran's model predicted a high probability of passing, and the student did pass, so the model is charged only a small penalty for this example.

WORKED EXAMPLE — COMPARE THE LOG LOSS IF STUDENT B HAD FAILED INSTEAD

Suppose, instead, Student B's true result had been Fail ($y = 0$), even though the model still predicted $p = 0.881$ as the probability of passing. Find the log loss for this alternative outcome.

1. Since the true label $y = 0$, only the second part of the formula matters: log loss $= -\ln(1 - p)$.
2. Substitute $p = 0.881$: $1 - 0.881 = 0.119$, so log loss $= -\ln(0.119)$.
3. Using a calculator, $\ln(0.119) \approx -2.129$.
4. Apply the negative sign from the formula: $-(-2.129) = 2.129$.

Answer: The log loss is about 2.129, more than 16 times larger than the 0.127 found in the previous example. The predicted probability, 0.881, did not change between the two examples; only the true outcome

changed. This is exactly what log loss is built to do: punish a confident prediction severely when it turns out to be wrong.

WORKED EXAMPLE — FIND THE LOG LOSS FOR A COMPLETELY UNSURE PREDICTION

A model predicts $p = 0.5$ for a student, leaning neither toward Pass nor toward Fail. Find the log loss if the student actually passed ($y = 1$), using $\ln(0.5) \approx -0.693$.

1. Since the true label $y = 1$, $\log \text{loss} = -\ln(p) = -\ln(0.5)$.
2. Substitute the given value and apply the negative sign: $-(-0.693) = 0.693$.

Answer: The log loss is about 0.693, worse than Student B's confident, correct score of 0.127, but far better than the confidently wrong score of 2.129. A model that always shrugs and answers "I don't know" pays this same moderate, steady price on every single example, which is exactly why a genuinely useful model needs to do better than a coin flip.

COMMON MISTAKE

✗ Wrong: Training a logistic regression model using mean squared error, the same **loss function** Chapter 6 used for a straight line.

✓ Right: Logistic regression trains using log loss, not mean squared error. Log loss reacts correctly to probabilities, and punishes a confident, wrong prediction far more harshly than mean squared error ever would.

Soran now has a way to score any single prediction. Training a full model means finding the coefficient and intercept that make the average log loss, across every training student, as small as possible. Lesson 7.4 uses a trained model's probabilities to make an actual Pass/Fail decision.

7.4 The Decision Boundary

A probability is useful on its own, but eventually Soran must decide: reach out to this student with extra support, or not. That requires turning a probability into one clear decision. The simplest rule uses 0.5 as a cutoff: predict Pass if the probability is 0.5 or higher, and predict Fail otherwise. Chapter 16 explores how to move this cutoff deliberately, for projects where one kind of mistake costs far more than the other; for now, 0.5 is a sensible default.

WORD TO KNOW

Decision boundary — The line, or surface, that separates the region where a classifier predicts one category from the region where it predicts another. For logistic regression, the decision boundary sits exactly where the predicted probability equals 0.5.

With a single feature, like Soran's hours-studied model, the decision boundary is just one point on the number line: the exact number of hours where the predicted probability crosses 0.5. With two features, the decision boundary becomes a full line, splitting a two-dimensional graph into a Pass region and a Fail region, exactly the way Lesson 7.7's case study uses hours studied and attendance together. With many features, it becomes a hyperplane, the same

word Lesson 6.5 used for a regression surface with many features. In every case, the recipe for finding it is the same: set the score, z , equal to 0, because Lesson 7.2 showed that a score of 0 always gives a probability of exactly 0.5. Everything on one side of the boundary gets one predicted category; everything on the other side gets the other.

WORKED EXAMPLE — FIND THE DECISION BOUNDARY FOR SORAN'S MODEL

Soran's logistic regression uses $\text{score} = -5 + 1 \times \text{hours studied}$. Find the exact number of study hours where the predicted probability of passing equals exactly 0.5.

1. A probability of exactly 0.5 happens only where the score, z , equals exactly 0.
2. Set the score formula equal to 0: $-5 + 1 \times \text{hours} = 0$.
3. Add 5 to both sides to solve for hours: $\text{hours} = 5$.

Answer: The decision boundary sits at exactly 5 hours studied per week. Soran's model predicts Pass for any student who studied more than 5 hours, and Fail for any student who studied fewer than 5 hours.

Students B and C from Lesson 7.2 confirm this rule. Student B studied 7 hours, above the boundary, and was predicted Pass with an 88.1% probability. Student C studied 3 hours, below the boundary, and was predicted Fail, since an 11.9% probability of passing means an 88.1% probability of failing.

COMMON MISTAKE

✗ Wrong: Assuming 0.5 is always the correct cutoff for turning a probability into a decision, for every project.

✓ Right: 0.5 is only a sensible default. Chapter 16 shows how to move this cutoff deliberately, for example lowering it to catch more true cases when missing one is especially costly, such as a serious illness or a fraud alert.

GOING FURTHER — OTHER CLASSIFIERS YOU WILL MEET

Logistic regression draws its decision boundary as a straight line, or a flat hyperplane once there are several features. Two other classifiers, common in real projects, draw their boundaries differently. A Support Vector Machine looks for the widest possible street between the two classes, not just any line that happens to separate them, which often makes it more resistant to a few unusual points near the edge. Naive Bayes multiplies simple probabilities together, feature by feature, and is a fast, surprisingly strong baseline for text such as emails or reviews, despite its rather bold, simplifying assumption that every feature acts independently of every other one. Both still follow the same fit-and-predict pattern you already know from this chapter, and both appear as options inside ML Studio's Play tab.

IMPORTANT

A predicted probability is not a fact about a student, and a Fail prediction is not a verdict. Soran's model can be wrong, and it never knows why a number is low: a student might be working a job, caring for

family, or simply new to independent study. Use a low predicted probability to offer support early, never to label a student or lower expectations of them.

Soran now knows how to compute a probability, score it honestly with log loss, and turn it into a decision. Lesson 7.5 explains how training actually finds a good coefficient and intercept in the first place.

7.5 Fitting Logistic Regression with Gradient Descent

Every coefficient and intercept used so far in this chapter was simply handed to you, to keep the focus on sigmoid and log loss. Training a real logistic regression model finds these numbers automatically, using the very same **gradient descent** method Lesson 6.4 already taught you, one small step at a time. Even though log loss looks very different from mean squared error, minimizing it with gradient descent turns out to follow almost the exact same recipe as before. The only change: compare the true label, y , to the sigmoid-transformed probability, p , instead of to a plain linear prediction.

EQUATION — ONE GRADIENT DESCENT STEP FOR LOGISTIC REGRESSION

$$w_{\text{new}} = w + \text{rate} \times \sum (x \times (y - p)), \quad \text{and} \quad b_{\text{new}} = b + \text{rate} \times \sum (y - p)$$

*In words: This is Lesson 6.4's exact recipe, with one change: p , the sigmoid-transformed probability, replaces the plain linear prediction. Multiply each example's feature by $(y - p)$, add those products together, multiply by the **learning rate**, then add the result to the current coefficient. Update the intercept the same way, using the plain $(y - p)$ values.*

w_{new} the coefficient after this step (same unit as w)

b_{new} the intercept after this step (same unit as b)

rate the learning rate, chosen before training begins (no unit, just a number)

p the predicted probability from the sigmoid function, using the CURRENT coefficient and intercept (probability, between 0 and 1)

y - p the true label minus the predicted probability, playing the same role a residual played in Chapter 6 (no unit, just a number)

Note: This close match between Chapter 6 and Chapter 7 is not a coincidence; it is one of the elegant facts of logistic regression. You do not need to derive it, only recognize and use it.

WORKED EXAMPLE — TAKE ONE ROUND OF GRADIENT DESCENT FOR LOGISTIC REGRESSION

A tiny practice dataset has 2 students: Student 1 studied 2 hours and failed ($y = 0$). Student 2 studied 8 hours and passed ($y = 1$). Starting from $w = 0$ and $b = 0$, with a learning rate of 0.1, find the updated coefficient and intercept after one round of gradient descent.

1. Compute each student's score with the current guess, $w = 0$ and $b = 0$: $0 \times 2 + 0 = 0$. $0 \times 8 + 0 = 0$.

2. Turn each score into a probability with the sigmoid function: $\sigma(0) = 0.5$ for both students, exactly as Lesson 7.2 showed.
3. Find $y - p$ for each student: Student 1: $0 - 0.5 = -0.5$. Student 2: $1 - 0.5 = 0.5$.
4. Add the two values together: $-0.5 + 0.5 = 0$.
5. Multiply each value by that student's hours, then add the results: $(2 \times -0.5) + (8 \times 0.5) = -1 + 4 = 3$.
6. Update the coefficient: $w_{\text{new}} = 0 + 0.1 \times 3 = 0.3$.
7. Update the intercept: $b_{\text{new}} = 0 + 0.1 \times 0 = 0$.

Answer: After one round, the coefficient is 0.3 and the intercept is still 0. Student 2 studied more hours than the model expected for someone predicted at only 50%, and passed anyway, pulling the coefficient upward. Student 1's and Student 2's pulls on the intercept canceled out exactly this round, since their $y - p$ values were equal and opposite.

Further rounds would keep nudging w and b , using the exact same four actions: score, sigmoid, compare to the truth, step. Training stops once the steps become small enough that the coefficient and intercept barely move at all. The full training loss being minimized is simply the average log loss across every training example, exactly the single-example loss from Lesson 7.3, repeated once per student and averaged.

Just like Chapter 6, this same training process can add a **regularization** penalty, ridge-style or lasso-style, discouraging overly large coefficients in exactly the same way. A learning rate that is too large causes the same overshooting problem Lesson 6.4 warned about; a learning rate that is too small simply takes longer to settle near the best line.

Python · NumPy

 runnable in the app

```
from sklearn.linear_model import LogisticRegression
import numpy as np

hours = np.array([[1], [2], [3], [6], [8], [9]])
passed = np.array([0, 0, 0, 1, 1, 1])

model = LogisticRegression()
model.fit(hours, passed)
print("coefficient:", model.coef_, "intercept:", model.intercept_)
print("predicted probability for 7 hours:", model.predict_proba([[7]]))
```

The hours array stores each past student's weekly study hours, one row per student, and passed stores the matching 0/1 outcome: Soran's original 6 students from Lesson 7.1. Fitting a LogisticRegression model works exactly like fitting the models from Chapter 6: create the model, then call fit with the

features and the labels. The `predict_proba` method returns a full probability, not just a final Pass/Fail label, exactly the number this chapter has been computing by hand since Lesson 7.2.

You now have the complete method: turn a score into a probability with sigmoid, grade that probability honestly with log loss, draw a decision boundary, and train the whole model automatically with gradient descent. Lesson 7.6 lets you watch this boundary bend, live, as you add data or change the regularization strength.

7.6 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Pass or Fail" dataset, and choose Logistic Regression as the model.
3. Press Fit, and look at the colored decision boundary that appears between the Pass points and the Fail points.
4. Open the Labs tab, choose the Sigmoid Lab, and drag a point left and right along the score axis. Watch its predicted probability rise and fall along the S-shaped curve.
5. Back in the Play tab, add a second feature, attendance rate, and press Fit again. Describe, in one or two sentences, how the decision boundary changes shape once a second feature is added.

You need: Computer, tablet, or phone with the ML Studio app

7.7 Case Study

CASE STUDY — AN EARLY-WARNING MODEL FOR A STATISTICS COURSE IN SULAYMANIYAH

Soran's single-feature model, using only hours studied, catches the most obvious cases, but he suspects it is missing something. Two students who report similar study hours sometimes have very different outcomes. He wonders whether attendance matters too, since a student who studies alone from the textbook, without attending class, may be missing lecture-only material that never appears in the readings.

He adds a second feature: each student's attendance rate over the first four weeks of term, as a percentage. His logistic regression now draws its decision boundary as a full line across a two-dimensional graph, one axis for hours studied and one for attendance, instead of a single point on a number line. Students in the upper-right region, high hours and high attendance, sit deep in the Pass region. Students in the lower-left region sit deep in the Fail region. The interesting cases, the ones Soran most wants to catch early, sit close to the line itself.

His dataset only covers 140 past students, spread across three previous terms, and with two features now instead of one, Soran worries about the same overfitting risk Chapter 5 and Chapter 6 both warned about. He tries three

regularization strengths and compares validation log loss at each one: a weak penalty scores 0.58, a moderate penalty scores 0.41, and a strong penalty scores 0.53, the same U-shaped pattern Chapter 5 taught him to expect. He chooses the moderate strength. He also checks the decision boundary by eye: without regularization, it bends in strange, sharp ways near a handful of unusual students; with the moderate penalty, it settles into a smooth, sensible line.

Soran then faces one more decision, beyond the model itself: where to set the cutoff. A missed at-risk student, one the model predicted as "likely to pass" who then fails, loses a chance at help that might have changed the outcome for the whole term. An unnecessary check-in with a student who was doing fine costs only a few minutes of both people's time, and rarely feels unwelcome when it is framed warmly. Given this imbalance between the two kinds of mistakes, Soran deliberately lowers his cutoff below 0.5, choosing to flag more students for a friendly conversation, even knowing some of them were never truly at risk.

The finished tool never appears on any official record, and it never assigns a grade or a warning letter on its own. It produces a private, weekly list for Soran's own use: students whose predicted probability of passing has dropped below his chosen cutoff, sorted so the lowest probabilities appear first. A flagged student receives a short, supportive message inviting a conversation about how the course is going, never a warning about failing. Soran retrains the model at the end of each term, folding in the newest results, because study habits, course difficulty, and even the students themselves change from one year to the next. A model trained on last year's class is a helpful starting point, not a permanent rule.

COMMON MISTAKES TO AVOID

- Reading a plain linear regression score as if it were already a probability.
- Assuming a negative score, z , must produce a negative probability from the sigmoid function.
- Training a logistic regression model with mean squared error instead of log loss.
- Assuming 0.5 is always the right cutoff for turning a probability into a decision.
- Treating a predicted probability as a fact about a person, rather than an estimate that can be wrong.
- Choosing a learning rate so large that gradient descent overshoots and never settles.
- Adding a second feature without checking for overfitting on a small dataset.

CHAPTER SUMMARY

- Logistic regression predicts a probability for a yes-or-no question, using the same coefficient-and-intercept recipe as linear regression, plus one more step.

- The sigmoid function squeezes any score into a probability between 0 and 1; a score of exactly 0 always gives a probability of exactly 0.5.
- Log loss, also called cross-entropy loss, grades a probability against the true 0/1 label, punishing a confident, wrong prediction far more than a small, honest mistake.
- The decision boundary is where the predicted probability equals 0.5: a point with one feature, a line with two features, a hyperplane with many.
- Gradient descent trains logistic regression using nearly the same recipe as Chapter 6, comparing the true label to the sigmoid-transformed probability instead of a plain prediction.
- Regularization works the same way for logistic regression as it does for linear regression, discouraging overly large coefficients.
- The 0.5 cutoff is only a default; Chapter 16 shows how to move it deliberately when one kind of mistake matters more than another.
- Naive Bayes and Support Vector Machines are two other common classifiers, each drawing a decision boundary in its own way.
- A predicted probability is an estimate, not a fact about a person; use it to offer help, never to hand down a verdict.
- Chapter 8 turns to a very different kind of classifier, one that predicts by looking at an example's closest neighbors, with no coefficients and no gradient descent at all.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Explain, in your own words, why a plain linear regression score cannot be used directly as a probability.

Problem 2 [Basic] Define the sigmoid function in your own words, in one or two sentences.

Problem 3 [Basic] A score of $z = 0$ is passed through the sigmoid function. State the resulting probability without doing any calculation, and explain why.

Problem 4 [Basic] Explain, in your own words, why log loss punishes a confident, wrong prediction more heavily than a small, honest mistake.

Problem 5 [Standard] A logistic regression model gives a student a score of $z = 3$. Using $\sigma(3) = 1 \div (1 + e^{-3})$ and $e^{-3} \approx 0.0498$, find the predicted probability, showing every step.

Problem 6 [Standard] A model predicts $p = 0.7$ for a student whose true label is Pass ($y = 1$). Using $\ln(0.7) \approx -0.357$, find the log loss, showing every step.

Problem 7 [Standard] A logistic regression model uses $\text{score} = -8 + 2 \times \text{hours studied}$. Find the decision boundary, showing every step.

Problem 8 [Standard] A tiny dataset has 2 examples: $x = 1, y = 0$, and $x = 3, y = 1$. Starting from $w = 0$ and $b = 0$, with a learning rate of 0.2, find w and b after one round of gradient descent, showing every step.

Problem 9 [Standard] Explain, in your own words, the difference between a decision boundary with one feature and a decision boundary with two features.

Problem 10 [Standard] A project must choose between 0.5 and 0.3 as the probability cutoff for flagging a rare disease during screening. Explain which cutoff is likely safer, and why.

Problem 11 [Challenge] (a) A logistic regression model has a training log loss of 0.20 and a validation log loss of 0.85. Which of Chapter 5's problems does this pattern suggest? (b) Suggest two specific fixes, from Chapter 6 or Chapter 7, that could help. (c) After adding regularization, training log loss rises slightly to 0.30, but validation log loss falls to 0.35. Explain why this trade is likely a good one.

Problem 12 [Challenge] Open ML Studio, fit Logistic Regression on the "Pass or Fail" dataset, and try at least three different regularization strengths. Record how the decision boundary's shape changes each time, and explain your observations in a few sentences.

Problem 13 [Challenge] Design your own yes/no prediction project for a setting you know well. Name your feature or features and your two possible labels, and explain in a few sentences whether missing a true case or raising a false alarm would cost more in your project, and how that would affect your choice of probability cutoff.

SUPERVISED LEARNING

CHAPTER 8

k-Nearest Neighbors

Predicting by asking your closest neighbors

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how k-Nearest Neighbors predicts a label by finding the closest training examples and letting them vote
- Compute the Euclidean distance between two examples by hand, using two or more features
- Explain why feature scaling changes a distance calculation, and standardize features before comparing distances
- Compare how a small k and a large k change a model's sensitivity to noise, using the same new example
- Use distance-weighted voting so that closer neighbors count more than farther ones
- Recognize the challenge that many features create for any distance-based method

8.1 Predict Like Your Neighbors

Imagine you just moved to a new neighborhood in Sulaymaniyah. You do not know the area yet, and you want to find a good, honest bakery nearby. You have no reviews app and no guidebook. So you do the simplest thing that actually works. You ask the neighbors who live closest to you. If four out of your five nearest neighbors all recommend the same bakery, you probably trust their answer, because people who live near you tend to shop in similar places and notice similar things.

This chapter turns that everyday habit into a complete machine learning method. Instead of asking human neighbors about a bakery, we ask a new example's closest neighbors inside the training data what label they carry, and we let those neighbors vote. You may already recognize the name from the ML Studio activities in earlier chapters. Now it is time to learn exactly how the method works.

WORD TO KNOW

k-Nearest Neighbors — A method that predicts a label for a new example by finding the k training examples most similar to it, then letting those k examples vote on the answer. The method is usually written kNN for short. The letter k simply stands for however many neighbors get a vote.

kNN is one of the simplest ideas in this book, and also one of the strangest compared with every model you have met so far. Linear regression and logistic

regression, from Chapters 6 and 7, each learn a fixed equation during training, made of coefficients and an intercept, and then set the original training data aside and use only that equation afterward. kNN does something else entirely. It keeps every single training example and waits. Only when a new example actually arrives does kNN do any real work: measure how far the new example sits from every stored training example, find the closest k of them, and let those neighbors vote. There is no equation to fit, and almost no training step at all, only storage.

Chapter 1 solved a similar puzzle a different way, using nearest-centroid classification. That method collapsed each class down into a single average point, a **centroid**, and compared a new example only to those two or three averaged points. kNN keeps every individual training example instead of averaging them away, and it compares a new example to real, individual neighbors, never to an average.

Let's build a running example for the whole chapter. A fruit export company runs a packing facility that sorts apples before shipping. Right now, an inspector grades each apple as Grade A, the premium grade, or Grade B, the standard grade, by picking it up and judging its weight and firmness by hand. The company wants a machine to help make this judgment faster and more consistent, starting with the simplest possible version of the idea: judging an apple using only its weight, in grams.

The company already has a small set of apples an inspector has graded by hand. Four of them are Grade A: weights of 115, 195, 210, and 220 grams. Three of them are Grade B: weights of 80, 90, and 100 grams. Notice that one Grade A apple, at 115 grams, is unusually small for its grade. Real inspectors sometimes grade an apple this way for reasons that weight alone does not explain, such as an unusually deep red color or a perfectly round shape. Keep this one small apple in mind. It becomes important later in this chapter.

Now a new apple arrives, weighing 205 grams, and it needs a grade before it can be boxed and shipped.

WORKED EXAMPLE — CLASSIFY A NEW APPLE USING K-NEAREST NEIGHBORS, $k = 3$

Grade A training apples weigh 115, 195, 210, and 220 grams. Grade B training apples weigh 80, 90, and 100 grams. A new apple weighs 205 grams. Using $k = 3$, find its three nearest neighbors by weight, and predict its grade by majority vote.

1. Find the distance from the new apple, 205 grams, to every training apple, by subtracting and dropping the sign: $|115 - 205| = 90$. $|195 - 205| = 10$. $|210 - 205| = 5$. $|220 - 205| = 15$. $|80 - 205| = 125$. $|90 - 205| = 115$. $|100 - 205| = 105$.
2. Sort all seven distances from smallest to largest: 5 (210 g, Grade A), 10 (195 g, Grade A), 15 (220 g, Grade A), 90 (115 g, Grade A), 105 (100 g, Grade B), 115 (90 g, Grade B), 125 (80 g, Grade B).

3. Because $k = 3$, keep only the three smallest distances: 5 (Grade A), 10 (Grade A), and 15 (Grade A).
4. Count the votes among these three nearest neighbors: 3 votes for Grade A, 0 votes for Grade B.

Answer: The new apple is classified as Grade A. All three of its nearest neighbors by weight are Grade A apples, so the vote is unanimous.

This one worked example already shows the entire kNN recipe: measure the distance to every training example, keep the k closest ones, and let them vote. Lesson 8.2 asks a question this example quietly skipped past. Why choose $k = 3$, and would a different choice of k ever change the answer?

COMMON MISTAKE

✗ Wrong: Believing kNN has no real work left to do at prediction time, just because it has almost no training step.

✓ Right: kNN shifts its work from training time to prediction time. Every single new example requires measuring its distance to every stored training example, which can get slow once the training set is very large.

8.2 Choosing k : Small vs. Large

The letter k is not decided by the learning algorithm. A person chooses it before making any prediction, which makes k a **hyperparameter**, exactly the kind of setting Chapter 5 warned you to choose carefully, using a validation set rather than a guess. Choosing k turns out to matter a great deal, because it controls how much a single unusual neighbor can influence the final vote.

Remember the unusually small Grade A apple from Lesson 8.1, weighing only 115 grams. A new apple now arrives weighing 112 grams, sitting very close to that one unusual apple, and also fairly close to the whole group of Grade B apples.

WORKED EXAMPLE — CLASSIFY A BOUNDARY APPLE AT THREE DIFFERENT VALUES OF k

Using the same training apples as Lesson 8.1 — Grade A at 115, 195, 210, and 220 grams; Grade B at 80, 90, and 100 grams — a new apple weighs 112 grams. Predict its grade using $k = 1$, then $k = 3$, then $k = 5$.

1. Find the distance from 112 grams to every training apple: $|115 - 112| = 3$. $|195 - 112| = 83$. $|210 - 112| = 98$. $|220 - 112| = 108$. $|80 - 112| = 32$. $|90 - 112| = 22$. $|100 - 112| = 12$.
2. Sort all seven distances from smallest to largest: 3 (115 g, Grade A), 12 (100 g, Grade B), 22 (90 g, Grade B), 32 (80 g, Grade B), 83 (195 g, Grade A), 98 (210 g, Grade A), 108 (220 g, Grade A).
3. At $k = 1$, keep only the single nearest neighbor: 115 g, Grade A. The vote is 1 for Grade A, 0 for Grade B, so the prediction is Grade A.
4. At $k = 3$, keep the three nearest neighbors: 115 g (A), 100 g (B), 90 g (B). The vote is 1 for Grade A, 2 for Grade B, so the prediction is Grade B.

5. At $k = 5$, keep the five nearest neighbors: 115 g (A), 100 g (B), 90 g (B), 80 g (B), 195 g (A). The vote is 2 for Grade A, 3 for Grade B, so the prediction is Grade B.

Answer: $k = 1$ predicts Grade A, but $k = 3$ and $k = 5$ both predict Grade B. A single very close neighbor was enough to control the entire prediction at $k = 1$, while asking more neighbors at $k = 3$ and $k = 5$ gave the steadier answer, Grade B, which matches the true local neighborhood of this apple far better than one unusual point does.

This is exactly the trade-off Chapter 5 called the **bias-variance trade-off**, now appearing inside a brand-new model. A very small k , especially $k = 1$, lets a single training example decide the whole prediction, so kNN reacts to every small accident in the data, including mislabeled or unusual points. This behaves like the wiggly, high-**variance** models from Chapter 5: it fits the training data closely but can be led astray by noise. A very large k asks so many neighbors for their opinion that the vote barely reacts to where the new example actually sits. Taken to an extreme, if k were set to 7, equal to the entire training set here, the vote would always be 4 Grade A votes against 3 Grade B votes, no matter where the new apple sits. At that point, k is so large that kNN has stopped looking at neighbors at all, and is simply predicting the majority class every single time, much like the Flat model from Chapter 5, which also ignored its input and always predicted the same value. This is **underfitting**, wearing a new disguise.

COMMON MISTAKE

✗ Wrong: Choosing an even number for k in a two-class problem, such as $k = 4$, which can produce an exact tie between the two classes with no clear winner.

✓ Right: Choose an odd k for a two-class problem whenever possible, so a majority vote is always possible. If a tie can still happen, such as in a three-class problem, decide in advance how ties will be broken, for example by falling back on the single nearest neighbor.

A real project rarely knows the best k in advance, just as Chapter 5 could not know the best tree depth or regularization strength in advance. The fix is the same one you already know: try several values of k , score each one on a validation set, and keep the value that scores best on data the model has not seen during training, never the value that merely looks best on training data alone.

8.3 Distance and Feature Scaling

Every example so far in this chapter used only one feature, weight, so "distance" simply meant subtracting two weights. Real packing lines measure more than weight. Let's add a second feature: a firmness score from 0 to 10, given by a handheld probe, where a higher score means a firmer apple. Grade A apples tend to be heavier and firmer. Grade B apples tend to be lighter and softer. To compare two apples that each have two features, distance needs a proper definition.

WORD TO KNOW

Euclidean distance — The straight-line distance between two points, found by squaring the difference in each feature, adding the squared differences together, and taking the square root of that sum. With two features, this is exactly the same idea as measuring the diagonal side of a right triangle.

EQUATION — EUCLIDEAN DISTANCE BETWEEN TWO POINTS

$$\text{distance} = \sqrt{(a_1 - a_2)^2 + (b_1 - b_2)^2}$$

In words: To find the Euclidean distance between two points, subtract their first-feature values and square the result, subtract their second-feature values and square that result, add the two squared results together, then take the square root of that sum.

a₁, a₂ the first feature of point 1 and of point 2 (same unit as that feature)

b₁, b₂ the second feature of point 1 and of point 2 (same unit as that feature)

distance how far apart the two points are, combining both features into one number (no natural unit when the two features use different scales — Lesson 8.3 shows why)

Note: With three or more features, add one more squared difference, one for each extra feature, before taking the square root.

WORKED EXAMPLE — FIND THE EUCLIDEAN DISTANCE BETWEEN TWO APPLES

Apple 1 weighs 206 grams with a firmness score of 9. Apple 2 weighs 200 grams with a firmness score of 1. Find the Euclidean distance between them.

1. Subtract the two weights and square the result: $206 - 200 = 6$, then $6^2 = 36$.
2. Subtract the two firmness scores and square the result: $9 - 1 = 8$, then $8^2 = 64$.
3. Add the two squared differences together: $36 + 64 = 100$.
4. Take the square root of that sum: $\sqrt{100} = 10$.

Answer: The Euclidean distance between the two apples is 10. Notice that this number mixes grams and firmness points together, so it has no real-world unit of its own — it is only useful for comparing against other distances measured the exact same way.

That last worked example used friendly numbers on purpose, so the square root would come out exactly. Real distances usually will not resolve so neatly, but that is not actually a problem: when kNN only needs to know which neighbor is closer, comparing the squared distances, before taking any square root, always gives the same ordering as comparing the true distances. Chapter 1's centroid example used this exact shortcut, and the rest of this lesson uses it too, to keep every number easy to check by hand.

Here is where a serious trap appears. Weight is measured in grams, with values in the hundreds. Firmness is measured on a scale from 0 to 10. When these two features are combined into one distance calculation without any adjustment, the feature with the much bigger numbers quietly takes over the entire result.

WORKED EXAMPLE — CLASSIFY AN APPLE USING RAW, UNSCALED DISTANCES

One Grade A reference apple weighs 200 g with firmness 8. One Grade B reference apple weighs 150 g with firmness 2. A new apple, Q, weighs 170 g with firmness 7. Using $k = 1$ and raw, unscaled features, which reference apple is Q closer to?

1. Find the squared distance from Q (170, 7) to the Grade A apple (200, 8).
Weight: $170 - 200 = -30$, squared is 900. Firmness: $7 - 8 = -1$, squared is 1. Add them: $900 + 1 = 901$.
2. Find the squared distance from Q (170, 7) to the Grade B apple (150, 2).
Weight: $170 - 150 = 20$, squared is 400. Firmness: $7 - 2 = 5$, squared is 25. Add them: $400 + 25 = 425$.
3. Compare the two squared distances: 425 is smaller than 901, so Q sits closer to the Grade B apple.

Answer: Using raw, unscaled features, apple Q is classified as Grade B. Notice that Q's firmness, 7, is nearly identical to the Grade A apple's firmness of 8, and quite far from the Grade B apple's firmness of 2. That strong signal barely mattered, because weight's much larger numbers dominated the squared distance almost completely.

This is precisely the trap Chapter 1 warned about when comparing a melon's weight against its sound score. The fix is the same one Chapter 3 already taught: put every feature on a shared, fair scale before comparing distances, using **standardization**. Recall the formula: subtract the training mean from a value, then divide by the training standard deviation. Suppose the packing facility's full training history gives a mean weight of 170 g with a standard deviation of 40 g, and a mean firmness of 5 with a standard deviation of 2.

WORKED EXAMPLE — CLASSIFY THE SAME APPLE AGAIN, AFTER STANDARDIZING

Using a training mean weight of 170 g and standard deviation of 40 g, and a training mean firmness of 5 and standard deviation of 2, standardize the Grade A apple (200, 8), the Grade B apple (150, 2), and the new apple Q (170, 7). Then repeat the $k = 1$ classification using the standardized values.

1. Standardize the Grade A apple's weight: $(200 - 170) \div 40 = 30 \div 40 = 0.75$. Standardize its firmness: $(8 - 5) \div 2 = 3 \div 2 = 1.5$. The Grade A apple becomes (0.75, 1.5).
2. Standardize the Grade B apple's weight: $(150 - 170) \div 40 = -20 \div 40 = -0.5$. Standardize its firmness: $(2 - 5) \div 2 = -3 \div 2 = -1.5$. The Grade B apple becomes (-0.5, -1.5).
3. Standardize apple Q's weight: $(170 - 170) \div 40 = 0 \div 40 = 0$. Standardize its firmness: $(7 - 5) \div 2 = 2 \div 2 = 1$. Apple Q becomes (0, 1).
4. Find the squared distance from standardized Q (0, 1) to standardized Grade A (0.75, 1.5). Weight: $0 - 0.75 = -0.75$, squared is 0.5625. Firmness: $1 - 1.5 = -0.5$, squared is 0.25. Add them: $0.5625 + 0.25 = 0.8125$.

5. Find the squared distance from standardized Q (0, 1) to standardized Grade B (-0.5, -1.5). Weight: $0 - (-0.5) = 0.5$, squared is 0.25. Firmness: $1 - (-1.5) = 2.5$, squared is 6.25. Add them: $0.25 + 6.25 = 6.5$.
6. Compare the two squared distances: 0.8125 is smaller than 6.5, so Q now sits closer to the Grade A apple.

Answer: After standardizing, apple Q is classified as Grade A, the opposite of the raw, unscaled answer. Once weight and firmness are placed on a fair, shared scale, firmness's strong signal, that Q's firmness closely matches the Grade A apple, finally gets to count.

IMPORTANT

Always compute the training mean and training standard deviation from the training set alone, exactly as Chapter 3 warned, then reuse those same two numbers to standardize every validation apple, test apple, and future new apple. Recomputing the mean and standard deviation later, using data that is supposed to stay hidden, is a form of data leakage, and it quietly breaks the fairness of every distance calculated afterward.

COMMON MISTAKE

✗ Wrong: Comparing raw distances between features that use very different scales, letting a large-number feature such as weight in grams quietly dominate a small-number feature such as a 0-to-10 firmness score.

✓ Right: Standardize every numerical feature before computing a distance for kNN. Distance-based methods are especially sensitive to feature scale, far more than most of the other models in this book.

The following short program repeats the exact same standardized classification as the worked example above, using scikit-learn.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X_train = np.array([[200, 8], [150, 2]])
y_train = ["Grade A", "Grade B"]
q = np.array([[170, 7]])

scaler = StandardScaler().fit(X_train)
X_scaled = scaler.transform(X_train)
q_scaled = scaler.transform(q)

model = KNeighborsClassifier(n_neighbors=1).fit(X_scaled, y_train)
print(model.predict(q_scaled))
```


The `StandardScaler` learns a mean and a standard deviation from `X_train`, then `transform` applies the same standardization formula from Chapter 3 to both the training points and the new apple, `q`. The `KNeighborsClassifier` then stores the scaled training points and, when asked to `predict`, finds the single nearest one, exactly like the hand calculation above. Running this code prints "Grade A," matching Lesson 8.3's standardized answer exactly. Notice that the scaler is fit only once, on the training data, and then reused for `q`, never refit on the new apple, exactly as the IMPORTANT box above insists.

8.4 Weighted Voting and High Dimensions

Every vote counted equally in every worked example so far. But think back to Lesson 8.2's boundary apple, weighing 112 grams. At $k = 3$, its three nearest neighbors were 115 g (Grade A, distance 3), 100 g (Grade B, distance 12), and 90 g (Grade B, distance 22). The Grade A neighbor sat far closer than either Grade B neighbor, yet a plain vote still only gave it one vote out of three, the same weight as neighbors sitting much farther away. A neighbor eight times closer arguably deserves more say than a distant one. This is exactly what distance-weighted voting fixes: instead of one vote per neighbor, each neighbor's vote counts more when it sits closer, and counts less when it sits farther away, usually by weighting each vote by one divided by its distance.

WORKED EXAMPLE — RECLASSIFY THE BOUNDARY APPLE USING DISTANCE-WEIGHTED VOTING

At $k = 3$, apple 112 g has neighbors 115 g (Grade A, distance 3), 100 g (Grade B, distance 12), and 90 g (Grade B, distance 22). Give each neighbor a weight of 1 divided by its distance, then predict the grade using weighted voting.

1. Find the weight of the Grade A neighbor: $1 \div 3 \approx 0.333$.
2. Find the weight of the first Grade B neighbor: $1 \div 12 \approx 0.083$.
3. Find the weight of the second Grade B neighbor: $1 \div 22 \approx 0.045$.
4. Add the weight of every Grade A neighbor: only one neighbor, so the Grade A total is 0.333.
5. Add the weight of every Grade B neighbor: $0.083 + 0.045 = 0.128$.
6. Compare the two totals: 0.333 is clearly larger than 0.128.

Answer: Weighted voting predicts Grade A, the opposite of the plain majority vote at $k = 3$, which predicted Grade B by two votes to one. The single Grade A neighbor sat so much closer than either Grade B neighbor that, once distance is allowed to matter, its opinion outweighs both farther neighbors combined.

Weighted voting does not remove the need to choose k sensibly, and it does not remove the need to standardize features first; it simply gives a fairer say to neighbors who are genuinely more similar to the new example, rather than treating every one of the k neighbors as equally trustworthy.

kNN's whole idea rests on one assumption: examples that are close together in feature space are usually similar in label too. That assumption starts to strain as more and more features are added. Picture the packing facility measuring not

just weight and firmness, but also color score, stem length, sugar content, skin texture, and a dozen other readings. With so many features at once, almost every pair of apples ends up moderately far apart in at least a few of those features, so nearly every apple starts to look almost equally distant from every other apple. "Nearest" stops being a meaningful idea once there are too many features competing at once, a problem researchers sometimes nickname the curse of dimensionality. This is one reason kNN often performs best with a modest, carefully chosen set of features, rather than every feature a team can think to measure. Chapter 15 returns to this exact question, choosing fewer, better features on purpose.

COMMON MISTAKE

✗ Wrong: Adding every available feature to a kNN model, assuming more measurements can only help the distance calculation.

✓ Right: Check whether each feature genuinely helps separate the classes before adding it. Extra, unhelpful features add noise to every distance calculation and can make truly similar examples look farther apart than they really are.

8.5 Try It in ML Studio and Case Study

Numbers on a page show you the mechanics of kNN one apple at a time. Watching the whole decision boundary shift as k changes, live on screen, makes the idea click in a different way.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Moons" dataset, which contains two curved, interleaving groups of colored points, and choose k-Nearest Neighbors as the model.
3. Set $k = 1$, press Fit, and look closely at the colored regions. Notice how tightly the boundary hugs every single point.
4. Increase k step by step, trying 5, 15, and 40, pressing Fit each time. Write down what happens to the boundary's shape, and what happens to the validation score shown on screen.
5. Find the value of k with the best validation score. Then switch the model to Logistic Regression on the same data. Explain in one or two sentences why a straight-line model struggles with this particular dataset in a way kNN does not.

You need: Computer, tablet, or phone with the ML Studio app

CASE STUDY — GRADING APPLES ON A PACKING LINE NEAR DUHOK

Lawand manages quality control at a fruit export company's packing facility near Duhok. Every day, thousands of apples pass along a conveyor belt, and a small team of inspectors grades each one by hand, by weight and by feel. The team is skilled, but two different inspectors do not always agree on a borderline

apple, and grading slows the whole line down during the busiest weeks of harvest season.

Lawand's team already owns a machine that records two numbers for every apple automatically: its weight in grams, and a firmness score from a small probe. Building on a season's worth of apples the inspectors already graded by hand, the team trains a kNN classifier, following this chapter's method closely. They first standardize both features, remembering the trap from Lesson 8.3, because weight's much larger numbers would otherwise swamp firmness entirely. They then try several values of k on a validation set of apples the model has not seen, exactly as Lesson 8.2 recommends, and settle on $k = 9$, which balances a steady vote against still noticing genuinely unusual apples.

The finished model runs on every apple as it crosses a sensor near the end of the line, instantly suggesting Grade A or Grade B before the apple reaches the packing boxes. Inspectors still review any apple the model is least confident about, which the team defines as any apple whose nearest neighbors are closely split, such as five votes to four rather than nine votes to zero. This keeps a trained human eye on precisely the borderline cases where a mistake is most likely, while letting the model handle the large majority of clear-cut apples on its own.

One detail from Lesson 8.1 comes back to matter here. The unusually small 115-gram apple that the team had graded Grade A, based on color and shape the model never measured, taught the team an important lesson about how kNN behaves. A single unusual training example like that one can pull a nearby prediction toward its own label, for reasons the model's two features cannot see or explain. Lawand's team responds by adding a written note to any training apple whose grade seems to depend on a feature the model does not measure, so that future retraining can consider adding that feature directly, rather than leaving the model to be quietly misled by an example it cannot fully understand.

IMPORTANT

A kNN model can only repeat the patterns already sitting in its training data, including any mistakes. If a past inspector mislabeled a batch of apples, kNN will happily copy that mistake onto every similar new apple, with no way to notice anything went wrong. Treat a model's prediction as a fast, consistent first opinion, not as a replacement for occasional human spot-checks on the training data itself.

COMMON MISTAKES TO AVOID

- Believing kNN has no real work to do at prediction time, just because it has almost no training step.
- Choosing an even number for k in a two-class problem, risking an exact tie with no clear winner.
- Comparing raw distances between features on very different scales, letting a large-number feature quietly dominate the whole calculation.
- Standardizing a new or test example using its own mean and standard deviation, instead of reusing the training set's mean and standard deviation.

- Assuming a very small k , such as $k = 1$, is always the most accurate choice, without checking a validation set.
- Assuming a very large k is always the safer choice, without noticing it can wash out real local patterns entirely.
- Adding every available feature to a kNN model, without checking whether each one truly helps separate the classes.

CHAPTER SUMMARY

- k-Nearest Neighbors predicts a new example's label by finding the k closest training examples and letting them vote, with almost no separate training step.
- Euclidean distance measures how far apart two points are by squaring each feature's difference, adding the squared differences, and taking the square root.
- Comparing squared distances gives the same ordering as comparing true distances, so the square-root step can be skipped when only the closest neighbor matters.
- A small k reacts strongly to individual, possibly noisy neighbors, behaving like a high-variance model; a very large k can wash out real local patterns and drift toward simply predicting the majority class, behaving like an underfit model.
- The best k is chosen using a validation set, the same honest method Chapter 5 used for every other hyperparameter.
- Standardizing every numerical feature before computing distance is essential for kNN, because raw features on different scales let one large-number feature dominate the whole calculation.
- Distance-weighted voting lets closer neighbors count more than farther ones, and can change a prediction compared with a plain, equal-weight vote.
- Too many features make every example look almost equally far from every other example, weakening the whole idea of a "nearest" neighbor.
- kNN can only repeat the patterns already present in its training data, including any past mistakes, so occasional human review of the training data itself still matters.
- Chapter 9 introduces a very different kind of model, one that predicts by asking a sequence of yes-or-no questions instead of measuring distance at all.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] In your own words, explain what the letter k means in k-Nearest Neighbors.

Problem 2 [Basic] Explain why kNN is sometimes described as having almost no training step.

Problem 3 [Basic] A two-class kNN model is set to $k = 6$. Explain, in one sentence, why this choice risks a tie vote.

Problem 4 [Basic] Training weights for Class X are 40 and 60 grams. Training weights for Class Y are 100 and 120 grams. A new example weighs 55 grams. Using $k = 1$ and weight only, which class does it belong to?

Problem 5 [Standard] Point P has features (12, 4). Point R has features (8, 1). Find the Euclidean distance between P and R, showing every step.

Problem 6 [Standard] A training point has features (30, 300), and a new point has features (35, 250). Explain, without doing the arithmetic, why comparing these two features' raw distance would be misleading, and name the fix.

Problem 7 [Standard] Using a training mean of 50 and a training standard deviation of 10, standardize the values 70 and 35, showing every step.

Problem 8 [Standard] At $k = 5$, a new example's five nearest neighbors are labeled Yes, No, Yes, No, and No, with distances 2, 3, 5, 6, and 9. Find the plain majority vote, then find the distance-weighted vote using $\text{weight} = 1 \div \text{distance}$, rounding each weight to three decimal places. State whether the two methods agree.

Problem 9 [Standard] Explain, in two or three sentences, why a very large k tends to behave like an underfit model.

Problem 10 [Challenge] A team builds a kNN model with 40 features for a small dataset of only 60 examples. (a) Explain one problem this many features could cause for a distance-based method. (b) Suggest one concrete step the team could take to fix it, and name the future chapter that covers this step in depth.

Problem 11 [Challenge] A hospital wants to use kNN to flag patients who may need a follow-up visit, based on features including age, blood pressure, and number of past visits. (a) Explain why standardizing these features matters here. (b) Describe one risk of trusting the model's prediction without any human review, and how the team could reduce that risk.

Problem 12 [Challenge] Open ML Studio, fit k-Nearest Neighbors on the "Blobs" dataset, and try at least four different values of k , recording the validation score shown on screen each time. Identify which values of k appear to underfit, which appear to overfit, and which value you would finally choose. Explain your reasoning in a few sentences.

SUPERVISED LEARNING

CHAPTER 9

Decision Trees

Making predictions through a sequence of questions

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how a decision tree makes a prediction by asking a sequence of yes-or-no questions
- Trace a new example from the root node to a leaf node, and read off its prediction
- Compute Gini impurity for a node, and use weighted impurity to compare two candidate splits
- Explain why decision trees are prone to overfitting, and describe how depth limits, leaf-size limits, and pruning control it
- Compute a regression tree's prediction as the average target value of the training examples in its leaf
- Recognize a decision tree's boxy, rectangular decision boundaries, and choose a depth using validation performance

9.1 A Tree Asks One Question at a Time

Imagine calling a repair hotline about a machine that has stopped working. A good technician does not ask everything at once. She asks one question, listens to your answer, then asks the next question based on what you just said. "Is the power light on?" If no, she asks about the cable. If yes, she asks something else entirely. Each answer narrows down the problem, one question at a time, until she reaches a clear next step.

A decision tree makes predictions the exact same way.

WORD TO KNOW

Decision tree — A model that predicts a label by asking a sequence of simple yes-or-no questions about an example's features, one question at a time, until it reaches a final answer. Each question narrows down the possibilities, exactly like a technician narrowing down a diagnosis.

Every decision tree has the same basic shape, borrowed from an upside-down real tree. It starts at a single starting point, asks its first and most important question there, and then branches downward into smaller and smaller groups until it reaches a final answer. Every question along the way sends an example down one of two branches, and every node that is not a leaf simply holds one more question, waiting to split its own group of examples even further.

WORD TO KNOW

Root Node and Leaf Node — The root node is the single starting point of a decision tree, where the very first question is asked, using the whole training set. A leaf node is an ending point that asks no further questions and simply gives a prediction, reached only after answering every question along the path from the root.

Let's build a running example for this chapter. A factory that manufactures small electric motors wants to prioritize which machines its maintenance team inspects first. The team already records two simple readings from each machine: its temperature in degrees Celsius, and its vibration level, recorded simply as High or Low by a sensor clipped to the machine's frame. Past experience already tells the team, for a set of machines, whether that machine actually turned out to need inspection soon afterward. A decision tree suits this problem well, because a tree's finished questions can be read aloud to a busy technician in plain language, unlike an equation full of coefficients.

Suppose a very small, already-trained tree looks like this. At the root node, the tree asks: is the machine's temperature 82°C or higher? If yes, the tree goes straight to a leaf node that predicts "Needs Inspection: Yes." If no, the tree is not done yet. It asks a second question: is the machine's vibration level High? If yes, it reaches a leaf node predicting "Needs Inspection: Yes." If no, it reaches a different leaf node predicting "Needs Inspection: No."

WORKED EXAMPLE — FOLLOW A NEW MACHINE FROM THE ROOT NODE TO A LEAF NODE

Using the small tree above — root question "temperature $\geq 82^{\circ}\text{C}$?", then, only if no, a second question "vibration = High?" — predict whether Machine X needs inspection. Machine X has a temperature of 78°C and a vibration level of High.

1. Start at the root node, and ask the first question: is 78°C greater than or equal to 82°C ? Since 78 is less than 82, the answer is no.
2. Because the answer was no, follow the "no" branch down to the second question: is the vibration level High? Machine X's vibration level is High, so the answer is yes.
3. Because the answer was yes, follow the "yes" branch down to a leaf node. This leaf node's prediction is "Needs Inspection: Yes."

Answer: Machine X needs inspection. Notice that reaching this answer took two questions, one at the root and one more beneath it, because the first question alone was not enough to decide.

Not every machine needs two questions. Machine Y has a temperature of 91°C and a vibration level of Low. At the root node, 91 is greater than or equal to 82, so the answer is yes, and the tree goes straight to a leaf node predicting "Needs Inspection: Yes," without ever asking about vibration at all. A decision tree only asks as many questions as it actually needs, and different examples can take paths of different lengths through the very same tree.

COMMON MISTAKE

✗ **Wrong:** Assuming every example must pass through the same number of questions on its way to a leaf node.

✓ **Right:** An example's path through a tree stops as soon as it reaches a leaf node, which can happen after just one question or after many. Two different examples can take paths of very different lengths through the same tree.

This tiny tree was simply handed to you, already built, so you could see how a finished tree makes a prediction. The far more interesting question is where its questions came from in the first place. Why did the tree ask about temperature before vibration, and why did it choose 82°C rather than some other number? Lesson 9.2 answers exactly that question.

9.2 A Good Split Makes Children More Pure

Building a decision tree means choosing the best possible question at every single node, starting at the root. A good question should sort examples so that the two resulting groups are each as one-sided as possible. Imagine sorting a basket of mixed fruit using a single question. A good question separates apples from oranges almost perfectly. A poor question leaves both groups just as mixed as the basket you started with. Decision trees measure exactly how "mixed" a group is using a single number.

WORD TO KNOW

Impurity — A number that measures how mixed together the labels are inside one node. A node with only one label present has zero impurity. A node with an even mix of labels has high impurity.

WORD TO KNOW

Gini impurity — The most common way to measure impurity in a decision tree. For each label present in a node, square the fraction of examples carrying that label, add these squares together, then subtract the total from 1.

EQUATION — GINI IMPURITY OF ONE NODE

$$\text{Gini} = 1 - \sum (p_i)^2$$

In words: Gini impurity equals 1 minus the sum, over every label present in the node, of that label's fraction of the node squared.

Gini the impurity of one node (ranges from 0 (pure) to 0.5 for two balanced classes)

p_i the fraction of examples in the node that carry label i (a proportion between 0 and 1)

Σ add up this quantity once for every label present in the node (no unit)

Note: A node where every example shares one label has $\text{Gini} = 0$, the purest possible node. A node split evenly between two labels has the highest possible Gini for two classes, 0.5.

Let's compute this for real, using eight machines from the maintenance team's records: their temperature readings, and whether each one turned out to need

inspection. Four machines, at 90°C, 95°C, 85°C, and 75°C, needed inspection. Four other machines, at 70°C, 65°C, 60°C, and 80°C, did not.

WORKED EXAMPLE — FIND THE ROOT NODE'S GINI IMPURITY

The root node contains all 8 machines: 4 labeled "Needs Inspection: Yes" and 4 labeled "Needs Inspection: No." Find this node's Gini impurity.

1. Find the fraction of machines labeled Yes: $4 \div 8 = 0.5$.
2. Find the fraction of machines labeled No: $4 \div 8 = 0.5$.
3. Square each fraction: $0.5^2 = 0.25$, and $0.5^2 = 0.25$.
4. Add the two squared fractions together: $0.25 + 0.25 = 0.5$.
5. Subtract that sum from 1: $1 - 0.5 = 0.5$.

Answer: The root node's Gini impurity is 0.5, the maximum possible value for two labels. This makes sense, because the root node is split exactly evenly between the two labels, the most mixed a two-label node can possibly be.

A Gini impurity of 0.5 is the worst possible starting point, so almost any sensible question should improve it. But "almost any" is not good enough; a real tree-building algorithm checks many candidate questions and keeps the one that lowers impurity the most. Comparing two children fairly requires one more idea: a large, pure child should count for more than a tiny, pure child, so each child's impurity must be weighted by how many examples actually landed inside it.

WORKED EXAMPLE — COMPARE TWO CANDIDATE SPLITS USING WEIGHTED GINI IMPURITY

Using the same 8 machines (Yes at 90°C, 95°C, 85°C, 75°C; No at 70°C, 65°C, 60°C, 80°C), compare two candidate splits. Split A asks "temperature $\geq 82^\circ\text{C}$?" Split B asks "temperature $\geq 68^\circ\text{C}$?" Find the weighted Gini impurity for each split, and decide which one the tree should choose.

1. Sort the machines into Split A's two children. The "yes" child (temperature $\geq 82^\circ\text{C}$) contains 90°C, 95°C, and 85°C, all three labeled Yes: 3 machines, 0 No. The "no" child contains 75°C, 70°C, 65°C, 60°C, and 80°C: 1 machine labeled Yes and 4 machines labeled No, 5 machines in total.
2. Find the Gini impurity of Split A's "yes" child: every one of its 3 machines shares the same label, so its impurity is $1 - (1^2 + 0^2) = 1 - 1 = 0$.
3. Find the Gini impurity of Split A's "no" child: the fraction labeled Yes is $1 \div 5 = 0.2$, and the fraction labeled No is $4 \div 5 = 0.8$. Its impurity is $1 - (0.2^2 + 0.8^2) = 1 - (0.04 + 0.64) = 1 - 0.68 = 0.32$.
4. Combine Split A's two children into one weighted score: $(3 \div 8) \times 0 = 0$, and $(5 \div 8) \times 0.32 = 0.2$. Add these together: $0 + 0.2 = 0.2$.
5. Sort the machines into Split B's two children. The "yes" child (temperature $\geq 68^\circ\text{C}$) contains 90°C, 95°C, 85°C, 80°C, 75°C, and 70°C: 4 machines labeled Yes and 2 machines labeled No, 6 machines in total. The "no" child contains 65°C and 60°C, both labeled No: 2 machines, 0 Yes.

6. Find the Gini impurity of Split B's "yes" child: the fraction labeled Yes is $4 \div 6 \approx 0.667$, and the fraction labeled No is $2 \div 6 \approx 0.333$. Its impurity is $1 - (0.667^2 + 0.333^2) \approx 1 - (0.445 + 0.111) = 1 - 0.556 = 0.444$.
7. Find the Gini impurity of Split B's "no" child: both of its 2 machines share the same label, so its impurity is $1 - (1^2 + 0^2) = 0$.
8. Combine Split B's two children into one weighted score: $(6 \div 8) \times 0.444 = 0.333$, and $(2 \div 8) \times 0 = 0$. Add these together: $0.333 + 0 = 0.333$.
9. Compare the two splits' weighted Gini impurities: Split A scores 0.2, and Split B scores 0.333.

Answer: Split A, asking "temperature $\geq 82^\circ\text{C}$ ", is the better split, because its weighted Gini impurity, 0.2, is lower than Split B's 0.333. A real tree-building algorithm checks many candidate questions exactly this way, and keeps whichever one produces the lowest weighted impurity — which is exactly why the tiny tree in Lesson 9.1 asked about 82°C at its root, instead of some other temperature.

COMMON MISTAKE

✗ Wrong: Comparing two children's impurity scores directly, without weighting each score by how many examples actually landed in that child.

✓ Right: Always weight each child's impurity by its share of the parent node's examples before adding the two together. A small, pure child should not count as much as a large, pure child.

A tree is built by repeating this exact same search at every new node it creates. After the root node splits into two children, the tree looks inside each child separately and searches for the best next question, using the same weighted Gini comparison, all over again. This continues, node by node, until every leaf is pure, or until some limit stops the tree early. That limit is the subject of Lesson 9.3, and it turns out to matter enormously.

GOING FURTHER — ENTROPY AND INFORMATION GAIN

Gini impurity is not the only way to measure how mixed a node is. Many decision tree tools also offer entropy, a measure borrowed from information theory that reaches zero for a pure node and its own maximum value for a perfectly mixed node, in a similar shape to Gini impurity. The improvement a split produces, measured using entropy, is often called information gain. In practice, trees built with Gini impurity and trees built with entropy usually choose very similar splits, and the choice between the two rarely changes a tree's real-world performance by much.

9.3 Trees Can Easily Overfit

Nothing stops a tree from splitting again and again, node after node, unless something tells it to stop. Taken to its extreme, a tree can keep splitting until every single leaf contains just one training example, each one perfectly pure, with a Gini impurity of exactly 0 everywhere. Such a tree scores a perfect 100% on its own training data. It has also almost certainly memorized noise, one-time

accidents, and unusual individual examples, instead of learning a real, general pattern. This is **overfitting**, in one of its purest and easiest-to-see forms.

Chapter 5 already named the exact setting that controls this risk: how many questions a tree is allowed to ask, one after another, before it must stop and make a prediction.

WORKED EXAMPLE — COMPARE TRAINING AND VALIDATION ACCURACY AT THREE TREE DEPTHS

The maintenance team fits three trees of different maximum depth on the same machine records. At maximum depth 1, training accuracy is 70% and validation accuracy is 68%. At maximum depth 3, training accuracy is 90% and validation accuracy is 87%. At maximum depth 8, training accuracy is 100% and validation accuracy is 76%. Find the training-validation gap at each depth, and choose the best depth.

1. Find the gap at depth 1: $70 - 68 = 2$ percentage points.
2. Find the gap at depth 3: $90 - 87 = 3$ percentage points.
3. Find the gap at depth 8: $100 - 76 = 24$ percentage points.
4. Compare the three validation accuracies, since validation performance, not the size of the gap, is what actually matters: 68%, 87%, and 76%.
5. Identify the highest validation accuracy among the three: 87%, reached at depth 3.

Answer: Depth 3 is the best choice, with the highest validation accuracy, 87%. Depth 8 reaches a perfect 100% on training data but its validation accuracy actually falls to 76%, a classic overfitting pattern, exactly like the Wiggly model from Chapter 5. Depth 1's small gap, only 2 points, might look reassuring, but both of its scores are simply low, a sign of underfitting, not of success.

The maximum depth a tree is allowed to reach is a **hyperparameter**, chosen by a person before training begins, exactly like the regularization strength from Chapter 5 or the number of neighbors, k , from Chapter 8. A second, closely related hyperparameter controls a different limit: the minimum number of training examples a leaf is allowed to contain. Recall the tiny tree from Lesson 9.1, where one leaf, reached only when vibration was High, was built around a single machine. A leaf built around just one example is a warning sign, because a single example could easily be an accident rather than a real pattern.

WORKED EXAMPLE — SEE WHAT A MINIMUM LEAF SIZE COSTS, AS WELL AS WHAT IT SAVES

A node contains 5 machines: 4 labeled No and 1 labeled Yes (this is the same "no" child that Lesson 9.2's Split A produced). Splitting further on vibration would perfectly separate the single Yes machine into its own leaf. Suppose a minimum leaf size of 2 forbids any split that would create a leaf with fewer than 2 examples. Find this node's prediction if that split is blocked.

1. Because a further split would create a leaf with only 1 example, and the minimum leaf size is 2, that split is not allowed. This node must become a leaf itself, with no further questions.

2. Count the labels inside this node: 4 machines labeled No, 1 machine labeled Yes.
3. A leaf predicts whichever label has the most votes. Since No has 4 votes and Yes has only 1, the leaf predicts No.

Answer: This leaf predicts "Needs Inspection: No" for every machine that reaches it, including the one machine that was actually labeled Yes. Minimum leaf size traded away one correct prediction on this particular training example, in exchange for a rule that is far less likely to be reacting to pure chance. This is the bias-variance trade-off from Chapter 5 again, now controlled by a tree's leaf-size hyperparameter instead of a regularization strength.

Limits like maximum depth and minimum leaf size are chosen before training begins, so they stop overly specific branches from ever being built in the first place. A second family of methods works the opposite way, letting a tree grow large first and only simplifying it afterward.

WORD TO KNOW

Pruning — Cutting back branches from an already-grown decision tree, removing splits that do not truly improve performance on a validation set. Pruning trades away a small amount of training accuracy in exchange for a tree that generalizes better to new examples, much like a gardener trimming a real tree's overgrown branches to keep it healthy.

Pruning works by checking each branch, usually starting from the bottom of the tree, and asking a simple question: if this branch were removed and replaced with a single leaf, would validation performance get worse, stay about the same, or actually improve? A branch that only helps training accuracy, while doing nothing for validation accuracy, is exactly the kind of branch pruning is designed to remove.

COMMON MISTAKE

✗ Wrong: Growing a tree to unlimited depth, then judging it only by its training accuracy.

✓ Right: Check validation accuracy at several different depths, or a validation-based pruning method, and choose whichever setting performs best on data the tree never trained on, exactly as Chapter 5 recommends for every hyperparameter.

COMMON MISTAKE

✗ Wrong: Assuming a deeper, more detailed tree must always be the more accurate, more powerful choice.

✓ Right: Depth is a hyperparameter like any other. A deeper tree has higher model capacity, but Lesson 9.3's own worked example shows the deepest tree of the three actually generalized the worst.

The following short program grows a small, depth-limited tree using the same machine records, with scikit-learn.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

X_train = np.array([[90], [95], [85], [75], [70], [65], [60], [80]])
y_train = ["Yes", "Yes", "Yes", "Yes", "No", "No", "No", "No"]

tree = DecisionTreeClassifier(criterion="gini", max_depth=1, random_state=0)
tree.fit(X_train, y_train)

print(tree.predict([[91], [68]]))
```

`X\_train` stores each machine's temperature, and `y\_train` stores its true label, in the same order used throughout this lesson. Setting `criterion="gini"` tells scikit-learn to choose splits using weighted Gini impurity, exactly as Lesson 9.2 computed by hand, and `max\_depth=1` limits the tree to a single root-node question, matching Split A's threshold search. Calling `predict` on a 91°C machine and a 68°C machine returns "Yes" and "No," matching the tree's single learned question, "temperature $\geq 82^\circ\text{C}$?", found automatically instead of by hand.

9.4 Regression Trees Predict Local Averages

Every tree in this chapter so far has predicted a category: Yes or No. A tree can just as easily predict a number instead, becoming a **regression** tree, the tree-based cousin of the linear regression from Chapter 6.

A regression tree is built almost exactly like a classification tree. It still asks a sequence of yes-or-no questions, and it still tries to make each child group as uniform as possible. The only real difference is what "uniform" means. Instead of searching for splits that make each child's labels more one-sided, a regression tree searches for splits that make each child's numbers sit closer together, rather than spread widely apart. A split that groups all of the expensive repairs into one child, and all of the cheap repairs into the other, is a good split, for exactly the same reason that grouping all the Yes labels together was a good split back in Lesson 9.2. The bigger change appears at the leaves. Instead of taking a majority vote, a regression tree's leaf simply predicts the plain average of the target values among the training examples that landed there.

WORKED EXAMPLE — PREDICT A REPAIR COST USING A REGRESSION TREE'S LEAF AVERAGE

A regression tree splits six machines by age using the question "age ≥ 6 years?" Machines aged 8, 9, and 10 years had repair costs of \$200, \$220, and \$210.

Machines aged 1, 2, and 3 years had repair costs of \$50, \$60, and \$55. Find each leaf's predicted repair cost, then use the tree to predict the repair cost for a new, 9-year-old machine.

1. Find the "age ≥ 6 " leaf's average repair cost. Add the three costs: $200 + 220 + 210 = 630$. Divide by the number of machines, 3: $630 \div 3 = 210$.
2. Find the "age < 6 " leaf's average repair cost. Add the three costs: $50 + 60 + 55 = 165$. Divide by the number of machines, 3: $165 \div 3 = 55$.
3. Check the new machine's age against the tree's question: is $9 \geq 6$? Yes, so the new machine follows the "yes" branch to the "age ≥ 6 " leaf.
4. Read off that leaf's prediction, found in step 1: \$210.

Answer: The regression tree predicts a repair cost of \$210 for the new, 9-year-old machine. Notice the prediction is not a fresh calculation for this one machine; it is simply the average repair cost already sitting inside the leaf the machine landed in, borrowed from three similar-aged machines the tree already knew about.

COMMON MISTAKE

✗ Wrong: Trusting a regression tree's prediction for a new example whose features fall far outside the range of anything in the training data, such as a 50-year-old machine when the oldest training machine was only 10 years old.

✓ Right: A regression tree can only predict averages it has already grouped inside a leaf. It cannot extrapolate a rising or falling trend beyond the data it has seen the way a straight regression line can. Always check whether a new example's features fall within the training data's range before trusting a tree's prediction.

Regression trees inherit every overfitting risk from Lesson 9.3, and the exact same fixes. An unlimited-depth regression tree can grow a separate leaf for every single training example, predicting each one's exact cost perfectly, and generalizing terribly. Maximum depth, minimum leaf size, and pruning all work on a regression tree exactly the way they work on a classification tree, chosen the same honest way, using validation performance rather than training performance alone.

9.5 Try It in ML Studio and Case Study

A decision tree's boundary looks different from every other model you have met so far. Because every question is a simple threshold on one feature at a time, the regions a tree draws always have straight, boxy, rectangular edges, never a diagonal line or a smooth curve. Seeing this shape live makes it easy to recognize a decision tree's work anywhere.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Blobs" dataset, and choose Decision Tree as the model.

3. Set the maximum depth to 1, press Fit, and look at the single straight, rectangular boundary that appears.
4. Increase the maximum depth one step at a time, trying 2, 4, and 10, pressing Fit each time. Write down how the boundary's rectangles change, and what happens to the training score and the validation score shown on screen.
5. Open the Labs tab, and turn on the pruning animation. Watch which branches get cut away, and write one sentence about whether pruning ever removes a branch that was helping the validation score.

You need: Computer, tablet, or phone with the ML Studio app

CASE STUDY — PRIORITIZING EQUIPMENT INSPECTIONS AT A MOTOR FACTORY

Priya manages equipment maintenance at a factory that manufactures small electric motors. The factory floor holds over three hundred machines, and her small team can only inspect a fraction of them closely each week. Right now, the team mostly inspects machines in a fixed rotation, regardless of which ones are actually showing warning signs. Priya wants a better way to decide which machines deserve attention first.

Her team already has sensors recording each machine's temperature and vibration level, along with hours of records showing which machines needed real repair work soon after being flagged, and which ones turned out fine. A decision tree appeals to Priya for a very practical reason beyond accuracy: whichever questions the tree learns to ask can be read aloud to her technicians directly, in plain language, unlike some more complicated models covered later in this book. "If temperature is above 82 degrees, inspect this week" is a rule a busy technician can actually remember and trust.

Priya's team follows this chapter's method closely. They compute Gini impurity to compare candidate splits, exactly like Lesson 9.2, and quickly find that temperature and vibration are both useful, while a third feature, the machine's paint color, barely lowers impurity at all and gets dropped. They try several maximum depths and compare validation accuracy at each one, finding that a tree limited to depth 4 performs best, just as Lesson 9.3's worked example found a moderate depth beating both a shallow tree and a fully grown one. A fully grown tree, with no depth limit at all, reached 100% training accuracy but performed clearly worse on validation machines, carving out lonely, one-machine leaves for unusual cases that never repeated.

The finished tree does not shut down any machine automatically. It produces a ranked list, refreshed every morning, showing which machines the model believes deserve inspection soonest, alongside the plain-language question path that led to each ranking. Technicians still walk the floor and make the final call themselves, and the team retrains the tree every few months as new inspection records come in, watching for early signs that the factory's machines are aging or changing in ways the tree has not learned yet.

Within the first two months, the team notices something the model quietly exposes: a handful of machines the fixed rotation used to check every week almost never needed real attention, while a few others, checked only

occasionally under the old system, kept reappearing near the top of the tree's list. Priya reports that the real win is not a single dramatic save, but a steady, unglamorous improvement: her small team now spends its limited inspection hours where the evidence actually points, instead of spreading that time out evenly across every machine on the floor.

IMPORTANT

A ranked list is a helpful starting point for a technician's morning, never a final verdict on its own. A tree trained on past inspection records can only repeat the patterns already present in those records, including any past oversights. Keep a trained person reviewing the ranked list and making the final inspection decision, especially before any action as serious as shutting a machine down.

COMMON MISTAKES TO AVOID

- Assuming every example must pass through the same number of questions on its way to a leaf node.
- Comparing two children's impurity scores directly, without weighting each score by how many examples actually landed in that child.
- Growing a tree to unlimited depth, then judging it only by its training accuracy.
- Assuming a deeper, more detailed tree must always be the more accurate, more powerful choice.
- Trusting a regression tree's prediction for a new example whose features fall far outside the range of anything in the training data.
- Pruning, or choosing any other tree hyperparameter, using training-set performance instead of a separate validation set.
- Forgetting that a single-example leaf is a warning sign of overfitting, not proof the tree found a genuine pattern.

CHAPTER SUMMARY

- A decision tree predicts a label by asking a sequence of yes-or-no questions, starting at the root node and ending at a leaf node.
- Different examples can take paths of different lengths through the same tree, stopping as soon as they reach a leaf.
- Impurity measures how mixed the labels are inside one node; Gini impurity is the most common way to compute it, reaching 0 for a pure node and 0.5 for an evenly split two-class node.
- A good split is chosen by comparing candidate questions using weighted Gini impurity, weighting each child by its share of the parent's examples, and keeping the split with the lowest score.
- Left unchecked, a decision tree can grow until every leaf is pure, memorizing the training data and overfitting badly.

- Maximum depth and minimum leaf size are hyperparameters chosen before training that limit how detailed a tree is allowed to become.
- Pruning cuts back an already-grown tree's branches afterward, using validation performance to decide what to remove.
- A regression tree predicts a number by averaging the target values of the training examples that land inside a leaf, and it cannot extrapolate beyond the range of data it has seen.
- Entropy and information gain, covered briefly in this chapter, offer an alternative to Gini impurity that usually chooses very similar splits in practice.
- Chapter 10 shows how combining many different trees together, instead of relying on just one, produces some of the strongest models in this entire book.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] In your own words, explain the difference between a root node and a leaf node.

Problem 2 [Basic] Explain why two different examples can take paths of different lengths through the same decision tree.

Problem 3 [Basic] A node contains 10 examples, all sharing the same label. State its Gini impurity, and explain your answer in one sentence.

Problem 4 [Basic] Explain, in your own words, what a leaf node predicts in a classification tree, and what a leaf node predicts in a regression tree.

Problem 5 [Standard] A node contains 20 examples: 12 labeled "Fail" and 8 labeled "Pass." Find this node's Gini impurity, showing every step.

Problem 6 [Standard] A split creates two children. Child 1 has 4 examples, all labeled "Yes," and Child 2 has 16 examples, 12 labeled "Yes" and 4 labeled "No." Find the weighted Gini impurity of this split, showing every step.

Problem 7 [Standard] Using the small tree from Lesson 9.1 (root: "temperature $\geq 82^{\circ}\text{C}$?"; if no, then "vibration = High?"), trace a machine with a temperature of 84°C and a vibration level of Low from the root node to its final prediction, showing each question and answer.

Problem 8 [Standard] A regression tree leaf contains three training examples with target values 40, 55, and 70. Find the leaf's prediction for any new example that lands there, showing every step.

Problem 9 [Standard] A tree at maximum depth 2 has training accuracy 82% and validation accuracy 80%. The same tree at maximum depth 12 has training accuracy 99% and validation accuracy 71%. Identify which depth is overfitting, and explain the evidence in one or two sentences.

Problem 10 [Standard] Explain, in your own words, why a regression tree cannot predict a sensible repair cost for a machine that is far older than any machine in its training data.

Problem 11 [Challenge] A team compares two candidate splits on the same 12 examples. Split A produces children with 6 and 6 examples, with weighted Gini impurity 0.28. Split B produces children with 2 and 10 examples, with weighted Gini impurity 0.19. (a) Which split should the tree choose, and why? (b) Explain, in your own words, why a split with a very small child, like Split B's 2-example child, deserves a second look before trusting it completely.

Problem 12 [Challenge] A hospital builds a decision tree to help prioritize which patients' lab results a doctor reviews first. (a) Explain one advantage a decision tree offers here compared with a model whose reasoning is harder to read aloud. (b) Describe one risk of trusting the tree's ranking without human review, and suggest one safeguard.

Problem 13 [Challenge] Open ML Studio, fit a Decision Tree on the "Moons" dataset, and try at least four different maximum depths, recording the training and validation score shown on screen each time. Identify which depths underfit, which overfit, and which depth you would finally choose. Explain your reasoning in a few sentences, and describe how the shape of the boundary changed as depth increased.

Problem 14 [Challenge] A regression tree predicting delivery time has one leaf containing 5 past deliveries with times of 20, 25, 30, 90, and 35 minutes. (a) Find the leaf's predicted delivery time, showing every step. (b) One of these five times looks very different from the rest. Explain, in a few sentences, what effect a single unusual value like this can have on a leaf's average, and suggest one question you would ask about that delivery before trusting the leaf's prediction.

SUPERVISED LEARNING

CHAPTER 10

Random Forests and Boosting

Combining many imperfect models

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what an ensemble is, and why combining several models can outperform any one of them alone
- Build a bootstrap sample by hand, and explain how bagging uses many bootstrap samples to reduce variance
- Explain how a random forest adds random feature subsets on top of bagging, and predict a new example by majority vote
- Use a tree's out-of-bag rows to compute a free, internal check on a forest's performance
- Walk a small boosted model through two rounds by hand, updating predictions with a learning rate
- Compare bagging and boosting side by side, and explain which kind of error each one mainly fixes
- Compute feature importance from a small ensemble, and explain why it needs careful reading, not blind trust

10.1 Many Trees Can Beat One

Chapter 9 taught you how to grow a single **decision tree**: ask one good question at a time, split the data, and keep going until you reach a confident prediction. A single tree is easy to read, but it has a weakness. Grow the same tree on a slightly different batch of data, say last month's deliveries instead of this month's, and it can pick different splits and end up making noticeably different predictions. A tree accurate enough to be useful often has high **variance**, the word Chapter 5 used for a model that swings depending on exactly which rows happened to be in the training data.

This chapter solves that weakness with one idea: instead of trusting one tree, grow many trees, and combine what they say. Combining many careless models rarely beats one carefully built model; the real power of this idea comes from combining models that are each reasonably good alone but make different kinds of mistakes, so their errors cancel out instead of piling up.

WORD TO KNOW

Ensemble — A model built by combining the predictions of many smaller models instead of relying on just one. An ensemble usually predicts by letting its members vote, for a category, or by averaging their guesses, for a number.

An ensemble sounds complicated, but the core idea is already part of daily life. Picture asking one friend to guess how many guests will come to a wedding; that guess might be too low or too high. Now picture asking twenty friends and averaging all twenty guesses. Some guess low and some guess high, but the mistakes tend to pull in different directions, so the average usually lands closer to the truth than almost any single guess did. Decision trees can be combined the exact same way.

This chapter builds two families of tree ensembles: a random forest, which grows many trees independently and lets them vote (Lessons 10.2 and 10.3), and boosting, which grows trees one at a time, each one fixing the mistakes the trees before it are still making (Lesson 10.4). Lesson 10.5 compares the two, and Lesson 10.6 shows how an ensemble can reveal which features mattered most.

10.2 How a Random Forest Creates Variety

Picture a small package delivery company called Sunrise Parcels, shipping parcels between Erbil, Baghdad, and Basra. The dispatch team wants to flag, before a driver leaves the depot, which deliveries are at risk of running late. Over the last two months, the team recorded four things for eight past deliveries: distance in kilometers, number of stops before this delivery, whether it rained that day, and whether the delivery arrived late. Delivery 1 traveled 5 km, made 1 stop, saw no rain, and was on time. Delivery 2 traveled 42 km, made 6 stops, saw rain, and was late. Delivery 3 traveled 8 km, made 2 stops, saw no rain, and was on time. Delivery 4 traveled 36 km, made 5 stops, saw no rain, and was late. Delivery 5 traveled 11 km, made 2 stops, saw no rain, and was on time. Delivery 6 traveled 30 km, made 6 stops, saw rain, and was late. Delivery 7 traveled 7 km, made 1 stop, saw rain, and was on time. Delivery 8 traveled 44 km, made 4 stops, saw rain, and was late.

One single decision tree grown on all eight deliveries gives one single set of splits. A random forest instead grows many trees, each with its own, slightly different training data. The tool that creates this difference has its own name.

WORD TO KNOW

Bootstrap sample — A new dataset built by drawing rows from an original dataset at random, with replacement, until the new dataset is the same size as the original. Because rows can be drawn more than once, a bootstrap sample usually repeats some rows and leaves other rows out completely.

The phrase "with replacement" is the important part. Imagine writing each delivery's number on its own slip of paper, dropping all eight slips into a bag, drawing one slip out, and putting it back before drawing again. Because the slip always goes back in, the same delivery can be drawn more than once, and some deliveries might never be drawn at all.

WORKED EXAMPLE — BUILD ONE BOOTSTRAP SAMPLE FROM EIGHT DELIVERIES

Sunrise Parcels has 8 past deliveries, numbered 1 through 8. To grow the first tree in a random forest, the computer draws 8 deliveries at random, with replacement, from these same 8 deliveries. Suppose the 8 draws come out in this order: Delivery 3, Delivery 1, Delivery 5, Delivery 8, Delivery 3, Delivery 6, Delivery 1, Delivery 4. List the bootstrap sample this tree will train on, and list which original deliveries were left out completely.

1. Count how many times each delivery number was drawn: Delivery 1 was drawn 2 times (draws 2 and 7). Delivery 3 was drawn 2 times (draws 1 and 5). Delivery 4, Delivery 5, Delivery 6, and Delivery 8 were each drawn 1 time.
2. Add up the draws to check the total equals 8: $2 + 2 + 1 + 1 + 1 + 1 = 8$, which matches the 8 draws made.
3. List the distinct deliveries that appear at least once in this bootstrap sample: Delivery 1, Delivery 3, Delivery 4, Delivery 5, Delivery 6, and Delivery 8. That is 6 distinct deliveries out of the original 8.
4. Find the deliveries that were never drawn at all, by comparing the full list 1 through 8 against the 6 distinct deliveries just found: Delivery 2 and Delivery 7 never appear.

Answer: This tree's bootstrap sample repeats Delivery 1 and Delivery 3 twice each, includes Deliveries 4, 5, 6, and 8 once each, and completely leaves out Delivery 2 and Delivery 7. Those two left-out deliveries will become useful again in Lesson 10.3.

COMMON MISTAKE

✗ Wrong: Believing a bootstrap sample is just the original training set reused, so every tree trains on identical data.

✓ Right: A bootstrap sample is drawn with replacement, so it repeats some rows and leaves out others — exactly what makes every tree in the forest a little different.

Growing many trees, each on its own bootstrap sample, and then combining their predictions, has its own name too.

WORD TO KNOW

Bagging — Short for "bootstrap aggregating": training many copies of a model, each on its own bootstrap sample, then combining every copy's prediction by voting or averaging. Bagging mainly reduces variance, because the different mistakes made by differently-trained models tend to cancel out when combined.

A random forest adds one more twist on top of bagging. At every split, a tree is not allowed to look at every feature; it must choose only from a small, randomly chosen subset, freshly picked at that one split. This sounds like it should make each tree worse, since it can no longer always use its favorite feature. In practice, restricting features usually helps the forest as a whole, because it stops every tree from leaning on the same one or two strongest features in exactly the same way. Two trees forced to notice different features make

different kinds of mistakes, exactly the extra variety bagging alone cannot fully provide.

WORD TO KNOW

Random forest — An ensemble of many decision trees, each trained on its own bootstrap sample, where each split is also chosen from only a random subset of the features. A random forest predicts by letting all of its trees vote, for classification, or by averaging their predictions, for regression.

EQUATION — A RANDOM FOREST'S PREDICTION

$$\text{prediction} = \frac{1}{B} \times \sum \text{tree}_b(x)$$

In words: A random forest's prediction is the average of every tree's prediction, divided by the number of trees, B. For a category, the forest instead takes the majority vote across all B trees.

B the number of trees in the forest (count)

tree_b(x) the prediction made by tree number b for input x (depends on the task)

Σ tree_b(x) the sum of every tree's prediction, added together (depends on the task)

Note: Classification forests almost always report the vote share too, such as "62 of 100 trees voted Late," which works like a rough confidence level for the prediction.

WORKED EXAMPLE — GROW THREE TREES WITH DIFFERENT DATA AND DIFFERENT FEATURES

A tiny random forest grows 3 trees for Sunrise Parcels, using the split-finding method from Chapter 9. Tree 1 trains on the bootstrap sample from the example above (Deliveries 1, 1, 3, 3, 4, 5, 6, 8) and may only split using Distance or Rain, not Stops. Tree 2 trains on a bootstrap sample of Deliveries 2, 2, 2, 4, 5, 6, 7, 8 and may only split using Stops or Rain, not Distance. Tree 3 trains on a bootstrap sample of Deliveries 1, 2, 3, 4, 4, 5, 6, 7 and may only split using Distance or Stops, not Rain. Find the one split each tree would choose, and confirm it sorts every delivery in that tree's own sample correctly.

1. Tree 1 can only use Distance or Rain. Sorted by distance, its on-time deliveries (1, 3, 5, at 5, 8, and 11 km) all sit below its late deliveries (4, 6, 8, at 36, 30, and 44 km), so "Distance ≤ 20 km" sorts all 8 draws in this sample correctly.
2. Tree 2 can only use Stops or Rain, so it cannot see Distance at all. Sorted by stops, its on-time deliveries (5, 7, at 2 and 1 stops) all sit below its late deliveries (2, 4, 6, 8, at 6, 5, 6, and 4 stops), so "Stops ≤ 2" sorts all 8 draws in this sample correctly.
3. Tree 3 can only use Distance or Stops. Sorted by distance, the same clean gap from Tree 1 appears again (on-time at 5, 8, 11, 7 km; late at 42, 36, 36, 30 km), so "Distance ≤ 20 km" sorts all 8 draws in this sample correctly.

Answer: All three trees perfectly sort their own bootstrap sample, but two trees leaned on Distance while one leaned entirely on Stops,

because Tree 2 was never even allowed to look at Distance. Three trees, three slightly different samples, and two different favorite features: this is exactly what "creating variety" means in a random forest.

WORKED EXAMPLE — LET THE MINI-FOREST VOTE ON A NEW DELIVERY

A new delivery, Delivery Q, is about to leave the depot: distance 25 km, 2 stops, no rain. Using the three trees and splits from the example above (Tree 1: Distance ≤ 20 km $\rightarrow$ On Time, otherwise Late. Tree 2: Stops $\leq 2 \rightarrow$ On Time, otherwise Late. Tree 3: Distance ≤ 20 km $\rightarrow$ On Time, otherwise Late), find the mini-forest's prediction for Delivery Q.

1. Apply Tree 1's rule to Delivery Q. Its distance is 25 km, which is greater than 20 km, so Tree 1 votes Late.
2. Apply Tree 2's rule to Delivery Q. Its stop count is 2, which is not greater than 2, so Tree 2 votes On Time.
3. Apply Tree 3's rule to Delivery Q. Its distance is 25 km, which is greater than 20 km, so Tree 3 votes Late.
4. Count the votes: Late received 2 votes (Tree 1 and Tree 3), and On Time received 1 vote (Tree 2).

Answer: The mini-forest predicts Late for Delivery Q, by a vote of 2 to 1. The three trees did not all agree: one tree, working from a different feature, saw the delivery differently, which is exactly why a forest reports a vote count instead of pretending every tree sees a situation the same way.

10.3 Out-of-Bag Examples Give a Free Check

Look back at Lesson 10.2's bootstrap draws. Tree 1 never saw Delivery 2 or Delivery 7. Tree 2 never saw Delivery 1 or Delivery 3. Tree 3 never saw Delivery 8. Those left-out deliveries are not wasted. They are about to give the forest a free, honest check on its own performance.

WORD TO KNOW

Out-of-bag (OOB) — For one tree in a random forest, the training rows that its own bootstrap sample happened to leave out. Because that tree never trained on its own out-of-bag rows, predicting on them shows how the tree handles data it has genuinely never seen.

To turn this into one score for the whole forest, take each row in the original data one at a time. Find only the trees that left that row out of their own bootstrap sample. Ask just those trees to predict that row, and combine their answers by voting, exactly like a mini-forest of its own. Compare that vote to the true label. Do this for every row that was left out by at least one tree, and count how many of those predictions were correct.

WORKED EXAMPLE — COMPUTE THE MINI-FOREST'S OUT-OF-BAG SCORE

Using the three trees and splits from Lesson 10.2, find each delivery's out-of-bag prediction, using only the trees that never trained on that delivery, then compute what fraction of those predictions were correct.

1. Delivery 1 was left out only by Tree 2 ($\text{Stops} \leq 2 \rightarrow \text{On Time}$). It has 1 stop, so Tree 2 predicts On Time, matching the true label. Correct.
2. Delivery 2 was left out only by Tree 1 ($\text{Distance} \leq 20 \text{ km} \rightarrow \text{On Time}$). Its distance is 42 km, so Tree 1 predicts Late, matching the true label. Correct.
3. Delivery 3 was left out only by Tree 2. It has 2 stops, so Tree 2 predicts On Time, matching the true label. Correct.
4. Delivery 7 was left out only by Tree 1. Its distance is 7 km, so Tree 1 predicts On Time, matching the true label. Correct.
5. Delivery 8 was left out only by Tree 3. Its distance is 44 km, so Tree 3 predicts Late, matching the true label. Correct.
6. Deliveries 4, 5, and 6 were never left out by any of the three trees, so none can be given an out-of-bag prediction with this small forest.
7. Count the checkable deliveries and the correct predictions among them: 5 deliveries could be checked, and all 5 were correct.

Answer: The mini-forest's out-of-bag score is 5 out of 5, or 100%, on the deliveries it was able to check. Only 5 of the 8 original deliveries ever got left out by at least one tree; with only 3 trees, Deliveries 4, 5, and 6 simply never had the chance to be tested this way.

EQUATION — OUT-OF-BAG SCORE

$$\text{OOB score} = \frac{c}{m}$$

In words: The out-of-bag score equals the number of correct out-of-bag predictions, divided by the number of rows that had at least one out-of-bag prediction available.

c number of correct out-of-bag predictions (count)

m number of rows that received at least one out-of-bag prediction (count)

Note: m is often smaller than the full training set, especially in a small forest. A real forest with hundreds of trees leaves almost every row out-of-bag for many trees, so m usually ends up close to the full training set size.

COMMON MISTAKE

✗ Wrong: Treating an out-of-bag score from a forest with only a handful of trees as just as trustworthy as a score from a large, separate test set.

✓ Right: A small forest may leave some rows in-bag for every tree, so its score rests on fewer rows and can bounce around. Use enough trees, typically at least 100, before leaning heavily on an out-of-bag score.

Out-of-bag scoring is especially useful when data is scarce. Chapter 2 taught you to set aside a separate **validation set** before tuning any model, the safe habit for most models in this book. A random forest gets something similar almost for free: every tree's own left-out rows act like a small, built-in validation

set, and combining every tree's out-of-bag checks gives one overall estimate for the whole forest. This is convenient, but it does not fully replace a final, untouched **test set** before a high-stakes decision. Treat the out-of-bag score as a fast, first check while improving the forest, and still confirm the finished model on real held-out data before reporting a final number.

10.4 Boosting Repairs Mistakes, Step by Step

Bagging grows many trees at the same time, independently. Boosting takes the opposite approach: it grows trees one at a time, in order, and every new tree's job is to fix whatever mistakes the current group is still making.

WORD TO KNOW

Boosting — Building an ensemble by training models one at a time, in sequence, where each new model focuses on correcting the mistakes the current ensemble is still making. The models' predictions are then added together, usually after scaling each new model's contribution down with a **learning rate**.

Boosting is easiest to see with a number to predict, so this lesson gives Sunrise Parcels' story one new twist: instead of asking only whether a delivery will be late, suppose dispatch wants to know how many minutes late it is likely to run. Four recent deliveries recorded both distance and actual minutes late: Delivery E1 traveled 10 km and was 2 minutes late. Delivery E2 traveled 50 km and was 18 minutes late. Delivery E3 traveled 15 km and was 4 minutes late. Delivery E4 traveled 45 km and was 16 minutes late.

Boosting starts with the simplest possible guess, then improves it in rounds. Round by round, it looks at how wrong the current guess still is for each delivery, called the **residual**, and grows one small tree that tries to predict that residual. Adding a scaled-down version of that tree's prediction to the running total nudges every guess closer to the truth. Repeat enough rounds, and the running total keeps improving.

EQUATION — BOOSTING'S RUNNING UPDATE

$$\begin{array}{l} \text{new prediction} \\ \text{new tree's output} \end{array} = \text{old prediction} + (\text{learning rate} \times$$

In words: After growing a new tree on the current residuals, boosting updates its running prediction by adding the learning rate times that tree's output to the prediction it already had.

old prediction the ensemble's running prediction before this round (same unit as the target)

new tree's output this round's small tree's predicted residual for this example (same unit as the target)

learning rate a small number, usually well below 1, that scales down how much this round is allowed to change the running prediction (no unit, just a number)

new prediction the updated running prediction, used as "old prediction" in the next round (same unit as the target)

Note: A smaller learning rate makes each round more cautious, usually needing more rounds to finish, but it is less likely to overshoot the correct answer.

WORKED EXAMPLE — BOOSTING ROUNDS ZERO AND ONE: START FLAT, THEN FIX IT

Boosting starts with the simplest possible prediction for every delivery: the average minutes late across the 4 training deliveries (2, 18, 4, and 16 minutes, for Deliveries E1 through E4, with distances 10, 50, 15, and 45 km). Find this starting prediction and the resulting residuals. Then grow a one-split tree that predicts the residual from distance, splitting at "distance ≤ 30 km," and update the running prediction using a learning rate of 0.5.

1. Add the four true values, then divide by 4 to find the starting prediction: $2 + 18 + 4 + 16 = 40$, and $40 \div 4 = 10$. Every delivery starts with a predicted 10 minutes late.
2. Find each delivery's residual, true value minus predicted value: Delivery E1: $2 - 10 = -8$. Delivery E2: $18 - 10 = 8$. Delivery E3: $4 - 10 = -6$. Delivery E4: $16 - 10 = 6$.
3. Group the deliveries by the split. Distance ≤ 30 km: Delivery E1 (10 km, residual -8) and Delivery E3 (15 km, residual -6). Distance > 30 km: Delivery E2 (50 km, residual 8) and Delivery E4 (45 km, residual 6).
4. Find the average residual in the " ≤ 30 km" group, which becomes that group's predicted residual: $(-8 + -6) \div 2 = -14 \div 2 = -7$.
5. Find the average residual in the " > 30 km" group: $(8 + 6) \div 2 = 14 \div 2 = 7$.
6. Update Delivery E1 and Delivery E3's running prediction: old prediction 10, plus learning rate 0.5 times the tree's output -7 : $10 + (0.5 \times -7) = 10 - 3.5 = 6.5$.
7. Update Delivery E2 and Delivery E4's running prediction: old prediction 10, plus learning rate 0.5 times the tree's output 7: $10 + (0.5 \times 7) = 10 + 3.5 = 13.5$.

Answer: After round 1, the running predictions are 6.5 minutes for Deliveries E1 and E3, and 13.5 minutes for Deliveries E2 and E4. Compare these to the true values, 2, 4, 18, and 16: every prediction has moved closer to the truth than the flat starting guess of 10 was.

WORKED EXAMPLE — BOOSTING ROUND TWO: FIX WHAT IS STILL WRONG

Find the new residuals after round 1 (predictions 6.5, 13.5, 6.5, and 13.5 for Deliveries E1 through E4, whose true values are 2, 18, 4, and 16), grow a second one-split tree at the same split, "distance ≤ 30 km," and update the running prediction again with the same learning rate of 0.5.

1. Find Delivery E1's new residual: true value 2 minus round-1 prediction 6.5: $2 - 6.5 = -4.5$.
2. Find Delivery E3's new residual: $4 - 6.5 = -2.5$.
3. Find Delivery E2's new residual: $18 - 13.5 = 4.5$.

4. Find Delivery E4's new residual: $16 - 13.5 = 2.5$.
5. Find the average residual in the " ≤ 30 km" group, Deliveries E1 and E3: $(-4.5 + -2.5) \div 2 = -7 \div 2 = -3.5$.
6. Find the average residual in the " > 30 km" group, Deliveries E2 and E4: $(4.5 + 2.5) \div 2 = 7 \div 2 = 3.5$.
7. Update Deliveries E1 and E3's running prediction: $6.5 + (0.5 \times -3.5) = 6.5 - 1.75 = 4.75$.
8. Update Deliveries E2 and E4's running prediction: $13.5 + (0.5 \times 3.5) = 13.5 + 1.75 = 15.25$.

Answer: After round 2, the running predictions are 4.75 minutes for Deliveries E1 and E3, and 15.25 minutes for Deliveries E2 and E4. The true values are 2, 4, 18, and 16. The mean absolute error has dropped from 7 minutes after round zero, to 3.5 minutes after round 1, to 1.75 minutes after round 2 — cut exactly in half at every round.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor

X = np.array([[10], [50], [15], [45]])
y = np.array([2, 18, 4, 16])

booster = GradientBoostingRegressor(n_estimators=2, learning_rate=0.5,
max_depth=1, random_state=0)
booster.fit(X, y)

for round_number, prediction in enumerate(booster.staged_predict(X),
start=1):
    print("after round", round_number, ":", prediction)
```

X stores the four deliveries' distances, one per row, and y stores their true minutes late. `n_estimators=2` grows exactly 2 trees in sequence, matching the two rounds worked out above, and `learning_rate=0.5` and `max_depth=1` match the same learning rate and one-split stumps used by hand.

`booster.staged_predict(X)` returns the running prediction after each tree is added, so this loop prints the ensemble's prediction after round 1 and after round 2. Running this code prints `[6.5, 13.5, 6.5, 13.5]` after round 1 and `[4.75, 15.25, 4.75, 15.25]` after round 2, matching the hand calculations exactly.

COMMON MISTAKE

✗ Wrong: Assuming that, just like a random forest, adding more boosting rounds can only help, or at least never hurts.

✓ Right: Unlike a forest's tree count, boosting's number of rounds is a capacity dial. Too many rounds, and boosting starts chasing tiny accidents in the

residuals, which is **overfitting**. Choose the rounds and the learning rate using a validation set, exactly the way Chapter 5 chose tree depth.

10.5 Bagging vs. Boosting, Side by Side

Bagging and boosting both combine many trees, and both usually beat a single tree, but they get there in almost opposite ways. Bagging grows every tree independently, on its own bootstrap sample, and only combines them at the end, by voting or averaging. Because the trees never depend on each other, bagging mainly attacks **variance**: it smooths out the wobble caused by any one tree's particular sample. Boosting grows every tree in strict order, and each new tree exists only because the trees before it were still wrong about something. Because each tree deliberately targets the previous group's mistakes, boosting mainly attacks **bias**: errors a single simple model keeps making in the same direction.

This difference also shapes how each family behaves as you add more trees. A random forest rarely gets worse from adding more trees; each new tree is just one more independent vote, and more independent opinions tend to help, or at least do no harm. A boosted model can get worse from too many rounds, because each new tree fits more tightly to whatever error is left, including small accidents that do not represent the real pattern. This is why a forest's tree count is usually set generously and left alone, while boosting's rounds and learning rate are tuned carefully against a validation set.

WORKED EXAMPLE — IDENTIFY BAGGING OR BOOSTING FOR FOUR DESCRIPTIONS

Identify each of these four descriptions as bagging or boosting. Description 1: two hundred trees grow at the same time, each on its own bootstrap sample, and the forest predicts by majority vote. Description 2: trees grow one at a time, each one trained to predict the previous trees' error. Description 3: adding more trees rarely hurts accuracy. Description 4: a learning rate controls how much each new tree changes the running prediction.

1. Description 1 grows trees independently, on bootstrap samples, combined by voting: bagging, specifically a random forest.
2. Description 2 grows trees in sequence, each one correcting the previous trees' errors: boosting.
3. Description 3 keeps working well with many more trees added: bagging, since independent trees mostly reduce variance without adding new mistakes.
4. Description 4 uses a learning rate to scale each new tree's contribution: boosting, since bagging has no such running update.

Answer: Descriptions 1 and 3 describe bagging; Descriptions 2 and 4 describe boosting. Bagging and boosting both combine many trees, but they build that combination in almost opposite ways, and for almost opposite reasons.

COMMON MISTAKE

✗ Wrong: Treating bagging and boosting as two names for the same idea, since both combine many decision trees into one ensemble.

✓ Right: Bagging trains trees independently and mainly reduces variance. Boosting trains trees in sequence and mainly reduces bias. Knowing which problem you have points you toward the right family before you even start tuning.

10.6 Feature Importance Needs Care

Besides predicting, a forest or a boosted ensemble of trees can also tell you which features it leaned on most. Every time any tree in the ensemble makes a split, that split reduces impurity, exactly the way Chapter 9 measured with **Gini impurity**. Adding up how much impurity each feature's splits removed, across every split in every tree, and turning that total into a share of the whole, produces a score for each feature.

WORD TO KNOW

Feature importance — A score given to each feature that estimates how much that feature helped a tree-based model make good splits, usually based on how much impurity that feature's splits removed, added up across the whole ensemble. A higher score means the model leaned on that feature more.

WORKED EXAMPLE — FIND EACH FEATURE'S IMPORTANCE IN THE MINI-FOREST

Across the three trees from Lesson 10.2, Tree 1's split on Distance removed 0.5 worth of impurity. Tree 2's split on Stops removed 0.5 worth of impurity. Tree 3's split on Distance removed 0.5 worth of impurity. Rain was never used in any split. Find the feature importance of Distance, Stops, and Rain, as a percentage.

1. Add up the impurity removed by Distance, across both trees that used it, Tree 1 and Tree 3: $0.5 + 0.5 = 1.0$.
2. Add up the impurity removed by Stops, from the one tree that used it, Tree 2: just 0.5.
3. Add up the impurity removed by Rain: 0, since no tree ever split on it.
4. Add up the impurity removed by all three features together, to get the whole forest's total: $1.0 + 0.5 + 0 = 1.5$.
5. Divide Distance's total by the whole forest's total, then convert to a percentage: $1.0 \div 1.5 \approx 0.667$, and $0.667 \times 100 \approx 66.7\%$.
6. Divide Stops's total by the whole forest's total, then convert to a percentage: $0.5 \div 1.5 \approx 0.333$, and $0.333 \times 100 \approx 33.3\%$.
7. Divide Rain's total by the whole forest's total: $0 \div 1.5 = 0$, or 0%.

Answer: Distance's importance is about 66.7%, Stops's importance is about 33.3%, and Rain's importance is 0% in this tiny forest. Distance mattered roughly twice as much as Stops to this mini-forest's splits, and Rain never helped a split at all, at least not in these three particular trees.

EQUATION — NORMALIZED FEATURE IMPORTANCE

$$\text{importance of feature } j = \frac{\text{impurity removed by feature } j}{\text{impurity removed by all features}}$$

In words: One feature's importance equals the total impurity its own splits removed, across every tree in the ensemble, divided by the impurity removed by every feature's splits combined.

feature j one particular feature, such as distance or stops (—)

impurity removed by feature j the sum of impurity decrease from every split, in every tree, that used feature j (*depends on the impurity measure*)

impurity removed by all features the sum of impurity decrease from every split in the whole ensemble, from every feature (*depends on the impurity measure*)

Note: Every feature's importance falls between 0 and 1 (or 0% and 100%), and all of the features' importances always add up to exactly 1 (or 100%).

Feature importance is genuinely useful, but it needs careful reading, for two reasons. First, features that carry similar information can split credit between them. In the Sunrise Parcels data, distance and stops tend to rise together, so if a tree splits on stops first, distance earns slightly less credit than it deserves alone. A low importance score does not always mean a feature is useless; sometimes a close relative of it simply got there first. Second, and more important, a high importance score is not proof that a feature causes the outcome. It only shows the feature was useful for making accurate splits inside this particular model, on this particular data.

IMPORTANT

A feature importance score describes this model's own splits, not the real world. It is not proof that a feature causes an outcome, and it should never be the only reason to take an action against a specific person, such as a driver or a customer. Use feature importance to guide a conversation and a human review, the same honest way Chapter 1 asked you to treat every prediction.

Chapter 18 introduces a more careful way to measure feature importance, called permutation importance, which checks a feature's true usefulness by scrambling it and watching the model's score drop. For now, treat impurity-based feature importance as a useful first look, not a final answer.

GOING FURTHER — FASTER BOOSTING LIBRARIES

Scikit-learn's boosting tools are excellent for learning, but most real production systems use faster, more specialized libraries built specifically for boosting: XGBoost, LightGBM, and CatBoost are three of the most popular. Each follows the same core idea taught in this chapter, sequentially correcting errors with a learning rate, but adds engineering tricks that make training large boosted models much faster.

A real forest, with 100 or more full-sized trees instead of our 3 tiny stumps, usually reshuffles a ranking like this a little: Distance and Stops can even swap

first place, and Rain, unused in our small example, typically earns a small nonzero score once more data and more trees are involved. This is a good reminder that a tiny hand example teaches the mechanism, but a real forest is where you should trust the actual numbers.

10.7 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Moons" dataset and fit a single Decision Tree first. Note its validation accuracy and the shape of its boundary.
3. Switch to Random Forest, keep everything else the same, and press Fit. Compare the new validation accuracy and the shape of the decision boundary to the single tree's.
4. Slide the number of trees from 10 up to 200, pressing Fit each time, and write down how the boundary and the validation accuracy change.
5. Now switch to Gradient Boosting, try learning rates of 1.0, 0.1, and 0.01 with the same number of rounds each time, and write one sentence describing what a learning rate that is too high does to the fitted boundary.

You need: Computer, tablet, or phone with the ML Studio app

A single tree's boundary looks blocky, chasing corners of the training data. A forest's boundary looks calmer and smoother, even though it is still built entirely out of blocky trees underneath — the fastest way to make Lesson 10.1's "many friends guessing a number" story feel real.

CASE STUDY — FLAGGING LATE-DELIVERY RISK AT SUNRISE PARCELS

Rawa manages dispatch for Sunrise Parcels, a small delivery company shipping parcels between Erbil, Baghdad, and Basra. Right now, a delivery is only recognized as "at risk" once it is already late and a customer calls to complain. Rawa wants dispatch to know earlier, while there is still time to add a driver to a busy route or warn a customer honestly, instead of finding out after the fact. Rawa's analyst, Karwan, starts with a single decision tree, trained on a season of past deliveries. It is easy to explain in a dispatch meeting, but its predictions shift noticeably depending on which month it trains on, exactly the high-variance behavior this chapter opened with. A random forest steadies the predictions immediately. Because Sunrise Parcels has only a modest amount of data, Karwan leans on the forest's out-of-bag score as a fast, honest check while trying different settings, saving a small, untouched test set for one final report. Karwan also trains a boosted model on the same data. After careful tuning of the learning rate and rounds, watched against the validation score to avoid overfitting, it edges out the random forest slightly on the held-out test set. Karwan chooses the boosted model for daily predictions, but keeps the forest running alongside it as a stable second opinion, since the two rarely make the same mistakes on the same deliveries.

Feature importance shows that distance and number of stops matter far more than any other detail, with weather a distant third. One column, which driver was assigned to each delivery, is deliberately left out of the model entirely: using a driver's identity risks quietly penalizing someone simply assigned the longest or hardest routes. The finished system flags a delivery for dispatch attention, never a driver for blame, and every flagged delivery still gets a quick look from a real dispatcher before any action is taken.

IMPORTANT

An ensemble of many trees is usually more accurate than any single tree, but it is also harder to fully explain, since no one tree tells the whole story anymore. When a prediction affects a real person's opportunity, such as a loan or a driver's assignment, that lost explanation is a real cost, not just a technical detail. Chapter 18 returns to this trade-off in depth; for now, choose the simplest model that meets your accuracy needs, not automatically the most complex one available.

COMMON MISTAKES TO AVOID

- Believing a bootstrap sample is the same as reusing the whole original training set, instead of a resampled set drawn with replacement.
- Trusting an out-of-bag score from a forest with only a few trees as if it were as reliable as a large, separate test set.
- Assuming more boosting rounds can only help, the way more trees rarely hurts a random forest.
- Mixing up bagging, which trains trees independently and mainly reduces variance, with boosting, which trains trees in sequence and mainly reduces bias.
- Treating a high feature importance score as proof that a feature causes the outcome.
- Forgetting that correlated features can split credit in a feature importance ranking, making a genuinely useful feature look weaker than it is.
- Choosing the most complex ensemble available instead of the simplest model that meets the project's accuracy needs.

CHAPTER SUMMARY

- An ensemble combines many models' predictions, usually by voting or averaging, so that different models' mistakes can cancel out.
- A bootstrap sample is drawn from the training data with replacement, which usually repeats some rows and leaves others out.
- Bagging trains many copies of a model, each on its own bootstrap sample, then combines them; it mainly reduces variance.
- A random forest is bagging applied to decision trees, with each split also limited to a random subset of features, adding a second source of variety.

- Out-of-bag rows are the training rows a tree's own bootstrap sample left out; predicting on them gives a free, built-in check, though it needs enough trees to be reliable.
- Boosting trains trees one at a time, each one fitted to the previous ensemble's remaining error, and adds their scaled contributions together using a learning rate; it mainly reduces bias.
- Boosting can overfit if given too many rounds or too large a learning rate, unlike a random forest, which rarely gets worse from adding more trees.
- Feature importance measures how much each feature's splits reduced impurity across an ensemble, but it is not proof of causation, and correlated features can split credit.
- Ensembles usually beat a single tree on accuracy, but they are harder to explain, a real cost when a prediction affects a real person.
- Chapter 11 leaves trees behind and builds prediction from a very different building block: a network of small, connected units called neurons.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define "ensemble" in your own words, using an example that is not from this chapter.

Problem 2 [Basic] Explain the difference between a bootstrap sample and the original training set.

Problem 3 [Basic] A random forest has 150 trees. For a classification problem, how does the forest turn 150 separate predictions into one final prediction?

Problem 4 [Basic] In your own words, explain why boosting needs a learning rate, but building a bootstrap sample by itself does not.

Problem 5 [Standard] A forest draws 6 rows with replacement from a training set of 6 rows, numbered 1 through 6. The draws are: 2, 4, 4, 1, 6, 2. List the distinct rows used in this bootstrap sample, and list which rows were left out.

Problem 6 [Standard] A tree in a random forest left out Row 5 and Row 9 from its bootstrap sample. Explain, in one or two sentences, how these two rows could be used to check that tree's performance.

Problem 7 [Standard] A boosting run starts with a flat guess of 50. Round 1's tree predicts -12 for a certain example. Using a learning rate of 0.4, find the updated prediction for that example, showing every step.

Problem 8 [Standard] A second round of boosting on the same example finds a new residual of -4 , and the tree assigned to that group predicts -4 as its output. Using the updated prediction from the previous problem and the same learning rate of 0.4, find the prediction after this second round.

Problem 9 [Standard] Explain, in your own words, why bagging mainly reduces variance while boosting mainly reduces bias.

Problem 10 [Standard] A forest's splits removed impurity of 0.8 (Feature A), 0.6 (Feature A again, in a different tree), and 0.6 (Feature B). Find the feature importance of Feature A and Feature B, as percentages.

Problem 11 [Challenge] A team compares a random forest and a boosted model on the same loan-approval dataset. The random forest's out-of-bag score is 88%, and its test score is 87%. The boosted model's training score is 99%, and its test score is 84%. (a) Which model shows a stronger warning sign of overfitting, and what evidence shows it? (b) Which model would you deploy, and what would you check before fully trusting your choice?

Problem 12 [Challenge] A hospital wants to use a random forest to help flag patients for a follow-up call, using features that include age, past visits, and test results. (a) Name one feature you would be cautious about including, and explain why. (b) Describe, in a few sentences, how the hospital could use feature importance responsibly, without treating it as medical proof.

Problem 13 [Challenge] Open ML Studio, fit a Random Forest and a Gradient Boosting model on the same dataset of your choice, and try at least three different settings for each (number of trees for the forest; learning rate for boosting). Report which model reached the higher validation score, and describe one difference you noticed in how each model responded to its setting change.

SUPERVISED LEARNING

CHAPTER 11

Neural Networks: A Short Bridge

One idea, stacked in layers

IN THIS CHAPTER YOU WILL LEARN TO

- Explain how a single neuron's calculation echoes logistic regression, and how neurons connect into layers
- Show why one straight-line neuron cannot solve a problem like the two-switch light, using real numbers
- Explain what an activation function does, and why stacking layers with no activation adds no power
- Walk the forward pass of a two-hidden-neuron network by hand, for all four corners of a square
- Explain, in plain words, what backpropagation calculates and what gradient descent then does with it
- Decide when a neural network is a good choice, and explain honestly what this book does and does not cover

11.1 From One Logistic Unit to a Network of Units

Chapter 7 taught you **logistic regression**: multiply each feature by its own **coefficient**, add the results together with an **intercept**, and pass the total through the **sigmoid function** to get a probability. That one calculation, it turns out, is also the basic building block of a neural network.

WORD TO KNOW

Neuron — A single computing unit inside a neural network that takes in several numbers, multiplies each by its own coefficient, adds them together with one extra bias number, and passes the result through an activation function to produce one output number. A neuron is really just a small logistic-regression-style calculation, reused as a building block.

EQUATION — WHAT ONE NEURON COMPUTES

$$z = (w_1 \times x_1) + (w_2 \times x_2) + b, \quad \text{then} \quad \text{output} = \text{activation}(z)$$

In words: A neuron multiplies each input by its own coefficient, adds the results together with one extra bias number to get a combined score z , then passes z through an activation function to produce its output.

x_1, x_2 the neuron's input values (*depends on the feature*)

w_1, w_2 the coefficient the neuron has learned for each input (*no unit, just a number*)

- b** the neuron's bias, an extra learned number added once, not tied to any input (no unit, just a number)
- z** the neuron's combined score, before the activation function bends it (no unit, just a number)

Note: This "bias" plays exactly the same role as the intercept from Chapter 6: a fixed adjustment added once. Lesson 11.3 explains the activation function.

A single neuron is just one calculation. A network connects many neurons together in layers, so that one layer's outputs become the next layer's inputs. When every neuron in one layer receives every value from the layer before it, each with its own separate coefficient, the layers are called fully connected.

WORKED EXAMPLE — COUNT THE NEURONS AND COEFFICIENTS IN A SMALL NETWORK

A network takes in 3 input features. It has one hidden layer with 4 neurons, fully connected to the 3 inputs. Its output layer has 1 neuron, fully connected to the 4 hidden neurons. How many neurons does this network have in total, and how many coefficients does the hidden layer alone contain?

1. Count the neurons in the hidden layer: 4 neurons.
2. Count the neurons in the output layer: 1 neuron.
3. Add the layers' neurons together: $4 + 1 = 5$ neurons in total. Input features are not neurons; they are only the data neurons receive.
4. Find the hidden layer's coefficients: each of its 4 neurons connects to all 3 inputs, so each has 3 coefficients: $4 \times 3 = 12$ coefficients.

Answer: This network has 5 neurons in total, and its hidden layer alone contains 12 coefficients, before even counting each neuron's own bias or the output layer's coefficients. Even a small network quickly accumulates a lot of numbers for training to find.

A single neuron, alone, is really just logistic regression again, with a new name. Its real power only appears once many neurons are connected together — which raises an obvious question: what can a network do that one neuron cannot? Lesson 11.2 answers it with a problem one straight line can never solve.

11.2 Hidden Layers and Learned Features

Many homes wire a staircase light to two switches: one at the bottom of the stairs, Switch A, and one at the top, Switch B. Either switch can turn the light on or off, which lets a person flip the light on at the bottom, walk upstairs, and turn it off again at the top, without ever walking back down in the dark. Each switch is either Down (0) or Up (1). The light turns ON when the two switches are in different positions, and stays OFF when they match. Four cases cover every possibility: both down, light OFF. Both up, light OFF. A up and B down, light ON. A down and B up, light ON. Notice that the two OFF cases and the two ON cases each pair up diagonally: (0,0) and (1,1) share OFF, while (1,0) and (0,1) share ON.

Can one straight-line neuron learn this rule?

WORKED EXAMPLE — TEST WHETHER ONE STRAIGHT-LINE NEURON CAN LEARN THE SWITCH PROBLEM

Try a single neuron with weights $w_1 = 1$, $w_2 = 1$, and bias $b = -0.5$, predicting ON whenever its combined score z is greater than 0, and OFF otherwise. Test it on all four switch positions.

1. Case (A=0, B=0): $z = (1 \times 0) + (1 \times 0) + (-0.5) = -0.5$. Since -0.5 is not greater than 0, the neuron predicts OFF. True answer: OFF. Correct.
2. Case (A=1, B=0): $z = (1 \times 1) + (1 \times 0) + (-0.5) = 0.5$. Since 0.5 is greater than 0, the neuron predicts ON. True answer: ON. Correct.
3. Case (A=0, B=1): $z = (1 \times 0) + (1 \times 1) + (-0.5) = 0.5$. Since 0.5 is greater than 0, the neuron predicts ON. True answer: ON. Correct.
4. Case (A=1, B=1): $z = (1 \times 1) + (1 \times 1) + (-0.5) = 1.5$. Since 1.5 is greater than 0, the neuron predicts ON. True answer: OFF. Wrong.

Answer: This neuron gets three of the four cases right, but always fails on (1, 1). No matter how w_1 , w_2 , and b are adjusted, a single straight-line neuron cannot get all four cases right at once, because the two ON cases and the two OFF cases sit on opposite corners of a square.

Picture the four cases as corners of a small square: (0,0) at bottom-left, (1,0) at bottom-right, (0,1) at top-left, (1,1) at top-right. The two ON corners sit diagonally opposite each other, and so do the two OFF corners. Try sketching any single straight line across this square. You can easily separate one corner from the other three, or two neighbors from the other two neighbors. You can never draw one straight line that puts both diagonal ON corners on one side and both diagonal OFF corners on the other, at the same time. A single neuron draws exactly one straight line, no matter how its numbers are tuned, so it is stuck with this same problem.

The fix is to stop asking one neuron to solve the whole problem alone. Use two neurons side by side, each drawing its own straight line, and then add one more neuron that combines their two answers.

WORD TO KNOW

Hidden layer — A layer of neurons sitting between a network's input features and its final output, so called because its output values are not directly observed in the data, only computed along the way. A network can have one hidden layer or many; this chapter mostly uses one.

A hidden layer's neurons compute new, learned features, not the original inputs themselves. Each hidden neuron still only draws one straight line, but that line is drawn in a way its training chooses to be useful. The output neuron then treats the hidden neurons' own outputs as brand-new inputs, and only needs a simple rule over these new, already-bent features. Lesson 11.4 shows this working, with real numbers, for the switch problem above. First, one missing ingredient: the bend itself.

11.3 Activation Functions: Why Bending Matters

If every neuron only computed a plain weighted sum, with no bending step, stacking layers would add no real power at all. Picture two rulers taped end to end: however you angle them, you still only get a straight line, never a curve or a corner. Combining one plain weighted sum with another plain weighted sum is still, after multiplying everything out, just one more plain weighted sum of the original inputs. Bending only enters the picture once a neuron's score is passed through a function that is not itself a straight line.

WORD TO KNOW

Activation function — A function applied to a neuron's combined score, z , that bends the result instead of leaving it a plain straight-line calculation. Without an activation function, stacking layers adds no power at all.

The **sigmoid function** from Chapter 7 is one activation function: it squeezes any score into a smooth curve between 0 and 1. This chapter's worked examples use a second, even simpler activation function.

WORD TO KNOW

ReLU — Short for "rectified linear unit," an activation function that keeps a positive number exactly as it is, and replaces any zero or negative number with 0. ReLU is simple to compute and is one of the most common activation functions used in hidden layers today.

EQUATION — RELU: THE RECTIFIED LINEAR UNIT

$$\text{ReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

In words: ReLU keeps a positive number exactly as it is, and replaces any zero or negative number with 0.

z the neuron's combined score, before the activation function bends it (no unit, just a number)

ReLU(z) the neuron's output after applying ReLU (no unit, just a number)

Note: ReLU looks almost like a straight line, but the sharp corner at $z = 0$ is enough bending to let a network learn shapes a single straight line never could.

COMMON MISTAKE

✗ Wrong: Mixing up an activation function with a loss function, since both involve a formula applied after some numbers are computed.

✓ Right: An activation function bends one neuron's own combined score, on the way to becoming that neuron's output. A **log loss** or similar loss function is different: it compares the whole network's final prediction to the true label, only at the very end.

GOING FURTHER — OTHER ACTIVATION SHAPES

Sigmoid and ReLU are only two of many bending shapes used in real networks. Tanh looks similar to sigmoid but stretches its output between -1 and 1 instead

of 0 and 1. Leaky ReLU behaves like ReLU for positive numbers, but lets a small negative slope through instead of flattening completely to 0, which can help a network keep learning even when a neuron's input is negative.

11.4 How Two Hidden Units Solve a Problem a Straight Line Cannot

Everything is now ready to solve the switch problem properly: two hidden neurons, each with a ReLU activation, feeding one output neuron.

EQUATION — A FULL PREDICTION THROUGH ONE HIDDEN LAYER

$$h_1 = g(w_{11}x_1 + w_{12}x_2 + b_1)$$

$$h_2 = g(w_{21}x_1 + w_{22}x_2 + b_2)$$

$$\hat{y} = v_1h_1 + v_2h_2 + c$$

In words: To get a final prediction, first compute each hidden neuron's own weighted sum and bend it with an activation function g , then feed both hidden outputs into the output neuron as if they were brand-new input features.

x_1, x_2 the network's original input values (*depends on the feature*)

$w_{11}, w_{12}, w_{21}, w_{22}$ the coefficients each hidden neuron applies to the two inputs (*no unit, just a number*)

b_1, b_2 each hidden neuron's own bias (*no unit, just a number*)

h_1, h_2 each hidden neuron's output, after bending — a new, learned feature (*no unit, just a number*)

v_1, v_2, c the output neuron's coefficients and bias, applied to h_1 and h_2 (*no unit, just a number*)

$\hat{y}$ the network's final prediction (*depends on the task*)

Note: This particular output neuron uses no activation function of its own, sometimes called a "linear" output — a common choice when the final answer is not meant to be squeezed into a 0-to-1 probability.

WORKED EXAMPLE — WALK THE FORWARD PASS THROUGH THE SWITCH NETWORK FOR ALL FOUR CORNERS

A small network solves the two-switch light problem. Hidden neuron 1 uses weights $w_{11} = 1$, $w_{12} = 1$, bias $b_1 = 0$, with ReLU. Hidden neuron 2 uses weights $w_{21} = 1$, $w_{22} = 1$, bias $b_2 = -1$, also with ReLU. The output neuron uses weights $v_1 = 1$, $v_2 = -2$, bias $c = 0$, with no activation: $\hat{y} = (1 \times h_1) + (-2 \times h_2)$. Compute the forward pass for all four switch positions, and compare each result to the true light status.

1. Corner (A=0, B=0): hidden neuron 1's score: $z_1 = (1 \times 0) + (1 \times 0) + 0 = 0$.
2. Apply ReLU to $z_1 = 0$: since 0 is not greater than 0, ReLU keeps it at 0, so $h_1 = 0$.
3. Hidden neuron 2's score: $z_2 = (1 \times 0) + (1 \times 0) + (-1) = -1$.
4. Apply ReLU to $z_2 = -1$: since -1 is negative, ReLU replaces it with 0, so $h_2 = 0$.

5. Output: $\hat{y} = (1 \times 0) + (-2 \times 0) = 0$. Matches the true light status, OFF (0).
6. Corner (A=1, B=0): hidden neuron 1's score: $z_1 = (1 \times 1) + (1 \times 0) + 0 = 1$.
7. Apply ReLU to $z_1 = 1$: positive, so ReLU keeps it unchanged, $h_1 = 1$.
8. Hidden neuron 2's score: $z_2 = (1 \times 1) + (1 \times 0) + (-1) = 0$.
9. Apply ReLU to $z_2 = 0$: not greater than 0, so $h_2 = 0$.
10. Output: $\hat{y} = (1 \times 1) + (-2 \times 0) = 1$. Matches the true light status, ON (1).
11. Corner (A=0, B=1): hidden neuron 1's score: $z_1 = (1 \times 0) + (1 \times 1) + 0 = 1$.
12. Apply ReLU to $z_1 = 1$: positive, so $h_1 = 1$.
13. Hidden neuron 2's score: $z_2 = (1 \times 0) + (1 \times 1) + (-1) = 0$.
14. Apply ReLU to $z_2 = 0$: not greater than 0, so $h_2 = 0$.
15. Output: $\hat{y} = (1 \times 1) + (-2 \times 0) = 1$. Matches the true light status, ON (1).
16. Corner (A=1, B=1): hidden neuron 1's score: $z_1 = (1 \times 1) + (1 \times 1) + 0 = 2$.
17. Apply ReLU to $z_1 = 2$: positive, so $h_1 = 2$.
18. Hidden neuron 2's score: $z_2 = (1 \times 1) + (1 \times 1) + (-1) = 1$.
19. Apply ReLU to $z_2 = 1$: positive, so $h_2 = 1$.
20. Output: $\hat{y} = (1 \times 2) + (-2 \times 1) = 2 - 2 = 0$. Matches the true light status, OFF (0).

Answer: The network gets all four corners exactly right: OFF, ON, ON, and OFF. The single neuron in Lesson 11.2 could only manage three out of four corners; two hidden neurons, working together, get all four.

It helps to notice what each hidden neuron learned to detect. Hidden neuron 1 computes $A + B$, so it turns positive whenever at least one switch is up, climbing to 2 only when both are up; think of it as a rough detector for "at least one switch is up." Hidden neuron 2 computes $A + B - 1$, so it stays at 0 unless both switches are up, in which case it turns positive; think of it as a detector for "both switches are up." The output computes $h_1 - 2 \times h_2$: when exactly one switch is up, $h_1 = 1$ and $h_2 = 0$, giving 1. When both are up, h_1 climbs to 2, but h_2 also switches on and subtracts twice its value, pulling the output back down to 0. Neither hidden neuron alone solves the problem; each one only draws one straight line across the square of four corners. Combined, their two lines carve the square into exactly the pieces the switch problem needs, which is the whole point of a hidden layer.

COMMON MISTAKE

✗ Wrong: Computing a neuron's weighted sum, z , and treating that number as the neuron's finished output, skipping the activation function entirely.

✓ Right: A neuron is not finished until its activation function is applied. At corner (0, 0) above, hidden neuron 2's combined score was -1 , but its output, after ReLU, was 0 — skip that step, and the wrong number flows into the next layer.

Python · NumPy

 runnable in the app

```

from sklearn.neural_network import MLPClassifier
import numpy as np

X = np.array([[0, 0], [1, 0], [0, 1], [1, 1]])
y = np.array([0, 1, 1, 0])

model = MLPClassifier(hidden_layer_sizes=(4,), activation="relu",
max_iter=5000, random_state=0)
model.fit(X, y)

print(model.predict(X))

```

X holds the four switch corners in the same order as the worked example, and y holds the true light status, 0 for OFF and 1 for ON. `hidden_layer_sizes=(4,)` builds one hidden layer, and `activation="relu"` matches the bending rule computed by hand — scikit-learn is given 4 hidden neurons rather than the 2 used above, so that its training search reliably finds a good answer on its own, unlike our carefully hand-chosen weights. Running this code prints [0, 1, 1, 0], exactly matching the four true labels and the hand-computed forward pass.

11.5 Backpropagation in Plain Words

A network's forward pass is easy to follow: numbers flow forward, layer by layer, from inputs to prediction, exactly like Lesson 11.4 walked through by hand. Training means adjusting every coefficient in every layer so predictions get closer to the true labels. But Lesson 11.4's tiny network already has ten separate numbers to adjust: six coefficients and two biases in the hidden layer, plus two coefficients and one bias in the output layer. A real network can have millions. Backpropagation is the method that figures out how to adjust every one of these numbers, efficiently, after just one forward pass.

WORD TO KNOW

Backpropagation — A method that works backward through a network, from the final loss toward the first layer, calculating exactly how much each coefficient contributed to the current mistake. Backpropagation does not change any coefficient itself; it only calculates the gradient that **gradient descent** then uses to make the actual update.

Picture a small factory line with three workers in a row, each adjusting a part before passing it on, until a finished product rolls off the end and gets inspected. If the product is flawed, the inspector cannot fix all three workers' habits directly. Instead, the flaw is reported backward: the last worker asks "was my adjustment part of the problem, and by how much," passes a share of the blame back to the second worker, who does the same, then back to the first. Backpropagation works through a network the same way, starting at the output,

where the loss (measured with log loss for a classifier, exactly as in Chapter 7) is known exactly, and working backward through each layer, calculating how much each coefficient is to blame. This backward calculation relies on a piece of mathematics called the chain rule, which this book does not derive, but the plain-English idea is exactly the assembly-line story: share the blame backward, one layer at a time.

WORKED EXAMPLE — UPDATE ONE COEFFICIENT USING ITS BACKPROPAGATED GRADIENT

After a forward pass and a backward pass, backpropagation reports that one particular coefficient, currently 0.40, has a gradient of 0.8. A gradient shows how much the loss would rise if the coefficient increased slightly, so gradient descent should move it the opposite way. Using a learning rate of 0.1, find the coefficient's updated value.

1. Recall gradient descent's update rule from Chapter 6: new value = old value – (learning rate $\times$ gradient).
2. Multiply the learning rate by the gradient: $0.1 \times 0.8 = 0.08$.
3. Subtract this amount from the coefficient's current value: $0.40 - 0.08 = 0.32$.

Answer: The coefficient's updated value is 0.32. Backpropagation supplied the gradient, 0.8, and gradient descent used it to make this small, careful step — the exact same update rule from Chapter 6, now applied to one coefficient buried inside a network instead of one coefficient in a single straight line.

WORKED EXAMPLE — SEE WHAT A LEARNING RATE TOO LARGE CAN DO

Using the same coefficient and gradient as the previous example (coefficient 0.40, gradient 0.8), find the updated value if the learning rate is 2.0 instead of 0.1.

1. Multiply the learning rate by the gradient: $2.0 \times 0.8 = 1.6$.
2. Subtract this amount from the coefficient's current value: $0.40 - 1.6 = -1.2$.

Answer: The coefficient jumps from 0.40 to -1.2 , a huge, unstable swing for one single update. A learning rate this large does not carefully approach a good answer; it overshoots past it, and can easily overshoot again on the very next round. This is exactly why Chapters 6 and 10 both treat the learning rate as a setting worth tuning carefully, not a number to guess at.

COMMON MISTAKE

✗ Wrong: Believing backpropagation is itself the method that changes a network's coefficients.

✓ Right: Backpropagation only calculates each coefficient's gradient. Gradient descent is the separate step that actually updates the coefficients, exactly as it did for a single straight line back in Chapter 6.

This whole cycle — forward pass, measure the loss, backward pass, update every coefficient a little — repeats many times, often thousands, each time nudging the network a little closer to predictions that match the true labels.

11.6 When to Choose a Network, and Where This Book Ends

When should you actually reach for a neural network instead of the models from Chapters 6 through 10? Three conditions make a network attractive. First, a large amount of data: a network with many coefficients needs many examples to learn from, or it memorizes instead of generalizing, exactly the **overfitting** risk Chapter 5 warned about. Second, a signal that is naturally layered: an image is built from edges, which build into shapes, which build into objects; hidden layers are especially good at this "features built from features" structure. Third, a willingness to spend more time tuning, and more computing power training, for the last bit of accuracy.

Most everyday business tables do not meet these conditions well. A table of loan applications or delivery records usually has a moderate number of rows, often a few thousand rather than millions, features already meaningful on their own (a distance in kilometers is already a useful number; it does not need several layers to "discover" what it means), and a real need for an answer that can be explained simply to a manager or a customer. This is exactly the territory where Chapter 10's random forests and boosted trees tend to shine, often matching or beating a small network, while training faster and staying easier to explain. This is not a small point: across many carefully-run comparisons on business tables, boosted trees are hard to beat, and a careful practitioner tries them before reaching for a network, not after.

Everything in this chapter used at most one hidden layer, with two or four neurons. Real image-recognition and language systems stack dozens, or even hundreds, of hidden layers, with millions or billions of coefficients between them.

WORD TO KNOW

Deep learning — Using a neural network with many hidden layers stacked one after another, so the network builds increasingly complex learned features on its own, layer by layer. Deep learning powers most modern image and language systems, but training it well needs far more data, computing power, and specialized technique than a short introduction can responsibly teach.

GOING FURTHER — NAMES YOU WILL MEET IN DEEP LEARNING

A convolutional neural network, or CNN, is built to find patterns in images, such as edges and shapes, wherever they sit in the picture. A transformer is built to handle sequences, such as sentences, by learning which earlier words matter most to each new word, and powers most modern language systems. Both are neural networks at heart, built from the same neurons, layers, and backpropagation taught in this chapter, just arranged in far more specialized, elaborate patterns, and trained on far more data.

This book, and its companion ML Studio app, stop here on purpose. ML Studio's Play tab, used in the activity below, lets you build a small network with a few hidden layers and watch its decision boundary bend in real time — exactly the right amount of hands-on practice for the ideas in this chapter. Going further, into deep learning for images and language, deserves its own dedicated, careful treatment: a separate companion volume and ML Studio's advanced labs are where that journey continues, with the same step-by-step, nothing-skipped approach this book has used from Chapter 1 onward. A single chapter that tried to teach deep learning properly would either take a dangerous shortcut through the hard parts, or stop being a book for beginners at all. This book chooses honesty over a false shortcut.

IMPORTANT

A neuron is a small calculation, not a brain cell, and a trained network is a set of numbers, not a mind. It is tempting to say a network "understands" a picture or "knows" a language, but underneath every impressive result is still the same forward pass you just computed by hand: weighted sums, activation functions, and nothing more mysterious than that.

IMPORTANT

This book stops at a shallow network on purpose, not because deep learning is unimportant, but because teaching it responsibly needs far more room than one chapter honestly allows. A companion volume and ML Studio's deeper labs pick up exactly where this chapter leaves off. Treat any single short lesson elsewhere, including this one, that promises to make you a deep learning expert in an hour with real suspicion.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Checkerboard" dataset, built from the same diagonal-corners pattern as this chapter's switch problem, and fit Logistic Regression first.
3. Look at the decision boundary. Confirm, on screen, the same failure this chapter worked out by hand: one straight line cannot separate the pattern.
4. Switch to Neural Network, set the hidden layer to 2 neurons, and press Fit. Then try 4, then 8. Write down how the boundary changes at each size.
5. Open the Labs tab and watch the backpropagation animation for a few rounds. Write one sentence describing what changes, round by round.

You need: Computer, tablet, or phone with the ML Studio app

CASE STUDY — A SMALL NETWORK, OR A BIGGER FOREST? AN ONLINE STORE DECIDES

Priya is a data scientist at a mid-size online electronics store serving customers across South Asia. The store wants to predict which customers are likely to buy again within 30 days, using each customer's order history, average order value, browsing time, and days since their last visit. Customers flagged as likely to

return get a simple thank-you email; customers flagged as unlikely get a modest, well-timed discount offer before they drift away for good.

Having just learned about neural networks, Priya builds a small network with one hidden layer, standardizes her features carefully, and spends a full afternoon choosing a sensible number of hidden neurons, a learning rate, and how long to train before stopping. It reaches a respectable validation accuracy, around 81%, only after several rounds of trial and error. Priya then spends twenty minutes fitting a gradient-boosted tree model, from Chapter 10, on the same data, barely touching its default settings. The boosted model reaches about 82% validation accuracy, essentially matching her network, trains in a fraction of the time, and hands her a ready-made feature importance ranking she can show the marketing team directly, without needing to explain what a hidden layer is first.

Priya's manager asks which model to deploy. She recommends the boosted model, and is honest about why: on this kind of tabular business data, with a moderate number of customers and features already meaningful on their own, a well-tuned ensemble of trees is simply the safer, stronger default. She does not delete her network's code, though. The store is planning a project next year to recommend visually similar products from photos, and Priya already knows that is exactly the kind of layered, image-shaped problem where a neural network, and eventually a deeper one, earns its extra complexity.

COMMON MISTAKES TO AVOID

- Believing a neuron is fundamentally different from the logistic regression unit already covered in Chapter 7, instead of the same weighted-sum-and-squash idea, reused as a building block.
- Forgetting to apply a neuron's activation function, and passing its raw weighted sum forward unchanged.
- Believing that stacking layers with no activation function, or a purely linear one, adds any power beyond a single straight line.
- Mixing up an activation function, which bends one neuron's own output, with a loss function, which scores the whole network's final prediction.
- Believing backpropagation itself changes a network's coefficients, instead of only calculating the gradients that gradient descent then uses.
- Assuming a neural network will automatically outperform simpler models, especially on small or ordinary tabular data.
- Treating a shallow, one-hidden-layer network taught in a single chapter as equivalent to full deep learning.

CHAPTER SUMMARY

- A neuron computes a weighted sum of its inputs plus a bias, then bends the result with an activation function, echoing the same calculation behind logistic regression.

- A hidden layer sits between a network's inputs and its final output, and its neurons compute new, learned features rather than directly observed ones.
- A single neuron can only draw one straight decision boundary; it cannot solve a problem like the two-switch light, where the two classes sit on opposite corners of a square.
- An activation function is what lets stacking layers add real power; without one, any number of layers still collapses into one straight line.
- ReLU is a simple, common activation function that keeps positive numbers unchanged and replaces negative numbers with 0.
- Two hidden neurons, each drawing their own line and bent by ReLU, can combine to solve the two-switch problem exactly, as the full forward-pass example showed for all four corners.
- Backpropagation works backward through a network, layer by layer, calculating how much each coefficient contributed to the current mistake; gradient descent then uses those gradients to update every coefficient.
- Neural networks tend to shine with large amounts of data and naturally layered signals, such as images and language; on ordinary tabular data, boosted trees are often just as strong, or stronger, and easier to explain.
- Deep learning stacks many hidden layers to build increasingly complex learned features, and needs far more data and computing power than one introductory chapter can responsibly teach.
- This book's coverage of building models ends here, on an honest note; a companion volume and ML Studio's advanced labs continue into deep learning, images, and language, while Part III turns to a different question: finding structure in data that has no labels at all.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Explain, in your own words, how a single neuron's calculation is similar to logistic regression from Chapter 7.

Problem 2 [Basic] Define "activation function" and explain why a network needs one.

Problem 3 [Basic] A network has one hidden layer with 3 neurons and one output neuron. How many neurons does the network have in total, not counting the input features?

Problem 4 [Basic] In your own words, explain what backpropagation calculates, and what it does NOT do by itself.

Problem 5 [Standard] A neuron has weights $w_1 = 2$, $w_2 = -1$, and bias $b = 0.5$. For inputs $x_1 = 3$ and $x_2 = 4$, find z , showing every step.

Problem 6 [Standard] Apply ReLU to each of these combined scores, showing your reasoning for each: $z = 5$, $z = 0$, $z = -3$, $z = 0.2$.

Problem 7 [Standard] A hidden neuron has weights $w_1 = 1$, $w_2 = 1$, and bias $b = -1.5$, with a ReLU activation. Find this neuron's output for inputs ($x_1 = 1$, $x_2 = 1$).

Problem 8 [Standard] Using the same neuron as the previous problem, find its output for inputs ($x_1 = 0$, $x_2 = 1$).

Problem 9 [Standard] A coefficient is currently 0.90. Backpropagation reports its gradient as -0.5 . Using a learning rate of 0.2, find the coefficient's updated value, showing every step.

Problem 10 [Standard] Give one example of a prediction problem where a large amount of naturally layered data, such as images or language, would make a neural network a strong choice, and explain your reasoning in a sentence or two.

Problem 11 [Challenge] A team has 300 rows of tabular customer data and is deciding between a small neural network and a gradient-boosted tree model. (a) Name two factors from this chapter that favor the boosted tree model here. (b) Describe one kind of dataset where the recommendation would likely flip toward a neural network.

Problem 12 [Challenge] Using the switch-network weights from Lesson 11.4 (hidden neuron 1: $w_{11}=1$, $w_{12}=1$, $b_1=0$; hidden neuron 2: $w_{21}=1$, $w_{22}=1$, $b_2=-1$; output: $v_1=1$, $v_2=-2$, $c=0$, ReLU on the hidden layer), verify by hand that the network still gets corner ($A=1$, $B=0$) correct. Show every step, including both hidden neurons and the final output.

Problem 13 [Challenge] Open ML Studio, fit a Neural Network on the "Checkerboard" dataset with 1 hidden neuron, then 2, then 8. Describe what happens to the decision boundary at each size, and explain, using this chapter's ideas, why 1 hidden neuron alone is not enough.

Part III — Unsupervised Learning

Chapter 12 — Clustering with k-Means

Chapter 13 — Principal Component Analysis

Chapter 14 — Anomaly Detection

UNSUPERVISED LEARNING

CHAPTER 12

Clustering with k-Means

Finding groups nobody labeled

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what clustering does differently from classification and regression, and recognize a project that calls for it
- Run the k-means algorithm by hand: assign points to the nearest centroid, recompute each centroid, and repeat until nothing changes
- Compute the inertia of a finished clustering, and explain what a lower inertia does and does not prove
- Use the elbow method to choose a sensible number of clusters, k
- Recognize cluster shapes and situations where k-means tends to perform poorly
- Describe, in plain terms, two other clustering methods you will meet elsewhere: hierarchical clustering and DBSCAN

12.1 Grouping Without Answers

Chapter 1 introduced three common learning tasks: classification, regression, and **clustering**. The first two, classification and regression, are **supervised learning**: every training example already carries a known, correct **label**, and a **model** learns to match new examples to the right answer. Clustering is different. It is **unsupervised learning**, which means the training examples carry no label at all. Nobody has already sorted them into groups. The whole job of a clustering method is to look at a pile of unlabeled examples and find groups inside it, using only the patterns already sitting in the data.

This chapter builds a complete, simple clustering method by hand, the same way Chapter 1 built a complete classifier by hand using nearest-centroid classification. The method is called k-means, and it is the most widely used clustering method in the world, partly because its idea is so easy to picture and partly because it works well on a very wide range of real problems.

Picture a small neighborhood grocery store that has been running a loyalty card program for a year. Every time a customer shops, the store's system quietly records two numbers: how many times that customer visited last month, and how much money that customer spent, on average, each visit. The store owner has never asked any customer to describe their own shopping habits. There is no column in the data called "customer type." And yet, the owner suspects that her customers are not all the same. Some customers probably shop often and spend a little each time. Others probably shop rarely but spend a lot when they

do. If she could find these natural groups, she could plan smarter promotions: a small discount to bring in the rare, big spenders more often, perhaps, and a loyalty bonus to reward the frequent small spenders.

This is exactly the kind of question clustering answers. There is no known right answer to check against, unlike a classification problem where every training example already has a correct label. There is only a table of numbers, and a question: which examples look similar enough to belong together?

With only a handful of customers and two features, a careful person could probably find sensible groups just by plotting the customers on a graph and looking at the picture. But real stores often track hundreds or thousands of customers, sometimes using five, ten, or twenty features at once: visits, spending, which product categories a customer buys from, how they pay, and more. A human eye cannot "just look" at twenty columns of numbers spread across thousands of rows and spot the groups. Clustering needs a repeatable, mechanical method that a computer can run automatically, no matter how large the table grows. That method is exactly what the rest of this chapter builds, one small, checkable step at a time.

WORKED EXAMPLE — DECIDE WHETHER FOUR STORE PROJECTS NEED CLUSTERING

The grocery store's data team is considering four small projects. Project 1: group customers into similar shopping patterns, with no existing group names to learn from. Project 2: predict whether a given customer will redeem a discount coupon, "yes" or "no," using past redemption records as labeled training examples. Project 3: predict how much a customer will spend next month, in dollars, using past monthly totals as labeled training examples. Project 4: group the store's suppliers by how similar their delivery schedules are, with no existing supplier categories to learn from. For each project, decide whether it needs clustering or a supervised method instead.

1. Project 1 has no labels to learn from, only a request to find similar customers, so Project 1 needs clustering.
2. Project 2 has a known, correct answer for every past customer, "yes" or "no," so Project 2 is supervised learning, specifically classification, not clustering.
3. Project 3 also has a known, correct answer for every past customer, a dollar amount, so Project 3 is supervised learning, specifically regression, not clustering.
4. Project 4 has no existing supplier categories to learn from, only a request to find similar suppliers, so Project 4 needs clustering, just like Project 1, even though its subject, suppliers, has nothing to do with customers.

Answer: Projects 1 and 4 need clustering. Projects 2 and 3 need supervised learning instead, because both already have a known, correct label attached to every training example. The presence or absence of a known label, not the subject of the project, is what decides which family of methods applies.

COMMON MISTAKE

✗ Wrong: Reaching for clustering any time a project involves "grouping" customers, products, or records, without first checking whether a known label already exists for each one.

✓ Right: Check for a label first. If every example already carries a known, correct answer, a supervised method, classification or regression, almost always gives a more precise, checkable result. Save clustering for the cases where no such label exists at all.

Clustering also leans on an idea Chapter 1 already introduced while building a classifier: the **centroid**, the average position of a group of points. Nearest-centroid classification in Chapter 1 computed one centroid per known class, because the classes were already labeled. Clustering flips this idea around. Instead of starting with known groups and finding their centroids, k-means starts with guessed centroids and uses them to discover the groups. Lesson 12.2 turns this one-sentence idea into a complete, repeatable algorithm.

12.2 The k-Means Algorithm, Step by Step

WORD TO KNOW

k-means — A clustering method that sorts examples into k groups by repeating two steps: assign every example to its nearest centroid, then move each centroid to the average position of the examples now assigned to it. The letter k is chosen ahead of time and simply means "however many groups we are looking for."

The name k-means describes the method almost completely on its own. The letter k is the number of groups you are asking for, chosen before the algorithm ever runs. "Means" refers to the averages, the centroids, that the algorithm keeps recomputing as it works. Put the two pieces together, and k-means simply means "find k averages that describe this data well."

In plain words, before any arithmetic, the whole algorithm is only four instructions, repeated in order:

First, choose k, the number of groups you want, and place k starting centroids somewhere among the data, often by picking k actual examples at random. Second, assign every single example to whichever centroid it currently sits closest to. Third, move each centroid to the average position of the examples now assigned to it. Fourth, check whether any example switched which centroid it is assigned to since the last round. If yes, go back to the second instruction and repeat. If no example switched, stop. The final centroids and their assigned groups are the finished clustering.

That fourth instruction is the heart of why k-means needs repeating at all. The very first round of centroids is often a rough guess, sometimes a poor one. Moving the centroids to match their assigned examples usually shifts them toward better positions, which can in turn change which centroid is nearest for a few borderline examples. The algorithm keeps repeating assign-then-move

until the groups finally settle down and stop changing. This settling point has its own short name: convergence. The worked example below walks through exactly this settling process, on real numbers, from the very first guess to the final, stable answer.

To decide which centroid is "nearest," k-means needs a distance rule. This chapter reuses the same **Euclidean distance** idea from Chapter 8, together with the same shortcut Chapter 1 and Chapter 8 already introduced: since we only ever need to know which centroid is closer, not the exact distance itself, comparing squared distances gives the identical answer as comparing true distances, without ever needing a square root.

EQUATION — SQUARED DISTANCE, THE RULE K-MEANS USES TO COMPARE CENTROIDS

$$\text{squared distance} = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In words: To compare how close a point is to two different centroids, subtract the two points' first-feature values and square the result, subtract their second-feature values and square that result, then add the two squared results together. Whichever centroid gives the smaller total is the nearer one.

x₁, x₂ the first feature of the point and of the centroid (same unit as that feature)
y₁, y₂ the second feature of the point and of the centroid (same unit as that feature)

Note: With three or more features, add one more squared difference, one for each extra feature, before comparing totals. No square root is needed anywhere in k-means, because only the ordering of the distances ever matters, never their exact size.

Now for the running example this whole lesson uses. Return to the neighborhood grocery store from Lesson 12.1. To keep every number checkable by hand, imagine the owner starts with just six loyalty-card customers, and only her two simplest features: visits last month, and average amount spent per visit, in dollars.

Customer A visited 2 times and spent \$18 per visit on average. Customer B visited 3 times and spent \$20. Customer C visited 4 times and spent \$22. Customer D visited 8 times and spent \$48. Customer E visited 9 times and spent \$50. Customer F visited 10 times and spent \$52. Written as (visits, average spend), the six customers are A = (2, 18), B = (3, 20), C = (4, 22), D = (8, 48), E = (9, 50), and F = (10, 52).

The owner decides to look for $k = 2$ groups. To start, she picks two customers at random to serve as the first two centroids: Customer B and Customer C. This is a genuinely reasonable-looking starting guess, since B and C are two ordinary, unremarkable customers, yet you are about to see that it is not quite the best possible starting point. Watching k-means correct this imperfect start is exactly what makes the next few pages worth walking through by hand.

WORKED EXAMPLE — K-MEANS ITERATION 1: ASSIGN THE SIX CUSTOMERS, THEN UPDATE THE TWO CENTROIDS

Six customers are $A = (2, 18)$, $B = (3, 20)$, $C = (4, 22)$, $D = (8, 48)$, $E = (9, 50)$, $F = (10, 52)$, using (visits, average spend in dollars). Starting centroids are $C1 = (3, 20)$ and $C2 = (4, 22)$. Assign every customer to its nearer centroid, then recompute both centroids as the average of their newly assigned customers.

1. Find each customer's squared distance to $C1 = (3, 20)$: $A: (2-3)^2 + (18-20)^2 = 1 + 4 = 5$. $B: (3-3)^2 + (20-20)^2 = 0 + 0 = 0$. $C: (4-3)^2 + (22-20)^2 = 1 + 4 = 5$. $D: (8-3)^2 + (48-20)^2 = 25 + 784 = 809$. $E: (9-3)^2 + (50-20)^2 = 36 + 900 = 936$. $F: (10-3)^2 + (52-20)^2 = 49 + 1024 = 1073$.
2. Find each customer's squared distance to $C2 = (4, 22)$: $A: (2-4)^2 + (18-22)^2 = 4 + 16 = 20$. $B: (3-4)^2 + (20-22)^2 = 1 + 4 = 5$. $C: (4-4)^2 + (22-22)^2 = 0 + 0 = 0$. $D: (8-4)^2 + (48-22)^2 = 16 + 676 = 692$. $E: (9-4)^2 + (50-22)^2 = 25 + 784 = 809$. $F: (10-4)^2 + (52-22)^2 = 36 + 900 = 936$.
3. Compare each customer's two distances and keep the smaller one: $A: 5 < 20$, so $C1$. $B: 0 < 5$, so $C1$. $C: 0 < 5$, so $C2$. $D: 692 < 809$, so $C2$. $E: 809 < 936$, so $C2$. $F: 936 < 1073$, so $C2$. Group 1 is now $\{A, B\}$, and Group 2 is now $\{C, D, E, F\}$.
4. Recompute $C1$'s visits value as the average of Group 1's visits, A and B: $(2 + 3) \div 2 = 2.5$.
5. Recompute $C1$'s spending value as the average of Group 1's spending, A and B: $(18 + 20) \div 2 = 19$. The new $C1$ is $(2.5, 19)$.
6. Recompute $C2$'s visits value as the average of Group 2's visits, C, D, E, and F: $(4 + 8 + 9 + 10) \div 4 = 31 \div 4 = 7.75$.
7. Recompute $C2$'s spending value as the average of Group 2's spending, C, D, E, and F: $(22 + 48 + 50 + 52) \div 4 = 172 \div 4 = 43$. The new $C2$ is $(7.75, 43)$.

Answer: After one full round of k-means, Group 1 = $\{A, B\}$ with centroid $(2.5, 19)$, and Group 2 = $\{C, D, E, F\}$ with centroid $(7.75, 43)$. Notice that Customer C, who visited 4 times and spent \$22, ended up in the same group as customers who visit far more often and spend far more, D, E, and F, simply because the starting centroids happened to sit close to C. The next iteration checks whether this still looks right once the centroids have moved.

Look closely at what just happened to Customer C. With the very first, rough centroids, C landed in Group 2, alongside three customers who visit and spend much more than C does. That grouping does not yet match what your own eyes would probably guess from the six customers' numbers. This is precisely why k-means does not stop after only one round. The centroids have now moved, using exactly the customers assigned to them, so it is time to check every customer against the new, moved centroids and see whether any assignment should change.

WORKED EXAMPLE — K-MEANS ITERATION 2: REASSIGN THE SIX CUSTOMERS TO THE MOVED CENTROIDS

After iteration 1, the centroids are $C1 = (2.5, 19)$ and $C2 = (7.75, 43)$. Using the same six customers as before, reassign every customer to its nearer centroid, then recompute both centroids again.

1. Find each customer's squared distance to $C1 = (2.5, 19)$: A: $(2-2.5)^2 + (18-19)^2 = 0.25 + 1 = 1.25$. B: $(3-2.5)^2 + (20-19)^2 = 0.25 + 1 = 1.25$. C: $(4-2.5)^2 + (22-19)^2 = 2.25 + 9 = 11.25$. D: $(8-2.5)^2 + (48-19)^2 = 30.25 + 841 = 871.25$. E: $(9-2.5)^2 + (50-19)^2 = 42.25 + 961 = 1003.25$. F: $(10-2.5)^2 + (52-19)^2 = 56.25 + 1089 = 1145.25$.
2. Find each customer's squared distance to $C2 = (7.75, 43)$: A: $(2-7.75)^2 + (18-43)^2 = 33.0625 + 625 = 658.0625$. B: $(3-7.75)^2 + (20-43)^2 = 22.5625 + 529 = 551.5625$. C: $(4-7.75)^2 + (22-43)^2 = 14.0625 + 441 = 455.0625$. D: $(8-7.75)^2 + (48-43)^2 = 0.0625 + 25 = 25.0625$. E: $(9-7.75)^2 + (50-43)^2 = 1.5625 + 49 = 50.5625$. F: $(10-7.75)^2 + (52-43)^2 = 5.0625 + 81 = 86.0625$.
3. Compare each customer's two distances and keep the smaller one: A: $1.25 < 658.0625$, so C1. B: $1.25 < 551.5625$, so C1. C: $11.25 < 455.0625$, so C1. D: $25.0625 < 871.25$, so C2. E: $50.5625 < 1003.25$, so C2. F: $86.0625 < 1145.25$, so C2. Group 1 is now {A, B, C}, and Group 2 is now {D, E, F}.
4. Recompute C1's visits value as the average of Group 1's visits, A, B, and C: $(2 + 3 + 4) \div 3 = 9 \div 3 = 3$.
5. Recompute C1's spending value as the average of Group 1's spending, A, B, and C: $(18 + 20 + 22) \div 3 = 60 \div 3 = 20$. The new C1 is (3, 20).
6. Recompute C2's visits value as the average of Group 2's visits, D, E, and F: $(8 + 9 + 10) \div 3 = 27 \div 3 = 9$.
7. Recompute C2's spending value as the average of Group 2's spending, D, E, and F: $(48 + 50 + 52) \div 3 = 150 \div 3 = 50$. The new C2 is (9, 50).

Answer: Customer C has now moved from Group 2 into Group 1, joining A and B, exactly the correction the algorithm needed to make. Group 1 = {A, B, C} with centroid (3, 20), and Group 2 = {D, E, F} with centroid (9, 50). This grouping now matches what a careful look at the six customers would already suggest: three light, occasional shoppers, and three frequent, bigger spenders.

One more round is needed to be sure the algorithm has truly finished, not just paused. The fourth instruction from earlier in this lesson said to keep repeating until no example switches groups. Iteration 2 produced a change, Customer C moving groups, so the algorithm is not yet allowed to stop. It must check again.

WORKED EXAMPLE — K-MEANS ITERATION 3: CHECK WHETHER ANY CUSTOMER STILL WANTS TO SWITCH

After iteration 2, the centroids are $C1 = (3, 20)$ and $C2 = (9, 50)$. Reassign every customer one more time, and compare the result with iteration 2's grouping.

1. Find each customer's squared distance to $C1 = (3, 20)$: A: $(2-3)^2 + (18-20)^2 = 1 + 4 = 5$. B: $(3-3)^2 + (20-20)^2 = 0$. C: $(4-3)^2 + (22-20)^2 = 1 + 4 = 5$. D: $(8-3)^2 + (48-20)^2 = 25 + 784 = 809$. E: $(9-3)^2 + (50-20)^2 = 36 + 900 = 936$. F: $(10-3)^2 + (52-20)^2 = 49 + 1024 = 1073$.
2. Find each customer's squared distance to $C2 = (9, 50)$: A: $(2-9)^2 + (18-50)^2 = 49 + 1024 = 1073$. B: $(3-9)^2 + (20-50)^2 = 36 + 900 = 936$. C: $(4-9)^2 +$

$(22-50)^2 = 25 + 784 = 809$. D: $(8-9)^2 + (48-50)^2 = 1 + 4 = 5$. E: $(9-9)^2 + (50-50)^2 = 0$. F: $(10-9)^2 + (52-50)^2 = 1 + 4 = 5$.

3. Compare each customer's two distances and keep the smaller one: A: $5 < 1073$, so C1. B: $0 < 936$, so C1. C: $5 < 809$, so C1. D: $5 < 809$, so C2. E: $0 < 936$, so C2. F: $5 < 1073$, so C2. Group 1 is {A, B, C}, and Group 2 is {D, E, F}.
4. Compare this grouping with iteration 2's grouping, Group 1 = {A, B, C} and Group 2 = {D, E, F}. The two groupings are exactly the same, so no customer switched.

Answer: No customer changed groups this round, so k-means has converged. The final clustering is Group 1 = {A, B, C}, the light, occasional shoppers, with centroid (3, 20), and Group 2 = {D, E, F}, the frequent, bigger spenders, with centroid (9, 50). Recomputing the centroids again would simply return these exact same two points, so there is nothing left to gain by repeating the process any further.

Step back and look at the whole journey. The very first, rough guess, using Customers B and C as starting centroids, briefly grouped C with the frequent big spenders. Two more rounds of assign-and-update quietly repaired that mistake, without anyone telling the algorithm the "right" answer, because the right answer does not exist yet in unsupervised learning. The algorithm only ever follows its own two simple rules, assign then average, and those two rules were enough to find a sensible, stable grouping on their own.

The following short program repeats the exact same clustering as the worked examples above, using scikit-learn.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[2, 18], [3, 20], [4, 22], [8, 48], [9, 50], [10, 52]])
start = np.array([[3, 20], [4, 22]])

model = KMeans(n_clusters=2, init=start, n_init=1)
model.fit(X)

print(model.labels_)
print(model.cluster_centers_)
print(model.inertia_)
```

The `X` array stores the six customers exactly as in the worked examples, and `start` stores the same two starting centroids, Customer B and Customer C. Setting `init=start` forces scikit-learn to begin from the exact same guess used by hand above, instead of picking its own random starting point. Calling `model.fit(X)` repeats the assign-then-update loop automatically, as many times

as needed until convergence. Running this code prints `[0 0 0 1 1 1]` for `labels_`, showing customers A, B, and C in one group and D, E, and F in the other, and prints centroids `(3, 20)` and `(9, 50)` for `cluster_centers_`, matching iteration 3's answer exactly.

COMMON MISTAKE

✗ Wrong: Assuming k-means always finds the one single best possible grouping, no matter which starting centroids happen to be chosen.

✓ Right: Different starting centroids can make k-means converge to a different final grouping, especially on messier, real-world data. In practice, k-means is usually run several times, from several different random starting points, and the run with the best score is kept. Lesson 12.3 gives that score its own name.

12.3 Choosing k: Inertia and the Elbow Method

The worked examples in Lesson 12.2 all assumed $k = 2$ from the very start. But nobody told the grocery store owner that two groups was the right number. She could just as easily have asked for three groups, or five. Before choosing k sensibly, this lesson needs one more idea: a single number that scores how well a finished clustering fits its own data.

WORD TO KNOW

Inertia — A clustering's total squared distance from every example to the centroid it is assigned to, added together across the whole dataset. A lower inertia means the examples sit closer, on average, to their own group's centroid.

EQUATION — INERTIA

$$\text{inertia} = \sum (\text{squared distance from each point to its assigned centroid})$$

In words: To find a clustering's inertia, find the squared distance from every single example to the centroid of the group it belongs to, then add every one of those squared distances together into one total.

inertia the total squared distance across the whole clustering (same unit as the features, squared)

Σ add up this quantity across every example in the dataset (no unit)

Note: Inertia can only be compared fairly between two clusterings that use the same value of k and the same dataset. A smaller k almost always produces a larger inertia than a larger k , simply because fewer centroids have to cover more ground.

WORKED EXAMPLE — FIND THE INERTIA OF THE FINISHED $k = 2$ CLUSTERING

The finished clustering from Lesson 12.2 has Group 1 = {A, B, C} with centroid (3, 20), and Group 2 = {D, E, F} with centroid (9, 50). Find the clustering's inertia.

1. Recall each Group 1 customer's squared distance to its centroid (3, 20), already found in iteration 3: A = 5, B = 0, C = 5.
2. Add Group 1's three squared distances together: $5 + 0 + 5 = 10$.

3. Recall each Group 2 customer's squared distance to its centroid (9, 50), also already found in iteration 3: $D = 5$, $E = 0$, $F = 5$.
4. Add Group 2's three squared distances together: $5 + 0 + 5 = 10$.
5. Add both groups' totals together to find the overall inertia: $10 + 10 = 20$.

Answer: The finished clustering's inertia is 20. This single number summarizes how tightly all six customers sit around their own group's centroid; Lesson 12.3 now uses this same number to help decide whether $k = 2$ was even a sensible choice in the first place.

A number like 20 is not very meaningful sitting by itself. It only becomes useful once it is compared with the inertia of other values of k on the exact same data. Suppose the store owner tries $k = 1$ as well, the simplest possible case, where every customer is forced into a single group with a single shared centroid.

WORKED EXAMPLE — FIND THE INERTIA WHEN $k = 1$

Using the same six customers, $A = (2, 18)$, $B = (3, 20)$, $C = (4, 22)$, $D = (8, 48)$, $E = (9, 50)$, $F = (10, 52)$, find the inertia of a single cluster containing all six.

1. Find the one shared centroid's visits value, the average of all six visit counts: $(2 + 3 + 4 + 8 + 9 + 10) \div 6 = 36 \div 6 = 6$.
2. Find the one shared centroid's spending value, the average of all six spending amounts: $(18 + 20 + 22 + 48 + 50 + 52) \div 6 = 210 \div 6 = 35$. The single centroid is (6, 35).
3. Find each customer's squared distance to (6, 35): A: $(2-6)^2 + (18-35)^2 = 16 + 289 = 305$. B: $(3-6)^2 + (20-35)^2 = 9 + 225 = 234$. C: $(4-6)^2 + (22-35)^2 = 4 + 169 = 173$. D: $(8-6)^2 + (48-35)^2 = 4 + 169 = 173$. E: $(9-6)^2 + (50-35)^2 = 9 + 225 = 234$. F: $(10-6)^2 + (52-35)^2 = 16 + 289 = 305$.
4. Add all six squared distances together: $305 + 234 + 173 + 173 + 234 + 305 = 1424$.

Answer: With $k = 1$, the inertia is 1424, dramatically higher than the 20 found for $k = 2$. Forcing every customer to share one single centroid leaves every point sitting far from it, especially the customers near either end of the shopping-habit spread.

Following exactly the same process for $k = 3$, $k = 4$, and $k = 5$ gives inertia values of 12.5, 5, and 2.5. Notice the pattern already forming: 1424, then 20, then 12.5, then 5, then 2.5. Inertia keeps falling every time k grows, which makes good sense, since more centroids can always divide the work of covering the data more finely. Taken to its extreme, if k were set to 6, equal to the number of customers, every single customer could become its own group, sitting exactly on its own centroid, making the inertia exactly 0. A score of 0 sounds perfect, but a "clustering" where every customer is their own private group is completely useless for planning promotions. This tension, lower inertia always sounds better, yet a lower inertia achieved only by adding pointless extra groups helps nobody, is exactly what the elbow method is built to resolve.

WORD TO KNOW

Elbow method — A way to choose a sensible k by plotting inertia against several values of k and looking for the point where adding another group stops helping very much. The plotted line usually drops sharply at first, then bends and flattens out, forming a shape like a bent arm, with the "elbow" marking a good choice of k .

WORKED EXAMPLE — USE THE ELBOW METHOD TO CHOOSE k FOR THE GROCERY STORE

Running k -means at several values of k on the same six customers gives these inertia values: $k = 1$, inertia 1424. $k = 2$, inertia 20. $k = 3$, inertia 12.5. $k = 4$, inertia 5. $k = 5$, inertia 2.5. Find the drop in inertia between each consecutive pair of k values, and use the pattern to choose a sensible k .

1. Find the drop from $k = 1$ to $k = 2$: $1424 - 20 = 1404$.
2. Find the drop from $k = 2$ to $k = 3$: $20 - 12.5 = 7.5$.
3. Find the drop from $k = 3$ to $k = 4$: $12.5 - 5 = 7.5$.
4. Find the drop from $k = 4$ to $k = 5$: $5 - 2.5 = 2.5$.
5. Compare the four drops: 1404, then 7.5, then 7.5, then 2.5. The very first drop is enormous compared with every drop that follows it.

Answer: $k = 2$ is the elbow. Moving from one group to two groups fixes almost the entire problem, a drop of 1404, while every later group added after that only chips away a small, steadily shrinking amount. Paying for three, four, or five separate promotions instead of two would cost the store real planning effort for very little extra benefit in how tightly the customers are grouped.

The elbow method is a visual, judgment-based tool, not a strict formula with one guaranteed right answer. Real inertia curves are sometimes smoother than this tidy example, with no single obvious bend, and two careful people can sometimes reasonably pick two different neighboring values of k from the same curve. Chapter 12's ML Studio activity lets you watch this exact curve get plotted automatically, for a dataset with far more than six points, so you can practice spotting the bend yourself.

COMMON MISTAKE

✗ Wrong: Choosing whichever k gives the lowest inertia, without noticing that inertia keeps falling, all the way down to exactly 0, as k grows to match the number of examples.

✓ Right: Use the elbow method instead: look for the value of k where inertia stops dropping sharply and starts flattening out. A lower inertia is not automatically a better choice of k , because it can simply mean the data was sliced into more, smaller, less useful pieces.

12.4 Limitations: Shapes k -Means Cannot Find

k -means is simple, fast, and works well on a huge range of everyday problems, but it carries a few honest limits worth knowing before you reach for it on a new dataset.

The clearest limit comes from the method's own distance rule. Because k-means always assigns a point to whichever centroid is nearest, and always measures "nearest" using the same squared-distance rule in every direction, it naturally draws round, roughly equal-sized clusters around each centroid. Picture two very different customer groups on a graph. One group forms a small, tight cluster of frequent visitors who all spend a similar amount. The other group forms a long, thin, curving streak, stretching from customers who visit rarely and spend little, gradually through to customers who visit rarely but occasionally spend quite a lot on a single large order. k-means has no way to notice that the long, curving streak is really one meaningful group. Because the streak stretches so far, k-means is likely to slice it into two or three round pieces instead, while possibly even swallowing part of the small, tight cluster into one of those pieces. The method is not doing anything wrong by its own rules; it is simply blind to any group shape that is not roughly round.

A second limit is sensitivity to starting centroids, already seen directly in Lesson 12.2's worked example, where a poor first guess needed two extra rounds to correct itself. Real datasets, with far more points and far less obviously separated groups than this chapter's six customers, can sometimes settle into a genuinely different, weaker final grouping depending on where the centroids started. This is exactly why Lesson 12.2's common mistake box recommended running k-means several times from different starting points and keeping the run with the lowest inertia.

A third limit is that k-means needs every feature to already be a comparable number, and it needs those numbers placed on a fair, shared scale before measuring any distance. This should sound familiar: it is exactly the same trap Chapter 8 uncovered for **k-Nearest Neighbors**. If the grocery store added a customer's total lifetime spending, in dollars, sitting in the thousands, alongside visits per month, sitting in the single digits, the spending column would quietly dominate every distance calculation, and the resulting clusters would barely reflect visit frequency at all. **Standardization** fixes this, exactly as it did in Chapter 3 and Chapter 8: subtract each feature's training mean, divide by its training standard deviation, and only then compute distances.

COMMON MISTAKE

✗ Wrong: Running k-means directly on raw features that use very different scales, such as a customer's visit count next to their total lifetime spending in dollars.

✓ Right: Standardize every numerical feature before running k-means, exactly as Chapter 8 recommends for k-Nearest Neighbors. Both methods make every decision using distance, so both are equally vulnerable to one large-number feature quietly taking over.

GOING FURTHER — OTHER CLUSTERING METHODS YOU WILL MEET

Two other clustering methods are common enough that you should recognize their names. Hierarchical clustering builds a whole tree of groupings at once, starting with every example in its own tiny group and repeatedly merging the

two closest groups together, one merge at a time, until only one giant group remains; a person can then "cut" this tree at any level to get however many clusters seems useful, without ever having to pick k in advance. DBSCAN groups together examples that are packed densely close to one another, and it is comfortable labeling a lonely, sparse example as noise instead of forcing it into any group at all, which lets it trace out long, curving, or oddly shaped clusters that k -means could never find, exactly the kind of shape described earlier in this lesson.

IMPORTANT

A cluster is a pattern found in data, not a fixed fact about any one person inside it. Two customers assigned to the very same cluster can still be genuinely different in ways the model never measured, and a single customer's habits can drift into a different cluster next season. Use clusters to guide broad decisions, such as which general kind of promotion to design, never to make a rigid, unreviewed decision about one specific, individual customer.

12.5 Try It in ML Studio

Six customers on paper make every step checkable by hand, but watching centroids actually slide across the screen, round after round, is what makes k -means truly click.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Labs tab, then choose the k -Means animation.
2. Pick the "Blobs" dataset and set $k = 3$. Press Step once, and watch the first assignment happen, then press Step again and watch the centroids move. Keep pressing Step and notice when the colors stop changing.
3. Reset the animation, and this time choose a deliberately poor set of starting centroids, all clustered together in one corner of the screen. Run it to the end. Does it still find the same three groups as before?
4. Switch to the Play tab, choose the same "Blobs" dataset, and turn on the Elbow Chart option. Try $k = 1$ through $k = 8$, and write down where you see the bend.
5. Now choose the "Moons" dataset instead, still with k -Means, and set $k = 2$. Look closely at where the boundary between the two colors falls. In one or two sentences, explain why this dataset gives k -means trouble.

You need: Computer, tablet, or phone with the ML Studio app

12.6 Case Study

CASE STUDY — SEGMENTING SHOPPERS FOR A SUPERMARKET CHAIN IN PUNE

A supermarket chain with a dozen branches across Pune has run the same loyalty program for years, and until now it has sent every member the exact same weekly discount coupon by text message. The marketing team suspects this one-size-fits-all approach is wasting money: a coupon that excites an

occasional big-basket shopper may barely interest someone who already visits three times a week for small top-up purchases. Nobody has ever labeled customers into "types," so there is no existing answer key to learn from, only the loyalty program's own transaction records. This is exactly the kind of question this chapter's method was built to answer.

The team starts, as this chapter recommends, by choosing a small, meaningful set of features rather than throwing in every column available: average number of visits per week over the last three months, average amount spent per basket, in rupees, and the share of a customer's visits that fall on a weekend. These three features arrive on very different scales, roughly 0 to 7 visits, hundreds of rupees per basket, and a percentage from 0 to 100. Following Lesson 12.4's warning closely, the team standardizes all three features first, so that basket size, sitting in the hundreds, cannot quietly drown out visit frequency the moment distances are calculated.

Running k-means across several values of k and plotting the elbow chart, the team finds a clear bend at $k = 4$. Looking at the four finished centroids, a category manager gives each group a working name, purely for the team's own convenience, never as an official label attached to any customer's account: frequent small-basket shoppers, who visit almost daily but spend little each time; weekend bulk buyers, who visit rarely but bring home a large basket, usually on a Saturday; steady mid-week regulars, who fall in between on every feature; and a small, occasional group whose basket sizes vary the most, visiting unpredictably. This last group's centroid sits noticeably far from the other three, which the team quietly flags for a closer look before trusting it too heavily, since a small, scattered group can sometimes be less a genuine "type" and more a mix of unrelated customers who simply did not fit elsewhere well.

Armed with these four groups, the marketing team designs four different weekly messages instead of one: a small loyalty-point bonus timed to a weekday for the frequent small-basket group, a bigger one-time discount aimed at the weekend bulk buyers, a simple thank-you note for the steady regulars, and, for the small unpredictable group, no automatic campaign at all, just a flag for a store manager to review by hand. The team also agrees to rerun the whole clustering every few months, because a customer's own shopping habits can drift from one cluster into another as their life changes, and a segment computed from last year's habits should never quietly control this year's messages forever.

COMMON MISTAKES TO AVOID

- Reaching for clustering when a known, correct label already exists for every example, when a supervised method would give a more precise, checkable answer.
- Assuming k-means always finds the one single best grouping, regardless of which starting centroids happen to be chosen.
- Stopping after only one assign-and-update round, instead of repeating until no example switches groups.

- Choosing k by picking whichever value gives the lowest inertia, instead of using the elbow method.
- Running k -means on raw features that sit on very different scales, letting one large-number feature dominate every distance.
- Treating a cluster label as a fixed, permanent fact about one specific person, instead of a broad pattern that can drift over time.
- Expecting k -means to find long, curving, or very unevenly sized groups, when its distance rule naturally favors round, similarly sized ones.

CHAPTER SUMMARY

- Clustering is unsupervised learning: it groups similar examples together with no known, correct labels to learn from.
- k -means repeats two steps, assign every example to its nearest centroid, then move each centroid to the average of its assigned examples, stopping only once no example switches groups.
- Comparing squared distances, without ever taking a square root, gives the same nearest-centroid answer as comparing true distances, exactly as in Chapters 1 and 8.
- Different starting centroids can lead k -means to different final groupings, so it is normal practice to run it several times and keep the best result.
- Inertia adds up the squared distance from every example to its own assigned centroid; a lower inertia means a tighter clustering, but it always keeps falling as k grows.
- The elbow method plots inertia against several values of k and looks for the point where adding another group stops helping much, giving a sensible, defensible choice of k .
- k -means assumes roughly round, similarly sized groups, and can fail badly on long, curving, or very unevenly shaped clusters.
- k -means needs numerical features on a shared, standardized scale, exactly like k -Nearest Neighbors, or one large-number feature will quietly dominate every distance.
- Hierarchical clustering and DBSCAN are two other clustering methods worth recognizing by name, each handling a weakness of k -means in its own way.
- Chapter 13 tackles a different unsupervised question: not how to group examples, but how to shrink a table with too many features down to the few directions that matter most.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] In your own words, explain the difference between clustering and classification.

Problem 2 [Basic] Explain what the letter k means in k-means, and state who chooses its value.

Problem 3 [Basic] Define inertia in your own words, in one or two sentences.

Problem 4 [Basic] A hospital team wants to group patients by symptom similarity, with no existing diagnosis categories to learn from. Name the learning task, and explain in one sentence why it is not classification.

Problem 5 [Basic] List, in order, the two steps that k-means repeats every round, once its starting centroids have first been chosen.

Problem 6 [Standard] Two centroids are $C1 = (1, 1)$ and $C2 = (6, 6)$. Find the squared distance from point $P = (2, 3)$ to each centroid, showing every step, and state which centroid P is assigned to.

Problem 7 [Standard] A group's two points are $(2, 4)$ and $(6, 8)$. Find the group's centroid, showing the average of each feature separately.

Problem 8 [Standard] A clustering has two groups. Group 1's three points have squared distances 3, 5, and 4 from their own centroid. Group 2's two points have squared distances 6 and 2 from their own centroid. Find the clustering's total inertia.

Problem 9 [Standard] A team records these inertia values: $k = 1$, inertia 800. $k = 2$, inertia 300. $k = 3$, inertia 260. $k = 4$, inertia 230. Find the drop in inertia between each consecutive pair of k values, and identify the elbow.

Problem 10 [Standard] Explain, in two or three sentences, why raw, unstandardized features can distort a k-means clustering, using an example of your own, different from the grocery store in this chapter.

Problem 11 [Challenge] Four points are $P1 = (1, 1)$, $P2 = (2, 2)$, $P3 = (8, 8)$, and $P4 = (9, 9)$. Starting centroids are $C1 = (1, 1)$ and $C2 = (2, 2)$. (a) Assign each point to its nearer centroid using squared distance, showing every calculation. (b) Recompute both centroids. (c) Reassign each point to the new centroids, and state whether the clustering has converged.

Problem 12 [Challenge] A retailer wants to group its stores by sales pattern, using two features: average daily transactions, and average transaction value in dollars. (a) Explain why standardizing these two features first matters. (b) Describe how you would use the elbow method to choose k. (c) Name one shape of grouping this method might fail to find, and one other clustering method that could handle it instead.

Problem 13 [Challenge] Open ML Studio, fit k-Means on the "Blobs" dataset, and try at least four different values of k, recording the inertia shown on screen each time. Identify the elbow, and explain in a few sentences why choosing a much larger k would not actually give you a "better" clustering, even though its inertia would be lower.

UNSUPERVISED LEARNING

CHAPTER 13

Principal Component Analysis

Finding the few directions that matter most

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a dataset with many features becomes hard to visualize and understand, and what dimensionality reduction is for
- Center a dataset's features and compute how much a group of numbers spreads out along one chosen direction
- Find the first principal component of a small dataset by hand, by testing candidate directions and keeping the one with the greatest spread
- Explain why the second principal component must sit at a right angle to the first, and describe how this pattern continues for more components
- Compute an explained variance ratio, and use it to decide how many components are worth keeping
- Recognize what Principal Component Analysis is used for, and name two related methods built specifically for visualization

13.1 Too Many Features, Too Little Insight

Every chapter so far has used a small, friendly number of features: a melon's weight and sound, an apartment's size and city, a customer's visits and spending. Real datasets are rarely so tidy. A university's student records might include a dozen exam scores. A hospital's patient file might include forty lab measurements. A factory's sensor log might include hundreds of readings taken every second from a single machine. Each new feature can genuinely add useful information, but each new feature also adds a new problem: it becomes harder and harder to actually look at the data and understand what it is showing you.

Think about why this happens. A dataset with two features can be plotted on an ordinary flat graph, one feature along the bottom and one feature up the side, and a person can immediately see clusters, outliers, and trends just by looking. A dataset with three features can still be plotted, awkwardly, as a 3D graph. A dataset with six features already has 15 different possible pairs of features you could plot against each other, and a dataset with twenty features has 190 such pairs, far more than anyone has the patience to look at one by one. Beyond three or four features, "just plotting the data and looking at it" quietly stops being possible at all.

There is a second, quieter problem hiding alongside the first one. Many real features move together. A student who scores highly in Math also tends to score highly in Physics, because both subjects lean on similar underlying skills. A

factory sensor reading a machine's temperature tends to rise and fall alongside a nearby sensor reading its vibration, because both are responding to the same underlying wear and tear. When two features move together this closely, much of the second feature's information is, in a real sense, already contained in the first. Keeping both in full does not always teach a model twice as much; sometimes it barely teaches it anything new at all, while still doubling the work of storing, plotting, and computing with the data.

This chapter's method, Principal Component Analysis, tackles both problems at once. It looks for a small number of new, carefully chosen directions through the data, directions that capture as much of the data's real spread as possible, and lets you describe every example using just those few new numbers instead of the original, larger pile of features. This general goal, shrinking the number of features while keeping as much real information as possible, has its own name.

WORD TO KNOW

Dimensionality reduction — Reducing the number of features used to describe each example, while trying to keep as much of the data's real, useful information as possible. Each original feature is often called a dimension, so cutting their number down is called reducing the dimensionality.

It helps to picture dimensionality reduction as packing for a long trip with only one small suitcase. You cannot bring your entire wardrobe, so you choose a handful of versatile items that, combined, cover the widest range of situations you might face. You are not simply throwing most of your clothes away at random. You are deliberately choosing the few items that carry the most real usefulness, so the small suitcase still serves you almost as well as the full wardrobe would have. Principal Component Analysis packs a dataset's features the very same way: not by deleting columns at random, but by building a small number of new, carefully chosen directions that carry as much of the original spread as possible. Lesson 13.2 begins building exactly this kind of direction, by hand, on a dataset small enough to compute with pencil and paper.

COMMON MISTAKE

✗ Wrong: Assuming the only way to reduce a dataset's number of features is to simply delete the columns that seem least important and keep the rest untouched.

✓ Right: Dimensionality reduction usually builds brand-new features, each one a careful blend of the original features, rather than just deleting some original columns outright. The next three lessons show exactly how a method can build such a blend, one useful direction at a time.

13.2 Finding the Direction of Greatest Spread

WORD TO KNOW

Principal Component Analysis (PCA) — A dimensionality-reduction method that finds new directions through a dataset, ranked by how much of the data's

spread each direction captures, so that a small number of these new directions can stand in for a much larger set of original features. The method is usually called PCA for short.

To see exactly how PCA finds its directions, this lesson uses a small, fully checkable example: just two features, for just five students, from a university department in Erbil. Suppose five students each sit a Math exam and a Physics exam, both graded on a 0-to-100 scale. Student 1 scores 73 in Math and 69 in Physics. Student 2 scores 67 in Math and 61 in Physics. Student 3 scores 76 in Math and 73 in Physics. Student 4 scores 64 in Math and 57 in Physics. Student 5 scores 70 in Math and 65 in Physics.

Plotted on a graph, with Math along the bottom and Physics up the side, these five students would not scatter randomly. A student who scores above average in Math also tends to score above average in Physics, and a student who scores below average in one tends to score below average in the other too. The five points would stretch out diagonally across the graph, exactly the "features that move together" pattern Lesson 13.1 described.

Before PCA can search for a useful direction through this diagonal stretch, it needs to know where the "center" of the data sits. This first step is one you already know.

WORKED EXAMPLE — CENTER FIVE STUDENTS' MATH AND PHYSICS SCORES

Student 1: Math 73, Physics 69. Student 2: Math 67, Physics 61. Student 3: Math 76, Physics 73. Student 4: Math 64, Physics 57. Student 5: Math 70, Physics 65. Center both features by subtracting each feature's class mean from every student's score.

1. Find the mean Math score across all five students: $(73 + 67 + 76 + 64 + 70) \div 5 = 350 \div 5 = 70$.
2. Find the mean Physics score across all five students: $(69 + 61 + 73 + 57 + 65) \div 5 = 325 \div 5 = 65$.
3. Subtract the Math mean, 70, from every student's Math score: $73 - 70 = 3$. $67 - 70 = -3$. $76 - 70 = 6$. $64 - 70 = -6$. $70 - 70 = 0$.
4. Subtract the Physics mean, 65, from every student's Physics score: $69 - 65 = 4$. $61 - 65 = -4$. $73 - 65 = 8$. $57 - 65 = -8$. $65 - 65 = 0$.

Answer: The centered scores, written as (Math deviation, Physics deviation), are Student 1 = (3, 4), Student 2 = (-3, -4), Student 3 = (6, 8), Student 4 = (-6, -8), and Student 5 = (0, 0). Every centered value now says how far above or below the class average a student scored, rather than the raw score itself, and the five centered points now sit evenly around (0, 0).

In this chapter's example, both Math and Physics already share the same 0-to-100 scale, so centering alone is enough to compare them fairly. When features arrive on very different scales, such as a factory sensor mixing degrees and volts, centering is not enough by itself; **standardization**, dividing by each

feature's standard deviation as well, exactly as in Chapter 3 and Chapter 12, is the safer habit to build before running PCA on real data.

With the data centered, PCA's actual question can finally be asked precisely: among every possible direction through this cloud of points, which one direction do the points spread out along the most? Measuring "spread along a direction" leans on an idea already familiar from this book, just used in a new setting.

Chapter 5 used the word **variance** to describe how much a model's predictions swing from one training sample to another. This chapter uses the very same word in a related, but not identical, everyday statistical sense: how spread out a group of numbers is around its own mean. Both meanings share the same core arithmetic, an average of squared distances from a center point, just applied to different things, a model's predictions in Chapter 5, and a list of plain data values here.

To measure a centered point's position along one chosen direction, PCA uses a simple combining rule called a projection.

EQUATION — PROJECTING A CENTERED POINT ONTO A DIRECTION

$$\text{projection} = (x \times u_1) + (y \times u_2)$$

In words: To project a centered point onto a direction, multiply the point's first feature by the direction's first number, multiply the point's second feature by the direction's second number, then add the two products together.

x, y the point's centered feature values (same unit as each feature)

u₁, u₂ the two numbers describing the chosen direction (no unit, just plain numbers)

projection one single number showing how far along that direction the point falls (same unit as the features)

Note: Every direction used in this chapter is written so that its two numbers, each squared and then added together, equal exactly 1, for example $0.6^2 + 0.8^2 = 1$. This keeps every direction the same "length," so that comparing spreads along different directions is always a fair comparison.

Once every point has been projected onto a chosen direction, that direction's spread is just the variance of those projected numbers, exactly the same average-of-squared-distances idea Chapter 5 used, only simpler here because the centered data's projections already average out to 0.

EQUATION — VARIANCE ALONG A DIRECTION

$$\text{variance along } u = \frac{\sum \text{projection}^2}{n}$$

In words: To find how much the data spreads out along one direction, square every point's projection onto that direction, add the squared projections together, then divide by the number of points.

variance along u how spread out the projected points are along direction u (same unit as the features, squared)

projection one point's projected position along direction u , from the equation above (same unit as the features)

n the number of points (count)

Note: A larger variance along a direction means the data spreads out more widely along that direction. PCA's whole search is simply a search for the direction with the largest possible variance.

Real PCA software searches every possible direction at once, using matrix arithmetic well beyond this book's scope. To see the same idea with only pencil-and-paper arithmetic, this lesson instead tests three sensible candidate directions by hand and compares their variance directly, exactly the way you might compare three candidate models in earlier chapters.

WORKED EXAMPLE — TEST THREE CANDIDATE DIRECTIONS FOR THE GREATEST SPREAD

Using the five centered scores from the previous worked example, (3, 4), (−3, −4), (6, 8), (−6, −8), and (0, 0), find the variance along three candidate directions: (1, 0), the original Math axis; (0, 1), the original Physics axis; and (0.6, 0.8), a diagonal direction. Identify which direction captures the greatest spread.

1. Project every centered point onto (1, 0): $3 \times 1 + 4 \times 0 = 3$. $-3 \times 1 + -4 \times 0 = -3$. $6 \times 1 + 8 \times 0 = 6$. $-6 \times 1 + -8 \times 0 = -6$. $0 \times 1 + 0 \times 0 = 0$.
2. Square and average those five projections: $3^2 + (-3)^2 + 6^2 + (-6)^2 + 0^2 = 9 + 9 + 36 + 36 + 0 = 90$, and $90 \div 5 = 18$. The variance along (1, 0) is 18.
3. Project every centered point onto (0, 1): $3 \times 0 + 4 \times 1 = 4$. $-3 \times 0 + -4 \times 1 = -4$. $6 \times 0 + 8 \times 1 = 8$. $-6 \times 0 + -8 \times 1 = -8$. $0 \times 0 + 0 \times 1 = 0$.
4. Square and average those five projections: $4^2 + (-4)^2 + 8^2 + (-8)^2 + 0^2 = 16 + 16 + 64 + 64 + 0 = 160$, and $160 \div 5 = 32$. The variance along (0, 1) is 32.
5. Project every centered point onto (0.6, 0.8): $3 \times 0.6 + 4 \times 0.8 = 1.8 + 3.2 = 5$. $-3 \times 0.6 + -4 \times 0.8 = -1.8 - 3.2 = -5$. $6 \times 0.6 + 8 \times 0.8 = 3.6 + 6.4 = 10$. $-6 \times 0.6 + -8 \times 0.8 = -3.6 - 6.4 = -10$. $0 \times 0.6 + 0 \times 0.8 = 0$.
6. Square and average those five projections: $5^2 + (-5)^2 + 10^2 + (-10)^2 + 0^2 = 25 + 25 + 100 + 100 + 0 = 250$, and $250 \div 5 = 50$. The variance along (0.6, 0.8) is 50.
7. Compare the three variances: 18 for the Math axis, 32 for the Physics axis, and 50 for the diagonal direction. 50 is the largest of the three.

Answer: The diagonal direction, (0.6, 0.8), captures the most spread, with a variance of 50, more than either original subject captures on its own. PCA keeps this winning direction as its first, most important new direction.

WORD TO KNOW

Principal component — One of the new directions PCA finds, ranked by how much of the data's spread it captures. The first principal component is the

single direction that captures the most spread possible; each later principal component captures the most spread still remaining, once the earlier ones are accounted for.

The winning direction, $(0.6, 0.8)$, is the first principal component of this small dataset, often written PC1 for short. Its two numbers have a plain-English reading: moving 0.6 steps in the Math-deviation direction goes hand in hand with moving 0.8 steps in the Physics-deviation direction, which matches exactly what the five students' scores already showed, a Physics score that rises a little faster than the matching Math score, for every student in this class. A new, single number along PC1 can now stand in for a student's whole pair of exam scores, with remarkably little lost, a promise Lesson 13.4 checks precisely.

COMMON MISTAKE

✗ Wrong: Running PCA directly on raw, uncentered data, or on features that use very different scales, without centering and, where needed, standardizing first.

✓ Right: Always center every feature, subtracting its own mean, before searching for directions of spread; when features use different units or very different scales, standardize them first too, exactly as Chapter 3 and Chapter 12 recommend. Most PCA software centers data automatically, but a careless mix of scales can still quietly tilt the winning direction toward whichever feature happens to use larger numbers.

13.3 Principal Components, One at a Time

Finding PC1 does not end PCA's work. A dataset with two original features can hold two principal components in total, a dataset with three original features can hold three, and so on, one new direction for every original feature. But these later directions are not found by simply repeating Lesson 13.2's search from scratch. Each new principal component has to earn its place with one extra rule: it must sit at a perfect right angle to every principal component already found.

This rule is easier to picture than it sounds. Think of PC1 as a line drawn straight through the middle of the data, the direction it stretches out along the most. PC2 must be perpendicular to that line, the way a north-south street sits at a right angle to an east-west street. This rule exists for a good reason. If PC2 were allowed to point in almost the same direction as PC1, it would mostly just measure the same spread all over again, teaching PCA nothing new. Forcing PC2 to sit at a right angle guarantees it can only capture spread that PC1 has not already claimed.

In this chapter's two-feature example, this rule leaves no freedom at all. Once PC1 is fixed at $(0.6, 0.8)$, there is exactly one direction, up to which way it points, that sits at a right angle to it: $(0.8, -0.6)$. No searching is even needed this time; the direction is completely determined.

WORKED EXAMPLE — FIND THE SECOND PRINCIPAL COMPONENT

The first principal component is $PC1 = (0.6, 0.8)$. The only direction at a right angle to it is $PC2 = (0.8, -0.6)$. Confirm this really is a right angle, confirm $PC2$ has the same "length" as every direction in this chapter, then find its variance using the same five centered scores from Lesson 13.2: $(3, 4)$, $(-3, -4)$, $(6, 8)$, $(-6, -8)$, and $(0, 0)$.

1. Confirm $PC1$ and $PC2$ sit at a right angle by multiplying their matching numbers and adding the results, the same combining rule used for every projection in this chapter: $(0.6 \times 0.8) + (0.8 \times -0.6) = 0.48 - 0.48 = 0$. A total of exactly 0 confirms a perfect right angle.
2. Confirm $PC2$'s two numbers, squared and added, equal 1, just like $PC1$'s: $0.8^2 + (-0.6)^2 = 0.64 + 0.36 = 1$.
3. Project every centered point onto $PC2 = (0.8, -0.6)$: $3 \times 0.8 + 4 \times (-0.6) = 2.4 - 2.4 = 0$. $-3 \times 0.8 + -4 \times (-0.6) = -2.4 + 2.4 = 0$. $6 \times 0.8 + 8 \times (-0.6) = 4.8 - 4.8 = 0$. $-6 \times 0.8 + -8 \times (-0.6) = -4.8 + 4.8 = 0$. $0 \times 0.8 + 0 \times (-0.6) = 0$.
4. Square and average these five projections: $0^2 + 0^2 + 0^2 + 0^2 + 0^2 = 0$, and $0 \div 5 = 0$. The variance along $PC2$ is 0.

Answer: $PC2$'s variance is exactly 0. Every one of the five students' centered scores lands precisely on the $PC1$ line, so once $PC1$ is accounted for, absolutely no spread is left over in the one remaining perpendicular direction. Lesson 13.4 turns this pair of numbers, 50 and 0, into a precise statement of how much information each principal component is really carrying.

Real datasets are almost never this perfectly tidy. This chapter's five students were deliberately chosen so that Math and Physics move together in perfect lockstep, which makes the arithmetic easy to check by hand and makes $PC2$'s variance land on a clean 0. An ordinary, real class would show some scatter around the $PC1$ line, so $PC2$ would usually capture a small but genuine amount of spread, not exactly zero. The method for finding it, however, never changes: search for the direction with the greatest remaining variance, subject to sitting at a right angle to every principal component already found.

With three or more original features, this same one-at-a-time process just continues for longer. $PC1$ is the single direction of greatest spread, chosen from every possible direction. $PC2$ is the direction of greatest remaining spread, chosen only from directions at a right angle to $PC1$, a whole flat sheet of possible directions rather than the single forced choice this chapter's two-feature example produced. $PC3$ is the direction of greatest remaining spread after that, chosen only from directions at a right angle to both $PC1$ and $PC2$, and so on, one component at a time, until as many principal components exist as there were original features.

Each principal component is a brand-new feature, built by blending the original features together using the two numbers of its direction, not a copy of any single original feature. Once PCA has found its components, a data table with many original columns can be rewritten using only a chosen handful of principal

components instead, each student, or each example, described by only a few new numbers rather than the full original list.

COMMON MISTAKE

✗ Wrong: Assuming the first principal component is simply the single most important original feature, such as thinking PC1 "is" the Math score.

✓ Right: A principal component is a blend of every original feature, combined using its own direction's numbers, not a stand-in for any one original column. PC1 in this lesson's example blends Math and Physics together, 0.6 parts Math-deviation and 0.8 parts Physics-deviation, and is a genuinely new feature that did not exist in the original data.

IMPORTANT

A principal component rarely has a clean, ready-made real-world meaning the way an original feature does. "Math score" is easy to explain to anyone. "0.6 times the Math deviation plus 0.8 times the Physics deviation" is much harder to explain, even though it is mathematically well defined. Before presenting principal components to someone outside your own team, be honest that they are computed blends, not natural, named quantities, and avoid overstating what any single component "means."

13.4 Explained Variance: How Much Did We Keep?

Every time PCA reduces a dataset's features down to a smaller number of principal components, some information is at risk of being left behind. This chapter's exam-score example turned out to lose nothing at all, an unusually tidy result, but any real project needs a precise, numeric way to check how much was actually kept, rather than simply hoping for the best.

WORD TO KNOW

Explained variance ratio — The share of a dataset's total variance that one principal component accounts for, found by dividing that component's own variance by the sum of every component's variance. A ratio close to 1, or 100%, means that component captures nearly all of the data's spread.

EQUATION — EXPLAINED VARIANCE RATIO

$$\text{explained variance ratio} = \frac{\text{variance of one component}}{\text{total variance of all components}}$$

In words: To find how much of the data's spread one principal component explains, divide that component's own variance by the sum of every principal component's variance added together.

explained variance ratio the share of the data's total spread that one component captures (no unit, often shown as a percentage)

variance of one component the variance found for that one principal component, from Lesson 13.2 or 13.3's method (*same unit as the features, squared*)

total variance every principal component's variance added together (*same unit as the features, squared*)

Note: Every dataset's explained variance ratios, added across all of its principal components, always sum to exactly 1, or 100%, because together the full set of components always recovers the data's entire spread.

WORKED EXAMPLE — FIND THE EXPLAINED VARIANCE RATIO FOR BOTH EXAM-SCORE COMPONENTS

Lesson 13.2 found PC1's variance to be 50. Lesson 13.3 found PC2's variance to be 0. Find the explained variance ratio for each component.

1. Find the total variance across both components: $50 + 0 = 50$.
2. Find PC1's explained variance ratio: $50 \div 50 = 1.00$, which is 100%.
3. Find PC2's explained variance ratio: $0 \div 50 = 0.00$, which is 0%.

Answer: PC1 explains 100% of the exam data's spread, and PC2 explains 0%. Keeping only PC1 loses no real information at all for this class, so each student's pair of exam scores could be replaced with a single PC1 number, with nothing meaningful thrown away.

A 100%-and-0% split is a rare, best-case outcome, produced here only because this chapter deliberately chose five students whose Math and Physics scores move together in perfect lockstep. A more realistic dataset spreads its variance more evenly across its components. Suppose an engineering team at a factory records two sensor readings on a single machine, a temperature deviation and a vibration deviation, both already centered. Running PCA on a full season of readings, the team finds a first principal component with variance 45, and a second principal component, forced to sit at a right angle to the first, with variance 5.

WORKED EXAMPLE — FIND THE EXPLAINED VARIANCE RATIO FOR THE FACTORY SENSOR DATA

A factory's two-sensor dataset has a first principal component with variance 45, and a second principal component with variance 5. Find the explained variance ratio for each component.

1. Find the total variance across both components: $45 + 5 = 50$.
2. Find PC1's explained variance ratio: $45 \div 50 = 0.90$, which is 90%.
3. Find PC2's explained variance ratio: $5 \div 50 = 0.10$, which is 10%.

Answer: PC1 explains 90% of the sensor data's spread, and PC2 explains the remaining 10%. Keeping only PC1 would lose a small but real 10% of the machine's true behavior, a realistic, everyday trade-off, quite different from the exam-score example's perfect 100%-and-0% split.

With more than two original features, a project usually keeps however many leading components are needed to reach a chosen target, often somewhere around 90% to 95% of the total variance, rather than insisting on keeping every single component.

WORKED EXAMPLE — DECIDE HOW MANY COMPONENTS TO KEEP FOR A FOUR-SUBJECT REPORT CARD

A university runs PCA on four subjects at once, Math, Physics, Chemistry, and Biology, producing four principal components with these explained variance ratios: PC1 = 60%, PC2 = 25%, PC3 = 10%, PC4 = 5%. Find the smallest number of components needed to reach at least 90% of the total variance.

1. Add PC1's ratio alone: 60%. This is below the 90% target, so one component is not enough.
2. Add PC1 and PC2 together: $60\% + 25\% = 85\%$. Still below the 90% target, so two components are not enough either.
3. Add PC1, PC2, and PC3 together: $85\% + 10\% = 95\%$. This has reached, and passed, the 90% target.

Answer: Keeping the first three components reaches 95% of the total variance, above the 90% target, so three components are enough. This reduces the report card from four original subject scores down to three new numbers per student, while keeping 95% of the real spread in the data.

GOING FURTHER — METHODS BUILT JUST FOR VISUALIZATION

Two other methods are common when the only goal is drawing a picture, rather than general-purpose dimensionality reduction. *t*-SNE looks for a 2D or 3D layout that keeps genuinely similar examples sitting close together, even if that means distorting the larger distances between less similar groups, which often produces strikingly clear, colorful cluster pictures. UMAP aims at a similar picture, is usually faster on large datasets, and tends to preserve a little more of the data's overall, large-scale shape alongside the local neighborhoods. Unlike PCA's principal components, the axes these two methods produce usually carry no reusable, independent meaning of their own, and are meant for looking at data, not for feeding into another model afterward.

COMMON MISTAKE

✗ Wrong: Insisting on keeping enough principal components to reach exactly 100% explained variance, no matter how many components that ends up requiring.

✓ Right: Choose a sensible target, often around 90% to 95%, and keep only as many leading components as needed to reach it. Requiring 100% keeps every original component, which defeats the entire purpose of dimensionality reduction.

The following short program repeats the exact same search as this chapter's worked examples, using scikit-learn.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[73, 69], [67, 61], [76, 73], [64, 57], [70, 65]])

pca = PCA(n_components=2)
pca.fit(X)

print(pca.components_)
print(pca.explained_variance_ratio_)
```

The `X` array stores the five students' raw Math and Physics scores. Calling `pca.fit(X)` centers the data automatically, exactly like the first worked example in Lesson 13.2, and then searches every possible direction for the greatest spread, exactly like the candidate-direction search that followed it. Running this code prints components very close to `[0.6, 0.8]` and `[0.8, -0.6]`, matching PC1 and PC2 by hand, though scikit-learn sometimes prints a direction with its sign flipped, such as `[-0.6, -0.8]` instead of `[0.6, 0.8]`; both point along the exact same line, just aimed in opposite directions, and represent the exact same principal component. The `explained_variance_ratio_` line prints values extremely close to `[1.0, 0.0]`, matching this lesson's hand-computed 100% and 0% exactly.

13.5 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab, then choose the "Report Card" dataset, which holds six subject scores for several hundred students.
2. Choose PCA as the method, and look at the bar chart of explained variance ratio for each component. Write down how many components it takes to pass 90%.
3. Switch the main view to a scatter plot of just PC1 and PC2, with points colored by each student's overall class rank. Describe, in one or two sentences, what pattern you notice.
4. Open the Labs tab and choose the PCA animation on a simple two-feature dataset. Watch the candidate direction rotate slowly around the data, and pause it at the exact moment the live variance number reaches its highest point.
5. Still in the Labs tab, switch the same dataset to the t-SNE view. Compare the resulting picture with the PCA scatter plot from step 3, and write one sentence about what looks different.

You need: Computer, tablet, or phone with the ML Studio app

13.6 Case Study

CASE STUDY — WATCHING A MACHINE'S HEALTH WITH TWO NUMBERS INSTEAD OF EIGHT, AT A FACTORY IN DA NANG

An electronics assembly plant in Da Nang runs a row of automated soldering machines around the clock. Each machine reports eight sensor readings every minute: temperature, vibration, motor current, humidity inside the housing, rotation speed, feed pressure, cooling-fan speed, and a wear sensor on the main belt. A maintenance engineer, Linh, is supposed to glance at these readings across all twelve machines on the floor and notice early signs of trouble, but eight numbers per machine, updated every minute, is simply too much for anyone to watch closely. Most days, the team only learns a machine was struggling once it has already stopped.

Linh's team decides to try Principal Component Analysis, aiming for a simple, glanceable health dashboard instead of a wall of raw numbers. The eight sensors arrive in very different units and ranges, degrees, millimeters per second, amps, percent humidity, and more, so the team standardizes every sensor first, exactly as this chapter's common mistake box warns, before searching for any principal component. Running PCA on several months of normal operating data, the team finds that the first two principal components together explain 93% of the sensors' total variance, comfortably above their chosen 90% target, so they keep just these two new numbers per machine, per minute, instead of the original eight.

Looking closely at the two components' own numbers, the team notices a loose, informal pattern. PC1 leans heavily on temperature, motor current, and rotation speed together, all rising and falling in step with how hard the machine is currently working, so the team nicknames it "overall load" for their own convenience. PC2 leans mostly on vibration and the belt-wear sensor, moving somewhat independently of load, so the team nicknames it "mechanical strain." Linh is careful to remind the newer technicians, exactly as this chapter's IMPORTANT box cautions, that these nicknames are a convenient shorthand the team invented for itself, not an official, guaranteed meaning baked into the mathematics; a future season of data could easily shift what each component leans on.

With just two numbers per machine, the team builds a single live scatter plot, PC1 across the bottom and PC2 up the side, with one dot per machine, refreshed every minute. Under normal conditions, all twelve machines' dots cluster together in a loose, familiar patch near the middle of the screen. When Machine 7 begins to wear out over the following weeks, its dot slowly drifts away from that familiar patch, days before any single one of its eight original readings crosses a limit strict enough to trigger the plant's old, one-sensor-at-a-time alarms. A technician catches the drift on the shared screen, inspects the belt by hand, and replaces it during a routine pause, avoiding an unplanned stoppage in the middle of a production run.

Linh's team does not treat this dashboard as the final word. A dot drifting away from the cluster is a prompt for a technician to look closer, never an automatic shutdown command, and the team retrains PCA every few months on fresh data, since a machine's own normal operating pattern can shift gradually as its parts

age. What began as an overwhelming wall of eight numbers per machine has become two numbers a busy engineer can genuinely watch, without losing the 93% of real information the team decided mattered most.

COMMON MISTAKES TO AVOID

- Deleting features that seem unimportant by hand, instead of letting PCA build carefully blended directions that keep information from every feature.
- Running PCA on raw, uncentered features, or on features that use very different scales without standardizing them first.
- Assuming a principal component is the same thing as one original feature, rather than a blend of several.
- Believing a later principal component, such as PC2, could point in nearly the same direction as an earlier one, instead of always sitting at a right angle to it.
- Insisting on keeping components until reaching exactly 100% explained variance, which quietly cancels out the whole benefit of dimensionality reduction.
- Presenting a principal component to someone outside the project as if it had one clean, obvious, guaranteed real-world meaning.

CHAPTER SUMMARY

- Dimensionality reduction shrinks the number of features used to describe each example, while trying to keep as much of the data's real information as possible.
- Principal Component Analysis, PCA, finds new directions through a centered dataset, each one built by blending the original features together.
- Projecting a centered point onto a direction multiplies matching numbers together and adds the results; squaring and averaging those projections gives the variance along that direction.
- The first principal component, PC1, is the single direction with the greatest possible variance; PCA finds it by searching every possible direction.
- Every later principal component must sit at a right angle to every component already found, so it only ever captures spread the earlier components missed.
- A principal component is a new, blended feature, not a copy of any single original column, and it often has no clean, ready-made real-world meaning.
- Explained variance ratio divides one component's variance by the total variance across every component; every dataset's ratios always add up to 100%.
- A project usually keeps only as many leading components as needed to reach a chosen target, often around 90% to 95%, not every component available.
- t-SNE and UMAP are related methods built specifically for visualization, favoring a clear picture of local groupings over components with a reusable, independent meaning.

- Chapter 14 turns to a different unsupervised question: not which few directions matter most, but which single examples look unusual enough to deserve a second look.

Practice Problems

Try each problem on paper first.

- Problem 1** [Basic] In your own words, explain what dimensionality reduction means.
- Problem 2** [Basic] Explain, in one or two sentences, why a dataset with many features is hard to plot and understand directly.
- Problem 3** [Basic] Define "principal component" in your own words.
- Problem 4** [Basic] Explain, in one or two sentences, why the second principal component must sit at a right angle to the first.
- Problem 5** [Basic] A dataset's three principal components have variances 40, 8, and 2. Without doing any division, state which component explains the most spread, and which explains the least.
- Problem 6** [Standard] Four students have raw Math scores of 82, 74, 90, and 78. Find the class mean, then find each student's centered score, showing every step.
- Problem 7** [Standard] A centered point is $(4, -2)$. Find its projection onto the direction $(1, 0)$, and separately onto the direction $(0, 1)$, showing every step.
- Problem 8** [Standard] Check whether $(0.6, -0.8)$ is a valid direction for this chapter's method, by confirming whether its two numbers, squared and added, equal 1.
- Problem 9** [Standard] A dataset's two principal components have variance 32 and variance 8. Find the explained variance ratio of each component, showing every step.
- Problem 10** [Standard] A dataset's three principal components have explained variance ratios of 70%, 20%, and 10%. Find the smallest number of components needed to reach at least 85% of the total variance.
- Problem 11** [Challenge] Three centered points are $(3, 6)$, $(-3, -6)$, and $(0, 0)$. (a) Find the variance along the direction $(1, 0)$, showing every step. (b) Find the variance along the direction $(0, 1)$, showing every step. (c) Without computing it directly, explain which direction, $(1, 0)$, $(0, 1)$, or a diagonal between them, is most likely to capture the greatest spread, and why.
- Problem 12** [Challenge] A hospital records twelve lab measurements per patient and wants to use PCA to help visualize patient similarity on a single 2D chart. (a) Explain why standardizing the twelve measurements first matters. (b) Explain, in your own words, why the resulting two axes might be hard to

explain to a doctor who has not studied PCA. (c) Suggest one safeguard the hospital should use before treating the chart as a basis for any real decision.

Problem 13 [Challenge] Open ML Studio, apply PCA to a dataset with at least five features, and record the explained variance ratio of each component. State how many components you would keep to reach 90% of the total variance, and explain, in a few sentences, one advantage and one limitation of reducing this dataset to just two components for a scatter plot.

UNSUPERVISED LEARNING

CHAPTER 14

Anomaly Detection

Finding the example that does not belong

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what an anomaly is, and how checking a whole example differs from checking one column for an outlier
- Compute the centroid of a set of normal examples and use distance from that centroid as a simple anomaly score
- Compute a z-score for a value, and explain what it reveals about how unusual that value is
- Combine several features' z-scores into one overall anomaly score
- Set and apply a threshold to turn a continuous anomaly score into a clear flagged-or-not decision
- Explain why a flagged example deserves a careful human review, never an automatic judgment

14.1 What Makes an Example Unusual?

Chapters 12 and 13 both studied a whole dataset at once. Chapter 12 grouped similar examples into clusters. Chapter 13 found the few directions that carried most of a dataset's real information. This chapter asks a smaller, sharper question. Forget the whole dataset for a moment, and look at just one example. Does it belong here at all?

Think about checking your family's electricity meter every month, for years. The reading climbs by a similar amount almost every month, a little higher in winter, a little lower in summer, but always inside a familiar range. Then one month the reading barely moves at all, or it jumps far higher than it has ever been before. You do not need a statistics course to notice that something is different about that one month. You already know your own normal pattern well enough to feel when a reading breaks it.

WORD TO KNOW

Anomaly — An example that looks noticeably different from most of the other examples in a dataset. An anomaly might point to a real problem, such as a broken sensor or a fraudulent charge, or it might simply be a rare, honest example. Finding an anomaly is only the first step; deciding why it is unusual takes a closer, human look.

Chapter 3 called one unusual value inside a single column an **outlier**. An anomaly is a related but bigger idea. An example can have every single feature sitting inside its normal range, and still be an anomaly, once you look at the

whole combination together. A 2-year-old child is not unusual. A height of 150 centimeters is not unusual, for a fully grown adult. But a 2-year-old child who is 150 centimeters tall would be highly unusual, even though neither fact alone raised any concern. Anomaly detection looks for exactly this kind of example: ordinary in isolation, strange in combination.

Real anomaly detection often has one more twist that makes it different from the classification you met back in Chapter 1. Chapter 1 imagined an online store with thousands of past orders, each one already labeled "fraudulent" or "genuine," ready to train a classifier. That labeled history is a luxury. A team monitoring a brand-new kind of sensor, or a bank watching for a brand-new kind of scam, often has plenty of normal examples but few or no confirmed examples of the problem itself, because the problem is exactly the thing nobody has seen enough of yet to label reliably. This chapter's methods do not need labels for the unusual class at all. They only need a good sense of what "normal" looks like, built from ordinary examples, and a way to measure how far a new example sits from that normal pattern.

WORKED EXAMPLE — SPOT THE UNUSUAL READING BY EYE

One household's smart electricity meter reports its daily usage, in kilowatt-hours (kWh), for five days: 42, 38, 45, 40, and 890. Which day's reading looks the most unusual, and why?

1. Compare the four smaller readings: 42, 38, 45, and 40. All four sit close together, each one within a few kWh of the others.
2. Compare the fifth reading, 890, against that group. It is far larger than any of the other four readings.
3. Find roughly how many times larger 890 is than the next-highest reading, 45: $890 \div 45 \approx 19.8$. This one day used almost 20 times more electricity than the household's next-highest day.
4. Notice that 890 does not fit the pattern the other four readings share, so it stands out immediately, with no formula needed at all.

Answer: The 890 kWh reading is the unusual one, standing far apart from the four ordinary readings near 40 kWh. With only five numbers, your eye alone is enough to spot it. The rest of this chapter builds a method that still works once a company must check millions of meter readings instead of five.

COMMON MISTAKE

✗ Wrong: Deciding an example is normal simply because every one of its individual features looks ordinary on its own, the way a height of 150 cm looks ordinary for an adult.

✓ Right: Check the combination of features together, not just each feature by itself. Lesson 14.2 builds a method that does exactly this, by comparing a whole example, every feature at once, to a "typical" example built from normal data.

Notice one more thing about the 890 kWh reading. Finding it does not, by itself, tell you why it happened. Perhaps a heater was left on by accident. Perhaps the meter itself is malfunctioning. Perhaps someone has illegally connected a business's equipment to a household line. An anomaly is a candidate worth checking, never an automatic verdict. Keep that idea close through the rest of this chapter, because every method here is built to answer only one question: how far does this example sit from normal, not what should be done about it.

14.2 A Simple Method: Distance from the Center

Lesson 14.1 spotted an anomaly by eye, using a single feature. Real monitoring almost always tracks several features at once, and a good anomaly detector needs a fair way to combine them. Chapter 1 already gave you half of exactly what is needed: the **centroid**, the average position of a group of points, found by averaging each feature separately. Chapter 8 gave you the other half:

Euclidean distance, the straight-line distance between two points, found by squaring each feature's difference, adding the squared differences together, and taking the square root of the sum. This lesson combines these two familiar tools into a new idea.

The plan is simple to say. Collect a set of examples you are confident are normal. Find their centroid. That centroid represents "typical" behavior, blended from every normal example you collected. Then, for any new example, measure its distance to that centroid. A small distance means the new example looks a lot like typical, normal behavior. A large distance means it does not.

WORD TO KNOW

Anomaly score — A number that measures how unusual one example looks, compared with normal data. A higher anomaly score means an example sits farther from typical behavior. This chapter builds anomaly scores from distance, though other methods, briefly named later in this chapter, build them in other ways.

EQUATION — ANOMALY SCORE BY DISTANCE FROM THE CENTROID

$$\text{anomaly score} = \text{distance}(\mathbf{x}, \text{centroid})$$

In words: One simple anomaly score for an example is just its Euclidean distance to the centroid of normal data: how far that example sits from the "typical" point, combining every feature together.

anomaly score how unusual one example looks, using this method (same unit as the features, when there is only one feature; otherwise no single natural unit)

x the example being checked (—)

centroid the average position of the normal training examples, one average per feature (same unit as the features)

Note: A larger anomaly score means an example sits farther from normal behavior. Deciding how large a score must be before it counts as "unusual enough" is the job of Lesson 14.3.

Let's build a running example for the rest of this chapter. An electricity distribution company tracks two numbers for every smart meter: daily usage, in kWh, and how far the meter's voltage strays from the standard 220-volt supply, called its voltage deviation, in volts. A small voltage deviation is completely normal; a large one can mean a wiring problem, aging equipment, or a tampered connection. Suppose the company studies six recent days from one household's meter, all confirmed normal by an inspector.

WORKED EXAMPLE — FIND THE CENTROID OF SIX NORMAL METER READINGS

One household's six most recent, confirmed-normal meter readings are: usage, in kWh: 40, 50, 60, 45, 55, 50. Voltage deviation, in volts: 2, 3, 4, 3, 3, 3. Find the centroid of these six readings.

1. Add the six usage values: $40 + 50 + 60 + 45 + 55 + 50 = 300$.
2. Divide by 6 to find the centroid's usage: $300 \div 6 = 50$.
3. Add the six voltage-deviation values: $2 + 3 + 4 + 3 + 3 + 3 = 18$.
4. Divide by 6 to find the centroid's voltage deviation: $18 \div 6 = 3$.

Answer: The centroid of this household's normal readings is (50 kWh, 3 V). This point is not any single real reading; it is the "typical day" this household's normal behavior centers around.

Now two new readings arrive. Reading P looks close to ordinary. Reading Q looks like it might be a problem.

WORKED EXAMPLE — COMPUTE THE ANOMALY SCORE OF TWO NEW METER READINGS

Using the centroid (50 kWh, 3 V) found above, find the anomaly score, the Euclidean distance to the centroid, for two new readings. Reading P: 54 kWh, 6 V deviation. Reading Q: 90 kWh, 12 V deviation.

1. Find Reading P's usage difference from the centroid: $54 - 50 = 4$. Square it: $4^2 = 16$.
2. Find Reading P's voltage-deviation difference from the centroid: $6 - 3 = 3$. Square it: $3^2 = 9$.
3. Add P's two squared differences, then take the square root: $16 + 9 = 25$, and $\sqrt{25} = 5$. Reading P's anomaly score is 5.
4. Find Reading Q's usage difference from the centroid: $90 - 50 = 40$. Square it: $40^2 = 1,600$.
5. Find Reading Q's voltage-deviation difference from the centroid: $12 - 3 = 9$. Square it: $9^2 = 81$.
6. Add Q's two squared differences, then take the square root: $1,600 + 81 = 1,681$, and $\sqrt{1,681} = 41$. Reading Q's anomaly score is 41.

Answer: Reading P has an anomaly score of 5, and Reading Q has an anomaly score of 41, more than eight times larger. Reading Q's usage is nearly double this household's normal 50 kWh, and its voltage deviation is four times the normal 3 V, and together those two mildly unusual numbers combine into a score that stands out clearly.

COMMON MISTAKE

✗ Wrong: Computing distance directly from raw features that use very different scales, such as usage in the tens of kWh sitting next to a voltage deviation of only a few volts, or the reverse, a feature measured in the thousands sitting next to one measured in single digits.

✓ Right: Whichever feature happens to use bigger raw numbers will quietly dominate a raw distance calculation, exactly the trap Chapter 8 warned about for k-Nearest Neighbors. Lesson 14.3 fixes this by putting every feature on a shared, fair scale before combining them.

The two readings above happened to use numbers close enough in size that this trap stayed hidden. Real features are rarely so cooperative. Lesson 14.3 reuses **standardization** to fix this properly, and gives the resulting, fairly-scaled distance a new name.

14.3 z-Scores and Setting a Threshold

Recall standardization's formula from Chapter 3: subtract a value's training mean, then divide by its training standard deviation. That one formula is about to earn a new name and a new job. Instead of using it only to prepare features before training a model, this lesson uses the very same number to directly answer a question: how unusual is this one value, all by itself?

WORD TO KNOW

z-score — How many standard deviations a value sits above or below the mean of normal data. A z-score of 0 means exactly average. A positive z-score means above average, and a negative z-score means below average, and the size of the number says how far.

EQUATION — Z-SCORE

$$z = \frac{x - \text{mean}}{\text{std}}$$

In words: A value's z-score equals the value itself, minus the mean of normal data for that feature, divided by the standard deviation of normal data for that feature.

z the z-score, how many standard deviations from average (no unit (a plain number))

x the value being checked (same unit as the feature)

mean the average value of this feature, across normal examples (same unit as the feature)

std the standard deviation of this feature, across normal examples (same unit as the feature)

Note: This is exactly Chapter 3's standardization formula. Standardization uses it to prepare features before training. This chapter uses the very same number as a ready-made anomaly score for one feature at a time.

Suppose the electricity company keeps a running mean and standard deviation for each meter, updated over many months, not just the six days from Lesson

14.2. For this household, the running mean usage is 50 kWh with a standard deviation of 10 kWh, and the running mean voltage deviation is 3 V with a standard deviation of 1 V.

WORKED EXAMPLE — FIND THE Z-SCORE OF TWO NEW USAGE READINGS

Using a training mean usage of 50 kWh and a training standard deviation of 10 kWh, find the z-score of a reading of 65 kWh, and separately, the z-score of a reading of 150 kWh.

1. Identify the three needed numbers for the first reading: $x = 65$, $\text{mean} = 50$, $\text{std} = 10$.
2. Subtract the mean from x : $65 - 50 = 15$.
3. Divide by the standard deviation: $15 \div 10 = 1.5$. The first reading's z-score is 1.5.
4. Identify the three needed numbers for the second reading: $x = 150$, $\text{mean} = 50$, $\text{std} = 10$.
5. Subtract the mean from x : $150 - 50 = 100$.
6. Divide by the standard deviation: $100 \div 10 = 10$. The second reading's z-score is 10.

Answer: The 65 kWh reading has a z-score of 1.5, only a little above average. The 150 kWh reading has a z-score of 10, ten full standard deviations above average, an extremely unusual value for this household's meter.

A z-score answers "how unusual," but a real system still has to make a decision: flag this reading for review, or let it pass quietly. That decision needs one more idea, a cutoff point separating the two outcomes.

WORD TO KNOW

Threshold — A cutoff value that turns a score into a decision. Anything at or beyond a threshold is treated one way; anything short of it is treated another way. This chapter uses a threshold on an anomaly score, but the same idea returns for very different kinds of scores later in this book.

A common starting convention treats any z-score of 3 or more, or -3 or less, as unusual enough to flag, since normal data rarely strays that far from its own mean. This convention is only a starting point, never a fixed law, and the IMPORTANT box below explains why.

IMPORTANT

A threshold is a choice, not a fact of nature. A strict threshold catches only the most extreme anomalies, but it can miss a real problem sitting just below the cutoff. A loose threshold catches more real problems, but it also flags many honest, merely unusual examples, wasting a team's limited time on false alarms. Choose a threshold by weighing which mistake costs more in your own situation, never by picking a number simply because it feels safe.

WORKED EXAMPLE — APPLY A THRESHOLD OF $|z| \geq 3$ TO FOUR METER READINGS

Using a training mean usage of 50 kWh and a training standard deviation of 10 kWh, find the z-score of four new readings, then flag any reading whose z-score is 3 or more, or -3 or less. Reading 1: 65 kWh. Reading 2: 150 kWh. Reading 3: 45 kWh. Reading 4: 18 kWh.

1. Find Reading 1's z-score: $(65 - 50) \div 10 = 15 \div 10 = 1.5$. Since 1.5 is between -3 and 3, Reading 1 is not flagged.
2. Find Reading 2's z-score: $(150 - 50) \div 10 = 100 \div 10 = 10$. Since 10 is beyond 3, Reading 2 is flagged.
3. Find Reading 3's z-score: $(45 - 50) \div 10 = -5 \div 10 = -0.5$. Since -0.5 is between -3 and 3, Reading 3 is not flagged.
4. Find Reading 4's z-score: $(18 - 50) \div 10 = -32 \div 10 = -3.2$. Since -3.2 is beyond -3 , Reading 4 is flagged.

Answer: Readings 2 and 4 are flagged; Readings 1 and 3 are not. Notice that Reading 4 was flagged for being unusually low, not high. An unusually low usage reading can mean a meter has been tampered with to under-report electricity use, so a good threshold must watch both directions, not only unusually large values.

COMMON MISTAKE

✗ Wrong: Comparing every meter to the same company-wide average usage, regardless of what kind of household or business owns it.

✓ Right: Compare each meter to its own normal history whenever enough history exists. A small apartment's normal usage is not a large workshop's normal usage, and judging both against one shared average would unfairly flag ordinary small households while missing real problems at heavy users.

COMMON MISTAKE

✗ Wrong: Treating a z-score threshold of 3 as a universal rule that must fit every dataset and every situation.

✓ Right: Treat 3 as a common starting convention, not a law. Chapter 16 returns to this exact question, choosing a threshold deliberately using how costly each kind of mistake is, rather than a single number borrowed from habit.

A single feature's z-score is useful, but Lesson 14.2's real strength was combining several features together. The same trick applies again here. Once every feature is standardized, the "typical" example's z-score is always 0 for every feature, since a value exactly equal to its own mean has a z-score of $(\text{mean} - \text{mean}) \div \text{std} = 0$. In other words, standardizing moves the centroid itself to (0, 0), one zero for every feature. Finding a distance to that centroid then becomes finding a distance to zero, using the exact same Euclidean-distance idea from Lesson 14.2, only now applied to z-scores instead of raw numbers.

WORKED EXAMPLE — COMBINE TWO FEATURES' Z-SCORES INTO ONE ANOMALY SCORE

Using a training mean usage of 50 kWh with a standard deviation of 10 kWh, and a training mean voltage deviation of 3 V with a standard deviation of 1 V, find the combined anomaly score for Reading R: 80 kWh with a voltage deviation of 7 V. The company flags any reading whose combined score is 4 or more.

1. Find Reading R's usage z-score: $(80 - 50) \div 10 = 30 \div 10 = 3$.
2. Find Reading R's voltage-deviation z-score: $(7 - 3) \div 1 = 4 \div 1 = 4$.
3. Square each z-score: $3^2 = 9$, and $4^2 = 16$.
4. Add the squared z-scores, then take the square root: $9 + 16 = 25$, and $\sqrt{25} = 5$.
5. Compare the combined score, 5, to the company's threshold, 4: 5 is greater than 4.

Answer: Reading R's combined anomaly score is 5, above the company's threshold of 4, so it is flagged for review. Neither raw number looked extreme on its own, but standardizing both features and combining them the way Lesson 14.2 combined raw features reveals a reading that truly does not match this household's normal pattern.

GOING FURTHER — OTHER WAYS TO SCORE AN ANOMALY

Distance from a single centroid works well when normal data forms one tidy cluster, but real data is not always so simple. An Isolation Forest scores an example by how few random yes-or-no splits it takes to separate that example from the rest of the data; anomalies typically isolate in very few splits, since they already stand apart. A Local Outlier Factor instead compares one example's local crowdedness to its neighbors' local crowdedness, catching an anomaly that sits inside a sparse pocket even when the overall dataset has several separate normal clusters, not just one. Both methods follow this chapter's same basic goal, a score that rises the less an example belongs, using a different rule to compute it.

The following short program repeats the same standardized, combined anomaly score as the worked example above, using scikit-learn.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.preprocessing import StandardScaler

normal = np.array([[40, 2], [50, 3], [60, 4], [45, 3], [55, 3], [50, 3]])
reading_r = np.array([[80, 7]])

scaler = StandardScaler().fit(normal)
z = scaler.transform(reading_r)

anomaly_score = np.sqrt((z ** 2).sum())
print(z, anomaly_score)
```

The ``normal`` array stores the six confirmed-normal readings from Lesson 14.2, and ``reading_r`` stores the new reading being checked.

``StandardScaler().fit(normal)`` learns the mean and standard deviation of each column from the normal readings alone, exactly the way Chapter 3 insists a scaler must be fit only on trustworthy data. Calling ``transform`` turns Reading R into its two z-scores, matching steps 1 and 2 of the worked example. Squaring, summing, and taking a square root repeats steps 3 and 4 exactly. Running this code prints z-scores of 3 and 4, and an anomaly score of 5, matching the hand calculation exactly.

14.4 Try It in ML Studio

Numbers on a page show you one meter, one reading, at a time. Watching hundreds of readings light up together, colored by anomaly score, makes it far easier to feel what a threshold is really trading off.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Meter Readings" dataset, which contains one dense cluster of ordinary readings and a handful of far-away points, and choose Anomaly Detection as the model.
3. Press Fit, and look at how the tool shades each point by its anomaly score, from calm blue for typical readings to warning red for extreme ones.
4. Drag the threshold slider from strict to loose, pressing Fit again at three or four different settings. Write down how many points get flagged at each one.
5. Choose the threshold you would actually recommend to a real monitoring team, and write one or two sentences explaining what you were trying to balance.

You need: Computer, tablet, or phone with the ML Studio app

14.5 Case Study

CASE STUDY — FLAGGING UNUSUAL SMART-METER READINGS FOR AN ELECTRICITY DISTRIBUTION COMPANY IN KIRKUK

Zainab works on the analytics team of a regional electricity distribution company serving Kirkuk and the surrounding towns. The company's smart meters report usage and voltage deviation every day, for hundreds of thousands of households and small businesses. A small fraction of these meters are genuinely faulty, and a smaller fraction still have been deliberately tampered with, bypassed to under-report usage or illegally shared with an unmetered neighbor. Sending an inspector to check every single meter is far too slow and far too expensive. Zainab's job is to help the company decide which meters most deserve a visit first.

Zainab follows this chapter's method closely, with one important refinement. Instead of comparing every meter to one company-wide average, she builds a

separate normal centroid for each meter, using that meter's own history from months confirmed problem-free. This matters because a small apartment and a busy workshop have very different honest "normal" usage, and comparing both against one shared average would bury the workshop's real problems while constantly, unfairly flagging ordinary apartments. For every new day's reading, Zainab standardizes usage and voltage deviation using that meter's own mean and standard deviation, then combines the two z-scores into one anomaly score, exactly as Lesson 14.3 showed.

Choosing a threshold turns out to be the hardest part of the whole project, and not a purely mathematical decision. A strict threshold sends inspectors only to the most extreme cases, but early testing shows it misses several meters the maintenance team already suspects are tampered with. A loose threshold catches those suspected meters, but it also flags hundreds of ordinary households, many of them simply hosting a large family celebration or running a new air conditioner for the first time that summer. Zainab's team settles on a middle threshold after talking with the inspection crew about how many visits they can realistically make in a week, then adds one more safeguard: every flagged meter gets a quick review by a senior analyst, who checks recent notes and nearby weather before any inspector is actually dispatched.

The finished system does not accuse anyone of anything. It produces a ranked list, most unusual first, and a short note explaining which feature drove each meter's score. An inspector who visits a flagged home still finds an honest explanation most of the time, a large family gathering, a new appliance, a summer heat wave, and closes the case with a friendly conversation instead of a penalty. For the smaller number of cases where the explanation is not innocent, the company now finds problems in weeks instead of the months it used to take, relying on customer complaints and chance discoveries alone.

IMPORTANT

A high anomaly score is a reason to look closer, never proof that something is wrong. A flagged meter, a flagged transaction, or a flagged medical test result all deserve a fast, respectful human review before any serious action, such as cutting off a service or shutting down a machine. Treat anomaly detection as a way to direct limited human attention efficiently, never as a replacement for human judgment.

COMMON MISTAKES TO AVOID

- Assuming every anomaly is automatically a mistake or a fraud, instead of a candidate that deserves a closer look.
- Deciding an example is normal because each single feature looks ordinary alone, without checking the whole combination together.
- Comparing raw distances across features on very different scales, letting one feature dominate an anomaly score.
- Comparing every example to one shared, global average, instead of a fair baseline built for each unit, meter, or customer.

- Treating a z-score threshold of 3 as a universal rule that fits every dataset and situation, instead of a starting convention.
- Treating a flagged example as proof of a problem, instead of a candidate for a respectful human review.

CHAPTER SUMMARY

- An anomaly is an example that looks noticeably different from most other examples, across its features taken together, not necessarily in any single column.
- A simple anomaly score can be built as the Euclidean distance from an example to the centroid of normal data.
- Comparing raw distances across features on different scales lets one feature dominate, so features are usually standardized before they are combined.
- A z-score tells how many standard deviations a value sits from the mean of normal data, reusing Chapter 3's standardization formula under a new name and a new job.
- A threshold turns a continuous score into a clear decision, flagged or not flagged; later chapters, including Chapter 16, reuse this same idea for a completely different kind of score.
- Standardizing every feature moves the centroid of normal data to zero, so combining several z-scores is just measuring distance from zero, the same way Lesson 14.2 measured distance from a centroid.
- Choosing a threshold means balancing missed problems against false alarms, and the right balance depends on how costly each kind of mistake is.
- An anomaly score is a reason to look closer, never proof of a problem on its own; a flagged example still needs a human review.
- Chapter 15 turns from finding unusual examples to building better features for every example a model sees.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define anomaly in your own words, and explain how it differs from an outlier, as Chapter 3 used that word.

Problem 2 [Basic] Five delivery times, in minutes, are 22, 25, 20, 24, and 95. Which one looks like the anomaly, and why?

Problem 3 [Basic] Explain, in one or two sentences, what a threshold does once a score has already been computed.

Problem 4 [Basic] A dataset's centroid is (10, 4). Explain in words, without computing anything, what this centroid represents.

Problem 5 [Standard] The centroid of a set of normal machine readings is (temperature 70, vibration 2). A new reading is (74, 5). Find its anomaly score, using Euclidean distance, showing every step.

Problem 6 [Standard] A feature has a training mean of 100 and a training standard deviation of 20. Find the z-score of a new value of 150, showing every step.

Problem 7 [Standard] Using a threshold rule of "flag if $|z| \geq 3$," decide whether each of these z-scores should be flagged: 1.2, -3.5 , 2.9, and 4.0.

Problem 8 [Standard] A bank transaction has a z-score of 3 for its amount and a z-score of 4 for its distance from the cardholder's home city. Find its combined anomaly score, using the same method as Lesson 14.3, showing every step.

Problem 9 [Standard] Explain, in your own words, why comparing raw distances between features on very different scales can be misleading for anomaly detection, and name the fix.

Problem 10 [Challenge] A hospital wants to flag unusual vital-sign readings for a quick nurse check. (a) Name two features you would use. (b) Explain why the "normal" centroid might need to be computed separately for different patient age groups, rather than one centroid for every patient. (c) Explain one risk of setting the threshold too strict, and one risk of setting it too loose.

Problem 11 [Challenge] A factory sets its anomaly threshold very strictly, flagging only the most extreme 0.1% of readings. (a) What kind of mistake does this choice risk making more often? (b) Suggest one situation where a strict threshold like this could still be the right choice, and explain why.

Problem 12 [Challenge] Open ML Studio, fit the anomaly detection tool on the "Meter Readings" dataset, and try at least three different threshold settings. Report how many points were flagged at each setting, and explain which setting you would recommend and why.

Part IV — Building Reliable Projects

- Chapter 15** — Feature Engineering and Selection
- Chapter 16** — Imbalanced Data and Useful Metrics
- Chapter 17** — Model Selection and Tuning
- Chapter 18** — Interpretability, Fairness, and Ethics
- Chapter 19** — From Notebook to Production

BUILDING RELIABLE PROJECTS

CHAPTER 15

Feature Engineering and Selection

Giving the model better questions to answer

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a well-engineered feature can matter more than a fancier model
- Create ratio, date-based, and interaction features from raw columns
- Recognize when a categorical column has too many values for one-hot encoding, and apply frequency encoding instead
- Build simple, useful features from a free-text column
- Select a smaller, more useful set of features, using feature importance and plain judgment together
- Explain how careless feature engineering can leak the label, or a sensitive attribute, into a model

15.1 A Good Feature Is Worth More Than a Fancy Model

Chapter 5 followed an agricultural researcher near Duhok, comparing three curves fitted to the very same rainfall and yield numbers. Every worked example in that chapter kept the columns themselves fixed, and only changed the model. This chapter finally changes the other half of the picture: the columns themselves.

Meet a new running example. The Zakho Farmers' Cooperative keeps careful yearly records for dozens of wheat fields: a planting date, a harvest date, total rainfall in millimeters, fertilizer used in kilograms, the village where the field sits, and each field's final yield, in tons per hectare. Once each season's records are complete, the cooperative's manager wants a model that predicts a field's yield accurately from these records, to help decide how much fertilizer to budget for and which villages most need irrigation investment the following year. A first attempt, a model trained directly on the raw columns exactly as they arrive, performs only weakly. The columns are not wrong, exactly. They are just not asking the model's favorite kind of question yet.

WORD TO KNOW

Feature engineering — Creating new features from a dataset's existing columns, using domain knowledge or simple arithmetic, so a model can find a useful pattern more easily. Good feature engineering can help a simple model perform as well as, or better than, a complex model working from raw features alone.

A model can only ever find patterns inside the exact numbers you hand it. If the real, useful pattern is a relationship between two raw columns, rather than

either column alone, a model may struggle to discover that relationship purely on its own, especially with a modest amount of data. Handing the model that relationship directly, already computed, is often far more effective than hoping a more complicated model will someday find it unaided.

WORKED EXAMPLE — SEE HOW A RATIO REVEALS A PATTERN RAW NUMBERS HIDE

Four wheat fields report rainfall, fertilizer, and yield. Field A: 300 mm rainfall, 30 kg fertilizer, yield 2.0 t/ha. Field B: 300 mm rainfall, 60 kg fertilizer, yield 1.2 t/ha. Field C: 600 mm rainfall, 60 kg fertilizer, yield 2.0 t/ha. Field D: 600 mm rainfall, 30 kg fertilizer, yield 2.8 t/ha. Find the ratio of rainfall to fertilizer for each field, and compare the four ratios to the four yields.

1. Find Field A's ratio: $300 \div 30 = 10$.
2. Find Field B's ratio: $300 \div 60 = 5$.
3. Find Field C's ratio: $600 \div 60 = 10$.
4. Find Field D's ratio: $600 \div 30 = 20$.
5. Compare the ratios to the yields. Field B has the smallest ratio, 5, and the smallest yield, 1.2. Field D has the largest ratio, 20, and the largest yield, 2.8. Fields A and C have completely different raw rainfall and fertilizer numbers, yet the exact same ratio, 10, and the exact same yield, 2.0.

Answer: The rainfall-to-fertilizer ratio lines up with yield far more cleanly than either raw column does alone. Field A and Field C look nothing alike in their raw numbers, but sharing the same ratio, they share the same yield, a pattern a model would need to work hard to discover from the two raw columns separately, but can use immediately once the ratio is handed to it directly.

COMMON MISTAKE

✗ Wrong: Reaching immediately for a more complex model when a project's accuracy disappoints, without first asking whether the features themselves carry enough information.

✓ Right: Check whether smarter, better-engineered features could reveal the pattern more directly first. A simple model with well-engineered features often beats a complex model working from raw, unhelpful features alone, and it is usually far easier to explain afterward too.

The rest of this chapter builds a small toolkit for exactly this kind of work: turning raw columns like the cooperative's planting dates, rainfall totals, and village names into features a model can actually use well, then trimming the result down to a smaller, stronger set.

15.2 Creating New Features from Old Ones

A raw date is one of the least useful columns a model ever receives. "2024-03-15" is not naturally bigger or smaller than "2024-08-15" in any way a model can use directly, even though the actual gap between two dates, the number of days, is often exactly what matters. The fix is simple: instead of handing a model the two raw dates, hand it the number of days between them.

WORKED EXAMPLE — ENGINEER A GROWING-SEASON-LENGTH FEATURE FROM TWO DATES

The Zakho Farmers' Cooperative records a planting date and a harvest date for each field, written here as a day number within the year. Field E was planted on day 74 (March 15) and harvested on day 227 (August 15). Field F was planted on day 91 (April 1) and harvested on day 208 (July 27). Engineer a new feature, growing-season length in days, for each field.

1. Find Field E's growing-season length: harvest day minus planting day: $227 - 74 = 153$ days.
2. Find Field F's growing-season length: harvest day minus planting day: $208 - 91 = 117$ days.

Answer: Field E grew for 153 days, and Field F grew for only 117 days, a difference of 36 days. Neither raw date alone showed this clearly; the engineered feature, days between planting and harvest, makes the real difference between the two fields immediately visible to a model, and to a person reading the data.

A raw date can also be split into simpler pieces, such as month of planting, or day of week for a retail order, whenever the calendar position itself, not just the gap between two dates, is likely to matter. A field planted in early spring behaves differently from a field planted a month late, even before a single drop of rain falls, so "planting month" can be a genuinely useful feature all on its own, sitting right alongside the growing-season length engineered above.

Ratios and date gaps each combine two features using one simple rule, division or subtraction. Some of the most useful engineered features instead capture how two features work together, in a way neither a ratio nor a simple difference can show.

WORD TO KNOW

Interaction feature — A new feature built by combining two or more existing features together, usually by multiplying them, so a model can see their combined effect directly, instead of only ever seeing each feature's separate effect.

EQUATION — INTERACTION FEATURE (TWO-WAY PRODUCT)

$$\text{interaction} = \text{feature}_1 \times \text{feature}_2$$

In words: A simple interaction feature multiplies two existing features together, producing one new number that is only large when both original features are large at the same time.

interaction the new, engineered feature (the product of the two features' units)

feature₁ the first raw feature (its own unit)

feature₂ the second raw feature (its own unit)

Note: An interaction feature can combine more than two features at once, though a two-way interaction, like this one, is by far the most common starting point.

WORKED EXAMPLE — DETECT AN INTERACTION BETWEEN RAINFALL AND FERTILIZER

Four more fields report rainfall, fertilizer, and yield. Field G: 300 mm rainfall, 20 kg fertilizer, yield 1.5 t/ha. Field H: 300 mm rainfall, 60 kg fertilizer, yield 1.6 t/ha. Field I: 600 mm rainfall, 20 kg fertilizer, yield 2.0 t/ha. Field J: 600 mm rainfall, 60 kg fertilizer, yield 3.5 t/ha. Find how much extra fertilizer, from 20 kg to 60 kg, changes yield at each rainfall level, then build an interaction feature that captures the difference.

1. At low rainfall, 300 mm, find the yield change from low to high fertilizer: Field H's yield minus Field G's yield: $1.6 - 1.5 = 0.1$ t/ha.
2. At high rainfall, 600 mm, find the same yield change from low to high fertilizer: Field J's yield minus Field I's yield: $3.5 - 2.0 = 1.5$ t/ha.
3. Compare the two changes: extra fertilizer added only 0.1 t/ha at low rainfall, but 1.5 t/ha at high rainfall, fifteen times more benefit from the exact same extra 40 kg of fertilizer.
4. Build an interaction feature by multiplying rainfall and fertilizer together for each field: Field G: $300 \times 20 = 6,000$. Field H: $300 \times 60 = 18,000$. Field I: $600 \times 20 = 12,000$. Field J: $600 \times 60 = 36,000$.
5. Notice that Field J, the only field with both rainfall and fertilizer high at the same time, has by far the largest interaction value, 36,000, matching its standout yield of 3.5 t/ha.

Answer: Fertilizer's benefit depends heavily on rainfall, a classic interaction effect: the same extra fertilizer helped fifteen times more when rainfall was already high. Multiplying rainfall and fertilizer into one interaction feature hands a model this combined effect directly, instead of hoping it discovers the connection on its own from two separate columns.

The following short program builds the same ratio and interaction features shown above, using the pandas library, which stores a table of data as a structure called a DataFrame.

Python · NumPy

 runnable in the app

```
import pandas as pd

fields = pd.DataFrame({
    "rainfall_mm": [300, 300, 600, 600],
    "fertilizer_kg": [20, 60, 20, 60],
    "yield_t_ha": [1.5, 1.6, 2.0, 3.5],
})

fields["rainfall_fertilizer_ratio"] = fields["rainfall_mm"] /
fields["fertilizer_kg"]
fields["rainfall_fertilizer_interaction"] = fields["rainfall_mm"] *
fields["fertilizer_kg"]

print(fields)
```

The ``pd.DataFrame`` line stores Fields G through J as one small table, with one column per raw measurement. The next two lines each add a brand-new column: dividing two existing columns builds the ratio feature, and multiplying them builds the interaction feature, exactly matching this lesson's hand arithmetic. Running this code prints an interaction value of 36,000 for Field J, matching step 4 of the worked example above.

COMMON MISTAKE

✗ Wrong: Engineering a feature using information that would not actually be available yet at the moment a real prediction must be made, such as a field's final total-season rainfall when the prediction must be ready before the season ends.

✓ Right: Ask exactly when the prediction must be made, then build every feature only from data that would genuinely be available by that moment. This is the same **data leakage** trap Chapter 2 warned about, now wearing a new disguise inside feature engineering itself.

15.3 Encoding Text and High-Cardinality Categories, Briefly

Chapter 3 introduced **one-hot encoding** for a categorical column with no natural order, and set text columns aside for a later chapter. This lesson returns to both, briefly, starting with a problem one-hot encoding handles badly.

The cooperative's records include a Village column, naming which of fifty different villages each field sits in. One-hot encoding would turn this single column into fifty new columns, almost all of them holding a 0 for almost every row. A column like this, with a very large number of possible categories, needs a different kind of care.

WORKED EXAMPLE — FREQUENCY-ENCODE A HIGH-CARDINALITY VILLAGE COLUMN

The Village column has fifty different names. "Zakho Center" appears 40 times in the training data, "Faysh Khabur" appears 25 times, "Batifa" appears 20 times, and 15 other villages each appear only once. Instead of one-hot encoding, use frequency encoding: replace each row's village name with how many times that name appears in the training data. Find the frequency-encoded value for a "Zakho Center" row, a "Batifa" row, and a row from one of the 15 rare villages.

1. Count how many times "Zakho Center" appears: 40 times, so every "Zakho Center" row becomes 40.
2. Count how many times "Batifa" appears: 20 times, so every "Batifa" row becomes 20.
3. Count how many times one of the 15 rare villages appears: only 1 time each, so each of those rows becomes 1.

Answer: Frequency encoding turns "Zakho Center" into 40, "Batifa" into 20, and each rare village into 1, using a single numeric column instead of fifty separate one-hot columns. Villages the model has barely seen

automatically end up with small, similar numbers, which is usually a more honest signal than fifty mostly-empty one-hot columns.

A second, even simpler fix works well when only a handful of categories truly matter: keep the few common categories by name, and lump every rare category together into a single new value, often written "Other." A third method, target encoding, replaces each category with the average label value seen for that category in the training data. Target encoding can be powerful, but it must be built with real care, using only training rows the model itself never sees during its own training pass, or the encoding quietly hands the model a shortcut straight to the answer.

COMMON MISTAKE

✗ Wrong: Building a target encoding by computing each category's average label using the very same rows the model will then train on.

✓ Right: Compute a target encoding using a separate fold of data, or a careful cross-validation scheme, so no row ever sees its own label smuggled back to it through the encoded category. Treat target encoding as a data leakage risk first, and a convenient shortcut second.

IMPORTANT

A high-cardinality category such as a village name, a postal code, or a customer ID can quietly stand in for something more sensitive, such as ethnicity, religion, or income level, even when nobody intended that. Chapter 18 studies this risk in depth, under the name proxy variable. For now, before adding a feature like this to a sensitive model, pause and ask what it might really be measuring underneath its surface meaning.

The cooperative's records also include one free-text column: a short note an agronomist writes after each visit, such as "Healthy stand, slight drought stress in the eastern corner." A model cannot read a sentence directly, but a few simple, engineered features can capture some of its value without any advanced text processing at all: the length of the note, in words, since a longer note often means the agronomist noticed more worth writing down, and whether the note contains a specific keyword, such as "drought," "pest," or "flood," recorded as a simple 0-or-1 column for each keyword worth tracking. These small, easy features rarely capture everything a skilled agronomist meant, but they are far better than ignoring the column completely, and they take only a few minutes to build.

WORKED EXAMPLE — BUILD TWO SIMPLE FEATURES FROM AN AGRONOMIST'S NOTES

Two field-visit notes: Note 1: "Healthy stand, slight drought stress in the eastern corner." Note 2: "Strong growth across the whole field, no concerns this visit." For each note, find its word count, and whether it contains the keyword "drought," recorded as 1 for yes or 0 for no.

1. Count Note 1's words one at a time: Healthy, stand, slight, drought, stress, in, the, eastern, corner. That is 9 words.

2. Count Note 2's words one at a time: Strong, growth, across, the, whole, field, no, concerns, this, visit. That is 10 words.
3. Check Note 1 for the keyword "drought": it appears once, so Note 1's drought flag is 1.
4. Check Note 2 for the keyword "drought": it does not appear, so Note 2's drought flag is 0.

Answer: Note 1 becomes (9 words, drought flag 1), and Note 2 becomes (10 words, drought flag 0). Neither number captures everything the agronomist meant, but the drought flag alone already hands a model a genuinely useful early warning it did not have before, straight from a column it previously could not read at all.

15.4 Selecting Fewer, Better Features

By this point, the cooperative's dataset has grown, not shrunk. Alongside the original raw columns, it now includes a rainfall-to-fertilizer ratio, a growing-season length, a rainfall-fertilizer interaction, a planting month, a frequency-encoded village, and a couple of simple text-based features. Chapter 8 warned that too many features can make every example look almost equally far from every other example, weakening a distance-based method like k-Nearest Neighbors. Chapter 10 showed how **feature importance** can reveal which features an ensemble of trees actually leaned on. This lesson puts both warnings to work, trimming a wide feature set back down to a smaller, stronger one.

WORD TO KNOW

Feature selection — Choosing a smaller, more useful subset of a dataset's features to actually give a model, instead of using every feature that happens to be available. Good feature selection can reduce overfitting risk and make a model both faster to train and easier to understand.

One of the simplest, most reliable ways to select features reuses a tool you already know: train a tree-based ensemble, read its feature importance scores, and drop the features that barely contributed.

WORKED EXAMPLE — SELECT FEATURES USING IMPORTANCE SCORES

A random forest trained to predict wheat yield for the Zakho Farmers' Cooperative reports these feature importance scores, using Chapter 10's method: Rainfall 45%, Fertilizer 30%, Growing-season length 20%, Farmer's shoe size 3%, Number of tractors owned 2%. Using a rule of keeping only features with importance of at least 5%, decide which features to keep and which to drop.

1. Compare Rainfall's importance, 45%, to the 5% rule: 45% is well above 5%, so keep Rainfall.
2. Compare Fertilizer's importance, 30%, to the rule: 30% is above 5%, so keep Fertilizer.

3. Compare Growing-season length's importance, 20%, to the rule: 20% is above 5%, so keep Growing-season length.
4. Compare Farmer's shoe size, 3%, to the rule: 3% is below 5%, so drop it.
5. Compare Number of tractors owned, 2%, to the rule: 2% is below 5%, so drop it.

Answer: Keep Rainfall, Fertilizer, and Growing-season length; drop Farmer's shoe size and Number of tractors owned. A farmer's shoe size was almost certainly recorded by accident, alongside genuinely useful survey questions, and feature selection is exactly how a team catches and removes a useless column like this before it adds noise to a model.

Feature importance is not the only useful filter. Two features that carry almost identical information, such as total rainfall and average monthly rainfall for a season of fairly steady rain, rarely both deserve a place in the final model; keeping just one usually loses little and keeps the model easier to read. You may also meet more automatic feature-selection tools elsewhere, with names such as recursive feature elimination, which repeatedly trains a model and drops its weakest feature, one round at a time. These tools follow the same goal as this lesson, a smaller, stronger feature set, using a more systematic search than checking one importance table by eye.

Trimming a feature set is not just about tidiness. Chapter 5 showed that extra, unearned capacity tends to widen the gap between training error and validation error. Removing unhelpful features is one of the simplest ways to pull a model back from that overfit edge.

WORKED EXAMPLE — COMPARE TRAINING AND VALIDATION ERROR BEFORE AND AFTER FEATURE SELECTION

A random forest trained on all 12 available columns, including several unhelpful ones like farmer's shoe size, reaches a training error of 0.05 t/ha and a validation error of 0.35 t/ha. The same model, retrained using only the 3 features feature selection kept, reaches a training error of 0.12 t/ha and a validation error of 0.18 t/ha. Find the training-validation gap for each version, and decide which model truly generalizes better.

1. Find the 12-feature model's gap: validation error minus training error: $0.35 - 0.05 = 0.30$ t/ha.
2. Find the 3-feature model's gap: $0.18 - 0.12 = 0.06$ t/ha.
3. Compare the two gaps: 0.30 t/ha is five times larger than 0.06 t/ha, a clear sign the 12-feature model was fitting noise that the 3-feature model was not.
4. Compare the two validation errors directly, since validation error is what actually predicts how the model will perform on next year's fields: 0.35 t/ha versus 0.18 t/ha.

Answer: The 3-feature model shows a higher training error but a much lower validation error, 0.18 t/ha against 0.35 t/ha, and a far smaller training-validation gap. Despite looking less impressive on the data it

trained on, it is the model that will actually serve the cooperative better, exactly the pattern Chapter 5 taught you to look for.

COMMON MISTAKE

✗ Wrong: Assuming more features can only help a model, since extra information seems like it should never hurt.

✓ Right: Extra, unhelpful features add noise and give a flexible model more ways to fit accidents in the training data, raising the risk of **overfitting**. Chapter 5's whole warning about model capacity applies here too: more features usually means more capacity, whether or not that capacity is actually earned.

15.5 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Farm Yield" dataset, and choose Random Forest as the model.
3. Press Fit using only the raw features, and write down the validation score shown on screen.
4. Turn on the "add engineered features" toggle, which adds ratio, date, and interaction features like the ones in this chapter, and press Fit again. Write down the new validation score.
5. Open the feature importance chart, and check whether any of the new engineered features rank among the most important. Write one or two sentences about what you notice.

You need: Computer, tablet, or phone with the ML Studio app

15.6 Case Study

CASE STUDY — ENGINEERING BETTER FEATURES FOR WHEAT YIELD ON A FARM COOPERATIVE NEAR ZAKHO

Diyar joined the Zakho Farmers' Cooperative last year as its first data analyst. The cooperative's records stretch back a decade: planting and harvest dates, monthly rainfall, fertilizer use, the village of each field, an agronomist's short visit notes, and each field's final yield. Diyar's first model, a random forest trained directly on these raw columns, barely improves on a plain baseline that simply predicts each village's average yield from past years. The cooperative's board is not impressed, and Diyar suspects the problem is not the model at all. Working through this chapter's toolkit, Diyar builds a handful of new features: growing-season length, from the planting and harvest dates; a rainfall-to-fertilizer ratio; a rainfall-fertilizer interaction feature; and planting month, split out from the raw planting date on its own. Retrained on the same raw columns plus these five new ones, the random forest's validation error falls from about 0.6 tons per hectare to about 0.4 tons per hectare, nearly a third lower, without changing the model itself at all. The growing-season length and the rainfall-

fertilizer interaction turn out to matter most, exactly the kind of relationship the raw columns alone had been hiding from the model.

Diyar then turns to the two columns still sitting untouched: Village, with fifty different names, and the agronomist's free-text notes. Frequency encoding replaces Village with a single, honest number, how often each village appears in the training data, instead of fifty mostly-empty one-hot columns. From the notes, Diyar builds two small features, note length and a "mentions drought" flag, without attempting anything more elaborate. A pass of feature selection, guided by the retrained forest's importance scores, then quietly drops three columns that were adding noise without adding insight: raw planting date, once growing-season length and planting month already capture what mattered about it, and, to the board's amusement, a long-forgotten "farmer's shoe size" column that had been sitting in the spreadsheet since the cooperative's very first paper survey.

Before presenting the finished model, Diyar checks one more thing: whether any engineered feature secretly depends on information that would not really have been known at the time a past decision was made, the same leakage check this chapter has raised twice already. Satisfied that every feature is honest, Diyar hands the board a shorter, stronger feature list than the one the project started with, together with a plain note that the model is a planning aid for next year's fertilizer and irrigation budget, not a guarantee for any single field.

Diyar also adds one habit to the cooperative's yearly calendar: revisiting the whole feature list every season, not just the model itself. A village can grow past fifty fields, a new fertilizer product can enter the local market, and a feature that mattered this year can quietly stop mattering the next. Treating the feature list as a living part of the project, not a decision made once and forgotten, turns out to matter almost as much as building it carefully in the first place.

IMPORTANT

An engineered feature is not automatically neutral just because it started as simple arithmetic. A ratio, a date gap, or a frequency count still needs the same honest checks as any other feature: was it truly available at the right time, and could it be quietly standing in for something sensitive it was never meant to measure? Treat every engineered feature as a new feature earning its place, not as a shortcut around this book's earlier warnings.

COMMON MISTAKES TO AVOID

- Reaching for a more complex model before checking whether better features could reveal the pattern more directly.
- Engineering a feature from information that would not actually be available yet at the moment a real prediction must be made.
- One-hot encoding a high-cardinality category into hundreds of near-empty columns, instead of using frequency encoding or grouping rare values together.

- Building a target encoding from the very same rows a model will then train on, risking a serious form of data leakage.
- Assuming every additional feature can only help a model, without checking whether it adds real information or just noise.
- Adding a feature built from a high-cardinality identifier, such as a village name, without asking what it might quietly stand in for.

CHAPTER SUMMARY

- Feature engineering creates new, more useful features from a dataset's existing raw columns, using domain knowledge and simple arithmetic.
- A ratio feature can reveal a pattern that neither of its two raw features shows clearly on its own.
- Date columns rarely help a model directly; a derived feature such as the number of days between two dates is usually far more useful.
- An interaction feature, usually built by multiplying two features together, hands a model their combined effect directly, instead of hoping the model discovers it alone.
- A high-cardinality category can overwhelm one-hot encoding with hundreds of near-empty columns; frequency encoding or grouping rare categories is often a better fit.
- Simple features built from a text column, such as its length or whether it contains a keyword, can be useful without any advanced text processing.
- Feature selection removes unhelpful features using tools such as feature importance, reducing overfitting risk and making a model easier to understand.
- Careless feature engineering can leak the future, the label, or a sensitive attribute into a model without anyone intending it.
- Chapter 16 turns to a different everyday challenge: what to do when the outcome you care about most is also the rarest one in the data.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Explain, in your own words, what feature engineering means.

Problem 2 [Basic] A dataset has separate columns for a shipment's weight in kilograms and its volume in cubic meters. Suggest one ratio feature that could be engineered from these two columns, and explain what it would measure.

Problem 3 [Basic] A dataset has a column for order date and a column for delivery date. Name one useful engineered feature these two columns could produce together.

Problem 4 [Basic] Explain, in one or two sentences, why one-hot encoding a column with 300 different city names could cause problems.

Problem 5 [Standard] Four fields report rainfall and fertilizer: Field K (rainfall 200 mm, fertilizer 10 kg), Field L (rainfall 400 mm, fertilizer 20 kg), Field M (rainfall 400 mm, fertilizer 10 kg), Field N (rainfall 800 mm, fertilizer 20 kg). Find the rainfall-to-fertilizer ratio for each field, showing every step, and identify any fields that share the same ratio.

Problem 6 [Standard] A shipment was placed on day 40 of the year and delivered on day 47. A second shipment was placed on day 100 and delivered on day 119. Engineer a "delivery time in days" feature for each shipment, showing every step.

Problem 7 [Standard] Build an interaction feature by multiplying these two raw features together: temperature = 30, humidity = 80. Show the calculation, and explain in one sentence what an interaction feature is meant to capture.

Problem 8 [Standard] A payment-method column has these training counts: "Cash" appears 500 times, "Card" appears 450 times, and 50 other rare payment types each appear once. Find the frequency-encoded value for a "Cash" row and for one rare row, showing every step.

Problem 9 [Standard] A model reports these feature importance scores: Location 40%, Price 35%, Store Size 20%, Day of Week 5%. Using a rule of keeping only features at or above 10%, decide which features to keep and which to drop.

Problem 10 [Challenge] A team predicting customer churn engineers a new feature, "total number of complaints this customer has ever filed," computed using the customer's entire history, including complaints filed after the churn decision already happened. (a) Explain what is wrong with this feature. (b) Suggest a corrected version of the same feature that would be safe to use.

Problem 11 [Challenge] A retail dataset has a "Product ID" column with 4,000 unique values. (a) Explain why one-hot encoding this column directly would be a poor choice. (b) Describe two different fixes this chapter suggests, and explain one advantage of each.

Problem 12 [Challenge] A hiring dataset includes an engineered feature, "distance from the office to the candidate's home address." (a) Explain one legitimate reason this feature might help predict an outcome like on-time attendance. (b) Explain one way this same feature could quietly act as a proxy for something the team never intended to use, such as economic background.

Problem 13 [Challenge] Open ML Studio, fit a model on the "Farm Yield" dataset with only its raw features, then again with engineered features turned on. Report the validation score both times, and describe, in a few sentences, which engineered feature seemed to matter most, using the feature importance chart.

BUILDING RELIABLE PROJECTS

CHAPTER 16

Imbalanced Data and Useful Metrics

When the rare case is the one that matters

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why accuracy can be a misleading metric when one class is far rarer than the other
- Define class imbalance, and calculate the imbalance ratio of a dataset
- Compare oversampling, undersampling, and SMOTE as ways to rebalance a training set
- Use class weights to make a rare class count for more during training, without changing the data itself
- Move a classifier's decision threshold to trade precision for recall, or recall for precision
- Check whether a model's predicted probabilities are well calibrated, and explain why that matters

16.1 When 99% Accuracy Means Nothing

A hospital network in Chengdu, China, wants to screen patients for a rare disease, using a simple, low-cost blood test. A laboratory machine reads each blood sample and reports several chemical measurements. A trained model studies those measurements and predicts whether the patient is likely to have the disease, so that only patients with a worrying prediction need to see a specialist for a slower, more certain test. Most people who walk into the clinic do not have this disease. For every 10,000 people screened, records from past seasons suggest that only about 100 of them, 1%, truly have it. These numbers are only an illustration, not a real medical statistic, but the pattern is extremely common in real projects. That one fact, that the disease is rare, matters more than almost anything else in this chapter.

Chapter 1 introduced **classification**, and Chapter 4 introduced **accuracy** as one way to grade a classifier: the fraction of predictions that are exactly correct. Both of those chapters mostly used examples where the two classes were reasonably close in size. Real projects are often not so tidy.

WORD TO KNOW

Class imbalance — A situation where one class in a classification dataset has far more examples than another class. The common class is called the majority class, and the rare class is called the minority class.

EQUATION — CLASS IMBALANCE RATIO

$$\text{imbalance ratio} = \frac{\text{majority class count}}{\text{minority class count}}$$

In words: The imbalance ratio compares the number of examples in the larger class to the number of examples in the smaller class, showing how rare the minority class really is.

majority class count number of examples in the more common class (count)

minority class count number of examples in the rarer class (count)

Note: A ratio written as 99-to-1 means the majority class has 99 times as many examples as the minority class.

WORKED EXAMPLE — FIND THE IMBALANCE RATIO OF THE DISEASE-SCREENING DATASET

Out of 10,000 patients screened, 100 truly have the disease, and 9,900 do not. Find the imbalance ratio.

1. Identify the majority class count. Patients without the disease are the majority class: 9,900.
2. Identify the minority class count. Patients with the disease are the minority class: 100.
3. Divide the majority class count by the minority class count: $9,900 \div 100 = 99$.

Answer: The imbalance ratio is 99-to-1. For every 1 patient who truly has the disease, there are 99 patients who do not, which is exactly why a model that ignores the disease entirely can still look accurate almost all of the time.

Suppose a new intern builds the laziest possible model. It never looks at a single measurement. It simply predicts "no disease" for every patient, every time, no matter what their blood test shows. How good does this lazy model look, judged only by accuracy?

WORKED EXAMPLE — COMPUTE THE ACCURACY OF A MODEL THAT ALWAYS PREDICTS 'NO DISEASE'

A lazy model predicts "no disease" for all 10,000 patients in the screening dataset, where 100 patients truly have the disease and 9,900 do not. Using "has the disease" as the positive class, find TP, FN, FP, and TN, then find the model's accuracy.

1. Find true positives (TP): patients truly positive who were also predicted positive. The model never predicts positive, so $TP = 0$.
2. Find false negatives (FN): patients truly positive who were predicted negative. All 100 truly positive patients were predicted negative, so $FN = 100$.
3. Find false positives (FP): patients truly negative who were predicted positive. The model never predicts positive, so $FP = 0$.
4. Find true negatives (TN): patients truly negative who were predicted negative. All 9,900 truly negative patients were predicted negative, so $TN = 9,900$.

5. Check the total: $TP + FN + FP + TN = 0 + 100 + 0 + 9,900 = 10,000$, matching the 10,000 patients screened.
6. Find accuracy: correct predictions divided by total predictions. Correct predictions = $TP + TN = 0 + 9,900 = 9,900$. Accuracy = $9,900 \div 10,000 = 0.99$.
7. Convert to a percentage: $0.99 \times 100 = 99$.

Answer: The lazy model reaches 99% accuracy, while catching exactly 0 of the 100 patients who truly have the disease. A number that sounds almost perfect is quietly hiding a model that is completely useless for the one job it exists to do.

This lazy model's **recall**, the fraction of truly positive patients it actually finds, is just as easy to check: $0 \div 100 = 0$, or 0%. A useful screening test must catch sick patients, not merely avoid bothering healthy ones, and this lazy model fails at the one job that matters most.

A genuinely trained model, one that actually studies the blood measurements, does far better at finding sick patients, though it is still not perfect. Suppose this real model is tested on the same 10,000 patients.

WORKED EXAMPLE — COMPUTE THE ACCURACY OF A GENUINELY USEFUL SCREENING MODEL

A real, trained model is tested on the same 10,000 patients. It correctly flags 80 of the 100 truly positive patients ($TP = 80$), missing the other 20 ($FN = 20$). It also wrongly flags 300 healthy patients as positive ($FP = 300$), while correctly clearing 9,600 healthy patients ($TN = 9,600$). Find this model's accuracy, recall, and precision.

1. Check the total: $TP + FN + FP + TN = 80 + 20 + 300 + 9,600 = 10,000$, matching the 10,000 patients screened.
2. Find accuracy: correct predictions divided by total. Correct predictions = $TP + TN = 80 + 9,600 = 9,680$. Accuracy = $9,680 \div 10,000 = 0.968$, or 96.8%.
3. Find recall: $TP \div (TP + FN) = 80 \div (80 + 20) = 80 \div 100 = 0.8$, or 80%.
4. Find precision: $TP \div (TP + FP) = 80 \div (80 + 300) = 80 \div 380 \approx 0.211$, or about 21.1%.

Answer: The real model's accuracy, 96.8%, is actually lower than the lazy model's 99%. Yet the real model catches 80% of true disease cases, while the lazy model catches none at all. Accuracy alone would rank the useless lazy model above the genuinely useful one, which is exactly why this chapter looks past accuracy whenever one class is this rare.

COMMON MISTAKE

✗ Wrong: Judging a classifier by accuracy alone, especially when one class is much rarer than the other.

✓ Right: On an imbalanced dataset, check **recall**, **precision**, and **F1 score** alongside accuracy, and always ask how many of the rare, important cases the model actually caught.

The following short program checks the lazy model's accuracy and recall using scikit-learn, on the same 10,000 patients used above.

Python · NumPy

 runnable in the app

```
import numpy as np
from sklearn.metrics import accuracy_score, recall_score

y_true = np.array([1] * 100 + [0] * 9900)
y_lazy = np.array([0] * 10000)

print("Lazy accuracy:", accuracy_score(y_true, y_lazy))
print("Lazy recall:", recall_score(y_true, y_lazy))
```

The array `y_true` stores 100 ones for the truly positive patients, followed by 9,900 zeros, matching the dataset exactly. The array `y_lazy` stores 10,000 zeros, since the lazy model always predicts "no disease." The two `print` lines repeat the accuracy and recall calculations from above, one line each. Running this code prints 0.99 and 0.0, the same numbers already found by hand.

Accuracy failed here because the training data itself, like the real world it was drawn from, is overwhelmingly one class. Lesson 16.2 attacks this problem at its root, by changing the balance of the training data before the model ever sees it.

16.2 Resampling: Oversampling and Undersampling

The 10,000 patients in Lesson 16.1 were used to test the finished model. Now rewind to before training even began. The hospital's data science team has a training set of 1,000 historical patients: 50 who truly had the disease, and 950 who did not, the same kind of imbalance seen throughout this chapter, just at a smaller scale. Two data-level strategies can change this training set's balance before a single model is trained.

WORD TO KNOW

Oversampling — Adding more copies, or new similar examples, of the minority class to a training set, so it makes up a larger share of the data the model learns from.

WORKED EXAMPLE — OVERSAMPLE THE MINORITY CLASS TO MATCH THE MAJORITY CLASS

The training set has 50 positive patients and 950 negative patients, 1,000 total. Oversample the positive class, by duplicating positive examples, until it is exactly as large as the negative class. Find the new size of the positive class, and the new total training set size.

1. Start with the current class counts: 50 positive, 950 negative.
2. Find the target size for the positive class: as large as the negative class, so 950.

3. Find how many copies of each positive example are needed: divide the target size by the current size, $950 \div 50 = 19$.
4. Multiply the original 50 positive examples by 19 copies each: $50 \times 19 = 950$.
5. Add the new positive count to the unchanged negative count: $950 + 950 = 1,900$.

Answer: After oversampling, the training set has 950 positive examples and 950 negative examples, 1,900 rows in total. Every original positive patient's record now appears 19 times, so the model can no longer shortcut its way to a good score by ignoring the disease.

WORD TO KNOW

Undersampling — Removing examples from the majority class in a training set, so it makes up a smaller share of the data the model learns from.

WORKED EXAMPLE — UNDERSAMPLE THE MAJORITY CLASS TO MATCH THE MINORITY CLASS

Using the same starting training set, 50 positive patients and 950 negative patients, undersample the negative class by randomly removing examples, until it is exactly as large as the positive class. Find how many negative examples are removed, and the new total training set size.

1. Start with the current class counts: 50 positive, 950 negative.
2. Find the target size for the negative class: as large as the positive class, so 50.
3. Find how many negative examples must be removed: subtract the target from the current count, $950 - 50 = 900$.
4. Add the unchanged positive count to the new negative count: $50 + 50 = 100$.

Answer: After undersampling, the training set has 50 positive examples and 50 negative examples, only 100 rows in total. The classes are now perfectly balanced, but 900 real patient records, all of them negative, were thrown away and never seen during training.

Oversampling and undersampling trade two different costs. Oversampling keeps every original record but repeats the same handful of positive patients, risking memorization instead of a pattern that generalizes. Undersampling avoids repeats entirely, but throws away 900 real, informative negative records. Neither choice is automatically better; it depends on how much data is available, and how much repetition or data loss a project can afford.

COMMON MISTAKE

✗ Wrong: Oversampling or undersampling the whole dataset first, then splitting the result into a training set and a test set.

✓ Right: Split into training and test sets first. Resample only the training set afterward. If a duplicated copy of a positive patient's record can land in both the training set and the test set, the test score becomes dishonestly optimistic, an echo of the **data leakage** problem from Chapter 2.

Plain duplication is not the only way to oversample. A more clever method creates brand-new, synthetic minority examples instead of exact copies.

WORD TO KNOW

SMOTE — Short for Synthetic Minority Oversampling Technique. A resampling method that creates new, synthetic minority-class examples by picking a point partway between a real minority example and one of its nearest neighbors of the same class, instead of copying an existing example exactly.

Picture one real patient whose measurements, simplified to two numbers, are age 45 and a blood-marker reading of 3.2. SMOTE finds a nearby same-class neighbor, using the same **Euclidean distance** idea from Chapter 8, say a second positive patient at age 41 with a reading of 3.6. It then invents a brand-new synthetic patient partway between them, such as age 43 with a reading of 3.4, and adds that synthetic patient to the training set. This new record is not a copy of any real patient, and it is not a real patient either, but a plausible new point among the real minority examples, which usually generalizes better than exact duplication. SMOTE is applied only to the training set, for the same leakage reason as plain oversampling, and this book leaves the fine details of choosing neighbors to the library that implements it.

Resampling changes the data itself. Lesson 16.3 reaches a similar goal through a completely different route: changing how costly a mistake is allowed to be, without touching a single row of data.

16.3 Class Weights: Telling the Model What Matters

Resampling changes which rows exist in the training set. Class weights leave every row exactly where it is, and instead change how much a mistake on that row is allowed to cost during training.

WORD TO KNOW

Class weight — A number attached to a class before training that multiplies how much a mistake on that class costs inside the training algorithm's loss. A high weight on the minority class pushes the algorithm to work harder to avoid missing it.

WORKED EXAMPLE — FIND A CLASS WEIGHT USING INVERSE FREQUENCY

Using the same 1,000-patient training set from Lesson 16.2, 50 positive and 950 negative, find a class weight for the positive class using a common method called inverse frequency: $\text{weight} = \frac{\text{the other class's count}}{\text{this class's count}}$.

1. Identify the two class counts: 950 negative, 50 positive.
2. Divide the negative count by the positive count: $950 \div 50 = 19$.
3. Set the positive class weight to this result, 19. The negative class keeps a baseline weight of 1.

Answer: The positive class gets a weight of 19, while the negative class keeps a weight of 1. Notice this is the same number, 19, found in the oversampling worked example in Lesson 16.2. Giving the positive class a

weight of 19 pushes training in a very similar direction to duplicating each positive example 19 times, without actually growing the dataset.

To see why this matters, imagine one small batch where the model makes 2 false negatives and 5 false positives. Unweighted, that is simply $2 + 5 = 7$ mistakes, counted equally. Weighted, the 2 false negatives now cost $2 \times 19 = 38$, and the 5 false positives still cost $5 \times 1 = 5$, for a weighted total of $38 + 5 = 43$. The 2 missed disease cases, only a small fraction of the 7 raw mistakes, now dominate the training algorithm's attention, which is exactly the pressure that pushes a model to stop missing positive cases.

COMMON MISTAKE

✗ Wrong: Setting a class weight during training, then judging the finished model using plain accuracy, and feeling reassured because the number looks high.

✓ Right: A class weight changes what happens during training, not how you should evaluate the finished model afterward. Still check **recall**, **precision**, and **F1 score** on the test set, exactly as Lesson 16.1 recommends.

Python · NumPy

 runnable in the app

```
from sklearn.linear_model import LogisticRegression

weights = {0: 1, 1: 19}
model = LogisticRegression(class_weight=weights)
```

The dictionary `weights` stores the same two numbers found by hand above: 1 for the negative class and 19 for the positive class. Passing `class_weight=weights` into `LogisticRegression` tells scikit-learn to multiply the cost of every mistake by these numbers during training, exactly like the weighted-cost example above. Many scikit-learn models also accept `class_weight="balanced"`, which calculates this same inverse-frequency weight automatically, instead of you typing the numbers in by hand.

Resampling and class weights both act before or during training. Lesson 16.4 works after training is already finished, on a model that has already learned to output a probability.

16.4 Moving the Decision Threshold

Every worked example so far in this chapter has used a model that already outputs a plain yes-or-no prediction. Most classifiers do not actually work that way underneath. They first output a probability, a number between 0 and 1, describing how likely a positive prediction is. Only afterward does a cutoff point turn that probability into a final "positive" or "negative" decision. Chapter 14 gave this cutoff point a name when flagging unusual sensor readings: a **threshold**. The same idea applies here. Whichever probability a model must

reach before its prediction counts as "positive" is a choice a team makes, not a fixed law of nature.

EQUATION — FROM PROBABILITY TO A CLASS PREDICTION

$$\text{prediction} = \begin{cases} \text{Positive} & \text{if } p \geq \text{threshold} \\ \text{Negative} & \text{otherwise} \end{cases}$$

In words: A model predicts the positive class whenever its predicted probability is at or above the chosen threshold. Otherwise, it predicts the negative class.

p the model's predicted probability for one example (value from 0 to 1)

threshold the cutoff point chosen by the team (value from 0 to 1)

Note: A threshold of 0.5 is a common default, but nothing forces a project to use it. Lowering the threshold makes a model flag more examples as positive; raising it makes the model flag fewer.

Ten patients from the screening clinic each receive a predicted probability of disease from the trained model, alongside their true, later-confirmed status. Patient 1: probability 0.05, truly negative. Patient 2: probability 0.10, truly negative. Patient 3: probability 0.15, truly negative. Patient 4: probability 0.22, truly negative. Patient 5: probability 0.30, truly positive. Patient 6: probability 0.40, truly negative. Patient 7: probability 0.55, truly positive. Patient 8: probability 0.60, truly negative. Patient 9: probability 0.75, truly positive. Patient 10: probability 0.90, truly positive.

WORKED EXAMPLE — CLASSIFY THE TEN PATIENTS AT THRESHOLD 0.5

Using the ten patients above and a threshold of 0.5, find each patient's predicted class, then find TP, FN, FP, TN, recall, and precision.

1. Apply the threshold: predict positive for every patient with probability ≥ 0.5 . That is patients 7 (0.55), 8 (0.60), 9 (0.75), and 10 (0.90). Predict negative for the other six patients.
2. Compare each predicted positive to its true label: patient 7 is truly positive (TP), patient 8 is truly negative (FP), patient 9 is truly positive (TP), patient 10 is truly positive (TP). This gives TP = 3 and FP = 1.
3. Compare each predicted negative to its true label: patients 1, 2, 3, 4, and 6 are truly negative (TN), and patient 5 is truly positive (FN). This gives TN = 5 and FN = 1.
4. Check the total: TP + FP + FN + TN = 3 + 1 + 1 + 5 = 10, matching the 10 patients.
5. Find recall: $\text{TP} \div (\text{TP} + \text{FN}) = 3 \div (3 + 1) = 3 \div 4 = 0.75$, or 75%.
6. Find precision: $\text{TP} \div (\text{TP} + \text{FP}) = 3 \div (3 + 1) = 3 \div 4 = 0.75$, or 75%.

Answer: At threshold 0.5, recall is 75% and precision is 75%. The model catches 3 of the 4 truly positive patients, and 3 of its 4 positive flags are correct.

WORKED EXAMPLE — MOVE THE THRESHOLD TO 0.2 AND RECLASSIFY THE SAME TEN PATIENTS

Using the same ten patients and probabilities, lower the threshold to 0.2, and find the new TP, FN, FP, TN, recall, and precision.

1. Apply the new threshold: predict positive for every patient with probability ≥ 0.2 . That is patients 4 (0.22), 5 (0.30), 6 (0.40), 7 (0.55), 8 (0.60), 9 (0.75), and 10 (0.90). Predict negative for patients 1, 2, and 3.
2. Compare each predicted positive to its true label: patients 5, 7, 9, and 10 are truly positive (TP); patients 4, 6, and 8 are truly negative (FP). This gives $TP = 4$ and $FP = 3$.
3. Compare each predicted negative to its true label: patients 1, 2, and 3 are all truly negative (TN). This gives $TN = 3$ and $FN = 0$.
4. Check the total: $TP + FP + FN + TN = 4 + 3 + 0 + 3 = 10$, matching the 10 patients.
5. Find recall: $TP \div (TP + FN) = 4 \div (4 + 0) = 4 \div 4 = 1.0$, or 100%.
6. Find precision: $TP \div (TP + FP) = 4 \div (4 + 3) \approx 0.571$, or about 57.1%.

Answer: At threshold 0.2, recall rises to 100%, catching every truly positive patient, including patient 5, who was missed at threshold 0.5. Precision falls to about 57.1%, because three healthy patients, 4, 6, and 8, are now also flagged.

IMPORTANT

A false negative and a false positive rarely cost the same amount in a screening program. Missing a genuine case, a false negative, can mean a real illness goes untreated until it is more advanced and harder to help. A false alarm, a false positive, usually means one more confirmatory test and some unwelcome worry, but it is rarely dangerous by itself. This is exactly why many screening programs deliberately choose a lower threshold, accepting more false alarms on purpose, in exchange for catching more genuine cases. Choose a threshold only after asking the people closest to the decision what each kind of mistake truly costs, never by defaulting to 0.5 out of habit.

WORD TO KNOW

Precision-recall curve — A plot showing precision and recall together at every possible threshold, not just one. The area underneath this curve, often shortened to PR-AUC, gives one summary score for how well a model balances precision and recall across every threshold at once.

One more threshold check completes the picture. Raise the threshold all the way to 0.8: only patient 10 (0.90) still qualifies as positive, giving $TP = 1$, $FP = 0$, and, among the nine predicted negatives, $FN = 3$ (patients 5, 7, and 9, wrongly missed) and $TN = 6$. That gives $\text{recall} = 1 \div (1 + 3) = 25\%$ and $\text{precision} = 1 \div (1 + 0) = 100\%$: fewer false alarms, but far more missed cases, the opposite trade from threshold 0.2.

Now look back at all three thresholds tried on these ten patients. At threshold 0.2, recall was 100% and precision was 57.1%. At threshold 0.5, recall and

precision were equal, both 75%. At threshold 0.8, precision reached 100% while recall fell to 25%. Plot each of these three results as one point on a graph, with recall along the bottom and precision up the side, and the shape of a precision-recall curve starts to appear. A real precision-recall curve uses many more thresholds than three, usually every threshold a model's probabilities could possibly produce, which is why software, not hand arithmetic, is used to calculate PR-AUC in practice. A PR-AUC close to 1.0 means a model keeps both precision and recall high across almost every threshold. A PR-AUC close to the dataset's overall positive fraction, 0.01 in this chapter's 10,000-patient dataset, is no better than guessing.

COMMON MISTAKE

✗ **Wrong:** Choosing a decision threshold by repeatedly checking scores on the test set, and keeping whichever threshold happens to score best there.

✓ **Right:** Choose a threshold using the **validation set**, or using cross-validation, the same way earlier chapters protect every other decision in a project. Save the test set for one last, honest check.

Moving the threshold changes how an already-trained model's probabilities get turned into decisions. Lesson 16.5 asks a different, deeper question: can those underlying probabilities themselves be trusted at all?

16.5 Calibration: Do the Probabilities Mean What They Say?

A model can be excellent at ranking patients from most at risk to least at risk, correctly sorting the order almost every time, while still being wrong about what its actual probability numbers mean. This is a separate concern from everything else in this chapter, and it deserves its own careful check.

WORD TO KNOW

Calibration — How well a model's predicted probabilities match real-world frequencies. A model is well calibrated if, among all the times it predicts a 70% chance of something, that thing genuinely happens about 70% of the time.

EQUATION — EXPECTED POSITIVES IN A CALIBRATION BUCKET

$$\text{expected positives} = p \times n$$

In words: If a model's probabilities are well calibrated, the expected number of true positives inside one bucket of examples equals that bucket's typical predicted probability, multiplied by the number of examples in the bucket.

p the model's typical predicted probability for examples in this bucket (value from 0 to 1)

n the number of examples in the bucket (count)

Note: Compare this expected count to the actual observed count of true positives in the same bucket, to check whether the model is well calibrated.

WORKED EXAMPLE — CHECK WHETHER A BUCKET OF '70% PROBABILITY' PATIENTS IS WELL CALIBRATED

Among all the patients the model gave a predicted probability close to 0.70, 20 patients fall into this bucket. If the model is well calibrated, find the expected number of truly positive patients in this bucket. Then, given that only 10 of these 20 patients were truly positive, find the observed frequency, and compare it to the predicted 70%.

1. Find the expected count if perfectly calibrated: multiply the predicted probability by the bucket size, $0.70 \times 20 = 14$.
2. Find the observed frequency: divide the actual positive count by the bucket size, $10 \div 20 = 0.5$, or 50%.
3. Compare the predicted probability, 70%, to the observed frequency, 50%: subtract to find the gap, $70 - 50 = 20$ percentage points.

Answer: This bucket is not well calibrated. The model claims a 70% chance of disease, but only 50% of these patients truly had it, a gap of 20 percentage points. A doctor who trusted the model's 70% figure at face value would overestimate these patients' real risk.

Checking one bucket at a time, as above, is the whole idea behind a calibration curve, sometimes called a reliability diagram: group every prediction into buckets by predicted probability, find the observed frequency inside each bucket, then plot predicted probability along the bottom and observed frequency up the side. A perfectly calibrated model produces points sitting exactly on a diagonal line, where predicted always equals observed. Points above the line mean the model is underconfident; points below the line, like the bucket just checked, mean it is overconfident.

COMMON MISTAKE

✗ Wrong: Oversampling the minority class, or applying a large class weight, and then trusting the model's output probabilities at face value afterward.

✓ Right: Both resampling and class weights change how common the positive class appears to be during training, which usually pushes predicted probabilities higher than the true real-world frequency. After using either technique, either recalibrate the model, or treat its probability outputs only as a useful ranking, not as literal real-world probabilities.

Calibration is a separate question from everything else in this chapter. A model can have excellent recall and precision at a chosen threshold, correctly ranking sick patients as more likely than healthy ones, while its raw probability numbers are still poorly calibrated. Calibration matters most when a probability is shown directly to a person, such as a doctor or a patient, as a trustworthy real-world chance, rather than used only to rank patients or compare against a threshold.

You now have a complete toolkit for a rare, important class: resampling, class weights, threshold choice, and calibration.

16.6 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Rare Disease Screening" dataset, built with a 99-to-1 imbalance ratio, and fit a classifier.
3. Open the confusion matrix panel, and write down accuracy, recall, and precision at the default threshold of 0.5.
4. Turn on the "Class Weight" toggle, using the balanced setting, and press Fit again. Compare the new recall and precision to your numbers from step 3.
5. Drag the threshold slider down until recall reaches 100%, then open the Calibration panel and check whether the model's points still sit on the diagonal line.

You need: Computer, tablet, or phone with the ML Studio app

Accuracy alone can never tell you whether a model is ready for a rare, high-stakes decision. Only a full confusion matrix, a deliberately chosen threshold, and a calibration check together can answer that honestly. The case study below follows one team through every tool in this chapter.

CASE STUDY — SCREENING FOR A RARE DISEASE AT A HOSPITAL NETWORK IN CHENGDU

The hospital network from this chapter serves more than forty clinics across Chengdu and the surrounding county towns, screening thousands of patients a month for a disease that, in this example, truly affects only about 1 in every 100 people tested. When the data team's first model reported 99% accuracy, a junior analyst nearly celebrated, until a senior colleague asked the exact question this chapter opened with: how many truly sick patients did it actually catch? Once the team built a full confusion matrix instead of trusting one number, the honest answer was almost none. The team started over, following the full toolkit this chapter has built.

The team compares two fixes on its training set of past patients: oversampling the positive class with SMOTE, and applying a class weight equal to the inverse class frequency. Both improve recall on the validation set, but the class-weighted model trains faster and keeps steadier predicted probabilities, so the team adopts class weighting as its main strategy, keeping SMOTE in reserve for when more confirmed positive patients become available.

Choosing a threshold is the next, most consequential decision, and not a purely mathematical one. The clinics' lead physician explains plainly what each mistake costs: a missed case means treatment starts later, when the disease is harder to manage, while a false alarm means one extra blood draw and a short, stressful wait for a confirmatory result. Weighing this real difference, the team deliberately chooses a threshold lower than the default of 0.5, accepting more false alarms in exchange for catching more genuine cases, checking every candidate threshold only against the validation set, never the untouched test set. Before trusting the model's probabilities in clinic conversations, the team also checks calibration and finds the model somewhat overconfident, much like the 70%-probability bucket checked earlier in this chapter. Rather than showing a raw "probability of disease," the team recalibrates the output and displays it

only as a three-level flag, low, medium, or high concern, with a plain note that the flag is a starting point for testing, not a finished diagnosis.

Three months after launch, the team reports both the model's strengths and its honest weaknesses to clinic staff: it now catches the large majority of true cases, at the cost of a noticeably higher confirmatory-test workload, exactly the trade the physicians asked for. Every flagged patient still receives a confirmatory laboratory test and a physician's own judgment before any diagnosis, and the team commits to rechecking recall and calibration every few months, since the screened population, and the disease itself, can both shift gradually over time.

IMPORTANT

A screening flag is a reason for a follow-up test and a clinician's own judgment, never a diagnosis by itself. Present a model's output as a starting point for human care, and be equally honest about what the model still misses, not only about what it catches.

IMPORTANT

Report a screening model's real recall and false-alarm rate honestly to the clinicians and patients who depend on it, not only its best-sounding number. A team that quietly hides a known weakness, hoping nobody notices, risks losing the trust that makes the whole screening program worth using in the first place.

COMMON MISTAKES TO AVOID

- Judging a classifier by accuracy alone when one class is far rarer than the other.
- Resampling the entire dataset before splitting into training and test sets, letting a duplicated row leak into both.
- Setting a class weight during training, then still trusting plain accuracy to judge the finished model.
- Choosing a decision threshold by repeatedly peeking at the test set instead of a validation set.
- Treating every screening mistake as equally costly, instead of asking what a false negative and a false positive truly cost in the real project.
- Trusting a model's raw probability outputs at face value after oversampling or class weighting, without checking calibration.
- Assuming a high accuracy number always means a model is ready to use.

CHAPTER SUMMARY

- Class imbalance means one class has far more examples than another; accuracy alone can hide a model that never finds the rare class.
- The imbalance ratio compares the majority class count to the minority class count, such as 99-to-1.

- Oversampling duplicates or synthesizes minority-class examples; undersampling removes majority-class examples; both are applied only to the training set, never before the train-test split.
- SMOTE creates synthetic minority examples partway between a real example and its nearest same-class neighbor, instead of copying exactly.
- A class weight multiplies how much a mistake on one class costs during training, pushing a model to avoid missing the minority class without changing the data itself.
- A threshold turns a predicted probability into a final decision; moving it trades recall against precision, and the right choice depends on what each mistake truly costs.
- A precision-recall curve plots precision and recall across every threshold; PR-AUC summarizes that curve in one number.
- Calibration checks whether a model's predicted probabilities match real-world frequency; resampling and class weights can both push probabilities out of calibration.
- In a screening setting, a false negative and a false positive are rarely equally costly, and a caring team designs its threshold and its process around that real difference.
- Chapter 17 turns to a related question: choosing a model's hyperparameters without cheating by tuning on the test set.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define class imbalance in your own words, and give one real-world example other than the ones used in this chapter.

Problem 2 [Basic] A dataset has 4,500 majority-class examples and 500 minority-class examples. Find the imbalance ratio.

Problem 3 [Basic] In one or two sentences, explain the difference between oversampling and undersampling.

Problem 4 [Basic] Explain, in your own words, what it means for a model to be "well calibrated."

Problem 5 [Standard] A training set has 50 positive examples and 750 negative examples. Find the class weight for the positive class using inverse frequency, showing every step.

Problem 6 [Standard] A lazy model always predicts "negative" on a dataset of 2,000 examples, where 30 are truly positive. Find TP, FN, FP, TN, and the model's accuracy, showing every step.

Problem 7 [Standard] A real model on the same 2,000 examples finds $TP = 24$, $FN = 6$, $FP = 150$, $TN = 1,820$. Find its accuracy and recall, showing every step, then compare both to the lazy model's numbers from problem 6.

Problem 8 [Standard] A training set has 90 positive examples and 810 negative examples. Find the new total size after oversampling the positive class to match the negative class, showing every step.

Problem 9 [Standard] Using the same training set as problem 8, find the new total size after undersampling the negative class to match the positive class, showing every step.

Problem 10 [Standard] A model predicts positive for 10 patients at a chosen threshold. Of these, 6 are truly positive and 4 are truly negative. There are 8 truly positive patients in total. Find recall and precision, showing every step.

Problem 11 [Challenge] A bucket of 50 patients all received a predicted probability near 0.40. (a) Find the expected number of true positives if the model is perfectly calibrated. (b) If only 15 of the 50 were truly positive, find the observed frequency and the calibration gap. (c) State whether the model is overconfident or underconfident for this bucket, and explain how you know.

Problem 12 [Challenge] A fraud-detection team must choose between two thresholds. Threshold A gives recall 60% and precision 90%. Threshold B gives recall 95% and precision 40%. (a) Explain one real situation where Threshold A would be the safer choice. (b) Explain one real situation where Threshold B would be the safer choice. (c) Explain, in your own words, why there is no single "correct" threshold that fits every project.

Problem 13 [Challenge] Open ML Studio, fit a classifier on an imbalanced dataset of your choice, and turn on the Calibration panel. Describe whether the model's points sit above, below, or on the diagonal line, and explain in a few sentences what that means for how the model's probabilities should be used.

BUILDING RELIABLE PROJECTS

CHAPTER 17

Model Selection and Tuning

Choosing settings without cheating

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a model's hyperparameters often need to be tuned together, not one at a time
- Count how many model fits a grid search requires, including cross-validation folds
- Explain how random search chooses a fixed budget of combinations instead of trying every one
- Compare grid search and random search, and choose between them for a given project
- Explain why tuning must never use the test set, and describe a safe alternative
- Use ML Studio to run a grid search and a random search, and compare their results

17.1 Every Model Has Dials to Turn

Chapter 5 introduced the **hyperparameter**: a setting chosen before training begins, rather than a value the learning algorithm learns from data on its own. Tree depth from Chapter 9, k from Chapter 8, the regularization strength λ from Chapter 6, the learning rate from Chapter 6, and the number of trees in a forest or a boosted ensemble from Chapter 10 are all hyperparameters you have already met. A real model rarely has just one dial to turn. It usually has several, turned at the same time, and the best setting for one dial often depends on where the other dials happen to be sitting.

Think back to Chapter 10's random forest, and its two dials: how many trees to grow, and how deep each tree is allowed to go. It is tempting to test a few tree counts with depth held at some convenient default, pick the best tree count, then test a few depths with that tree count held fixed. This feels efficient, since it only requires testing a handful of settings instead of every combination. The worked example below shows exactly why that shortcut can quietly fail.

WORKED EXAMPLE — SEE WHY TUNING ONE DIAL AT A TIME CAN MISS THE BEST COMBINATION

A team tunes a random forest with two hyperparameters: number of trees (50 or 200) and max depth (4 or 10). All four combinations are tested on a validation set, giving these accuracies: 50 trees and depth 4 = 88%. 50 trees and depth 10 = 84%. 200 trees and depth 4 = 85%. 200 trees and depth 10 = 90%. The team tunes one dial at a time: first fix trees at 50 and choose the

better depth, then fix depth at that choice and choose the better tree count. Find the combination this method lands on, then compare it to the true best combination.

1. Hold trees at 50, and compare the two depths: depth 4 gives 88%, depth 10 gives 84%. Since 88% is higher, choose depth 4.
2. Hold depth at 4, the choice just made, and compare the two tree counts: 50 trees gives 88%, 200 trees gives 85%. Since 88% is higher, choose 50 trees.
3. The one-at-a-time method lands on 50 trees and depth 4, scoring 88%.
4. Compare all four combinations directly: 88%, 84%, 85%, and 90%. The highest of the four is 90%, from 200 trees and depth 10, a combination the one-at-a-time method never tried.

Answer: One-at-a-time tuning finds only 88% accuracy, but the true best combination, 200 trees and depth 10, reaches 90%. Because the two dials interact, checking every combination together, not one dial at a time, is the safer approach. That is exactly what grid search does, starting in Lesson 17.2.

Why would depth 4 beat depth 10 at 50 trees, yet depth 10 beat depth 4 at 200 trees? A shallow forest of only 50 trees may need every one of those trees to reach a reasonable depth just to capture the pattern at all, so a shallower limit of 4 is not much of a restriction. Once the forest grows to 200 trees, there is enough variety among the trees that letting each one grow deeper, to 10, adds real, useful detail instead of just noise. Neither dial has one fixed "best" value in isolation; the right value for one depends on the other.

COMMON MISTAKE

✗ Wrong: Assuming the best value of one hyperparameter, found while holding every other hyperparameter fixed, stays best no matter what the other hyperparameters are set to.

✓ Right: Hyperparameters can interact. The best max depth for a forest with 50 trees is not always the best max depth for the same forest with 200 trees. Check combinations together, not one dial in isolation.

Testing every combination by hand, the way the worked example above just did, works fine with two dials and two values each, only four combinations in total. Real projects are rarely that small. Lesson 17.2 gives this idea a name, and a way to count exactly how large it grows.

17.2 Grid Search: Trying Every Combination

Lesson 17.1's four-combination check already has a name. Picture those four combinations arranged in a small table: one row for each tree count, one column for each depth, with every cell holding one accuracy score. That table is exactly why this method is called a grid: values for one hyperparameter form the rows, values for another form the columns, and every cell is one combination the search must try. With three or more hyperparameters, the

same idea still holds, even though a simple two-dimensional picture no longer fits neatly on a page.

WORD TO KNOW

Grid search — A tuning method that tries every possible combination of hyperparameter values from a chosen list for each hyperparameter, training and validating one model for every combination.

EQUATION — NUMBER OF COMBINATIONS IN A GRID SEARCH

$$\text{total combinations} = v_1 \times v_2 \times v_3 \times \dots$$

In words: The total number of combinations a grid search must try equals the number of values chosen for each hyperparameter, all multiplied together.

v_1 number of values chosen for the first hyperparameter (count)

v_2 number of values chosen for the second hyperparameter (count)

Note: Multiply in one more count for every additional hyperparameter, following the same pattern.

WORKED EXAMPLE — COUNT COMBINATIONS FOR A TWO-HYPERPARAMETER GRID

A team tunes a k-Nearest Neighbors model over 4 values of k (3, 5, 7, 9) and 2 weighting schemes (uniform, distance-weighted). Find the total number of combinations, then find the total number of model fits if every combination is scored with 5-fold cross-validation.

1. Identify the number of values for each hyperparameter: k has 4 values, weighting scheme has 2 values.
2. Multiply the two counts to find the total combinations: $4 \times 2 = 8$.
3. Multiply the total combinations by the number of cross-validation folds, 5: $8 \times 5 = 40$.

Answer: The grid has 8 combinations, and scoring all of them with 5-fold cross-validation requires 40 separate model fits.

Larger grids grow fast. The next worked example tunes a boosted-tree model, the same family Chapter 10 introduced, over three hyperparameters at once.

EQUATION — TOTAL MODEL FITS, INCLUDING CROSS-VALIDATION

$$\text{total fits} = \text{total combinations} \times k$$

In words: The total number of times a model must actually be trained equals the number of hyperparameter combinations, multiplied by the number of cross-validation folds, k.

total combinations every combination of hyperparameter values in the grid (count)

k number of cross-validation folds used to score each combination (count)

Note: Each of these fits is a separate, complete training run, which is exactly why grid search can become slow as the grid grows.

WORKED EXAMPLE — COUNT COMBINATIONS AND TOTAL FITS FOR A THREE-HYPERPARAMETER GRID

A team tunes a boosted-tree model over three hyperparameters: number of trees, with 4 choices (50, 100, 200, 400); max depth, with 3 choices (3, 5, 7); and learning rate, with 3 choices (0.01, 0.1, 0.3). Every combination is scored with 5-fold cross-validation. (a) Find the total combinations in the grid. (b) Find the total model fits required. (c) If each fit takes about 2 minutes, find the total training time in hours.

1. Identify the number of values for each hyperparameter: 4 for number of trees, 3 for max depth, 3 for learning rate.
2. Multiply the three counts together: $4 \times 3 \times 3 = 36$.
3. Multiply the total combinations by the number of cross-validation folds, 5: $36 \times 5 = 180$.
4. Multiply the total fits by 2 minutes per fit: $180 \times 2 = 360$ minutes.
5. Convert minutes to hours by dividing by 60: $360 \div 60 = 6$.

Answer: The grid contains 36 combinations. Training every one with 5-fold cross-validation requires 180 model fits, and at 2 minutes per fit that is 360 minutes, or 6 hours of computer time. This is exactly why grid search can quickly become impractical as more hyperparameters, or more values per hyperparameter, are added.

COMMON MISTAKE

✗ Wrong: Assuming that adding "just one more value" to a hyperparameter's list barely changes how long a grid search takes.

✓ Right: Every extra value multiplies the total; it does not just add to it. Growing the max depth list above from 3 choices to 4, with the other two hyperparameters unchanged, raises the grid from $4 \times 3 \times 3 = 36$ combinations to $4 \times 4 \times 3 = 48$ combinations, and from 180 total fits to $48 \times 5 = 240$ total fits, 60 more full training runs from one extra number on one list.

One practical way to keep a grid manageable is to search in two rounds instead of one. The first round uses a coarse grid, with only a few, widely spaced values for each hyperparameter, to find roughly which region of settings looks promising. The second round then builds a smaller, finer grid, with closely spaced values, centered only on that promising region. Two small rounds like this can cost far fewer total fits than one enormous grid covering every value at fine detail from the very start, while still landing close to the same best setting.

COMMON MISTAKE

✗ Wrong: Choosing hyperparameter values for a grid completely at random, with no reasoning behind the range, then hoping the search finds something useful.

✓ Right: Base a grid's values on domain knowledge, a model's documentation, or a quick coarse search first, so the grid's limited combinations are spent on plausible settings, not wasted on values unlikely to ever help.

Python · NumPy

 runnable in the app

```

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import GradientBoostingClassifier

param_grid = {"n_estimators": [50, 100, 200, 400],
              "max_depth": [3, 5, 7],
              "learning_rate": [0.01, 0.1, 0.3]}
search = GridSearchCV(GradientBoostingClassifier(), param_grid, cv=5)
search.fit(X_train, y_train)

```

The dictionary `param\_grid` lists the same three hyperparameters and values used in the worked example above: 4 tree counts, 3 depths, and 3 learning rates. `GridSearchCV` builds and scores all 36 combinations automatically, using `cv=5` to run 5-fold **cross-validation** on each one, the same 180 fits found by hand. After `fit` finishes, `search.best\_params\_` holds the winning combination.

Grid search is thorough, but the common mistake box above already showed how fast it can grow. Lesson 17.3 introduces a different strategy: choosing a budget in advance, instead of letting the grid decide the cost.

17.3 Random Search: Trying Many, Not All

Grid search's cost is fixed the moment the value lists are chosen: every combination, no exceptions. A different tuning method lets a team fix the cost directly instead.

WORD TO KNOW

Random search — A tuning method that tries a fixed number of randomly chosen hyperparameter combinations, instead of every possible combination, and keeps the best one found.

WORKED EXAMPLE — COMPARE GRID SEARCH TO A RANDOM SEARCH BUDGET OF 20

Using the same boosted-tree grid from Lesson 17.2, 36 total combinations, 180 total fits with 5-fold cross-validation, about 360 minutes of computer time, a team instead runs a random search that samples 20 random combinations from the same set of possible values, still scored with 5-fold cross-validation. (a) Find the total model fits the random search requires. (b) Find its total time, at 2 minutes per fit. (c) Find how many fewer fits it runs than the full grid search, and that savings as a percentage.

1. Multiply the 20 sampled combinations by the 5 cross-validation folds: $20 \times 5 = 100$ total fits.
2. Multiply the 100 fits by 2 minutes per fit: $100 \times 2 = 200$ minutes.
3. Convert 200 minutes to hours and minutes: $200 \div 60 = 3$, remainder 20, so 200 minutes is 3 hours and 20 minutes.

4. Find the difference in fits from the full grid search: $180 - 100 = 80$ fewer fits.
5. Find this savings as a percentage of the grid search's fits: $80 \div 180 \approx 0.444$, or about 44.4%.

Answer: The random search needs only 100 model fits, about 3 hours and 20 minutes, compared to the grid search's 180 fits and 6 hours. That is 80 fewer fits, a savings of about 44.4% of the computing time, while still trying 20 genuinely different combinations spread across the same set of possible values.

A larger or smaller budget simply changes the multiplication. A budget of 40 needs $40 \times 5 = 200$ fits, more than this chapter's grid, while a budget of 10 needs only $10 \times 5 = 50$ fits. The team chooses the budget directly, based on how much computer time is available, instead of waiting to find out how large the grid turned out to be.

WORKED EXAMPLE — FIND THE FITS FOR A LARGER RANDOM SEARCH BUDGET

A different project uses a random search with a budget of 50 combinations, scored with 8-fold cross-validation. Find the total number of model fits required.

1. Identify the budget and the number of folds: 50 combinations, 8 folds.
2. Multiply the two together: $50 \times 8 = 400$.

Answer: This random search requires 400 model fits. The same rule from grid search, total fits = combinations tried $\times$ folds, applies here too; only the source of the combinations changes, chosen at random instead of listed out in a grid.

Random search can afford to try many combinations precisely because it does not have to try every combination. Suppose only one of the three hyperparameters in this chapter's boosted-tree example actually changes the score much, say the learning rate, and the other two barely matter. A grid search still spends its combinations testing every mix of tree count and depth against only 3 distinct learning rate values, the 3 originally chosen for the grid, no matter how large the rest of the grid grows. A random search with a similar-sized budget of, say, 36 draws, by contrast, picks a fresh value for every hyperparameter on every single try, so it can end up trying as many as 36 different learning rate values, spread across the same range, a far finer look at the one dial that actually matters.

COMMON MISTAKE

✗ Wrong: Assuming random search is always worse than grid search, simply because it does not try every combination.

✓ Right: With a reasonable budget, random search often finds a combination nearly as good as the full grid, in a fraction of the time, especially when only a few hyperparameters truly matter for a given dataset.

So which should a team actually choose? Grid search is a reasonable choice when there are only one or two hyperparameters, each with just a few sensible

values, since the full grid stays small and guarantees the single best combination inside that grid is found. Random search becomes the safer choice once three or more hyperparameters are involved, or once any one hyperparameter has many possible values, since the grid's combinations would otherwise multiply faster than most teams' computer time allows.

Python · NumPy

 runnable in the app

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import GradientBoostingClassifier

param_grid = {"n_estimators": [50, 100, 200, 400],
              "max_depth": [3, 5, 7],
              "learning_rate": [0.01, 0.1, 0.3]}
search = RandomizedSearchCV(GradientBoostingClassifier(), param_grid,
                           n_iter=20, cv=5, random_state=0)
search.fit(X_train, y_train)
```

This code reuses the exact same `param\_grid` dictionary from Lesson 17.2, but swaps `GridSearchCV` for `RandomizedSearchCV`, and adds `n\_iter=20` to set the budget at 20 combinations, matching the worked example above. `cv=5` still runs 5-fold cross-validation on every combination tried, for the same 100 fits found by hand. Setting `random\_state=0` makes the random draws repeatable, so running this code twice picks the exact same 20 combinations both times.

IMPORTANT

A very large search, whether grid or random, checks many settings against the same validation folds. With enough attempts, one setting can score well by luck alone, the same danger as checking the test set too many times, covered next in Lesson 17.4. Keep the search space reasonable, and treat a surprisingly perfect validation score with healthy suspicion, not celebration.

Grid search and random search both protect themselves with cross-validation or a **validation set**. Lesson 17.4 names the one mistake that can quietly undo that protection, no matter which search method a team chooses.

17.4 Avoiding a Common Trap: Tuning on the Test Set

Every safeguard in this chapter so far, cross-validation inside a grid search, cross-validation inside a random search, only works if one rule is never broken: the test set stays completely closed until tuning is finished.

It is tempting to check the test set "just once more" after trying a new setting, especially close to a deadline. Each time a team peeks at the test set to decide what to try next, the test set quietly stops being a fair, one-time check and starts becoming another validation set, except one nobody labeled as such. The

final reported score then reflects some luck, not only genuine quality, a hidden form of **overfitting**, this time to the test set itself rather than to the training data.

WORKED EXAMPLE — SEE HOW TEST-SET PEEKING INFLATES A REPORTED SCORE

A team tunes a model by trying 5 different hyperparameter settings, checking accuracy on their 200-example test set after every single try: Setting A scores 82%, Setting B scores 85%, Setting C scores 79%, Setting D scores 88%, Setting E scores 84%. The team picks Setting D and reports "88% test accuracy." Setting D is later checked again, on a completely fresh batch of 200 new examples collected after launch, and scores 81%. Find the drop between the two numbers, and explain what caused it.

1. List the five scores seen while tuning: 82%, 85%, 79%, 88%, 84%.
2. Identify the highest of the five: 88%, from Setting D, the one the team chose and reported.
3. Compare the reported score, 88%, to Setting D's score on the completely fresh batch collected after launch: 81%.
4. Find the drop: $88 - 81 = 7$ percentage points.

Answer: The reported 88% was too optimistic by about 7 percentage points. Setting D was not necessarily the genuinely best setting; it was simply the one that happened to score highest on that one test set, out of five separate looks. A test set checked only once, after tuning is already finished using cross-validation or a validation set, would not have this problem.

Why does using cross-validation during tuning avoid this same trap? Every one of its folds is carved out of the training data itself, never from the test set, so a team can check as many settings as it likes against those folds without ever touching the one number meant to be a final, honest surprise. The test set only enters the picture once, after the winning setting from cross-validation is already locked in.

WORKED EXAMPLE — SEE THE SAME SCENARIO DONE SAFELY WITH CROSS-VALIDATION

The same 5 settings from the previous worked example are instead compared using 5-fold cross-validation on the training data only, never the test set: Setting A averages 81%, Setting B averages 84%, Setting C averages 78%, Setting D averages 83%, Setting E averages 85%. The team picks Setting E, the highest cross-validation average, and checks it against the test set exactly once, scoring 84%. Find the gap between the cross-validation estimate and the real test score, and compare it to the 7-point gap found in the previous worked example.

1. Identify Setting E's cross-validation average: 85%.
2. Identify Setting E's one-time test score: 84%.
3. Find the gap between the two: $85 - 84 = 1$ percentage point.

4. Compare this gap to the previous worked example's gap of $88 - 81 = 7$ percentage points.

Answer: The cross-validation estimate, 85%, sits only 1 percentage point away from the honest test score, 84%, far closer than the 7-point gap that test-set peeking produced. Choosing between settings using cross-validation, then checking the test set only once at the very end, gives a team an estimate it can actually trust.

Some especially careful projects go one step further, repeating this entire test-once process across several different outer splits of the data, so that even the choice of which examples end up in the final test set is not left to a single lucky draw. That extra layer of care is beyond what this book covers in detail, but it follows the same discipline this lesson already teaches: never let the number you report be the same number you used to choose.

IMPORTANT

A test set can honestly be used only once. The moment its score is used to choose between hyperparameter settings, it quietly becomes another validation set, and the final number reported is no longer a fair estimate of how the model will perform on genuinely new data. Finish all tuning first, using cross-validation or a validation set, then open the test set exactly one time.

COMMON MISTAKE

✗ **Wrong:** Believing that using cross-validation during grid search or random search means the test set no longer matters.

✓ **Right:** Cross-validation protects the tuning process itself. The test set still plays its own, separate, final role: one honest, last check, opened only after tuning is completely finished.

You now have two ways to search for good hyperparameters, and one rule that protects both of them: never let the test set influence a single choice along the way. Lesson 17.5 lets you run both searches yourself, in ML Studio.

17.5 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Tune tab.
2. Choose the "Retail Demand" dataset and a boosted-tree model.
3. Build a small grid with two hyperparameters, and press Run Grid Search. Write down the total number of combinations ML Studio reports, and the best validation score found.
4. Switch to Random Search, set a budget smaller than the grid's total combinations, and press Run. Compare its best score and its total run time to the grid search.
5. Open the test set panel only after both searches are finished, check the final score once, and write one sentence about why you waited until now to look.

You need: Computer, tablet, or phone with the ML Studio app

A search method is only as good as the discipline around it: combinations counted honestly, a fair validation method, and a test set opened only once. The case study below follows one team applying all three together.

17.6 Case Study

CASE STUDY — TUNING A DEMAND-FORECASTING MODEL FOR A SUPERMARKET CHAIN IN MEXICO

A supermarket chain with stores across several Mexican cities wants to forecast how much of each product to stock at every store, every week. Order too little, and shelves sit empty during a weekend rush. Order too much, and fresh produce and dairy spoil before they sell, a direct loss on every unsold item. The chain's data team builds a boosted-tree model, the same family introduced in Chapter 10, using two years of past sales, weather, and local holiday data across more than eighty stores.

An untuned model, left at its default settings, reaches a mean absolute error of about 18 units per product per week, barely better than the chain's old rule-of-thumb ordering method, and its errors are worst around holidays, exactly the weeks when getting the order right matters most. The team suspects the default settings either overfit the noisy week-to-week swings of a single store or underfit the clear seasonal pattern shared by every store. A full grid search across four hyperparameters, tree count, depth, learning rate, and minimum leaf size, with four values each, would require $4 \times 4 \times 4 \times 4 = 256$ combinations, and with 5-fold cross-validation, $256 \times 5 = 1,280$ model fits, far more computer time than the team has before the next planning cycle.

The team instead runs a random search with a budget of 60 combinations, sized to fit inside one overnight computing run on the chain's shared server, scoring every candidate with cross-validation and never once opening the held-out test set. Sixty draws, spread across four hyperparameters, still give a reasonably fine look at each one individually, especially the learning rate, which an early, coarser check had already suggested mattered more than the other three. Halfway through the search, one analyst grows impatient and asks whether they could just peek at the test set to see how the best setting so far is doing. The team says no, exactly as this chapter recommends, and waits for the full search to finish.

The winning combination is checked against the test set exactly once, after every tuning decision is already locked in, and the tuned model's mean absolute error falls to about 11 units per product per week, an improvement the team can defend honestly, precisely because the test set was never used to help find it. Before rolling the model out to every store, the team documents the exact hyperparameter values chosen, so a future analyst can reproduce the same search instead of starting over, and schedules a review in three months to check whether the same settings still hold up as new stores and new products are added.

Even a well-tuned forecast stays a suggestion, not a command. Each store manager can still adjust the suggested order up or down before it is submitted,

using local knowledge no dataset captures, such as a school holiday just announced or a delivery truck running late that week.

COMMON MISTAKES TO AVOID

- Tuning one hyperparameter at a time and assuming the result still holds once another hyperparameter changes.
- Forgetting to multiply by the number of cross-validation folds when estimating how many model fits a search will require.
- Assuming random search is automatically worse than grid search, even on a reasonable budget.
- Letting a search space grow so large that some setting scores well by luck alone.
- Checking the test set repeatedly while tuning, and reporting the best of several looks as if it were a single, fair score.
- Believing cross-validation during tuning means the test set no longer serves any purpose.
- Choosing a hyperparameter's candidate values with no reasoning behind them, instead of using domain knowledge or a quick coarse search first.

CHAPTER SUMMARY

- Hyperparameters often interact, so the best setting for one dial can depend on where another dial is set.
- Grid search tries every combination of a chosen set of hyperparameter values; total combinations multiply, they do not add.
- Total model fits during a search equal the total combinations tried, multiplied by the number of cross-validation folds.
- Grid search can grow impractically large very quickly as more hyperparameters, or more values per hyperparameter, are added.
- Random search tries a fixed budget of randomly chosen combinations, letting a team control cost directly.
- With a reasonable budget, random search often performs nearly as well as a full grid search, in far less time.
- Grid search suits a small number of hyperparameters with few values each; random search suits larger, richer search spaces.
- The test set must never influence a tuning decision; use cross-validation or a validation set for every comparison during tuning.
- Checking the test set repeatedly while tuning inflates the reported score, a hidden form of overfitting to the test set itself.
- Chapter 18 turns from choosing a model's settings to understanding what a finished model has actually learned.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define hyperparameter in your own words, and name one hyperparameter from an earlier chapter.

Problem 2 [Basic] In one or two sentences, explain the difference between grid search and random search.

Problem 3 [Basic] A grid has 3 values for one hyperparameter and 5 values for another. Find the total number of combinations.

Problem 4 [Basic] Explain, in your own words, why the test set must not be used while choosing hyperparameters.

Problem 5 [Standard] A grid has 4 values for hyperparameter A, 2 values for hyperparameter B, and 3 values for hyperparameter C. Find the total number of combinations, showing every step.

Problem 6 [Standard] Using the grid from problem 5, find the total number of model fits required with 10-fold cross-validation, showing every step.

Problem 7 [Standard] A random search uses a budget of 25 combinations, scored with 4-fold cross-validation. Find the total number of model fits required.

Problem 8 [Standard] A grid search requires 300 total fits, using 6-fold cross-validation. A random search on the same problem uses a budget of 30 combinations, scored with the same 6 folds. Find how many fits the random search requires, and how many fewer fits that is than the grid search.

Problem 9 [Standard] A team tunes one hyperparameter at a time on a model with two dials, each with 3 possible values. Explain, in two or three sentences, why testing only $3 + 3 = 6$ combinations one at a time is not the same as testing all $3 \times 3 = 9$ combinations together.

Problem 10 [Standard] A team checks their test set 8 times while tuning, and reports the highest of the 8 scores, 93%, as the model's accuracy. Explain, in your own words, why 93% is probably not an honest estimate of the model's real performance.

Problem 11 [Challenge] A grid has 5 hyperparameters, each with 4 possible values. (a) Find the total number of combinations. (b) Find the total number of model fits with 5-fold cross-validation. (c) Suggest one practical way to reduce this number without simply guessing at good values.

Problem 12 [Challenge] A team must choose between a full grid search and a random search with a fixed budget for a project with very limited computer time. (a) Explain one situation where the grid search is still worth running. (b) Explain one situation where random search is clearly the better choice. (c) Explain, in your own words, why neither method should ever be scored using the test set.

Problem 13 [Challenge] Open ML Studio, build a small grid for a model of your choice, and run both a grid search and a random search with a budget smaller than the grid's total size. Compare the best validation score and total run time for each, and describe, in a few sentences, whether the random search's savings in time were worth its small drop, if any, in score.

BUILDING RELIABLE PROJECTS

CHAPTER 18

Interpretability, Fairness, and Ethics

Understanding a model, and using it responsibly

IN THIS CHAPTER YOU WILL LEARN TO

- Explain why a strong validation score alone does not prove a model is safe, fair, or trustworthy
- Read a linear model's coefficients and a tree ensemble's feature importance, and explain what each one does and does not tell you
- Compute permutation importance by hand, and explain why correlated features can hide each other's true importance
- Tell apart this chapter's social meaning of "bias" from Chapter 5's statistical meaning, and explain both in your own words
- Compare group-level outcomes, such as selection rate and false-negative rate, to check a model for unfairness
- Identify a protected attribute, and recognize when a proxy variable could quietly reintroduce it

18.1 Why "It Works" Is Not Enough

Imagine a doctor who prescribes you a new medicine every morning, but refuses to explain what it treats or why it should help you. The medicine might even work, most of the time, for most patients. Would you keep taking it forever without ever asking why? Most people would want to understand a lot more before trusting something that affects their health this directly.

A machine learning model that makes decisions about people deserves the same kind of question. Chapters 1 through 17 gave you careful tools to build a model and to measure how well it performs, using a validation set and honest metrics. But a strong validation score only answers one question: does the model perform well on data like the data it was tested on? It does not answer two other questions this chapter cares about deeply. Can a person explain why the model made a particular decision? And does the model treat every group of people it affects fairly? A model can score beautifully on Chapter 4's metrics and still fail both of these tests.

WORD TO KNOW

Interpretability — How well a person can understand why a model made a particular prediction, and what patterns it relies on in general. A highly interpretable model is easy for a person to check, question, and explain to someone else.

Some models are naturally easy to read. A **linear regression** model, from Chapter 6, hands you a short list of coefficients, one per feature, and each coefficient's sign and size already tell you most of the story. A shallow **decision tree**, from Chapter 9, is a short list of yes/no questions a person can follow from root to leaf by hand. Other models are much harder to read this way. A **random forest** or a **boosting** model, both from Chapter 10, might combine hundreds of trees, and a **neural network**, from Chapter 11, might combine thousands of numbers spread across several hidden layers. Nobody can trace a single clean path through that much machinery just by eye. These harder-to-read models are sometimes called "black box" models, not because they are hiding anything on purpose, but because their internal calculation is too large and too tangled for a person to walk through directly, one step at a time. This chapter's first tools were built mainly for that second kind of model, though they work on any model at all.

A natural question is why anyone would use a harder-to-read model at all, if a linear regression or a shallow tree is so much easier to check. The honest answer is that flexible models often predict more accurately, especially when the real pattern in the data is genuinely complicated, which is the same trade-off Chapter 5 first introduced between a model's capacity and how easy it is to trust at a glance. This chapter does not ask you to give up accuracy for the sake of a simpler model. It gives you tools that open the black box far enough to check it carefully, even while you keep the more flexible, more accurate model running underneath.

People affected by a model's decision often want an explanation too, not just a number. A rejected loan applicant, a screened-out job candidate, and a flagged bank transaction all deserve some honest account of why, even when that account is necessarily a simplified summary of a much larger calculation.

COMMON MISTAKE

✗ Wrong: Trusting a model completely because it reached a high validation score, without ever asking why it makes the predictions it makes.

✓ Right: Treat a strong validation score as necessary, but not sufficient. Also check which features the model relies on, and whether it treats every group of people fairly, before trusting it with a real decision.

Chapter 10 already gave you one honest warning about this, right after introducing **feature importance**: a score that shows what a model leaned on is not proof that the feature actually causes the outcome, and it should never be the only reason to take an action against one specific person. This chapter picks that warning up, builds it into a full toolkit, and then asks an even bigger question. Even a model that is fully and correctly explained can still end up treating one group of people worse than another. Explaining a model and making it fair are two related but separate jobs, and this chapter teaches both, one at a time.

IMPORTANT

A model can be highly accurate and still be unsafe to use without a person watching over it. Accuracy tells you how often a model is right. It does not tell you why the model is right, and it does not tell you whether the model is equally right for every person it affects. Keep a trained, caring human in the loop for any decision that meaningfully affects a person's life, health, money, or opportunities.

18.2 Reading Coefficients and Feature Importance

The most direct way to understand a model is to read the numbers it already carries inside it. Different kinds of models hand you different numbers, and it matters a great deal which kind you have in front of you.

Chapter 6 introduced the **coefficient**: the number a linear regression model multiplies a feature by before adding everything together. A coefficient is one of the most directly readable numbers in all of machine learning, because its sign tells you the direction of its effect, and its size tells you exactly how much the prediction moves for every extra unit of that feature, with every other feature held still.

WORKED EXAMPLE — READ WHAT A COEFFICIENT PREDICTS, AND WHAT CHANGING IT WOULD DO

Sunrise Parcels, the delivery company from Chapter 10, also tries a plain linear regression model that predicts a delivery's delay, in minutes: $\text{predicted delay} = 4 + 1.5 \times (\text{distance in kilometers}) + 2 \times (\text{number of stops before this delivery})$. Find the predicted delay for a delivery that is 12 kilometers away with 3 stops before it. Then find the new predicted delay if that same delivery instead had 4 stops before it, with distance unchanged.

1. Multiply the distance coefficient by the distance: $1.5 \times 12 = 18$.
2. Multiply the stops coefficient by the number of stops: $2 \times 3 = 6$.
3. Add the intercept and both terms together: $4 + 18 + 6 = 28$.
4. Now change only the stops, from 3 to 4, and multiply again: $2 \times 4 = 8$.
5. Add the intercept, the unchanged distance term, and the new stops term: $4 + 18 + 8 = 30$.

Answer: The first delivery's predicted delay is 28 minutes. With one extra stop added before it, the predicted delay rises to 30 minutes, exactly 2 minutes more, matching the stops coefficient exactly. Reading a coefficient directly like this tells you both the direction, delay goes up, and the size, 2 minutes per stop, without retraining or rechecking anything.

Chapter 10 gave you a different kind of number for a random forest or a boosted ensemble: **feature importance**, a score built from how much each feature's splits reduced impurity, added up across every tree. Feature importance is useful, but it answers a narrower question than a coefficient does, and it is worth being precise about exactly what that question is. Knowing which question a number actually answers matters well beyond a classroom exercise. A manager reading a chart in a meeting, or a customer asking why a

decision went a certain way, deserves an answer built on the right number, not just any number that happened to be easy to print.

WORKED EXAMPLE — NOTICE WHAT A FEATURE IMPORTANCE SCORE CANNOT TELL YOU

Lesson 10.6 found that Distance's feature importance in Sunrise Parcels' mini-forest was about 66.7%, the highest of the three features, with Stops at 33.3% and Rain at 0%. Karwan wants to know one more thing: does a longer distance make a late delivery more likely, or less likely? Can the importance score alone answer this?

1. Recall what 66.7% actually measures: how much impurity Distance's splits removed, as a share of all the impurity removed by every feature, added up across every tree.
2. Notice that this share is always a positive number, or zero. A feature importance score can never come out negative, no matter which direction that feature actually pushes a prediction.
3. Because the number can never be negative, it cannot, by itself, say whether long distances push predictions toward "late" or toward "on time." It only ranks how much each feature helped the trees make good splits.
4. To learn the direction, Karwan must look further, such as checking the trees' actual split rules, or comparing the model's average prediction for short-distance deliveries against long-distance deliveries directly.

Answer: A feature importance score alone cannot tell you which direction a feature pushes a prediction, only how much that feature mattered to the model's splits. A linear regression coefficient's sign gives you the direction for free, inside the very same number; a tree ensemble's importance score does not, and needs an extra check before anyone acts on it.

COMMON MISTAKE

✗ Wrong: Comparing two coefficients, such as 1.5 for distance measured in kilometers and 2 for a count of stops, and concluding that stops "matters more" just because 2 is the bigger number.

✓ Right: A coefficient's size depends on the scale of its feature. Compare coefficients fairly only after standardizing features, from Chapter 3, onto a similar scale, or compare the realistic range each feature actually moves across, not just the raw coefficient.

18.3 Permutation Importance: Testing a Feature by Breaking It

Chapter 10 already promised a sharper tool for measuring feature importance, one that checks a feature's true usefulness directly, by breaking it on purpose and watching what happens next. This lesson delivers that tool.

The idea is refreshingly simple, once you see it laid out in order. Fit a model, exactly as you already know how to do. Measure its score on a validation set,

the same honest check Chapter 4 taught you to trust. Then take just one feature's column and shuffle its values, scrambling which value belongs to which row, while every other feature and the true label stay completely untouched. This scrambling destroys any real relationship that one feature had with the outcome, while leaving the rest of the data exactly as it was. Measure the model's score again, on this scrambled version. If the score barely changes, the model was not depending on that feature very much. If the score drops sharply, the model was depending on it a great deal.

WORD TO KNOW

Permutation importance — How much a trained model's validation score drops after one feature's values are randomly shuffled, breaking that feature's link to the outcome while leaving every other feature untouched. A bigger drop means the model depended on that feature more.

EQUATION — PERMUTATION IMPORTANCE

$$\text{permutation importance} = \text{baseline score} - \text{shuffled score}$$

In words: A feature's permutation importance equals the model's validation score before shuffling, minus its validation score after that one feature's values are randomly shuffled. This assumes a score where a higher number means better performance, such as accuracy or F1 score.

baseline score the model's validation score with every feature left exactly as it was (same unit as the score)

shuffled score the model's validation score after one feature's column is randomly reordered (same unit as the score)

Note: For an error measure where a lower number is better, such as MAE, reverse the subtraction instead: importance = shuffled score – baseline score, so a bigger positive number still means the feature mattered more.

WORKED EXAMPLE — FIND THE PERMUTATION IMPORTANCE OF THREE DELIVERY FEATURES

Karwan checks Sunrise Parcels' full random forest on a validation set of past deliveries. Its baseline F1 score is 0.80. After shuffling Distance alone, the F1 score falls to 0.62. After shuffling Stops alone, it falls to 0.71. After shuffling Rain alone, it falls to 0.79. Find each feature's permutation importance, and rank the three features.

1. Find Distance's importance: baseline minus shuffled: $0.80 - 0.62 = 0.18$.
2. Find Stops's importance: $0.80 - 0.71 = 0.09$.
3. Find Rain's importance: $0.80 - 0.79 = 0.01$.
4. Rank the three drops from largest to smallest: 0.18, then 0.09, then 0.01.

Answer: Distance has the largest permutation importance, 0.18, followed by Stops at 0.09, and Rain barely matters at 0.01. This matches the same basic ranking Chapter 10 found using impurity, now confirmed a second way, by directly testing what happens to real predictive performance when each feature is broken.

Permutation importance has one blind spot of its own, and it is worth seeing exactly how it happens. Karwan adds a new feature, estimated travel time, which mostly just restates distance in a different unit, since a longer distance almost always means a longer estimated travel time.

WORKED EXAMPLE — SEE HOW TWO CORRELATED FEATURES CAN HIDE EACH OTHER

With the new feature added, the model's baseline F1 score is 0.81. Shuffling Distance alone drops it to 0.76. Shuffling Estimated Travel Time alone drops it to 0.75. Shuffling both of them together, in the same shuffled copy of the data, drops it to 0.60. Find all three importances, and compare shuffling them separately with shuffling them together.

1. Find Distance's importance, shuffled alone: $0.81 - 0.76 = 0.05$.
2. Find Estimated Travel Time's importance, shuffled alone: $0.81 - 0.75 = 0.06$.
3. Add these two separate importances together: $0.05 + 0.06 = 0.11$.
4. Find the importance of shuffling both features together, in the same pass: $0.81 - 0.60 = 0.21$.
5. Compare the combined shuffle, 0.21, with the sum of the two separate shuffles, 0.11.

Answer: Shuffled separately, Distance and Estimated Travel Time each look only mildly important, 0.05 and 0.06. Shuffled together, the drop is 0.21, almost twice the sum of the two separate drops. When one of the pair is shuffled alone, the model can still lean on its closely correlated twin to cover for the missing information. Only breaking both at once reveals how much the model truly depended on that shared information.

Python · NumPy

 runnable in the app

```
from sklearn.inspection import permutation_importance
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=0).fit(X_train, y_train)
result = permutation_importance(
    model, X_val, y_val, scoring="f1", n_repeats=10, random_state=0
)

for name, score in zip(feature_names, result.importances_mean):
    print(name, round(score, 2))
```

The `permutation_importance` function automates the exact same steps you just did by hand. Setting `n_repeats=10` shuffles each feature ten separate times and averages the drop, because any single shuffle carries some luck of its own, and averaging several shuffles gives a steadier answer. The `scoring="f1"` argument tells it which score to watch shrink, matching the F1 score used

throughout this lesson. The result holds one average importance number per feature, in the same units as the hand-calculated drops above.

Permutation importance also has a practical advantage that impurity-based feature importance does not share. Because it only needs to shuffle a column and re-score the model, it works exactly the same way on a linear regression, a k-Nearest Neighbors model, a random forest, or a neural network. Impurity-based importance, by contrast, only makes sense for trees. This is one reason permutation importance has become one of the most broadly useful tools in this entire chapter.

An unusually large permutation importance score is also a useful warning sign for a much older problem: Chapter 2's **data leakage**. If one feature's importance towers far above every other feature's, larger than the story reasonably explains, it is worth asking whether that feature could only exist after the true answer is already known. A feature that leaked the label into training would often show up here as a strangely dominant score, well before it ever became an unpleasant surprise once the model met real, live data.

COMMON MISTAKE

✗ Wrong: Treating a feature's large permutation importance as proof that changing that feature in the real world would change the outcome.

✓ Right: Permutation importance only shows how much the fitted model depended on a feature for making predictions on this data. It is a measure of dependence, not a measure of cause. Chapter 10 made exactly this same point about impurity-based importance, and it applies here too.

IMPORTANT

Do not turn a permutation importance score directly into a real-world policy. If Distance has high permutation importance in a late-delivery model, that does not automatically mean shortening routes will reduce late deliveries. It only means the model leans on distance heavily to make its predictions. Testing a real change in the real world, such as a small controlled trial on a few routes, is a very different kind of evidence than a model's internal dependence on a feature.

18.4 Fairness: The Same Word, a Different Meaning

The last two lessons asked what a model depends on. This lesson asks a different, and in some ways larger, question: even once you fully understand a model, could it still be treating one group of people worse than another? Answering this well needs one careful vocabulary pause first.

Historical data is not a neutral photograph of the world. It is a record of past decisions, made by people, inside systems that were not always equally fair to everyone. A model trained faithfully on that record can learn its patterns just as faithfully, including the unfair ones, without a single person involved intending any harm. This is exactly why checking fairness cannot be skipped just because nobody meant for a problem to happen.

This lesson uses the word bias again, and it is worth pausing on purpose, because the word does not mean the same thing here as it did in Chapter 5. Chapter 5 gave bias one precise, statistical meaning: a model's tendency to make the same kind of mistake, in the same direction, again and again, because its assumptions are too simple for the real pattern in the data — like the Flat model from that chapter, which always under-predicted wet fields and over-predicted dry ones. That kind of bias is a mismatch between a model's predictions and the true numbers it is trying to predict. Fairness work uses the word bias too, but for a related, different idea: a pattern in a model's predictions or errors that systematically disadvantages one group of people more than another, often because the historical data already contained that same unfair pattern, or because of a careless design choice nobody meant to make. Both meanings share one thing worth noticing: a consistent, repeated pattern, not a one-time accident. But hold the two meanings side by side, and the difference becomes clear. Chapter 5's bias is a mismatch between a model and the numbers it is trying to predict. This chapter's bias is a mismatch between how a model treats one group of people and how it treats another. From here on, whenever this book uses the word "bias" while discussing fairness, it means this second, social meaning, and the sentence around it will always make that clear.

WORD TO KNOW

Fairness — The requirement that a model's benefits, errors, and decisions do not systematically disadvantage one group of people more than another, in ways that matter for the situation. What counts as fair depends on context, and usually needs more than one kind of check.

Checking fairness starts with a habit you already know from Chapter 4: instead of trusting one combined score for an entire dataset, break that score apart by group, and compare the pieces.

WORKED EXAMPLE — COMPARE A SCHOLARSHIP MODEL'S FALSE-NEGATIVE RATE ACROSS TWO GROUPS

Peshraw University's scholarship-ranking model is checked against 200 applicants a review committee already judged, by hand, to be genuinely qualified for a scholarship. Of these, 40 attended a rural secondary school, and the model's shortlist missed 12 of them. The other 160 attended an urban secondary school, and the model's shortlist missed 8 of them. Find the false-negative rate for each group, and the combined false-negative rate for all 200 applicants together.

1. Find the rural group's false-negative rate: missed applicants divided by truly qualified applicants: $12 \div 40 = 0.30$, or 30%.
2. Find the urban group's false-negative rate: $8 \div 160 = 0.05$, or 5%.
3. Find the combined false-negative rate for both groups together: total missed divided by total qualified: $(12 + 8) \div (40 + 160) = 20 \div 200 = 0.10$, or 10%.
4. Compare all three numbers: 30%, 5%, and a combined 10%.

Answer: The combined false-negative rate, 10%, looks reasonably low on its own. But it hides a large gap: qualified rural applicants are missed 30% of the time, six times the rate for qualified urban applicants, who are missed only 5% of the time. A single combined number would have hidden this gap completely.

Alarmed by this gap, Peshraw University's committee adjusts the model so both groups reach a similar false-negative rate. A natural next question is whether fixing this one measure quietly fixes fairness altogether, or whether a different, equally honest measure might still tell a different story.

WORKED EXAMPLE — CHECK A SECOND FAIRNESS MEASURE AFTER FIXING THE FIRST

After adjusting the model, both groups now have a false-negative rate near 10%, matching the combined figure found just above. The committee also checks the overall selection rate: the share of every applicant from each group, qualified or not, who actually receives a scholarship offer. Among 200 rural applicants, 24 receive an offer. Among 800 urban applicants, 168 receive an offer. Find each group's selection rate, and compare them.

1. Find the rural selection rate: offers divided by total rural applicants: $24 \div 200 = 0.12$, or 12%.
2. Find the urban selection rate: $168 \div 800 = 0.21$, or 21%.
3. Compare the two rates: 12% for rural applicants against 21% for urban applicants.

Answer: Even after equalizing the false-negative rate at about 10% for both groups, the overall selection rate is still noticeably lower for rural applicants, 12%, than for urban applicants, 21%. Equal treatment on one honest measure did not automatically create equal treatment on a different, equally honest measure. The committee, not a formula, must decide which measures matter most for this particular decision, and say so openly.

IMPORTANT

There is no single formula that settles whether a model is fair. Equalizing one measure across groups, such as the false-negative rate above, can leave a different measure, such as the overall selection rate, unequal between the very same two groups, exactly as the example above just showed. Different fair-sounding rules can genuinely conflict with each other when groups differ in other ways too. The right choice depends on context, consequences, and often law, which is exactly why a human review process belongs in every high-impact decision, not a single automatic score.

Comparing outcomes by group is the right habit. But it only works if the model, and the team studying it, can even tell which applicants belong to which group in the first place — and that turns out to be a more complicated question than it first appears.

18.5 Protected Attributes and Proxy Variables

WORD TO KNOW

Protected attribute — A personal characteristic, such as gender, age, ethnicity, religion, or disability status, that law or an organization's own policy says must not be the reason someone receives a worse outcome. Protected attributes vary somewhat by country and by situation.

A common, understandable instinct is to keep a protected attribute completely out of a model's features, reasoning that a model cannot discriminate by gender, for instance, if it never sees a gender column at all. This instinct is not wrong, exactly, but it is not enough by itself, because of a second kind of feature.

WORD TO KNOW

Proxy variable — A feature that is not itself a protected attribute, but is so closely related to one that it carries almost the same information. A model can rebuild a pattern connected to a protected attribute through a proxy, even after the protected attribute itself is removed from the data.

A student's home postcode, school name, or even first name can each act as a proxy in different situations, because each one can be closely tied to income, ethnicity, or region, depending on where the university is and how its towns and neighborhoods are organized. Removing the protected attribute's own column does nothing, by itself, to stop a proxy from carrying nearly the same information back in.

WORKED EXAMPLE — MEASURE HOW WELL A SCHOOL DISTRICT PREDICTS A REMOVED ATTRIBUTE

Peshraw University's data team removes the rural/urban flag from their scholarship model entirely, satisfied that this solves the fairness gap from Lesson 18.4. A colleague checks the remaining features first. Of the same 200 applicants, 190 attend a school district that matches their rural or urban status perfectly: applicants from two specific districts are always rural, and applicants from three other specific districts are always urban. Only 10 applicants, who attend an urban boarding school despite living in a rural area, break this pattern. Find what percentage of applicants their school district alone would correctly classify as rural or urban.

1. Find how many of the 200 applicants district correctly predicts: subtract the 10 exceptions from the total: $200 - 10 = 190$.
2. Divide the correct predictions by the total number of applicants: $190 \div 200 = 0.95$.
3. Convert the result to a percentage: $0.95 \times 100 = 95\%$.

Answer: School district alone would correctly guess an applicant's rural or urban status about 95% of the time. Simply deleting the rural/urban column did not remove this information from the dataset. It was still sitting there, almost perfectly, inside the district column, which the model remained completely free to use.

Nobody on Peshraw University's data team set out to rebuild the rural/urban flag on purpose. The district column was included for an entirely ordinary reason, to help the admissions office route paperwork to the correct regional branch. A proxy rarely announces itself. It usually hides inside a feature that was added for a completely different, entirely innocent purpose, which is exactly what makes proxies easy to miss without a deliberate check.

COMMON MISTAKE

✗ Wrong: Believing that deleting a protected attribute's column from the training data is enough, by itself, to make a model fair.

✓ Right: Check remaining features for proxies, such as location, school, or employer, that could rebuild the same pattern. Also test the model's actual group-level outcomes directly, from Lesson 18.4, rather than trusting that removing one column solved the problem.

IMPORTANT

Sometimes the safest choice is the opposite of deleting a protected attribute. Keeping it in a separate, carefully controlled dataset, used only to measure group-level outcomes like Lesson 18.4's false-negative rates, lets a team check for unfairness directly instead of guessing at it through proxies. Access to that data should be limited, logged, and used only for this auditing purpose, never handed to the prediction model itself as a feature.

18.6 Try It in ML Studio

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Scholarship Ranking" dataset and fit a Random Forest.
3. Open the Explain tab and read the impurity-based feature importance chart.
4. Switch to the Permutation tab, compare the new ranking with the impurity-based one, and write down whether any feature's rank changed.
5. Open the Fairness tab, choose a group column, and compare the selection rate and the false-negative rate across groups. Write one sentence describing what you notice.

You need: Computer, tablet, or phone with the ML Studio app

Watching a bar chart shrink after a shuffle, or watching two group bars sit at different heights on the Fairness tab, tends to make both ideas click far faster than reading about them alone. Notice, too, that ML Studio never tells you whether a gap is acceptable. That judgment call belongs to a person, informed by the numbers, never handed off to the software itself.

The case study below follows one team through this entire chapter, from a first, purely technical model to a fuller review that a real organization could stand behind.

18.7 Case Study

CASE STUDY — AUDITING A HIRING-SCREENING MODEL IN CHENNAI

Chalo Rides, a ride-hailing company with a large customer-support center in Chennai, receives thousands of applications every month for a small number of customer-support openings. For years, a small team of recruiters read every application by hand, a slow process that left strong candidates waiting for weeks for a first reply. The company builds a screening model, trained on years of past applications together with each hired employee's first-year performance rating, to produce a shortlist of applicants for a recruiter to interview. The model itself never makes a hiring decision. It only decides who gets a closer look.

Before the model goes anywhere near a real applicant, the recruiting team runs it through this chapter's full checklist. Reading the model's feature importance shows it leans most heavily on typing-test score and months of prior customer-service experience, both features that connect sensibly to the job. It also shows a smaller, curious weight on distance from the office, which sounds harmless on its own, a simple commute-time question. Permutation importance confirms the same ranking is real, not just an accident of the trees' internal splits.

The team then checks group-level outcomes, exactly as Lesson 18.4 taught, comparing the shortlist rate for men and women applicants who scored equally well on the company's own structured interview questions in past hiring rounds. Among equally strong past applicants, the model shortlists men at a noticeably higher rate than women. Investigating further, the team finds the cause: distance from the office is acting as a proxy, exactly like Lesson 18.5's school-district example, because past women applicants more often lived farther from the office, in neighborhoods the company had not previously served well with evening transport, and the training data's old outcome labels already reflected that old, unrelated barrier rather than any real difference in job performance.

Chalo Rides does not throw the model away, and it does not ignore the finding either. It removes distance from the office as a feature entirely, retrains, and confirms on a fresh check that the shortlist gap closes. It also keeps gender in a separate, access-controlled file used only to run this same fairness check every quarter, never as a feature the model can see. Most importantly, the model still only produces a shortlist. A human recruiter reviews every application on that shortlist personally, and any applicant who is not shortlisted may still ask a person to review their case by hand. Chalo Rides publishes a short, plain-language description of what the tool does and does not do, so applicants are never left guessing about how they were assessed. The team also commits to rerunning this entire audit every quarter, not only once, since new hiring rounds keep adding fresh data that could quietly reopen the same gap. Chapter 19 explains exactly why that ongoing habit matters: a model's fairness, like its accuracy, needs to be watched continuously after launch, not checked once and then trusted forever.

COMMON MISTAKES TO AVOID

- Trusting a model because of a high validation score, without ever checking which features it relies on or whether it treats every group fairly.
- Comparing raw coefficient sizes across features that sit on very different scales, and mistaking the bigger number for the more important feature.
- Reading only an impurity-based feature importance score and assuming it also reveals which direction a feature pushes the prediction.
- Treating a large permutation importance score as proof that a feature causes the outcome, rather than proof that the model merely depends on it.
- Believing that deleting a protected attribute's column from the data automatically makes a model fair, without checking for proxy variables.
- Reporting only one combined, dataset-wide fairness number instead of comparing outcomes separately across groups.
- Assuming a single fixed formula can settle what counts as "fair" for every situation, regardless of context or consequences.

CHAPTER SUMMARY

- Interpretability tools help explain what a model does, but they do not automatically prove a model is safe, fair, or trustworthy on their own.
- A linear model's coefficient shows both the direction and the size of a feature's effect; a tree ensemble's feature importance shows only how much a feature helped, never which direction it pushes predictions.
- Permutation importance measures a feature's true contribution to a fitted model's score by shuffling that feature and watching the score drop.
- Correlated features can hide each other's importance when shuffled one at a time; shuffling them together reveals their true combined effect.
- A high importance score, or a strong coefficient, shows dependence, not proof of a real-world cause.
- This chapter's "bias" is a social, fairness-related idea, a pattern of decisions that disadvantages a group of people. It is a different meaning from Chapter 5's statistical bias, even though both describe a consistent, repeated pattern rather than a one-time accident.
- Fairness has no single formula; comparing group-level outcomes, such as selection rate and false-negative rate, can reveal gaps that one combined score hides.
- A protected attribute cannot always be protected simply by deleting its column, because a proxy variable, such as location or school, can carry almost the same information back in.
- High-impact decisions need a human review step and ongoing group-level monitoring, not a single automatic score.
- Chapter 19 continues this responsibility after a model goes live, watching for the ways a deployed model's world can quietly change.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define interpretability in your own words.

Problem 2 [Basic] State whether this situation matches Chapter 5's "bias" or this chapter's "bias," and explain your choice in one sentence: "A scholarship model scores rural districts several points lower on average than urban districts, even after accounting for exam performance."

Problem 3 [Basic] Define protected attribute, and give two examples of your own that were not used in this chapter.

Problem 4 [Basic] Explain, in one or two sentences, why deleting a protected attribute's column from a dataset does not guarantee a fair model.

Problem 5 [Standard] A linear model predicts a delivery's delay in minutes: $\text{predicted delay} = 6 + 2 \times (\text{distance in kilometers}) + 3 \times (\text{stops})$. Find the predicted delay for a delivery with distance 10 km and 2 stops, showing every step.

Problem 6 [Standard] Using the same model as Problem 5, find the predicted delay if the same delivery instead had 5 stops, with distance unchanged. By how many minutes did the prediction rise, and does that match the stops coefficient?

Problem 7 [Standard] A model's baseline validation accuracy is 0.88. After shuffling Feature X, accuracy falls to 0.81. After shuffling Feature Y, accuracy falls to 0.86. Find the permutation importance of each feature, and state which one the model depends on more.

Problem 8 [Standard] Group A has 50 truly qualified applicants, and the model misses 5 of them. Group B has 30 truly qualified applicants, and the model misses 9 of them. Find each group's false-negative rate, and the combined false-negative rate for both groups together.

Problem 9 [Standard] Explain, in your own words, why shuffling two correlated features one at a time can understate how important they really are, using an example different from the one in this chapter.

Problem 10 [Standard] A colleague says, "Our hiring model doesn't use gender, so it cannot be biased." Write a two- to three-sentence reply, using at least one term from this chapter.

Problem 11 [Challenge] A hospital's readmission-risk model has an overall false-negative rate of 12%. (a) Explain one reason this single number might hide a problem. (b) Describe two specific group-level checks you would run before trusting the model. (c) Name one non-model factor that should also be part of the decision to use this tool.

Problem 12 [Challenge] (a) Name a feature that could act as a proxy for income in your own country or region. (b) Explain why simply removing income itself

from a dataset would probably not remove this proxy's effect. (c) Suggest one process-based safeguard, besides removing a column, that a team could add.

Problem 13 [Challenge] Open ML Studio, fit a Random Forest on a dataset with at least one demographic column, and compare its impurity-based feature importance with its permutation importance. Report any features whose rank changed, and suggest one reason why.

BUILDING RELIABLE PROJECTS

CHAPTER 19

From Notebook to Production

Keeping a model healthy after it goes live

IN THIS CHAPTER YOU WILL LEARN TO

- Explain what deployment means, and why a production system needs more than just a saved model file
- Recognize data drift, and check whether a feature's distribution has shifted using a simple percent-change calculation
- Describe the layers a monitoring plan should watch, from incoming data to real outcomes, and compute a live error measure by hand
- Explain training-serving skew, and describe why reusing the exact same pipeline for training and live prediction helps prevent it
- Explain what a rollback is, and why it is usually the safer first response to a serious monitoring alert
- Design a basic monitoring and rollback plan for a real deployed model

19.1 From a Script to a Real System

Every chapter so far has worked with one dataset that sits still. You load it once, split it, train on it, and check a score, and the data never changes while you work. Real projects are rarely this calm. A model that goes into real use meets new data every day, sometimes every second, and that new data almost never behaves as politely as the tidy file you trained on.

Think about the difference between cooking a new recipe once for your own family, on a quiet evening, and cooking that same recipe every night for a busy restaurant. At home, you control every ingredient and can taste as you go. In a restaurant kitchen, ingredients sometimes arrive late, an oven runs a little hot, and a cook might reach for the wrong-sized spoon without ever noticing. The recipe has not changed, but the kitchen around it is far less forgiving of small mistakes. Turning a trained model into a real, running system is a similar jump. The model itself does not change, but everything around it suddenly becomes far less forgiving.

WORD TO KNOW

Deployment — Making a trained model available so it produces real predictions that people or other systems actually use, instead of sitting inside a notebook or script that only its author runs by hand.

A deployed model usually runs in one of two rhythms. A batch system runs on a schedule, processing many examples at once. Roj Solar Supply, a solar-equipment retailer with stores across the Kurdistan Region, runs its demand-

forecasting model this way: every Sunday night, the system forecasts next week's expected sales for every product line, and the purchasing team reads the results Monday morning before placing orders. An on-demand system instead answers one request at a time, the moment it is asked. A loan-approval model that scores one application the instant a customer submits it is a good example of this second rhythm, and so is a ride-hailing app estimating one trip's arrival time the moment a rider requests a car. Both rhythms follow the same underlying idea: a saved, trained model, waiting to turn new input into a prediction whenever it is called on, whether that call comes once a week or a thousand times a minute.

Whichever rhythm a system uses, one rule stays constant. Chapter 3 introduced the **pipeline**, an ordered chain of preprocessing steps and a model, bundled together into one single object. Deployment should always run that exact same saved pipeline, never a hand-rewritten copy of its steps. A live system that recomputes preprocessing from scratch, even carefully, risks a subtle mismatch with what the model actually learned during training, a problem serious enough to earn its own name later in this chapter.

Python · NumPy

 runnable in the app

```
import joblib

joblib.dump(pipeline, "demand_model_v3.joblib")
loaded_pipeline = joblib.load("demand_model_v3.joblib")

check = loaded_pipeline.predict(X_check)
original = pipeline.predict(X_check)
assert (check == original).all()
```

Saving a pipeline with `joblib.dump` freezes every preprocessing step and the model together, in one single file, exactly as Chapter 3 built it. Loading it back with `joblib.load`, then predicting on a small, familiar check set, `X_check`, confirms the reloaded pipeline still behaves identically to the original. The `assert` line simply stops the program if the two sets of predictions ever disagree, a cheap, fast habit worth running every time a new version is about to replace an old one.

Notice the file name also carries a version number, `v3`. Keeping every past version available, not only the newest one, is what makes Lesson 19.4's rollback possible later. A system that only ever keeps "the current model," with no record of what came before it, has nothing safe to fall back to when something eventually goes wrong.

Saving the pipeline correctly is only half the job. The live system also needs to decide what to do with input that never should have reached the model in the first place.

WORKED EXAMPLE — DECIDE WHICH INCOMING ORDERS A SYSTEM SHOULD ACCEPT

Roj Solar's live ordering system checks four incoming purchase requests before handing them to its pricing model. Each request must have a positive quantity, a known product code, and a delivery city inside the company's service area. Request A: quantity 5, product code SP-100, city Erbil. Request B: quantity -2, product code SP-100, city Duhok. Request C: quantity 3, product code ZZ-999, city Sulaymaniyah. Request D: quantity 8, product code SP-210, city Mosul, which is outside the service area. Decide which requests the system should accept, and which it should reject or flag for review.

1. Check Request A: quantity 5 is positive, SP-100 is a known product code, and Erbil is inside the service area. Every check passes, so accept it.
2. Check Request B: quantity is -2, a negative number, which is not a sensible quantity for a purchase. Reject it.
3. Check Request C: quantity 3 is fine, and Sulaymaniyah is inside the service area, but ZZ-999 is not a known product code. Reject or flag it.
4. Check Request D: quantity 8 is fine and SP-210 is a known code, but Mosul is outside the service area the model was built for. Reject or flag it.

Answer: Only Request A should pass straight through to the pricing model. Requests B, C, and D each fail one specific check, and each should be rejected or sent for human review, rather than quietly forced through a model that was never built to handle them.

COMMON MISTAKE

✗ Wrong: Treating a model that scored well in a notebook as automatically ready for real customers or real decisions to depend on.

✓ Right: Real input can be messy, late, or shaped differently than the tidy training file ever was. A production system needs to handle unusual input gracefully, not just repeat whatever worked once on a clean, carefully prepared dataset.

IMPORTANT

A live system should check every incoming input before handing it to a model, and reject or flag anything outside a sensible range, such as a negative price or a missing required field. Never let a broken or nonsensical input quietly turn into a confident-looking prediction. A plain refusal, or a request for human review, is far safer than a fluent-looking guess built on bad information.

19.2 Data Drift: The World Keeps Changing

A model learns its rule from one snapshot of the world: whatever the training data happened to look like at the time it was collected. The world does not hold still just because a model has finished training.

WORD TO KNOW

Data drift — A change, over time, in the data a deployed model sees, compared with the data it was trained on. Drift can show up in the input features themselves, or in how those features relate to the true answer, and either kind can make old predictions less trustworthy.

Roj Solar Supply's demand-forecasting model uses several features to predict next week's battery sales, including the number of inquiries the company's website receives each week. For months, this number moves inside a narrow, predictable band. Then a government program announces a new subsidy for home solar batteries, and customer interest changes almost overnight. Roj Solar's analytics team treats any feature whose weekly average moves by more than 20% as worth a closer look, rather than dismissing the change as ordinary week-to-week noise.

WORKED EXAMPLE — DETECT DRIFT IN WEEKLY WEBSITE INQUIRIES

In the 8 weeks before the subsidy announcement, weekly website inquiries were 36, 40, 42, 38, 41, 39, 43, and 41. In the 8 weeks after the announcement, weekly inquiries were 66, 70, 72, 68, 71, 69, 73, and 71. Find the average for each period, the percent change between them, and state whether this counts as drift under Roj Solar's 20% rule.

1. Add the 8 "before" weekly counts and divide by 8: $36 + 40 + 42 + 38 + 41 + 39 + 43 + 41 = 320$, and $320 \div 8 = 40$.
2. Add the 8 "after" weekly counts and divide by 8: $66 + 70 + 72 + 68 + 71 + 69 + 73 + 71 = 560$, and $560 \div 8 = 70$.
3. Find the difference between the two averages: $70 - 40 = 30$.
4. Divide the difference by the original "before" average, then convert to a percentage: $30 \div 40 = 0.75$, and $0.75 \times 100 = 75\%$.
5. Compare this 75% change with the team's 20% threshold for a closer look.

Answer: Weekly website inquiries rose by about 75%, far past Roj Solar's 20% threshold. This is real, meaningful drift, not ordinary noise, and it earns a closer look before the team keeps trusting the forecasting model's next few predictions.

Left unnoticed, drift rarely announces itself with one dramatic failure. It usually shows up as a slow, easy-to-explain-away slide: a few more stockouts than usual, a few more unsold pallets sitting in a warehouse, each one small enough to blame on an ordinary bad week. By the time anyone notices a pattern, weeks of purchasing decisions may already rest on a forecast the world has quietly outgrown.

COMMON MISTAKE

✗ Wrong: Assuming a model that performed well when it first launched will keep performing well forever, without ever checking again.

✓ Right: Plan to check for drift regularly, on a fixed schedule, not only when someone happens to notice a problem. The data a model meets in month six is rarely identical to the data it trained on.

19.3 Monitoring What Matters

Knowing that drift can happen is not the same as catching it in time. Catching it, and other problems besides, is the job of a deliberate, ongoing habit.

WORD TO KNOW

Monitoring — Regularly checking a deployed model's incoming data, its predictions, and its real-world outcomes, so that problems are caught quickly instead of discovered by accident, much later.

A useful monitoring plan checks three different layers, and each layer catches a different kind of problem. The first layer watches the incoming data itself: are website inquiries, prices, and product categories arriving in the ranges and formats the model expects, with no sudden flood of missing values? The second layer watches the model's predictions: is the forecasting model suddenly predicting far higher or far lower demand than usual, across many products at once, in a way that looks suspicious on its own, before anyone even knows the true sales figures yet? The third layer waits for real outcomes, then compares them honestly with what the model predicted, exactly the way Chapter 4 taught you to score a model in the first place, just repeated on a schedule, forever, instead of only once.

These three layers form a kind of early-warning chain. The first layer can flag a problem within minutes, since it needs nothing but the input itself. The second layer can flag a problem within hours, once enough predictions accumulate to look unusual as a group. The third layer is the slowest, sometimes taking days or weeks until real outcomes are finally known, but it is also the most convincing of the three, because it checks the one thing that actually matters in the end: was the model right?

If a deployed model is a classifier producing probabilities rather than a plain yes-or-no answer, Chapter 16 introduced **calibration**: checking whether a predicted probability of 70% really does come true about 70% of the time. Calibration can quietly drift too, even while a model's plain accuracy looks unchanged, so a full monitoring plan for a classifier rechecks calibration on fresh data now and then, not only once, back when the model was first built.

WORKED EXAMPLE — CHECK WHETHER A LIVE CLASSIFIER IS STILL CALIBRATED

A bank's loan-risk model predicted a 20% chance of default for a group of 150 loans issued three months ago. Enough time has now passed to know the true outcome for all 150 loans. Of these, 42 actually defaulted. Compare the predicted default rate with the actual default rate, and decide whether the model's calibration still looks healthy.

1. Find how many defaults the model's 20% prediction implies for this group of 150: $0.20 \times 150 = 30$ expected defaults.
2. Find the actual number of defaults that happened: 42.
3. Find the actual default rate: $42 \div 150 = 0.28$, or 28%.

4. Compare the predicted rate, 20%, with the actual rate, 28%, a gap of $28 - 20 = 8$ percentage points.

Answer: The model predicted a 20% default rate for this group, but 28% of the loans actually defaulted, an 8-percentage-point gap. A model that was well calibrated at launch is no longer matching reality this closely, a sign worth investigating alongside any drift or skew checks already underway.

Roj Solar's third monitoring layer, real outcomes, is the most convincing one, because it compares a prediction with what actually happened, not just with what seems reasonable.

WORKED EXAMPLE — COMPARE RECENT FORECAST ERROR WITH THE ORIGINAL VALIDATION ERROR

When Roj Solar's demand-forecasting model was first built, its validation MAE was 12 units per week. Over the most recent 4 live weeks, true and predicted battery demand were: Week 1, true 100, predicted 98. Week 2, true 132, predicted 101. Week 3, true 90, predicted 95. Week 4, true 150, predicted 108. Find the model's recent MAE, and compare it with its original validation MAE.

1. Find each week's absolute error, true minus predicted, ignoring the sign: $|100 - 98| = 2$. $|132 - 101| = 31$. $|90 - 95| = 5$. $|150 - 108| = 42$.
2. Add the four absolute errors together: $2 + 31 + 5 + 42 = 80$.
3. Divide by the number of weeks, 4, to find the recent average error: $80 \div 4 = 20$.
4. Compare this recent MAE, 20 units, with the model's original validation MAE, 12 units.

Answer: The model's recent error, 20 units a week, is well above its original validation error of 12 units a week, nearly double. This is exactly the kind of signal the third monitoring layer, real outcomes, is built to catch, and it deserves investigation, not a shrug.

COMMON MISTAKE

✗ Wrong: Monitoring only the health of incoming data, such as missing values and sensible ranges, but never checking real prediction errors once true outcomes become available.

✓ Right: A model can look perfectly healthy on the data layer for weeks while quietly making much worse predictions than before. Always close the loop with the third layer, real outcomes, as soon as they exist.

19.4 Training-Serving Skew and Safe Rollback

Lesson 19.2 described a problem caused by the real world changing. This lesson describes a different problem, one that can strike even when the real world has not changed at all.

WORD TO KNOW

Training-serving skew — A mismatch between how a feature is computed during training and how that same feature is computed for real, live predictions.

The deployed model ends up receiving different information than it learned from, even though nothing about the real world has actually changed.

Skew is almost the mirror image of Chapter 2's **data leakage**. Leakage sneaks extra, unavailable information into training that would not really be available at prediction time. Skew does the opposite: it quietly feeds the live model different, often poorer, information than the model ever learned to expect, usually by accident, through two pieces of code that were each written separately and have quietly stopped agreeing with each other.

Skew is particularly hard to catch because it hides in plain sight. The feature still has the same name, still holds a plausible-looking number, and the model still produces a confident, ordinary-looking prediction. Nothing about the output looks obviously broken. Only a careful, side-by-side comparison, like the one below, reveals that the number actually feeding the model no longer means what it meant back during training.

Roj Solar's forecasting model uses a feature called "average daily inquiries," meant to be a full 7-day average. A quiet bug reveals exactly how skew can happen.

WORKED EXAMPLE — COMPARE HOW TRAINING AND LIVE SYSTEMS COMPUTE THE SAME FEATURE

During training, "average daily inquiries" was correctly computed as the average of 7 full days: 40, 45, 38, 42, 44, 41, and 40 inquiries. Because of a timing bug, the live system instead reports only the current, still-incomplete day's count, captured early each morning: 15 inquiries. Compute the feature the way training computed it, compare it with the value the live system actually sends the model, and state whether this looks like drift or like skew.

1. Add the 7 full days used during training: $40 + 45 + 38 + 42 + 44 + 41 + 40 = 290$.
2. Divide by 7 to find the training-time value of the feature: $290 \div 7 \approx 41.43$.
3. Compare this with the value the live system actually sends: 15.
4. Find the size of the gap between the two: $41.43 - 15 = 26.43$.
5. Ask whether real customer behavior changed. It did not; only the calculation changed. This points to skew, not drift.

Answer: Training computed "average daily inquiries" as about 41.43, while the live system was actually sending 15 for the very same feature name, a gap of more than 26. No customer behavior caused this difference. A quiet mismatch in how the two systems compute the same feature did, which is training-serving skew, not data drift. Left uncaught, a bug this size could push every forecast far lower than it should be, no matter how good the original model was.

Reusing Chapter 3's saved **pipeline** for both training and live prediction is the strongest defense against this exact problem. When one single, saved object computes every feature the same way every time, there is no second copy of the logic left to quietly drift out of agreement with the first.

WORD TO KNOW

Rollback — Switching a live system back to its last known-good model or version, quickly, when monitoring shows the current version is behaving badly, rather than leaving the faulty version live while investigating.

Deciding when a problem is serious enough to roll back is much easier when a team agrees on a clear rule in advance, before any incident, rather than debating it in the middle of one.

WORKED EXAMPLE — DECIDE WHETHER AN INCIDENT CROSSES ROJ SOLAR'S ROLLBACK LINE

Roj Solar's team agrees in advance that if live MAE ever reaches 1.5 times the validation MAE for two consecutive weeks, they roll back immediately rather than keep investigating live. Validation MAE is 12 units. In Week 1 of an incident, live MAE is 18 units. In Week 2, live MAE is 24 units. Decide whether this incident crosses the rollback line, and in which week.

1. Find the rollback threshold: $1.5 \times 12 = 18$ units.
2. Compare Week 1's live MAE, 18, with the threshold, 18. It meets the threshold exactly, so it counts as crossing it.
3. Compare Week 2's live MAE, 24, with the threshold, 18. 24 is well above 18, so it also crosses the line.
4. Since both consecutive weeks meet or exceed the threshold, the team's two-week rule is satisfied.

Answer: The incident crosses Roj Solar's rollback line as early as Week 1, and Week 2 confirms it was not a one-week fluke. Under the team's own pre-agreed rule, this calls for an immediate rollback to the last known-good version, not more open-ended investigation while live customers are still affected by the problem.

IMPORTANT

When monitoring shows a clear, serious problem, the safest first move is usually to roll back to the last known-good version, and investigate calmly afterward. Do not leave a misbehaving model live while slowly debugging it in front of real customers, and do not silence or dismiss a monitoring alert just because its cause is not obvious yet.

COMMON MISTAKE

✗ Wrong: Retraining on the newest data the moment a monitoring alert fires, as an automatic, reflexive fix.

✓ Right: Diagnose first. A retrain on top of an undiagnosed skew bug can bake the bug in even deeper. Confirm whether the true cause is real drift, a skew bug, or a short-lived blip, and only then decide whether retraining is the right repair.

19.5 Try It in ML Studio and Case Study

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Fit any regression model on the "Demand Forecast" dataset, and write down its validation MAE.
3. Go to the Labs tab and open the "Drift Simulator."
4. Slide the drift amount up from 0%, pressing Run at each step, and watch the live MAE line pull away from the validation MAE line.
5. Find the drift level where you would personally choose to roll back rather than keep watching, and write one sentence explaining your choice.

You need: Computer, tablet, or phone with the ML Studio app

Watching the live MAE line drift away from the validation line, a little more with every step of the slider, tends to make Lesson 19.2's abstract idea of drift feel concrete in a way that a page of numbers alone cannot.

The case study below gathers this chapter's whole story into one place, following a real deployed model from a quiet launch, through a genuine incident, to a calm and deliberate fix.

CASE STUDY — MONITORING A DEMAND-FORECASTING MODEL AT ROJ SOLAR SUPPLY

Roj Solar Supply sells solar panels, batteries, and inverters from stores in Erbil, Duhok, and Sulaymaniyah. Two years ago, its analytics team built a weekly demand-forecasting model, following the same careful workflow this book has taught from Chapter 2 onward: a clear baseline, a validated pipeline, and an honest test score before anyone trusted it with a real decision. The finished model was deployed as a batch job, exactly as Lesson 19.1 described, running every Sunday night and handing the purchasing team a fresh forecast every Monday morning. For four months, it performed close to its validation score, and the purchasing team grew to rely on it.

Then, in the fifth month, the national government announces a new subsidy for home solar battery installations. Within two weeks, the model's monitoring dashboard, watching all three of Lesson 19.3's layers, shows its performance layer sliding: weekly forecast MAE climbs from its usual 12 units to 18, then to 24, over three consecutive weeks. The data layer shows something too: website inquiries have jumped far past the team's 20% drift threshold, echoing Lesson 19.2's own example almost exactly.

Before assuming a cause, the team follows Lesson 19.4's order of operations. First, they rule out training-serving skew, confirming that the live system is still calling the exact same saved pipeline used during training, with no separately rewritten feature logic hiding anywhere. Second, they compare feature distributions directly, the same kind of before-and-after check as Lesson 19.2's worked example, and confirm the shift is real: customers genuinely are inquiring far more often, driven by authentic new demand, not a bug.

With the cause clearly identified as real drift, not skew, the team does not retrain in a panic. For two weeks, they roll back the purchasing team's reliance to a simpler, more conservative baseline forecast, leaning more heavily on a regional manager's own judgment, while they collect enough post-subsidy data

to retrain properly. When they do retrain, they repeat the full workflow from Chapter 2: a fresh train-validate-test split, a careful comparison against the old model on the same recent weeks, and a gradual redeployment rather than an overnight swap. Monitoring stays switched on through all of it, watching quietly for whatever surprise the world eventually sends next.

Months later, the analytics team looks back on the incident not as a failure, but as proof the system worked the way it was designed to. The monitoring dashboards caught a real change within two weeks. The rollback protected the purchasing team from acting on a run of bad forecasts while the true cause was still being confirmed. The eventual retrain followed the same careful workflow as the original model, rather than a rushed shortcut born of panic. Roj Solar's manager likes to tell new hires that the forecasting model is not a finished product sitting quietly on a shelf. It is closer to a solar panel itself: something that keeps producing value only if somebody checks on it, cleans it off, and adjusts it as the seasons keep changing.

COMMON MISTAKES TO AVOID

- Treating a model that scored well in a notebook as automatically ready for real-world use, without planning for messy, late, or invalid input.
- Assuming a model that performed well at launch will keep performing well forever, without any ongoing monitoring.
- Monitoring only input-data health, and never checking real prediction errors once true outcomes are known.
- Rewriting preprocessing steps by hand for a live system instead of reusing the exact same saved pipeline used in training.
- Reflexively retraining the moment a monitoring alert fires, without first diagnosing whether the cause is drift, skew, or a short-lived blip.
- Leaving a misbehaving model live while slowly debugging it, instead of rolling back first and investigating calmly afterward.

CHAPTER SUMMARY

- Deployment means making a trained model produce real predictions inside an actual running system, not just inside a notebook, and it should always reuse the exact same pipeline used for training.
- Production input can be messy, late, or invalid; a safe system validates or flags bad input rather than silently turning it into a confident-looking prediction.
- Data drift is a change over time in the data a deployed model sees, compared with what it trained on, and it can make old predictions less trustworthy even though the model itself never changed.
- Monitoring checks several layers: the health of incoming data, the pattern of the model's predictions, and, once true outcomes are known, the model's real error or calibration.

- Comparing a recent error measure, such as MAE, against its original validation-time value is a simple, checkable way to catch a real performance drop.
- Training-serving skew happens when a feature is computed differently for training than for live prediction, feeding the model different information than it learned from, even without any real-world change.
- Rollback means switching back to the last known-good version quickly once monitoring shows a real problem, rather than leaving a faulty version live or retraining blindly.
- A deployed model is an ongoing responsibility, not a one-time deliverable — Chapters 20 and 21 now walk through two complete projects, start to finish, using everything you have learned across this entire book.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define deployment in your own words.

Problem 2 [Basic] Explain the difference between a batch prediction system and an on-demand prediction system, using an example of your own.

Problem 3 [Basic] Define data drift in your own words.

Problem 4 [Basic] Define rollback, and explain why it is often the safer first move compared with immediately debugging a live, misbehaving system.

Problem 5 [Standard] A model's weekly website-visit feature averaged 500 over 6 weeks before a marketing campaign: 480, 510, 495, 505, 515, 495. It averaged 650 over the 6 weeks during the campaign: 640, 660, 645, 655, 660, 640. Find both averages and the percent change, showing every step.

Problem 6 [Standard] A model's validation MAE was 8 units. Over the most recent 5 days, true and predicted values were: (52, 50), (48, 47), (60, 44), (55, 53), (50, 49). Find the recent MAE and compare it with the validation MAE.

Problem 7 [Standard] Explain, in your own words, the difference between data drift and training-serving skew, using an example different from the ones in this chapter.

Problem 8 [Standard] A team retrains a model the moment its weekly accuracy drops. Explain two reasons this could be a risky habit, using vocabulary from this chapter.

Problem 9 [Standard] Explain why saving and reusing the exact same pipeline for training and for live prediction helps prevent training-serving skew.

Problem 10 [Challenge] A ride-hailing app's arrival-time model shows rising error every day for a week after a new city map update. (a) List three checks you would run, in order, before deciding whether to roll back. (b) Explain what result from each check would point to drift, and what result would point to

skew. (c) State one condition under which you would roll back immediately rather than keep investigating.

Problem 11 [Challenge] (a) Name one feature whose live computation could plausibly differ from its training computation in a system you are familiar with. (b) Explain exactly how the mismatch could happen. (c) Suggest one safeguard that would have caught it before release.

Problem 12 [Challenge] Open ML Studio's Drift Simulator, run it at three different drift levels, and record the validation MAE and the live MAE at each level. Describe the pattern you see, and state the drift level at which you would alert a human.

Part V — Complete Case Studies

Chapter 20 — Case Study: Predicting House Prices

Chapter 21 — Case Study: Predicting Customer Churn

COMPLETE CASE STUDIES

CHAPTER 20

Case Study: Predicting House Prices

Putting the whole workflow to work on one real project

IN THIS CHAPTER YOU WILL LEARN TO

- Frame a regression project as one plain decision sentence, and name its unit of observation, target, and prediction time
- Audit a real dataset for missing values, duplicates, impossible values, and data leakage before building anything
- Split historical sales data by time, and explain why a random split would be dishonest here
- Build a baseline, then compare ridge regression, a random forest, and a gradient boosting model against it using validation MAE
- Read a final test score honestly, and break error down by a subgroup to find where a model is weakest
- Explain a finished model's predictions in plain language, and describe how to monitor it safely after launch

20.1 Frame the Decision: A Starting Price, Not an Appraisal

Chapters 1 through 19 gave you every tool this book has to offer. You can frame a project around a decision instead of an algorithm. You can split data safely, build a **baseline**, and choose a metric that matches the real goal. You can fit a straight line, grow a forest of trees, and correct a model's mistakes round by round with **boosting**. You can catch **overfitting**, tame it with **regularization**, and explain a finished model's predictions instead of just trusting them. This chapter puts all of it to work on one real project, start to finish, the way a real team actually works: not one lesson at a time, but the whole toolbox at once.

Meet Lana. She is a data analyst at Hawler Homes, a small real estate service in Erbil. Homeowners come to Hawler Homes when they want to sell a house or an apartment, and an agent helps them set an asking price, market the home, and manage the sale. Right now, that starting price comes from one agent's memory and a quick look at a handful of recent listings nearby. An experienced agent, in a neighborhood they know well, does this skillfully. A newer agent, or a fast-changing neighborhood with new buildings going up every year, has a harder time. Hawler Homes' director asks Lana whether years of the company's own past sales records could give every agent a faster, steadier starting point.

Lana already knows, from Chapter 2, exactly where a safe project begins: not with an algorithm, but with one plain sentence. "At prediction time, the model will use ___ to predict ___, so that ___."

WORKED EXAMPLE — WRITE THE DECISION SENTENCE FOR HAWLER HOMES

Before touching any data, Lana wants to fill in the sentence: "At prediction time, the model will use ___ to predict ___, so that ___." Prediction happens the moment a seller first sits down with a Hawler Homes agent, before the home is listed anywhere. At that moment, the agent already knows the home's size, its number of bedrooms and bathrooms, its age, its neighborhood, and its distance from the city center. Nobody yet knows what the home will actually sell for. Fill in the three blanks.

1. Decide what information is genuinely available at that exact moment: the home's size, bedrooms, bathrooms, age, neighborhood, and distance from the city center, since an agent can note all of these during one visit, before any listing goes up.
2. Decide the target, the answer the model must produce: the price this home is likely to sell for.
3. Decide the purpose, why the prediction matters: so the agent can offer the seller an honest starting range for discussion, instead of a guess based on memory alone.
4. Combine the three parts into one sentence.

Answer: "At prediction time, the model will use the home's size, bedrooms, bathrooms, age, neighborhood, and distance from the city center to predict the price it is likely to sell for, so that the agent can offer the seller an honest starting range for discussion." One sentence, written before any data is even opened, already tells Lana what her unit of observation is, what her target is, and what her prediction time is.

This one plain sentence already carries three separate ideas. Lana's **unit of observation** is one completed home sale. Her **target** is the price that home is likely to sell for. Her prediction time is the first meeting between the seller and the agent, before anything is listed anywhere.

IMPORTANT

Hawler Homes' tool is a starting range for a conversation, not a legal appraisal and not a promise of what a home will sell for. A bank, a court, or a tax office may require a formal appraisal that follows rules this project never claims to follow. Every price range this model produces should say so plainly, right next to the number.

COMMON MISTAKE

✗ Wrong: A junior teammate suggests "let's use gradient boosting, since it's usually the strongest model," before anyone has written down what decision the prediction actually supports.

✓ **Right:** Write the plain decision sentence first, exactly as Chapter 2 recommends. Choose a model only after the target, the unit of observation, and the prediction time are completely clear.

20.2 Audit the Data Before Trusting It

Lana pulls 2,400 past sales from Hawler Homes' records, covering the last 24 months. Each row is one unit of observation: one completed sale, with that home's features and its final price sitting together. Before fitting anything at all, she does exactly what Chapters 2 and 3 both told her to do first: look at the data honestly, with no model running yet.

The spreadsheet is not clean. Forty rows are missing a bathroom count entirely, probably skipped by whichever agent entered the sale that week. Twelve rows are exact duplicates of another row, most likely the same sale typed in twice by two different people. One row lists a size of 0 square meters next to a sale price of \$180,000, clearly a typing mistake, since no home is truly sized zero. Chapter 3 already gave Lana a plan for problems exactly like these: fix what can be sensibly fixed, drop what cannot, and never guess at a fix using information that would not have been available before this home's price was already known. For the forty rows missing a bathroom count, she fills each one in using the typical bathroom count for homes of a similar size, computed only from the training rows, rather than guessing a single number for every home or, worse, peeking at validation or test rows to make a better guess. She drops the twelve duplicate rows entirely, keeping only the first copy of each, and drops the one impossible zero-size row rather than inventing a size for it.

Lana also lists every column in the raw spreadsheet as a candidate feature, then checks each one against the prediction time from Lesson 20.1, exactly the way Chapter 2 taught her to check for **data leakage**.

WORKED EXAMPLE — CHECK FOUR CANDIDATE FEATURES FOR LEAKAGE

Lana's raw spreadsheet has more columns than she needs. Four candidates: (1) the home's size in square meters, (2) the average sale price of that same neighborhood, using only sales that closed before this one, (3) the number of days this home stayed listed before it sold, and (4) the asking price the agent originally listed the home at. For each one, decide whether it is safe to use as a feature.

1. Check candidate 1 against the prediction time, the first meeting with the seller: size is known the moment an agent walks through the home, so it is safe.
2. Check candidate 2: an average built only from sales that already closed before this one is fully known at the first meeting too, so it is safe.
3. Check candidate 3 against the prediction time: the number of days a home stays listed is only known after it finally sells, long after a starting price is needed, so using it would be data leakage.
4. Check candidate 4 more carefully: the agent's original asking price is technically known before the sale closes, so it is not leakage in Chapter 2's

strict sense. But Hawler Homes wants this exact model to help set that very number. Using it as an input would be circular: the model would partly just be predicting the number it is supposed to help replace.

Answer: Candidates 1 and 2 are safe features. Candidate 3 must be dropped for data leakage. Candidate 4 must also be dropped, for a different, more subtle reason: not leakage, but circularity, since using it would let the model quietly lean on the very number Hawler Homes built this project to improve.

COMMON MISTAKE

✗ Wrong: Keeping "days on the market" in the feature list because it happens to improve the training score, without asking whether it would truly be known before a starting price is needed.

✓ Right: Check every candidate feature against the exact prediction time from Lesson 20.1. A feature that only exists after the sale closes can never be used to predict a starting price before that sale happens.

With leakage risks removed, Lana settles on nine safe features: size in square meters, bedrooms, bathrooms, age in years, neighborhood, distance to the city center in kilometers, whether the home has a garden, whether it has covered parking, and whether it is a house or an apartment. The age feature is not stored directly in the raw data; Lana builds it herself by subtracting each home's construction year from its sale year, a small piece of **feature engineering** in the spirit of Chapter 15: turning a raw date into a feature that actually means something to a regression model.

20.3 Split by Time, Not at Random

Real estate prices in a growing city do not sit still. Over Lana's 24 months of data, construction costs rose, new roads opened up formerly distant land, and a new shopping center pulled prices up in one neighborhood faster than in others. If Lana split her 2,400 sales randomly, a handful of next month's sales could land in her training set purely by chance, letting the model quietly learn about a price trend before that trend was supposed to be knowable. Chapter 2 gave this exact problem a name and a fix: a **time-based split**.

WORKED EXAMPLE — SPLIT 2,400 HOME SALES BY TIME

Lana's 2,400 sales are spread evenly across the last 24 months, 100 sales per month. She decides to hold out the most recent 3 months as a test set, the 3 months before that as a validation set, and use everything older for training. How many sales fall into each group?

1. Find how many months belong to training: 24 total months, minus 3 test months, minus 3 validation months, leaves $24 - 3 - 3 = 18$ months.
2. Find the training sales: 18 months $\times$ 100 sales per month = 1,800 sales.
3. Find the validation sales: 3 months $\times$ 100 sales per month = 300 sales.
4. Find the test sales: 3 months $\times$ 100 sales per month = 300 sales.

5. Check the total: $1,800 + 300 + 300 = 2,400$, which matches the original number of sales.

Answer: The training set holds 1,800 sales from the oldest 18 months, the validation set holds 300 sales from the next 3 months, and the test set holds 300 sales from the most recent 3 months. Every training sale happened before every validation sale, and every validation sale happened before every test sale, exactly the protection a time-based split is built to give.

COMMON MISTAKE

✗ **Wrong:** Using a plain random split on the 2,400 sales, since it is the fastest, most familiar option.

✓ **Right:** Sort the sales by date first, then cut the three groups along that timeline. A random split would let a handful of next month's prices slip into training, making validation scores look better than the finished tool will actually perform once it is live.

Lana also considers a time-respecting rolling-window **cross-validation**, the kind Chapter 4 hinted at when it warned against ordinary random folds on data where order matters. For a first version of the tool, she keeps it simple: one clean time-based split, exactly like the one above. A more elaborate rolling check can wait for a later version, once the basic pipeline is already working end to end.

20.4 Build a Baseline First

Before fitting anything clever, Lana builds the simplest possible predictor: always guess the average sale price from the training data, no matter what the home looks like. Chapter 2 called this a baseline, and insisted it comes first, precisely because it is so easy to build that skipping it has no excuse.

WORKED EXAMPLE — BUILD AND TEST A BASELINE PRICE PREDICTOR

Five representative training sales sold for \$80,000, \$95,000, \$110,000, \$130,000, and \$160,000. Find the baseline's prediction. Then find its absolute error on a validation-set home that actually sold for \$124,000.

1. Add the five training prices together: $80,000 + 95,000 + 110,000 + 130,000 + 160,000 = 575,000$.
2. Divide by the number of sales, which is 5, to find the average: $575,000 \div 5 = 115,000$.
3. The baseline predicts \$115,000 for every home, no matter its size, neighborhood, or age.
4. Find the absolute error on the validation home: true price \$124,000, so the error is $|124,000 - 115,000| = \$9,000$.

Answer: The baseline predicts \$115,000 for every home. On this one validation home it was off by \$9,000. Across the complete training set of 1,800 sales, not just these five, the average works out to this same

figure, so \$115,000 is the number the baseline predicts for every home in the validation and test sets too.

Lana runs this flat \$115,000 guess against all 300 validation homes and finds a **mean absolute error** of \$28,400, with a **root mean squared error** of \$35,700. Neither number means much by itself, exactly as Chapter 4 warned. What it gives Lana is a floor: any real model she builds afterward must clear a validation MAE of \$28,400 by a healthy margin, or it is not worth the extra complexity. She reports this floor next to every later score, so nobody at Hawler Homes can mistake a fancier model's good-looking number for real progress until it has actually beaten \$28,400 by a wide, honest margin.

20.5 Build One Safe Pipeline

Lana now has safe features, a clean time-based split, and a baseline to beat. Chapter 3 already warned her about a trap hiding at this exact step: any cleaning, scaling, or encoding rule must be learned only from the training set, then applied unchanged to validation and test. The safest way to guarantee that is to chain every step together into one **pipeline**, so training and applying preprocessing can never accidentally happen out of order.

Two kinds of features need attention before any model can use them. Size, bedrooms, bathrooms, age, and distance to the city center are already numbers, so Chapter 3's **standardization** puts them on a shared scale. Neighborhood and home type are categories, so they need Chapter 3's **one-hot encoding** instead: one new 0-or-1 column per category.

WORKED EXAMPLE — ONE-HOT ENCODE THREE HOMES' NEIGHBORHOODS

Hawler Homes' data covers four neighborhoods: Ankawa, Dream City, Empire World, and Iskan. Three homes are listed in Ankawa, Dream City, and Iskan. One-hot encode the neighborhood feature for these three homes.

1. Create one new column for each of the four neighborhoods: Ankawa, Dream City, Empire World, and Iskan.
2. For the first home, in Ankawa, put a 1 in the Ankawa column and a 0 in the other three columns.
3. For the second home, in Dream City, put a 1 in the Dream City column and a 0 in the other three columns.
4. For the third home, in Iskan, put a 1 in the Iskan column and a 0 in the other three columns.

Answer: Home 1: Ankawa 1, Dream City 0, Empire World 0, Iskan 0. Home 2: Ankawa 0, Dream City 1, Empire World 0, Iskan 0. Home 3: Ankawa 0, Dream City 0, Empire World 0, Iskan 1. Every home gets exactly one 1 among the four new columns, and a random forest or a boosted model can now use neighborhood the same way it uses any numeric feature.

COMMON MISTAKE

✗ Wrong: Computing each numeric feature's average and typical spread using all 2,400 homes, training, validation, and test together, before standardizing.

✓ Right: Fit the standardization rule, and the one-hot encoder's list of known categories, using only the 1,800 training homes. Apply that exact same fitted rule to the validation and test homes afterward, without ever refitting it on them.

Python · NumPy

 runnable in the app

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import GradientBoostingRegressor

numeric_cols = ["size", "bedrooms", "bathrooms", "age", "distance_km"]
category_cols = ["neighborhood", "home_type"]

preprocess = ColumnTransformer([
    ("scale", StandardScaler(), numeric_cols),
    ("encode", OneHotEncoder(handle_unknown="ignore"), category_cols),
])

pipe = Pipeline([("preprocess", preprocess), ("model",
GradientBoostingRegressor(random_state=0))])
pipe.fit(X_train, y_train)
```

The `ColumnTransformer` applies `StandardScaler` only to the numeric columns and `OneHotEncoder` only to the two category columns, so neither step touches a column it was not meant for. Setting `handle_unknown="ignore"` tells the encoder what to do if a brand-new neighborhood shows up after launch: give it all zeros instead of crashing the whole pipeline. Wrapping both stages inside one `Pipeline` means a single call to `fit` learns the scaling, the categories, and the model together, always in the correct order, and the exact same fitted object can later transform validation or test homes without ever refitting on them.

CASE STUDY — EXPLAINING THE PIPELINE TO HAWLER HOMES' AGENTS

Before going further, Lana walks Hawler Homes' agents through what she has built so far, since they are the ones who will eventually use this tool with real sellers. One senior agent, who has sold homes in Ankawa for eleven years, asks a sharp question: why does the computer need to see 1,800 old sales just to guess one new price, when he could guess faster from memory alone? Lana explains that the pipeline is not trying to replace his judgment. It is trying to give every agent, including the newest hire who has never worked in Ankawa before, a fast, consistent starting point built from the same kind of pattern he already carries in his head. The agent is not fully convinced yet, but he agrees to try the tool on his next ten listings and compare its starting range with his

own estimate. Lana writes down his request in her project notes: nothing about this model matters more, in the end, than whether the agents who use it every day trust it enough to actually open the app before writing down a number.

20.6 Compare Models on the Validation Set

With one safe pipeline in hand, Lana can swap in any model she likes without touching the preprocessing at all. She trains three real candidates on the 1,800 training homes and checks every one of them on the same 300 validation homes the baseline was already checked on.

Her first real candidate is plain linear regression, the method Chapter 6 built from scratch. On the full training set, it gives home size a sensible **coefficient**, but the coefficient for distance to the city center swings to an oddly large number, pulled off course by three unusually expensive homes far from downtown. This is the same trap Chapter 6's own case study fell into with a building's age. Lana reaches for the same fix: **ridge regression**, with its penalty strength λ chosen by trying a few values and keeping whichever one gives the lowest validation MAE, exactly as Chapter 5 taught her to choose any **hyperparameter**. She tries $\lambda = 1$, $\lambda = 5$, and $\lambda = 20$, and keeps $\lambda = 5$, the middle value, which gives the lowest validation MAE of the three: strong enough to calm the distance coefficient, not so strong that it flattens the useful part of the pattern.

Her third and fourth candidates are tree-based: a **random forest**, built from many bootstrap-sampled trees voting together, and a gradient boosting model, built from trees added one at a time, each one correcting the mistakes the trees before it are still making. Neither one needs standardized features to work well, but Lana keeps them inside the same pipeline anyway, so every candidate model is judged on identical, cleanly split data. For the boosting model, she tries a small **grid search** over a few **learning rates** and tree depths, exactly as Chapter 17 describes, and keeps whichever combination scores best on the **validation set**, never on training data.

Lana briefly considers a small neural network too. Chapter 11 already previewed an honest lesson worth remembering here: a well-tuned boosted-tree model usually beats a shallow network on a modest, tabular business dataset like this one. She spends her limited time tuning the boosting model instead, and leaves the network untried.

WORKED EXAMPLE — COMPARE FOUR MODELS USING VALIDATION MAE

Lana checks the baseline and three real candidates on the same 300 validation homes. The baseline, always predicting the average price, scores a validation MAE of \$28,400 and a validation RMSE of \$35,700. Ridge regression scores a validation MAE of \$16,200 and a validation RMSE of \$21,000. The random forest scores a validation MAE of \$12,800 and a validation RMSE of \$17,500. Gradient boosting scores a validation MAE of \$11,900 and a validation RMSE of \$16,400. Rank the four models from worst to best, then find how much lower gradient boosting's MAE is than the baseline's, as a percentage.

1. Rank the four validation MAE values from highest (worst) to lowest (best): baseline \$28,400, ridge regression \$16,200, random forest \$12,800, gradient boosting \$11,900.
2. Find the drop in MAE from baseline to gradient boosting: $28,400 - 11,900 = 16,500$.
3. Divide that drop by the baseline's MAE: $16,500 \div 28,400 \approx 0.581$.
4. Convert to a percentage: $0.581 \times 100 \approx 58.1$.

Answer: Gradient boosting has the lowest validation MAE, \$11,900, about 58.1% lower than the baseline's \$28,400. Every real candidate beat the baseline by a wide margin, and the two tree ensembles beat plain ridge regression too, though by a smaller amount than ridge beat the baseline.

Look closer at where the improvement actually came from. Ridge regression alone recovers \$12,200 of the baseline's \$28,400 error, since $28,400 - 16,200 = 12,200$, and $12,200 \div 28,400 \approx 0.430$, a 43.0% drop, just by fitting one well-regularized straight-line relationship. The two tree ensembles add real value on top of that, but with a smaller extra gain each time: random forest's MAE, \$12,800, is only \$900 lower than gradient boosting's \$11,900, and $900 \div 12,800 \approx 0.070$, about a 7.0% difference between the two ensembles themselves. Lana keeps this pattern in mind going forward: most of the improvement over doing nothing came from the simplest real model, not the fanciest one.

COMMON MISTAKE

✗ **Wrong:** Choosing gradient boosting because its training MAE is nearly zero, without checking whether that same low error holds up on the validation set.

✓ **Right:** Compare validation MAE, never training MAE, when choosing between candidate models. Chapter 5 already showed how a flexible model can fit training data almost perfectly while still doing worse than a simpler model on data it has never seen.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.
2. Choose the "Home Prices" dataset, and fit Ridge Regression, Random Forest, and Gradient Boosting in turn, using a time-based split.
3. Write down each model's validation MAE, and rank all three against the baseline score ML Studio shows automatically.
4. Open the settings panel for Gradient Boosting, and try three different learning rates. Record which one gives the lowest validation MAE.
5. Switch the split from time-based back to random, refit your best model, and write one sentence about whether its validation score looks better or worse, and why that difference might be misleading.

You need: Computer, tablet, or phone with the ML Studio app

20.7 Final Test and Error Analysis

Gradient boosting won on the validation set, so it is the one model Lana checks on the 300 test homes, the most recent 3 months of sales, opened for the first and only time. Every setting inside the pipeline, its scaling rule, its encoder, its learning rate and tree depth, was already locked in before this moment. The test set exists only to answer one honest question: how will this finished tool actually perform on sales it never influenced in any way?

WORKED EXAMPLE — READ THE FINAL TEST SCORE HONESTLY

On the 300 test homes, the finished gradient boosting pipeline scores a mean absolute error of \$12,400 and a root mean squared error of \$17,100. Its validation MAE, found in Lesson 20.6, was \$11,900. Compare the two MAE values, and decide whether this result should worry Lana.

1. Compare the two MAE values: validation MAE was \$11,900, and test MAE is \$12,400.
2. Find the size of the rise: $12,400 - 11,900 = 500$.
3. Judge the size of that rise against the scale of the numbers involved: \$500, on errors already around \$12,000, is a small change, not a sign of a badly broken pipeline.

Answer: Test MAE, \$12,400, is a little higher than validation MAE, \$11,900, which is the small, expected rise Chapter 2 already prepared Lana for: the validation set helped choose the model, so the test set's slightly tougher, completely untouched score is the one Lana reports to Hawler Homes as the honest, final number.

A single overall number, \$12,400, hides an uneven pattern underneath it, so Lana asks one more question Chapter 4 always pushes her toward: honest for whom? She sorts the 300 test homes into three price bands and checks MAE separately inside each one.

WORKED EXAMPLE — BREAK TEST ERROR DOWN BY PRICE BAND

Lana sorts the 300 test homes into three price bands. The 90 homes priced under \$80,000, typically around \$70,000, have a band MAE of \$7,000. The 150 homes priced between \$80,000 and \$150,000, typically around \$115,000, have a band MAE of \$9,000. The 60 homes priced over \$150,000, typically around \$200,000, have a band MAE of \$29,000. Find each band's error as a percentage of its typical price, then decide which band the model serves worst.

1. Find the relative error for the lowest band: $7,000 \div 70,000 = 0.10$, or 10%.
2. Find the relative error for the middle band: $9,000 \div 115,000 \approx 0.078$, or about 7.8%.
3. Find the relative error for the highest band: $29,000 \div 200,000 = 0.145$, or 14.5%.
4. Compare the three relative errors: 10%, 7.8%, and 14.5%.

Answer: The middle band, \$80,000 to \$150,000, has the smallest relative error, about 7.8%, because it is where most of Hawler Homes' past sales come from, giving the model the most practice. Homes over

\$150,000 are the model's least reliable predictions by far, at 14.5% relative error, most likely because far fewer expensive homes appear in the training data, and each one tends to carry unique features, such as a custom finish or a rare view, that plain columns like size and bedroom count cannot fully capture.

IMPORTANT

An overall test score can hide a model that quietly serves one group of homes, or one group of people, far worse than another. Whenever a prediction feeds into a real decision, check its error inside meaningful subgroups, not only its one headline number.

COMMON MISTAKE

✗ Wrong: Noticing the weak result on homes over \$150,000, then retuning the gradient boosting model and checking the same 300 test homes a second time to see if the new settings look better.

✓ Right: Write down the weakness honestly and plan it as a goal for the next version of the project. Once the test set has been opened and reported, reusing it to guide more changes quietly turns it into another validation set, exactly the trap Chapter 2 warned about.

20.8 Explain the Model, Then Monitor It After Launch

A price range with no explanation is hard for an agent to defend to a skeptical seller, and hard for Snur to trust enough to launch. Chapter 10 already gave Lana a first, quick look inside her gradient boosting model: **feature importance**, built from how much impurity each feature's splits removed across every tree.

Reading it off the trained pipeline, Lana finds size responsible for about 30% of the model's total importance, neighborhood about 38%, distance to the city center about 12%, bedrooms and bathrooms together about 12%, and age about 8%. At a glance, neighborhood looks like the single strongest driver of price, ahead of even size.

Chapter 10 already warned Lana not to stop there: impurity-based importance needs care, and a feature spread across several one-hot columns, like neighborhood's four columns, gets more chances to be picked for a split than a single numeric column does. Chapter 18's fix, **permutation importance**, checks a feature's true usefulness a different way: shuffle just that one feature's values across the validation set, keep everything else the same, and watch how much worse the model's score gets.

WORKED EXAMPLE — COMPARE TWO FEATURES USING PERMUTATION IMPORTANCE

On the 300 validation homes, the gradient boosting pipeline's ordinary MAE is \$11,900. Lana shuffles the size column only, keeping every other feature and the model itself unchanged, and the MAE rises to \$19,300. She repeats this, shuffling only the neighborhood columns instead, and the MAE rises to \$14,600.

Find each feature's permutation importance, measured as the increase in MAE, and as a percentage increase over the original MAE.

1. Find the increase in MAE from shuffling size: $19,300 - 11,900 = 7,400$.
2. Find that increase as a percentage of the original MAE: $7,400 \div 11,900 \approx 0.622$, or about 62.2%.
3. Find the increase in MAE from shuffling neighborhood: $14,600 - 11,900 = 2,700$.
4. Find that increase as a percentage of the original MAE: $2,700 \div 11,900 \approx 0.227$, or about 22.7%.

Answer: Shuffling size hurts the model far more than shuffling neighborhood does, \$7,400 versus \$2,700, or about 62.2% versus 22.7%. Permutation importance reverses the impurity-based ranking from earlier: size is really the model's most load-bearing feature, and neighborhood's earlier lead was partly an accident of how many one-hot columns it happened to occupy.

Python · NumPy

 runnable in the app

```
from sklearn.inspection import permutation_importance

result = permutation_importance(
    pipe, X_val, y_val, scoring="neg_mean_absolute_error", n_repeats=10,
    random_state=0
)
for name, score in zip(X_val.columns, result.importances_mean):
    print(name, round(-score, 0))
```

`permutation\_importance` repeats the shuffle-and-rescore idea from the worked example automatically, `n\_repeats` times per feature, then averages the results so one lucky or unlucky shuffle cannot skew the ranking. Scikit-learn reports scores as "the more negative, the worse," since it always tries to maximize a score, so Lana flips the sign back with a minus before printing, matching the plain increase-in-MAE numbers used above.

IMPORTANT

Neither kind of importance proves that a feature causes a higher price, and neighborhood deserves special caution. A neighborhood can quietly become a proxy variable for other things a fair pricing tool should never use directly, and treating "this neighborhood raises prices" as a fixed rule risks freezing old patterns in place instead of pricing each home on its own features. Lana uses importance scores to guide a conversation with agents, never as a rule an agent or a seller is not allowed to question.

Beyond this global picture, Lana wants something more useful for one specific seller sitting across the table from an agent: why did the model land on this particular number for this particular home?

WORKED EXAMPLE — EXPLAIN ONE HOME'S PREDICTED PRICE IN PLAIN LANGUAGE

A seller's home is 140 m², in Dream City, 5 years old, with 3 bedrooms. The gradient boosting model predicts \$167,000. Lana approximates the model's reasoning for the agent as a series of adjustments starting from the \$115,000 baseline: +\$28,000 for its above-average size, +\$21,000 for its neighborhood, +\$6,000 for being newer than most homes in the data, and −\$3,000 because its bedroom count is unremarkable for a home this size. Confirm that these adjustments add up to the model's predicted price.

1. Start from the baseline: \$115,000.
2. Add the size adjustment: $115,000 + 28,000 = 143,000$.
3. Add the neighborhood adjustment: $143,000 + 21,000 = 164,000$.
4. Add the age adjustment: $164,000 + 6,000 = 170,000$.
5. Subtract the bedroom adjustment: $170,000 - 3,000 = 167,000$.

Answer: The adjustments add up to \$167,000, matching the model's actual prediction exactly. The agent can now tell the seller a short, honest story instead of a bare number: this home's size and its neighborhood are doing most of the work, its newer age helps a little more, and its bedroom count is not pulling the price up or down by much.

A gradient boosting model does not truly add up clean, separate pieces like this behind the scenes, but breaking a prediction into approximate, plain-language pieces this way is a fair and honest way to explain it to someone who has never studied a decision tree, and it captures the same spirit as the explanation tools Chapter 18 introduced.

Snur reviews Lana's full report before agreeing to launch anything: the decision sentence, the leakage checks, the time-based split, the baseline, the comparison table, the price-band breakdown, and the plain-language explanation for a sample home. What convinces him is not any single number, but that every number comes with an honest account of where it came from and where it might still be wrong. He signs off on a limited launch, starting with agents who volunteer to try it first, rather than rolling it out to the whole office at once.

A finished, well-explained model is still not a finished project. Chapter 19 already taught Lana that a live model needs watching, not just launching. Lana and Snur agree on a **monitoring** plan before launch, not after: track the live MAE every month, compare it against the \$12,400 test score, and investigate if it climbs much higher. **Data drift** worries Lana most, since Erbil keeps building, and a new phase of a development, or a sudden jump in construction material costs, can shift prices in ways her 24 months of training data never saw. She also checks for **training-serving skew**, confirming that the exact same fitted pipeline object used for testing is the one running inside the agent's

app, not a hand-rebuilt copy that could quietly preprocess a new home slightly differently. And she keeps the previous, already-tested pipeline ready for a quick **rollback**, in case a future retraining run turns out worse instead of better on a fresh test period.

CASE STUDY — THREE MONTHS AFTER LAUNCH AT HAWLER HOMES

Hawler Homes launches the pricing tool inside the same app agents already use to log new listings. Every price range comes with one sentence attached, matching Lesson 20.1's promise: this is a starting point for discussion, not an appraisal. Three months in, Lana checks the live numbers against her test-set report. The overall MAE has crept up to \$15,800, noticeably worse than the \$12,400 she reported at launch, worth investigating rather than ignoring. Digging in, she finds the drift concentrated in one place: Empire World opened a new construction phase halfway through those three months, and homes there are now selling for noticeably more than anything in the original training data. The rest of the city's predictions are holding up close to their test-time accuracy. Rather than retraining blindly, Lana adds the newest sales to her training set, confirms the refreshed pipeline scores better on a fresh, more recent test period before replacing anything, and keeps the older pipeline saved and ready in case the new one underperforms once it, too, is live. The senior agent from Ankawa, still testing the tool on his own listings, tells Lana its starting ranges now land close to his own estimate more often than not; he still overrides it sometimes, and Lana is glad he does, since a caring, experienced agent catching a model's occasional bad guess is exactly the kind of human check this project was built to keep in place, never to remove.

IMPORTANT

Publication-Ready Conclusion. Hawler Homes' pipeline is suitable for one specific, modest job: giving an agent a data-informed starting range to open a pricing conversation with a seller, alongside the agent's own judgment. It is trained on 24 months of one city's sales, so it should not be trusted for a different city, a commercial property, or a sale type it has never seen, without new data and a new test. It is not a legal appraisal, not a bank valuation, not a substitute for a licensed professional where one is legally required, and not a promise of what any home will actually sell for. Its error grows for expensive, unusual homes, so any range it offers should be paired with human review, especially at the high end. A model like this earns continued use only by staying honest about where it works well, where it does not, and by being watched, tested, and retrained, never launched once and forgotten.

COMMON MISTAKES TO AVOID

- Choosing an algorithm before writing the plain decision sentence that names the target, the unit of observation, and the prediction time.
- Using a feature that would not truly be known before a starting price is needed, such as days on the market or a home's final negotiated price.

- Splitting home sales randomly instead of by time, letting future market trends leak into training.
- Skipping the baseline, so nobody can tell whether a complex model actually earned its extra cost and complexity.
- Fitting a scaling or encoding rule on the full dataset before splitting it into training, validation, and test.
- Picking a model by its training score instead of its validation score.
- Reporting one overall test score without checking whether error is much worse for one subgroup, such as expensive homes.

CHAPTER SUMMARY

- A real project starts with one plain decision sentence, naming the unit of observation, the target, and the prediction time, before any model is chosen.
- Auditing raw data catches missing values, duplicates, and impossible values, and checking every candidate feature against the prediction time catches both data leakage and, sometimes, a more subtle circularity.
- A time-based split protects a project where the target can drift over months or years; training always happens on the oldest data, and the test set is always the most recent.
- A baseline that predicts the average training target costs almost nothing to build, and it sets the bar every real model must clear.
- A pipeline chains standardization, encoding, and a model into one object, so the exact preprocessing rule learned from training data is always the one applied to validation and test data.
- Comparing several models, a baseline, ridge regression, a random forest, and a gradient boosting model, on the same validation set shows how much each added layer of complexity is really worth.
- A model is chosen using validation score, not training score, and checked on the test set exactly once, honestly reporting even a small, expected rise from validation to test.
- Breaking a single test score down by subgroup, such as a price band, can reveal that a model serves some groups far better than others.
- Feature importance and permutation importance both describe what a trained model leaned on, not proof of what causes a higher price in the real world; permutation importance is the more careful of the two.
- A finished model is not a finished project: monitoring for data drift, checking for training-serving skew, and keeping an earlier version ready for rollback keep a live model trustworthy long after launch.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] In your own words, explain why Hawler Homes' tool should never be called a "legal appraisal."

Problem 2 [Basic] Name the unit of observation, the target, and the prediction time for the Hawler Homes project.

Problem 3 [Basic] A dataset has 3,600 home sales spread evenly across 36 months. The team holds out the most recent 4 months for testing and the 4 months before that for validation. How many months, and how many sales, remain for training?

Problem 4 [Basic] Explain, in one or two sentences, why "days on the market" cannot be used as a feature to predict a starting price.

Problem 5 [Basic] A baseline predicts the average training price, \$102,000, for every home. A validation home actually sold for \$95,000. Find the absolute error.

Problem 6 [Standard] Five training sales sold for \$60,000, \$75,000, \$90,000, \$105,000, and \$120,000. Find the baseline's prediction, showing every step.

Problem 7 [Standard] A model's validation MAE is \$14,500, and the baseline's validation MAE is \$29,000 on the same homes. Find how much lower the model's MAE is than the baseline's, as a percentage, showing every step.

Problem 8 [Standard] One-hot encode the "home type" feature for four homes, where the possible types are House, Apartment, and Villa, and the four homes are House, Villa, Apartment, and House, in that order.

Problem 9 [Standard] A model's validation MAE is \$13,200, and its test MAE is \$13,900. Explain, in your own words, whether this small rise should worry the team, and why.

Problem 10 [Standard] A test set of 200 homes is split into two price bands. Band A has 140 homes and a MAE of \$8,000. Band B has 60 homes and a MAE of \$22,000. Find the overall MAE across all 200 homes, showing every step.

Problem 11 [Standard] A feature's ordinary validation MAE is \$10,000. After shuffling that one feature's values, MAE rises to \$15,500. Find the feature's permutation importance, as a dollar increase and as a percentage of the original MAE.

Problem 12 [Challenge] A colleague suggests using the agent's original asking price as a feature, since it is always known before the home sells. (a) Explain why this would not count as data leakage under Chapter 2's definition. (b) Explain the separate reason it should still be left out of this particular project.

Problem 13 [Challenge] A random forest reaches a training MAE of \$200 and a validation MAE of \$13,500. A gradient boosting model reaches a training MAE of \$9,800 and a validation MAE of \$12,100. (a) Which model shows a stronger warning sign of overfitting, and what evidence shows it? (b) Which model would you choose for Hawler Homes, and what would you still want to check before fully trusting it?

Problem 14 [Challenge] Open ML Studio, fit a Random Forest and a Gradient Boosting model on the "Home Prices" dataset using a time-based split, and record each model's validation MAE. Then use ML Studio's permutation importance panel to rank the features for your best model. Write a short paragraph explaining, in plain language a first-time seller could understand, why the model predicted the price it did for one home of your choosing.

COMPLETE CASE STUDIES

CHAPTER 21

Case Study: Predicting Customer Churn

One complete project, every idea working together

IN THIS CHAPTER YOU WILL LEARN TO

- Frame a churn-prediction project around one helpful action, not just around a prediction
- Prevent both time leakage and customer leakage when splitting a dataset with many rows per customer
- Audit a churn label and a list of candidate features for hidden shortcuts before training any model
- Build two opposite baselines, and explain why 90% accuracy can still mean 0% recall
- Compare models, check calibration, and choose a decision threshold using real capacity and real value
- Design a monitoring plan that checks a decision's real effect, not only a model's score

You have now met almost every idea this book has to offer: how to frame a decision, split data safely, build a **baseline**, train and compare models, choose a fair **metric**, and watch a system after it goes live. This chapter puts every one of those ideas to work together, on one realistic project, from the very first sentence to the last line of a monitoring plan. Almost nothing here is new for its own sake. The goal is to show you what a real project actually feels like, start to finish, using tools you already own.

21.1 Frame the Prediction Around a Helpful Action

Meet Darya, a data analyst at Zagros Mobile, a mobile phone company serving customers across Iraq, from Duhok in the north to Basra in the south. Zagros Mobile sells monthly postpaid plans: a customer signs up, uses data and voice service all month, and receives a bill at the end of it. Like every subscription business, Zagros Mobile loses some customers every month. Some cancel outright. Some simply stop paying and let their line lapse. Darya's manager, Ronahi, leads the customer retention team, the small group whose job is to keep customers happy enough to stay. Ronahi asks a simple question: could a model predict, early enough to matter, which customers are at risk of leaving, so her team can reach out and help before they go?

This kind of leaving-the-business event has its own name, and this book has not used it before now.

WORD TO KNOW

Churn — A customer ending their relationship with a business, such as canceling a subscription, closing an account, or simply not renewing. In a churn project, the label usually marks whether a customer churns within a fixed future window, such as the next 30 days.

A churn prediction is only useful if it leads to a helpful action. Chapter 2 warned you against starting a project with an algorithm's name instead of a plain decision sentence, and that warning matters more here than almost anywhere else in this book. It would be easy for Darya to jump straight to "let's train a random forest on customer data." It is far more useful to start exactly where Chapter 2 taught you to start: with the decision, the **unit of observation**, the **target**, and the prediction time, written down in plain sentences before any model is chosen.

WORKED EXAMPLE — WRITE THE DECISION SENTENCE FOR ZAGROS MOBILE'S CHURN PROJECT

Ronahi wants a model that helps her retention team act early, not a model built for its own sake. Using Chapter 2's pattern, "At prediction time, the model will use ___ to predict ___, so that ___, " name the unit of observation, the target, and the prediction time, then write the full decision sentence.

1. Decide the moment of prediction: the first day of each month, using only data collected through the end of the previous month.
2. Decide the unit of observation: one customer, checked at one monthly snapshot.
3. Decide the information available at that moment: the customer's tenure, recent usage, recent bill amounts, recent customer service contacts, and recent late payments, all measured only through the end of last month.
4. Decide the target: whether that customer churns, meaning cancels or lets their line lapse for nonpayment, within the next 30 days.
5. Decide the purpose: so the retention team can call the customers most at risk, while there is still time to help them before they leave.
6. Combine these pieces into one decision sentence: "At prediction time, on the first day of the month, the model will use a customer's tenure, usage, billing, and support history through the end of last month, to predict whether that customer will churn in the next 30 days, so that the retention team can call the customers most at risk while there is still time to help."

Answer: **Unit of observation: one customer, at one monthly snapshot. Target: churn within the next 30 days. Prediction time: the first day of the month. Writing this down first, exactly as Chapter 2 recommends, keeps the whole project pointed at a real decision instead of at an impressive-sounding model.**

Notice what the decision sentence already rules out. It rules out a model that only produces a score with no owner and no next step. It rules out using any information that would not genuinely be sitting in Zagros Mobile's systems on the first day of the month. And it quietly answers the question every churn

project must answer honestly before it starts: what will actually happen to a customer once the model flags them? At Zagros Mobile, the answer is a phone call offering real help, never a penalty and never a locked account.

IMPORTANT

A churn prediction is not a fact about a customer's loyalty, and it is not proof that a customer has already decided to leave. Use a high-risk prediction to offer a genuinely helpful reason to stay, such as fixing a real problem or offering a fair deal, never to pressure, guilt, or trap a customer into staying.

21.2 Prevent Customer Leakage and Time Leakage

Chapter 2 introduced **data leakage**: information entering training that would not truly be available at prediction time. Zagros Mobile's data adds a wrinkle that a simple one-row-per-customer table does not have. Darya's table has one row per customer per month, so the same customer can appear many times: once for January, once for February, once for March, and so on, for as long as that customer stays active. This repeated structure creates two separate leakage risks, and a safe project needs to guard against both of them.

The first risk is the one Chapter 2 already covers in full: time leakage. If Darya is not careful, a row from next month could end up training a model that is then tested on a row from last month, letting the model quietly learn from the future. The fix is the same **time-based split** Chapter 2 taught: order every row by its snapshot date, and keep every training row earlier in time than every validation or test row.

The second risk is new, and it comes directly from having many rows per customer. Suppose one customer's January row lands in the training set, and that exact same customer's March row lands in the **test set**. The two rows are not truly independent of each other. They share the same city, the same plan, a very similar tenure, and often a very similar usage pattern, because they describe the very same person only two months apart. A flexible model can partly learn to recognize that particular customer from the training row, then score suspiciously well on the test row, not because it has genuinely learned to predict churn, but because it has partly memorized one customer's own pattern. This risk deserves a plain name too, even without a formal box: customer leakage, information about one specific customer's identity crossing from training into testing, even when no single column ever states a name.

WORKED EXAMPLE — SPLIT ZAGROS MOBILE'S CUSTOMERS, NOT THEIR ROWS

Darya's team collects one snapshot row per active customer, per month. Over 6 months this produced 42,000 snapshot rows from 7,000 unique customers, each contributing up to 6 rows, one per month they stayed active. To avoid customer leakage, the team splits the 7,000 customers, not the 42,000 rows, into 70% training, 15% validation, and 15% test groups, moving every row that belongs to one customer into that customer's single assigned group. How many customers land in each group, and roughly how many rows travel with them?

1. Find the training customers: $0.70 \times 7,000 = 4,900$.
2. Find the validation customers: $0.15 \times 7,000 = 1,050$.
3. Find the test customers: $0.15 \times 7,000 = 1,050$.
4. Check the customer total: $4,900 + 1,050 + 1,050 = 7,000$, which matches the original 7,000 customers.
5. Since every customer's rows travel together into that same group, estimate the rows using about 6 rows per customer: training rows $\approx 4,900 \times 6 = 29,400$. Validation rows $\approx 1,050 \times 6 = 6,300$. Test rows $\approx 1,050 \times 6 = 6,300$.
6. Check the row total: $29,400 + 6,300 + 6,300 = 42,000$, matching the original 42,000 rows.

Answer: 4,900 customers, about 29,400 rows, go to training. 1,050 customers, about 6,300 rows, go to validation. 1,050 customers, about 6,300 rows, go to test. No customer's rows are ever split across two groups, so the model is always tested on customers it has genuinely never trained on.

Darya's team applies both fixes together: split by customer to stop customer leakage, and keep each customer's own test-set snapshots later in time than their training-set snapshots, to stop time leakage too. Both problems, and both fixes, are really the same idea Chapter 2 already taught, applied honestly to a dataset shaped like Zagros Mobile's.

COMMON MISTAKE

✗ Wrong: Splitting a dataset of monthly customer snapshots randomly, row by row, so the very same customer's snapshots can land in both the training set and the test set.

✓ Right: Split by customer, not by row. Assign every row belonging to one customer to only one group, training, validation, or test, so a model is always tested on customers it has genuinely never seen before.

21.3 Audit the Label and the Features

With a safe split in place, Darya still cannot train anything yet. Two more things need a careful look first: the label itself, and the list of candidate features. Both can hide problems that no split, however careful, can fix.

A churn label sounds obvious until you try to write down its exact rule. Does a customer count as churned the moment they stop using their phone, or only once they formally cancel? What about a customer whose card payment fails and is never fixed? Zagros Mobile's team agrees on one precise, written definition before touching any data: a customer counts as churned if their subscription status becomes "canceled," or becomes "suspended for nonpayment" and stays suspended for more than 30 days, with the check made within the 30 days after prediction time.

WORKED EXAMPLE — DECIDE WHETHER FOUR CUSTOMER STORIES COUNT AS CHURN

Using Zagros Mobile's definition above, decide whether each of these four customers counts as churn, checked within the 30-day window of March 1 to March 31. Customer 1 formally canceled their plan on March 12. Customer 2's card payment failed on March 5, and service was restored on March 9, after only 4 days, once the customer updated their card. Customer 3 simply stopped using data and voice service after March 2, but the plan bills automatically either way, so the account was never canceled and never missed a payment. Customer 4's account was suspended for nonpayment on March 20 and was still suspended when the window closed on March 31, only 11 days later.

1. Customer 1 formally canceled on March 12, matching the definition's first condition directly: churn.
2. Customer 2's suspension lasted only 4 days, March 5 to March 9, far short of the 30-day rule, and the account was never canceled: not churn.
3. Customer 3 never canceled and was never suspended for nonpayment; the account's status never changed at all, even though real usage dropped to zero: not churn, at least not under this definition.
4. Customer 4's suspension had lasted only 11 days by March 31, still short of the 30-day rule at the moment the label must be recorded: not churn, at least not yet.

Answer: Only Customer 1 counts as churn under Zagros Mobile's written definition. Customers 2, 3, and 4 do not, even though a manager glancing at their accounts might reasonably guess otherwise. A precise, written label definition, agreed on in advance, stops different team members from silently disagreeing about which customers "really" churned.

Customer 3 deserves a second look, because this definition has a real blind spot. A customer who quietly stops using the service but keeps paying automatically never trips this label at all, even though that customer's business is, in every practical sense, already lost. Teams sometimes call this pattern silent churn. Zagros Mobile's team cannot fix this blind spot by writing a cleverer formula; they can only be honest about it, and consider tracking usage decline as a second, related project later.

IMPORTANT

A label definition is a choice, not a fact of nature, and every choice leaves something out. Zagros Mobile's label misses customers who quietly stop using the service without ever canceling. Write your label definition down, share it with the team that will act on it, and say plainly what it does not catch, instead of letting a hidden gap surprise someone later.

Once the label is precise, the feature list needs the same kind of scrutiny Chapter 2 gave the clinic's candidate features. A column can look perfectly ordinary and still leak the very answer the model is supposed to predict.

WORKED EXAMPLE — CHECK FOUR CANDIDATE FEATURES FOR LEAKAGE

Darya's team is choosing which columns to use as features. Four candidates are suggested: (1) average monthly data usage over the trailing 3 months, (2) the number of days since the customer's account was marked "pending cancellation," (3) the number of late payments in the last 90 days, and (4) the final closing balance recorded on a customer's account. For each candidate, decide whether it is safe to use or whether it would cause data leakage.

1. Check candidate 1 against prediction time: 3 months of past usage is fully known on the first day of the month, so it is safe to use.
2. Check candidate 2 against prediction time: this column is only ever filled in once a customer has already started canceling, which happens at or after the very event being predicted, so using it would be data leakage.
3. Check candidate 3 against prediction time: 90 days of past late payments are fully known before the month begins, so it is safe to use.
4. Check candidate 4 against prediction time: a "final closing balance" only exists once an account has already closed, long after the prediction must be made, so using it would be data leakage.

Answer: Candidates 1 and 3 are safe features. Candidates 2 and 4 must be thrown out, because both are only ever recorded once a customer has already left, or is already leaving. Including either one would let the model quietly peek at the answer, making test scores look wonderful while the real, deployed system could never see that column filled in before it needed a prediction.

Not every candidate is this clear-cut. Zagros Mobile's support system also logs a tag, "retention save attempt," whenever an agent notices a customer sounding unhappy during a call and tries to fix the problem on the spot. This tag is genuinely known before the 30-day window closes, so it passes the timing check. But it is also suspicious for a different reason: an agent typically applies this tag only after already noticing the same warning signs the model is trying to learn on its own, so it risks teaching the model to lean on a human's hunch instead of on the underlying pattern. Darya's team keeps this column out of the first version of the model, and plans to study it separately later. Not every leakage risk is a simple yes-or-no question, and a careful audit sometimes ends with "we are not sure yet, so we leave it out for now."

21.4 Build Two Baselines Before Any Real Model

Chapter 2 taught you never to trust a fancy model's score until you have compared it to a baseline, a deliberately simple method that any real model must beat. Churn prediction is exactly the kind of project where skipping this step can quietly hide a useless model behind an impressive-looking number, because most customers, in any given month, do not churn.

WORKED EXAMPLE — EVALUATE THE 'ALWAYS PREDICT STAY' BASELINE

Zagros Mobile's validation set has 3,000 customers. In a typical month, 300 of them truly churn and 2,700 truly stay. A baseline model always predicts "Stay" for every customer, no matter what. Find this baseline's accuracy, then find its

recall for catching true churners, using the confusion-matrix definitions from Chapter 4.

1. Count correct predictions: the baseline is correct on every customer who truly stays, 2,700 customers, and wrong on every customer who truly churns, 300 customers.
2. Find accuracy: correct predictions divided by total predictions: $2,700 \div 3,000 = 0.9$.
3. Convert to a percentage: $0.9 \times 100 = 90$.
4. Find true positives: the baseline never predicts "Churn" for anyone, so it never correctly catches a true churner: $TP = 0$.
5. Find false negatives: every one of the 300 true churners is missed, so $FN = 300$.
6. Find recall: $TP \div (TP + FN) = 0 \div (0 + 300) = 0 \div 300 = 0$.
7. Convert the result to a percentage: $0 \times 100 = 0$.

Answer: The "always predict Stay" baseline reaches 90% accuracy and exactly 0% recall. It looks impressive on a slide, and it is completely useless for the retention team's real goal, because it never flags a single one of the 300 customers who are about to leave.

COMMON MISTAKE

✗ Wrong: Reporting a churn model's accuracy alone, without also reporting recall, especially when churners make up a small share of all customers.

✓ Right: Always report recall and precision next to accuracy for a churn model. A high accuracy number, by itself, can hide a model that never catches a single true churner, exactly like the baseline above.

The opposite baseline is just as instructive, because it shows the other extreme the retention team must avoid.

WORKED EXAMPLE — EVALUATE THE OPPOSITE BASELINE: 'ALWAYS PREDICT CHURN'

Using the same validation set, 300 true churners and 2,700 true stayers out of 3,000 customers, consider the opposite baseline: always predict "Churn" for every customer. Find this baseline's recall and its precision.

1. Every customer is predicted "Churn," so every true churner is correctly caught: $TP = 300$, and $FN = 0$.
2. Find recall: $TP \div (TP + FN) = 300 \div (300 + 0) = 300 \div 300 = 1.0$.
3. Convert to a percentage: $1.0 \times 100 = 100$.
4. Every customer who truly stays is also wrongly flagged as "Churn": $FP = 2,700$.
5. Find precision: $TP \div (TP + FP) = 300 \div (300 + 2,700) = 300 \div 3,000 = 0.1$.
6. Convert to a percentage: $0.1 \times 100 = 10$.

Answer: This baseline catches all 300 true churners, 100% recall, but only 1 flagged customer in 10 is actually a true churner, 10% precision. If the retention team tried to call all 3,000 flagged customers, they

would waste 9 calls for every 1 that truly mattered. A useful model, and later a well-chosen threshold, must land somewhere between these two useless extremes.

These two baselines are not just a warm-up exercise. They mark the two ends of a line every real classifier lives somewhere on: catch nothing and waste no effort, or catch everything and waste enormous effort. Lesson 21.5 compares three real models against both of these extremes, then Lesson 21.6 shows how to choose a landing point on that line that is actually worth using.

21.5 Compare Models and Check Calibration

With both baselines in hand, Darya trains three real candidate models on the training set, using the features audited in Lesson 21.3: a **logistic regression**, a **random forest**, and a gradient boosting model, the two ensemble methods Chapter 10 built from many small decision trees. She compares all three on the untouched **validation set**, at the ordinary default **threshold** of 0.5, using **precision**, **recall**, and **F1 score** together, exactly as Chapter 4 recommends.

The logistic regression reaches a precision of 40% and a recall of 50%, for an F1 score of about 44.4%. The random forest reaches a precision of 50% and a recall of 70%, for an F1 score of about 58.3%. The gradient boosting model reaches a precision of 60% and a recall of 75%, for an F1 score of about 66.7%, the strongest of the three, and a clear improvement over even the better of Lesson 21.4's two baselines. This result echoes something Chapters 10 and 11 already told you honestly: on tabular business data like this, a boosted tree model very often wins. Darya moves forward with the gradient boosting model as her leading candidate, though she keeps the other two trained and saved, in case a future fairness check, like Lesson 21.8's, ever favors a simpler model instead.

A strong F1 score is not the last question to ask. Before Darya trusts this model's probabilities enough to build the currency-based case in Lesson 21.6, she needs to know whether those probabilities actually mean what they say. A model can rank customers correctly, riskiest first, while its actual probability numbers still run too high or too low. **Calibration**, the check Chapter 16 introduced for exactly this problem, compares a model's predicted probabilities to how often those customers really did churn.

WORKED EXAMPLE — CHECK WHETHER THE MODEL'S PROBABILITIES ARE CALIBRATED

Darya groups every validation customer whose predicted churn probability fell between 0.70 and 0.80 into one bucket. This bucket contains 50 customers. If the model's probabilities are trustworthy, close to the middle of that range, about 75%, should truly churn. In reality, 44 of these 50 customers truly churned. Find the bucket's true churn rate, and compare it to the model's implied 75%.

1. Divide the true churners by the bucket size: $44 \div 50 = 0.88$.
2. Convert to a percentage: $0.88 \times 100 = 88$.

3. Compare this true rate, 88%, to the roughly 75% the model's own predicted probabilities implied for this bucket.

Answer: The true churn rate in this bucket, 88%, is noticeably higher than the roughly 75% the model's probabilities suggested. In this range, the model is underconfident: real risk runs higher than its own numbers admit. Lesson 21.6's currency-based worked examples need trustworthy probabilities, so a gap like this must be fixed, often with a calibration step added after training, before those numbers are used to compute real money.

COMMON MISTAKE

✗ **Wrong:** Choosing a model using only its overall accuracy or F1 score, then trusting its predicted probabilities without ever checking whether they are calibrated.

✓ **Right:** Check calibration separately from ranking ability. A model can sort risky customers correctly while still reporting probability numbers that are systematically too high or too low, which matters enormously once those numbers are used to estimate real money, as in Lesson 21.6.

21.6 Choose a Threshold Using Capacity and Value

A trained model produces a probability for every customer, not yet a decision. Chapter 7 used 0.5 as a default cutoff, and Chapter 16 showed that this cutoff can be moved deliberately when one kind of mistake costs more than another. Zagros Mobile's retention team faces a very concrete version of this choice, and it has two parts, not one: how many customers can the team actually call, and is calling them worth what it costs?

Ronahi's retention team has 4 staff members, and each one can make about 200 thoughtful, unrushed retention calls a week. That gives the team a hard capacity of 800 calls a week, scored against roughly 20,000 active customers checked every week. Whatever threshold Darya chooses must flag close to 800 customers: comfortably more would overwhelm the team, and needlessly fewer would waste the capacity Ronahi already has.

Darya studies the model's **precision-recall curve**, the tool Chapter 16 introduced for reading precision and recall at every possible threshold, and finds the exact threshold that flags close to 800 of this week's 20,000 scored customers. At that threshold, validation data shows a precision of 0.75: three out of every four flagged customers are genuinely at risk.

EQUATION — EXPECTED CUSTOMERS SAVED AT A THRESHOLD

$$\text{saved} \approx \text{flagged} \times \text{precision} \times \text{save rate}$$

In words: The expected number of customers saved equals the number of customers flagged at this threshold, times the threshold's precision, the share of those flags that are truly at risk, times the save rate, the share of truly at-risk, contacted customers who stay because of the contact.

flagged number of customers the model flags at this threshold (*count*)

precision share of flagged customers who are genuinely about to churn, measured on a validation or test set (*value from 0 to 1*)

save_rate share of genuinely at-risk, contacted customers who stay because of the contact, usually estimated from past campaigns (*value from 0 to 1*)

saved expected number of customers this campaign keeps who would otherwise have left (*count*)

Note: Save rate is a business assumption, not something the model itself predicts. Treat it as a clearly labeled estimate until a real experiment, like Lesson 21.9's, measures it directly.

WORKED EXAMPLE — FIND THE EXPECTED NUMBER OF CUSTOMERS SAVED AT THE CHOSEN THRESHOLD

Darya's chosen threshold flags 800 of this week's 20,000 customers, with a validation precision of 0.75 at that threshold. Past retention campaigns show that a phone call from the retention team successfully keeps 30% of the true at-risk customers it reaches, persuading them to stay another month. Find the expected number of true churners among the 800 flagged customers, then find the expected number of customers this week's campaign will actually save.

1. Find the expected number of true churners among the 800 flagged customers, using the threshold's precision: $0.75 \times 800 = 600$.
2. Find the flagged customers who were never truly at risk: $800 - 600 = 200$. A call to these 200 customers cannot "save" anyone who was never leaving.
3. Apply the retention call's 30% save rate to only the 600 true churners, since a call can only change the mind of someone who was actually about to leave: $0.30 \times 600 = 180$.

Answer: The team expects to save about 180 customers this week, out of 800 calls. The other 620 calls either reach a customer who was never going to churn, 200 of them, or reach a true churner who leaves anyway despite the call, $600 - 180 = 420$ of them.

Python · NumPy

 runnable in the app

```
flagged = 800
precision = 0.75
save_rate = 0.30

true_churners = flagged * precision
saved = true_churners * save_rate

print(true_churners, saved)
```

The variables ``flagged``, ``precision``, and ``save_rate`` store the same three numbers used by hand above. ``true_churners`` repeats step 1, multiplying the flagged count by precision. ``saved`` repeats step 3, multiplying ``true_churners``

by the save rate. Running this code prints 600.0 and 180.0, the same two numbers already found with a pencil.

Saving 180 customers is a meaningful number on its own, but Ronahi's real question is simpler and blunter: is this campaign worth running? Answering that means turning customers saved into money. Every dollar figure in the rest of this lesson is a clearly labeled illustration for Zagros Mobile's planning, not a verified real-world statistic, exactly the way Chapter 2 and Chapter 4 both used illustrative numbers in their own worked examples.

EQUATION — NET VALUE OF A RETENTION CAMPAIGN

$$\text{net value} = (\text{saved} \times \text{value per save}) - (\text{flagged} \times \text{cost per contact})$$

In words: Net value equals the money gained from saved customers, minus the money spent contacting everyone the threshold flagged, including the flagged customers who were never truly at risk.

saved expected number of customers kept, from the previous formula (count)

value per save the average future value of keeping one customer, in currency (currency)

flagged number of customers contacted at this threshold (count)

cost per contact the cost of reaching one customer, such as staff time or a discount offered (currency)

net value the campaign's total expected benefit, after subtracting its cost (currency)

Note: A threshold with more flagged customers is not automatically better. More flags mean more chances to save someone, but also more cost spent on customers who were never at risk.

WORKED EXAMPLE — FIND THE EXPECTED NET VALUE OF THIS WEEK'S CAMPAIGN

Zagros Mobile estimates that keeping one customer is worth \$90 in future revenue, on average, since a saved customer typically stays several more months. Each retention call costs about \$5, counting staff time and any discount offered. Using the 180 customers saved and the 800 customers flagged from the worked example above, find this week's expected net value.

1. Find the gross value from saved customers: $180 \times \$90 = \$16,200$.
2. Find the total cost of contacting every flagged customer, saved or not: $800 \times \$5 = \$4,000$.
3. Subtract the cost from the gross value: $\$16,200 - \$4,000 = \$12,200$.

Answer: This week's campaign has an expected net value of about \$12,200, or about \$15.25 of net value for every call the team makes, since $\$12,200 \div 800 = \15.25 . The campaign is comfortably worth running at this threshold, even after paying for the 620 calls that do not end up saving anyone.

Before locking in this threshold, Darya checks one more thing: would a different threshold, if the team had more capacity, create even more value, or does 800 calls happen to be a genuinely good stopping point?

WORKED EXAMPLE — CHECK WHETHER A LOOSER THRESHOLD WOULD CREATE MORE NET VALUE

Suppose Zagros Mobile considered hiring extra help to allow 1,200 calls a week instead of 800. At that looser threshold, precision on the validation set falls to 0.55, since more of the newly flagged customers are less certain cases. Using the same \$90 value per save, \$5 cost per contact, and 30% save rate as before, find this option's expected customers saved and its expected net value, then compare it to the 800-call option's net value of \$12,200.

1. Find the expected true churners among the 1,200 flagged customers: $0.55 \times 1,200 = 660$.
2. Find the expected customers saved: $0.30 \times 660 = 198$.
3. Find the gross value from saves: $198 \times \$90 = \$17,820$.
4. Find the total cost of 1,200 calls: $1,200 \times \$5 = \$6,000$.
5. Find the net value: $\$17,820 - \$6,000 = \$11,820$.
6. Compare the two options side by side: \$11,820 for 1,200 calls, against \$12,200 for 800 calls.

Answer: The looser, 1,200-call option saves more customers in total, 198 against 180, but its net value, \$11,820, is actually lower than the 800-call option's \$12,200. Extra calls cost money even when they reach a customer who was never at risk, and past a certain point those extra costs outweigh the extra saves. The threshold with the single highest number of saves is not always the threshold with the highest net value.

IMPORTANT

Every number in this lesson's formulas beyond the model's own precision, the save rate and the value of one saved customer, is a business estimate, not a measurement the model produced. Label these numbers clearly as assumptions whenever you present them, and revisit them once real evidence arrives. Lesson 21.9 shows exactly how Zagros Mobile checks whether its 30% save rate assumption was actually true.

COMMON MISTAKE

✗ Wrong: Leaving a churn model's threshold at the default of 0.5, or choosing whichever threshold maximizes recall, without ever checking a team's real capacity or a campaign's real net value.

✓ Right: Choose a threshold using both capacity, how many customers a team can actually contact, and value, what each save is worth against what each contact costs. The best threshold is rarely the one with the single highest recall or the single highest precision.

ACTIVITY — TRY IT IN ML STUDIO

1. Open ML Studio and go to the Play tab.

2. Choose the "Telecom Churn" dataset, and fit a Gradient Boosting model.
3. Open the threshold slider next to the precision-recall panel, and move it until the flagged count sits near a target capacity of your own choosing.
4. Note the precision shown at that threshold, then use pencil and paper to estimate customers saved and net value, using this lesson's two formulas and your own assumed save rate and value per save.
5. Move the threshold higher, then lower, and write down whether your estimated net value goes up or down each time, and why.

You need: Computer, tablet, or phone with the ML Studio app

21.7 Train the Full Pipeline, Then Test Once

Every piece is ready: audited features, a safe customer-and-time split, a chosen model, and a threshold picked for real capacity and real value. Chapter 3 taught you to wrap preprocessing and a model together into one **pipeline**, so that every step, fit only on training data, travels together and never leaks information from validation or test rows. Darya builds exactly this kind of pipeline for the final model.

Python · NumPy

 runnable in the app

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import GradientBoostingClassifier

preprocess = ColumnTransformer([
    ("scale", StandardScaler(), numeric_columns),
    ("encode", OneHotEncoder(handle_unknown="ignore"), category_columns),
])

pipeline = Pipeline([
    ("preprocess", preprocess),
    ("model", GradientBoostingClassifier(random_state=42)),
])

pipeline.fit(X_train, y_train)
probabilities = pipeline.predict_proba(X_test)[: , 1]
flagged = probabilities >= chosen_threshold
```

The `ColumnTransformer`, from Chapter 3, applies `StandardScaler` to numeric columns like tenure and usage, and `OneHotEncoder` to category columns like city and plan type. Wrapping this preprocessing step and the `GradientBoostingClassifier` together inside one `Pipeline` means calling `fit` once, on training data only, runs both steps safely in the right order. `predict_proba` returns a full probability for every test customer, the same kind of output Chapter 7's code block introduced, and the last line turns those

probabilities into flags using the threshold chosen in Lesson 21.6, not the default of 0.5.

With training complete, Darya's team opens the test set for the first and only time, exactly as Chapter 2's golden rule requires.

WORKED EXAMPLE — REPORT THE FINAL TEST-SET RESULT, ONE TIME ONLY

On the untouched test set of 3,000 customers, 300 of whom truly churn, the final pipeline's predictions at the chosen threshold produce $TP = 210$, $FN = 90$, $FP = 140$, and $TN = 2,560$. Find the test-set recall, precision, and F1 score, then compare them to the validation numbers from Lesson 21.5: 75% recall and 60% precision.

1. Check the total: $TP + FN + FP + TN = 210 + 90 + 140 + 2,560 = 3,000$, matching the 3,000 test customers.
2. Find recall: $TP \div (TP + FN) = 210 \div (210 + 90) = 210 \div 300 = 0.70$, or 70%.
3. Find precision: $TP \div (TP + FP) = 210 \div (210 + 140) = 210 \div 350 = 0.60$, or 60%.
4. Find F1: $2 \times \text{precision} \times \text{recall} \div (\text{precision} + \text{recall}) = 2 \times 0.60 \times 0.70 \div (0.60 + 0.70) = 0.84 \div 1.30 \approx 0.646$, or about 64.6%.

Answer: The final test-set score is 70% recall, 60% precision, and an F1 of about 64.6%. Recall dropped a little from the validation set's 75%, while precision held steady at 60%. A small drop like this from validation to test is normal and expected, exactly as Chapter 2 taught: it is a healthy sign of an honest process, not a failure.

A single honest test score is not the end of the story. Before this pipeline reaches a single real customer, it still needs two more checks: whether it treats different groups of customers fairly, and how its decision will be watched once it goes live.

21.8 Fairness and the Human in the Loop

A model that works well on average can still fail one group of customers quietly. Chapter 18 introduced **fairness**: checking whether a model's errors, and its benefits, land evenly across different groups of people, not just checking one overall score. Darya runs exactly this kind of check before Ronahi's team calls a single customer.

She compares the model's recall across Zagros Mobile's five largest service regions: Erbil, Sulaymaniyah, Duhok, Baghdad, and Basra. Recall in four regions sits close to the overall 70% found in Lesson 21.7. In Basra, recall is noticeably lower, closer to 55%, meaning true churners there are quietly missed more often than customers elsewhere. Darya digs into why, and finds a likely cause: Zagros Mobile launched a new prepaid-to-postpaid conversion plan in Basra only eight months ago, so the training data holds far fewer full customer histories from that region, and the model has had less to learn from there.

This kind of gap deserves a second look at the features themselves, because a column can encode unfairness without ever naming a group directly. Chapter 18 called this a **protected attribute** when a column names a group directly, such as a customer's gender or age, and a **proxy variable** when an ordinary-looking column stands in for one indirectly. Handset price tier is one such risk at Zagros Mobile: a customer's phone model can closely track their income, and income can closely track other things a fair retention process should never quietly penalize a customer for. Darya keeps handset price tier in the model, since it does carry real, useful signal about usage patterns, but the team commits to something else instead: whatever offer a customer receives, a discount, a plan upgrade, or simply a helpful phone call, must depend only on that customer's predicted risk and their own account history, never on their region, their phone's price, or any group they belong to.

IMPORTANT

Checking one overall score is not enough. Compare a model's recall, precision, and calibration separately across regions, plans, and any group you have reason to care about, not just across the whole customer base at once. A gap like Basra's is not always a sign of true unfairness, but it is always worth a second look before the model reaches real customers.

Fairness is not only a property of a model's numbers. It is also a property of what happens after the model speaks. Every flagged customer at Zagros Mobile is reviewed by a retention agent before any offer is made, never contacted by an automatic message alone. An agent can see that a flagged customer just had one genuinely hard month, a late payment because of a hospital visit, say, and choose a warmer, more patient conversation than the model's bare number would suggest on its own. The model decides who gets a closer look; a person still decides what to actually say.

COMMON MISTAKE

✗ Wrong: Assuming a model is fair simply because its overall accuracy, precision, or recall looks strong across the whole customer base.

✓ Right: Check a model's key metrics separately for each region, plan, or group that matters. A strong overall number can hide a much weaker result for one particular group, exactly like Basra's lower recall.

21.9 Monitor the Decision, Not Only the Model

A model's honest test score, from Lesson 21.7, describes one moment. Chapter 19 taught you that a live model needs continued watching, because the world it was trained on keeps changing. Zagros Mobile's churn model is no exception, and it needs three separate kinds of watching, not just one.

The first is familiar: **monitoring** for **data drift**, an ongoing check that new customers' features still look like the ones the model trained on. When Zagros Mobile launches a new unlimited-data plan next year, average usage across the

whole customer base will shift upward almost overnight, and a model trained before that launch may quietly grow less accurate without anyone deciding to break it. Darya's team tracks each feature's average value every week and compares it to the training data's own average, watching for a drift large enough to matter.

The second is a direct check on the model's own honesty. As each month's 30-day window closes, the true churn outcome becomes known for every customer who was scored a month earlier. Darya recomputes recall, precision, and calibration on this freshly arrived, real outcome data, the same three checks from Lessons 21.5 through 21.7, but now measured on live customers instead of a held-out test set. A steady drop in any of the three is an early warning sign, worth investigating before it grows serious enough to require a **rollback** to an earlier, trusted version of the pipeline.

The third kind of watching is the most important, and it is the easiest to get wrong. Suppose Darya notices, after a few months, that customers the retention team called churned at a lower rate than customers who were never flagged at all. It is tempting to read this as proof the calls work. It is not proof of any such thing. The customers who were called were, by design, the riskiest customers in the whole base; comparing them to customers who were never flagged at all compares two very different groups from the start, not two groups that only differ in whether they received a call.

IMPORTANT

A prediction is not proof that acting on it changes anything. Watching flagged, contacted customers do better than everyone else proves very little on its own, because the two groups were already different before any call was ever made. Only a controlled experiment, comparing similarly at-risk customers who are contacted against similarly at-risk customers who are not, can show whether a retention call is truly changing customer behavior.

Zagros Mobile fixes this the same way any careful team would. Each week, among the customers the threshold flags, a small, random slice, say 1 in every 10, is deliberately held back and not called, purely so the team has an honest comparison group of equally at-risk customers. A month later, the team compares the churn rate of the called group against the churn rate of this held-back group. Because both groups were chosen the same way, equally risky by the model's own scoring, any real difference between them can now be credited to the phone call itself, not to some difference that was already sitting there before anyone picked up the phone. The result of exactly this experiment closes out Zagros Mobile's story below.

CASE STUDY — LAUNCHING THE RETENTION MODEL AT ZAGROS MOBILE

Six months after Darya first wrote her decision sentence, Zagros Mobile's churn model is fully live. Every Monday morning, the pipeline scores all 20,000 active customers, flags close to 800 of them using the threshold Lesson 21.6 chose,

and sends that list to Ronahi's retention team, sorted so the highest-risk customers appear first. Each flagged customer's account history travels with the list, so an agent can open a conversation informed, not cold. Nothing about a flag ever appears on a customer's bill or account status; it only ever triggers a private, internal task for a human to follow up on.

The team also builds in the controlled experiment Lesson 21.9 described, holding back a random 1 in 10 of each week's flagged customers as an untouched comparison group. Three months of real results arrive. Among customers who were called, 8% churned over the following month. Among the held-back comparison group, equally high-risk by the model's own scoring but never contacted, 15% churned. The gap, about 7 percentage points, is smaller than the optimistic 30% save rate the team had assumed while planning in Lesson 21.6, but it is real, and now it rests on a genuine experiment instead of a guess. Darya updates the net-value formula with this measured, more modest number, and the campaign still comes out clearly worth running, just not quite as valuable as the first hopeful estimate had suggested.

The Basra recall gap from Lesson 21.8 also gets a real fix, not just a note in a report. Once six more months of Basra customer histories accumulate, Darya retrains the pipeline, and recall in Basra rises from 55% to 66%, closer to the rest of the network, though still not fully equal. She flags this as an open item to keep watching, rather than a solved problem to forget about.

Ronahi's own verdict, delivered in a short review meeting, sounds a lot like the honest, measured tone this book has tried to model from Chapter 1 onward: the system genuinely helps, it did not solve everything on the first try, it needed a real experiment before anyone could honestly claim it changed a single customer's mind, and it will keep needing quiet attention, every month, for as long as Zagros Mobile keeps using it.

IMPORTANT

Publication-Ready Conclusion: This churn model is suitable for one job: ranking customers by estimated risk, so a limited retention team can spend its time where it likely matters most. It is not suitable for deciding a customer's fate automatically, for explaining with certainty why any one customer might leave, or for proving that a phone call changes behavior. A high predicted risk is a reason to offer a human conversation and real help, never a verdict, a penalty, or a public label. And no matter how carefully a threshold is chosen using capacity and value, a prediction alone can never prove that contacting someone changes what they do next. Only a controlled experiment, like the one Zagros Mobile ran, comparing contacted and uncontacted customers who were equally at risk, can show that.

COMMON MISTAKES TO AVOID

- Splitting a dataset of repeated customer snapshots randomly, row by row, instead of by customer, letting the same customer appear in both training and test.

- Using a feature that is only ever recorded after a customer has already started leaving, such as a cancellation date or a final closing balance.
- Trusting a single accuracy number for a churn model, when the "always predict Stay" baseline alone can reach 90% accuracy with 0% recall.
- Choosing a model by its accuracy or F1 score alone, without ever checking whether its predicted probabilities are calibrated.
- Setting a decision threshold at the default of 0.5, or for maximum recall alone, without checking a team's real capacity or a campaign's real net value.
- Judging fairness from one overall score, instead of checking recall, precision, and calibration separately across regions or groups.
- Treating a correlation between "was contacted" and "stayed" as proof that contacting someone changed their behavior, instead of running a controlled experiment.

CHAPTER SUMMARY

- A churn project should start with a decision sentence and a named unit of observation, target, and prediction time, exactly like every other project in this book.
- Churn is a customer ending their relationship with a business; a written, precise label definition prevents disagreement and reveals blind spots such as silent churn.
- Repeated customer snapshots create two leakage risks: time leakage, fixed by a time-based split, and customer leakage, fixed by splitting whole customers, not rows.
- A feature audit checks every candidate column against the true prediction time, throwing out any column only ever recorded after the outcome is already known.
- An "always predict Stay" baseline can reach 90% accuracy with 0% recall; an "always predict Churn" baseline reaches 100% recall with very low precision. A useful model must beat both.
- Comparing models needs more than one score: precision, recall, F1, and calibration together, since a model can rank customers well while its probabilities still run too high or too low.
- A good decision threshold balances a team's real capacity to act with the real value of acting, not just the highest recall or precision available.
- A full pipeline keeps preprocessing and a model together, trained once on training data and evaluated once, honestly, on a test set that was never touched before.
- Fairness means checking a model's errors separately across regions and groups, and keeping a human reviewer between every flagged customer and any real action.

- Monitoring a churn system means watching for data drift, watching live recall and precision, and running a controlled experiment, because a prediction alone can never prove that an action changed anyone's behavior.

Practice Problems

Try each problem on paper first.

Problem 1 [Basic] Define churn in your own words, and give one example of a churn event that is not a formal cancellation.

Problem 2 [Basic] Explain, in one or two sentences, the difference between time leakage and customer leakage.

Problem 3 [Basic] A validation set has 2,000 customers, of whom 200 truly churn. Find the accuracy of an "always predict Stay" baseline, showing every step.

Problem 4 [Basic] Using the same 2,000 customers from problem 3, find the recall of the "always predict Stay" baseline, showing every step.

Problem 5 [Standard] A dataset has one row per customer per month, covering 5,000 customers over 4 months, giving 20,000 rows. The team splits by customer into 80% training and 20% test. How many customers, and roughly how many rows, land in each group?

Problem 6 [Standard] A candidate feature records the date a customer's line was permanently disconnected. Explain, in your own words, why this feature would cause data leakage, and state the earliest point in a customer's history when it could honestly become known.

Problem 7 [Standard] A churn model flags 500 customers out of 10,000 scored this week, with a validation precision of 0.60 at that threshold. Past campaigns show a 25% save rate. Find the expected number of customers saved, showing every step.

Problem 8 [Standard] Using the 500 flagged customers and the customers-saved result from problem 7, and a value of \$80 per saved customer and a cost of \$6 per contact, find this week's expected net value, showing every step.

Problem 9 [Standard] A model's predicted-probability bucket of 0.40 to 0.50 contains 80 customers, of whom 28 truly churned. Find the bucket's true churn rate, and state whether the model looks overconfident or underconfident in this bucket.

Problem 10 [Standard] Explain, in your own words, why comparing called customers to never-flagged customers cannot prove that a retention call changed anyone's behavior.

Problem 11 [Challenge] A team scores 15,000 customers and must choose between two thresholds. Threshold A flags 600 customers at a precision of 0.70. Threshold B flags 1,000 customers at a precision of 0.50. Using a 30%

save rate, a value of \$100 per save, and a cost of \$8 per contact, find the expected net value of each threshold, showing every step, and state which one the team should choose.

Problem 12 [Challenge] A churn model reaches an overall recall of 72%, but recall in one region is only 48%. (a) Suggest one possible cause, drawn from this chapter. (b) Describe two responsible next steps the team should take before quietly ignoring the gap.

Problem 13 [Challenge] Design your own churn-style project for a subscription service you know, such as a gym, a streaming service, or a mobile data plan. Name your unit of observation, your target, one feature you would exclude for leakage, and one reason contacting a flagged customer might not, on its own, prove your intervention works.

Problem 14 [Challenge] Open ML Studio, fit a Gradient Boosting model on the "Telecom Churn" dataset, and move the decision threshold until the flagged count matches a capacity of your own choosing. Record the precision at that threshold, compute an expected-saved and net-value estimate by hand using this chapter's formulas, and explain in a few sentences whether you would recommend that threshold.

Reference

Appendix A — Mathematics When You Need It

Appendix B — Python, NumPy, and scikit-learn Field Guide

Glossary

REFERENCE

APPENDIX A

Mathematics When You Need It

A calm, step-by-step reference for the math behind every chapter

This appendix collects the mathematics used across the whole book and gathers it in one place, for whenever you want a slower, more patient look at it. You do not need to read this appendix from beginning to end before Chapter 1, and most readers never will. Instead, dip into one short section whenever a chapter mentions an idea you would like to see explained more carefully, with every arithmetic step spelled out. Every single idea in this appendix already appeared informally in an earlier chapter, doing real work on a real example. This appendix does not add anything new; it only slows each idea down, gives it a proper definition, and works through one or two more examples by hand, so the symbols never feel mysterious.

A.1 Scalars, Vectors, and Matrices

Every chapter in this book worked with numbers arranged in different ways. Sometimes there was just one number. Sometimes there was a short list of numbers describing one example. Sometimes there was a whole table of examples. Mathematics gives each of these three arrangements its own name, and this book has quietly been using all three since Chapter 1.

WORD TO KNOW

Scalar — A single number, on its own, with no other numbers attached to it. A melon's weight, 4.2 kilograms, is a scalar. A model's accuracy, 85%, is also a scalar.

A single scalar is often not enough to describe something interesting. One melon has more than one measurement worth recording: its weight, and also a sound score from a ripeness meter. Writing both numbers together, in a fixed order, gives that melon its own small, ordered list of numbers.

WORD TO KNOW

Vector — An ordered list of numbers. A vector often describes one example, one number per feature, in a fixed, agreed order. The melon (4, 8) is a vector: weight 4 kilograms, sound score 8, always in that order.

A single vector describes one melon. A real project needs many melons at once, so many vectors are naturally stacked together, one on top of the next, into a rectangular table.

WORD TO KNOW

Matrix — A rectangular table of numbers, arranged in rows and columns. A matrix often stores a whole dataset: one row per example, and one column per feature.

A matrix's size has its own short description, called its shape, written as two numbers in parentheses: the number of rows, then the number of columns. A matrix with 4 rows and 2 columns has shape (4, 2). This book's code already followed a quiet convention worth naming here directly: a whole feature matrix is usually stored under a capital letter, *X*, and a single column of labels or targets is stored under a lowercase letter, *y*. A capital letter for a full table, a lowercase letter for a single list — Chapter 6's code used exactly this convention, and Appendix B uses it throughout.

WORKED EXAMPLE — FIND THE SHAPE OF A SMALL MATRIX, AND READ ONE ENTRY

Chapter 1 recorded weight and sound score for four melons: ripe melon 1 is (4, 8), ripe melon 2 is (6, 10), unripe melon 1 is (2, 2), and unripe melon 2 is (2, 4). Arrange these four melons into a matrix *X*, one row per melon, in that order, and one column per feature, weight then sound score. State the shape of *X*, and read the entry in row 3, column 2.

1. Count the rows: one row per melon, and there are four melons, so *X* has 4 rows.
2. Count the columns: one column per feature, weight and sound score, so *X* has 2 columns.
3. Write the shape as (rows, columns): (4, 2).
4. Find row 3 by counting from the top: row 1 is (4, 8), row 2 is (6, 10), and row 3 is (2, 2), the first unripe melon.
5. Find column 2 of row 3: column 1 of that row holds 2 (weight), and column 2 holds 2 (sound score).

Answer: *X* has shape (4, 2), and the entry in row 3, column 2 is 2, the sound score of the first unripe melon. This appendix counts rows and columns starting from 1, the usual math convention. Appendix B.2 shows that Python and NumPy instead count starting from 0, so this exact same entry is written *X*[2, 1] in code.

A.2 Mean, Variance, and Standard Deviation

Several chapters have already computed an average without pausing to name the general rule behind it. Chapter 1's centroid averaged each feature separately. Chapter 4's mean absolute error averaged a list of errors. This section names that general rule precisely, then adds two new ideas that describe not just the center of a group of numbers, but how spread out those numbers are.

EQUATION — MEAN (AVERAGE) OF A LIST OF NUMBERS

$$\text{mean} = \frac{\sum x}{n}$$

In words: The mean equals the sum of every value in the list, divided by how many values the list has.

mean the average value of the list (same unit as the data)

Σx every value in the list, added together (same unit as the data)

n the number of values in the list (count)

Note: This is the exact rule behind Chapter 1's centroid: average each feature column separately, one column at a time.

WORKED EXAMPLE — FIND THE MEAN OF FIVE DAILY TEMPERATURES

A weather station in Sulaymaniyah records five daily high temperatures, in °C: 30, 32, 28, 35, and 30. Find the mean temperature.

1. Add the five temperatures: $30 + 32 + 28 + 35 + 30 = 155$.

2. Divide by the number of temperatures, 5: $155 \div 5 = 31$.

Answer: The mean temperature is 31°C. On average, across these five days, the temperature was 31°C, even though no single day was exactly 31°C.

The mean gives one single "typical" value, but it hides an important detail: how tightly the real values cluster around that typical value, or how widely they scatter away from it. Two groups of numbers can share the exact same mean while looking completely different up close. **Variance**, and its close relative standard deviation, measure exactly this spread.

This is the same word Chapter 5 used for a model's variance, the way a model's predictions can swing around when trained on different data. The word is the same, and the underlying math, an average of squared differences, is the same too. Only what is being measured differs: here, variance measures the spread of a plain list of numbers, not the spread of a model's own behavior.

EQUATION — VARIANCE AND STANDARD DEVIATION

$$\text{variance} = \frac{\sum (x - \text{mean})^2}{n}, \text{ and } \text{standard deviation} = \sqrt{\text{variance}}$$

In words: Variance is the average of each value's squared distance from the mean. Standard deviation is the square root of variance, which brings the result back to the data's original unit.

variance the average squared distance from the mean (same unit as the data, squared)

$x - \text{mean}$ one value's distance from the mean, before squaring (same unit as the data)

standard deviation the square root of the variance (same unit as the data)

n the number of values in the list (count)

Note: Squaring, exactly as in Chapter 6's least squares, stops positive and negative distances from cancelling out, and it punishes a value far from the mean more heavily than one close to it.

WORKED EXAMPLE — FIND THE VARIANCE AND STANDARD DEVIATION OF THE SAME FIVE TEMPERATURES

Using the same five temperatures, 30, 32, 28, 35, and 30°C, and their mean of 31°C found above, find the variance and the standard deviation.

1. Subtract the mean from each temperature: $30 - 31 = -1$. $32 - 31 = 1$. $28 - 31 = -3$. $35 - 31 = 4$. $30 - 31 = -1$.
2. Square each of these differences: $(-1)^2 = 1$. $1^2 = 1$. $(-3)^2 = 9$. $4^2 = 16$. $(-1)^2 = 1$.
3. Add the five squared differences: $1 + 1 + 9 + 16 + 1 = 28$.
4. Divide by the number of temperatures, 5, to find the variance: $28 \div 5 = 5.6$.
5. Take the square root of the variance to find the standard deviation: $\sqrt{5.6} \approx 2.37$.

Answer: The variance is $5.6 (\text{°C})^2$, and the standard deviation is about 2.37°C . A typical day's temperature in this small group differs from the mean, 31°C , by roughly 2.37°C , either above or below.

Standard deviation is the more useful number to picture, because it returns to the original unit, degrees Celsius, instead of degrees squared. Chapter 3's standardization step uses exactly this pair of numbers: it subtracts each feature's mean, then divides by that feature's standard deviation, so every feature ends up measured on a similar, comparable scale. Chapter 14's z-score reuses the same two ingredients again, this time to ask how many standard deviations away from the mean one single example sits.

A.3 The Dot Product

Chapter 6 built a prediction by multiplying each feature by its own coefficient, then adding every one of those products together, before adding one final intercept. That "multiply matching entries, then add" pattern is useful enough, in enough different corners of machine learning, that mathematics gives it its own short name and its own short piece of notation.

WORD TO KNOW

Dot product — A way to combine two vectors of the same length into a single number: multiply each pair of matching entries together, then add all of the products. Written with a small dot between the two vectors, such as $w \cdot x$, said "w dot x."

EQUATION — THE DOT PRODUCT OF TWO VECTORS

$$w \cdot x = w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

In words: The dot product multiplies each pair of matching entries, first entry with first entry, second with second, and so on, then adds every one of those products together into one single number.

$w \cdot x$ the dot product of vector w and vector x (depends on what w and x measure)

$w_1, w_2, \dots, w_n$ the entries of vector w , in order (depends on the vector)

$x_1, x_2, \dots, x_n$ the entries of vector x , in order (depends on the vector)

n the number of entries in each vector; both vectors must have the same number of entries (count)

Note: A dot product only makes sense between two vectors of the exact same length, one entry pairing with exactly one other entry.

WORKED EXAMPLE — REWRITE CHAPTER 6'S BILL PREDICTION AS A DOT PRODUCT

Chapter 6 predicted a home's bill using $\text{predicted bill} = 5 + 0.4 \times \text{size} + 9 \times \text{appliances}$, for a 120 m² home with 4 appliances, and found an answer of \$89. Write the coefficients as a vector $w = (0.4, 9)$ and the features as a vector $x = (120, 4)$. Find the dot product $w \cdot x$, then add the intercept, 5, and check the result against Chapter 6's answer.

1. Multiply the first matching pair: $0.4 \times 120 = 48$.
2. Multiply the second matching pair: $9 \times 4 = 36$.
3. Add the two products together: $48 + 36 = 84$. This is the dot product, $w \cdot x$.
4. Add the intercept, 5: $84 + 5 = 89$.

Answer: $w \cdot x = 84$, and adding the intercept gives 89, exactly matching Chapter 6's prediction of \$89 for this home. Every linear regression prediction is really a dot product between the coefficients and the features, plus the intercept, written out in full detail in Chapter 6 and now written compactly with one single operation.

WORKED EXAMPLE — FIND A SECOND DOT PRODUCT

Find the dot product of $w = (2, -1, 3)$ and $x = (4, 5, 2)$.

1. Multiply the first matching pair: $2 \times 4 = 8$.
2. Multiply the second matching pair: $-1 \times 5 = -5$.
3. Multiply the third matching pair: $3 \times 2 = 6$.
4. Add the three products together: $8 + (-5) + 6 = 9$.

Answer: $w \cdot x = 9$. Even with three entries instead of two, the recipe never changes: multiply each matching pair, then add every product together.

A.4 Distance and Norm

Chapter 1's nearest-centroid method and Chapter 8's k-Nearest Neighbors both had to answer the same question: how far apart are two points? Both chapters computed a squared distance and compared sizes, without ever taking a final square root, because only the order of the distances mattered, not their exact value. This section finally takes that last step and gives the general idea a name.

WORD TO KNOW

Norm — A single number that measures the length of a vector: the straight-line distance from the all-zero point to the point the vector describes. The most

common norm, used throughout this book, squares every entry, adds the squares, then takes the square root.

EQUATION — THE NORM OF A VECTOR

$$|v| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

In words: The norm of a vector squares every entry, adds all of the squares together, then takes the square root of that sum. The bars on each side are the same style Chapter 6 used for lasso's absolute value, $|w|$; both answer a similar question, how big is this, ignoring direction or sign.

v

$v_1, v_2, \dots, v_n$ the entries of vector v (depends on the vector)

n the number of entries in v (count)

Note: The distance between two points a and b is the norm of their difference: distance $= |a - b|$. This is exactly what Chapter 1's squared-distance steps were building toward, one square root away.

WORKED EXAMPLE — FIND THE NORM OF A VECTOR

Find the norm of the vector (3, 4).

1. Square each entry: $3^2 = 9$. $4^2 = 16$.
2. Add the squares: $9 + 16 = 25$.
3. Take the square root: $\sqrt{25} = 5$.

Answer: $|(3, 4)| = 5$. This vector's straight-line length, from (0, 0) to (3, 4), is exactly 5 units.

WORKED EXAMPLE — TURN CHAPTER 1'S SQUARED DISTANCES INTO TRUE DISTANCES

Chapter 1 classified melon Q at (5, 7) using nearest centroid, and found its squared distance to the Ripe centroid (5, 9) was 4, and its squared distance to the Unripe centroid (2, 3) was 25, skipping the square root because only the smaller value mattered. Find the true, straight-line distance from melon Q to each centroid.

1. Take the square root of the squared distance to the Ripe centroid: $\sqrt{4} = 2$.
2. Take the square root of the squared distance to the Unripe centroid: $\sqrt{25} = 5$.

Answer: Melon Q sits 2 units from the Ripe centroid and 5 units from the Unripe centroid. This matches Chapter 1's conclusion that melon Q is closer to Ripe, now expressed as an actual distance instead of a squared distance.

A.5 Probability Basics

Several chapters have already produced a number between 0 and 1 and called it a probability: Chapter 7's sigmoid function, Chapter 16's calibration check, and every confusion-matrix rate from Chapter 4. This section gives that number a proper, careful definition.

A **probability** is a number between 0 and 1 that measures how likely an event is. A probability of 0 means the event never happens. A probability of 1 means the event always happens. A probability of 0.5 means the event happens about half of the time. Probabilities are often written as percentages instead, simply by multiplying by 100, so a probability of 0.12 and 12% mean exactly the same thing.

One common, honest way to estimate a probability is to look at how often an event actually happened in past data.

EQUATION — ESTIMATING A PROBABILITY FROM PAST DATA

$$P(A) \approx \frac{\text{count of } A}{\text{total count}}$$

In words: Estimate the probability of an event by dividing how many times it happened by the total number of chances it had to happen.

P(A) the estimated probability of event A (probability, between 0 and 1)

count of A the number of past cases where event A happened (count)

total count the total number of past cases (count)

Note: This is an estimate built from past data, not a guaranteed law. Chapter 16's calibration lesson checks whether a model's own probabilities behave this same honest way.

WORKED EXAMPLE — ESTIMATE A PROBABILITY, AND ITS COMPLEMENT, FROM PAST DATA

Out of 200 past loan applicants, 24 failed to repay their loan. Estimate the probability that a new, similar applicant fails to repay, and the probability that a new, similar applicant repays successfully.

1. Identify the count of the event and the total: 24 applicants failed to repay, out of 200 applicants in total.
2. Divide to estimate the probability of failing to repay: $24 \div 200 = 0.12$.
3. Find the complement, the probability of repaying successfully, by subtracting from 1: $1 - 0.12 = 0.88$.
4. Check that the two probabilities add to 1: $0.12 + 0.88 = 1.00$.

Answer: The probability of failing to repay is about 0.12, or 12%, and the probability of repaying successfully is about 0.88, or 88%. Out of every 100 similar applicants, about 12 are expected not to repay, and about 88 are expected to repay.

Sometimes a probability is conditioned on already knowing something else. Conditional probability, written $P(A | B)$ and said "the probability of A given B," means the probability that A happens, once B is already known to have happened. Chapter 4's **precision** is secretly a conditional probability in disguise: precision asks, out of every case the model flagged positive, what fraction were truly positive — exactly $P(\text{truly positive} | \text{flagged positive})$.

WORKED EXAMPLE — READ PRECISION AS A CONDITIONAL PROBABILITY

A spam filter flags 50 emails as spam. Of those 50, 42 really are spam, and 8 are not. Find the precision, and state it as a conditional probability in words.

1. Identify true positives, flagged spam that really is spam: 42. Identify the total number flagged: 50.
2. Divide: $42 \div 50 = 0.84$.

Answer: Precision = 0.84, or 84%. In the language of this section, this is $P(\text{really spam} \mid \text{flagged as spam}) = 0.84$: given that an email was flagged, there is an 84% chance it truly is spam.

A model's predicted probability is always an estimate built from patterns in past data. It is never a certainty about one single future case — the same honest limit that applies to every prediction in this book.

A.6 Derivatives and Gradients

Chapter 6 described gradient descent as walking downhill in the dark, feeling the ground's slope with each step, without ever giving that slope its own precise name. This section gives it one, in plain, careful language: how steep is the hill exactly where you are standing, and which direction is downhill.

WORD TO KNOW

Derivative — A number that says how much one output changes for a tiny change in one input, measured at one exact point. Another name for a derivative is the slope, or the steepness, of a curve at that exact point.

EQUATION — A DERIVATIVE, ESTIMATED FROM TWO POINTS

$$\text{derivative} \approx \frac{\text{change in output}}{\text{change in input}}$$

In words: A derivative estimates how much an output changes for a small change in the input, by dividing how much the output changed by how much the input changed.

change in output the difference between the output at the new input and the output at the original input (same unit as the output)

change in input the difference between the new input and the original input (same unit as the input)

Note: For a perfectly straight line, this ratio is exactly the same no matter where on the line it is measured, and that constant ratio is exactly the line's coefficient, w , from Chapter 6.

WORKED EXAMPLE — FIND THE DERIVATIVE OF A STRAIGHT LINE

Chapter 6 found that predicted cost = $2 \times \text{air conditioners} + 1$ fits its tiny 3-home dataset perfectly. Find the derivative of this line with respect to "air conditioners": how much predicted cost rises for one extra air conditioner.

1. Find the predicted cost at 1 air conditioner: $2 \times 1 + 1 = 3$.
2. Find the predicted cost at 2 air conditioners: $2 \times 2 + 1 = 5$.
3. Find the change in output: $5 - 3 = 2$.

4. Find the change in input: $2 - 1 = 1$.
5. Divide the change in output by the change in input: $2 \div 1 = 2$.

Answer: The derivative is 2. For every one extra air conditioner, predicted cost rises by exactly \$2, and because this line is perfectly straight, this rate of change is exactly the same no matter where on the line it is checked — it exactly matches the coefficient, $w = 2$, that Chapter 6's gradient descent was searching for.

Most loss functions are not straight lines. Chapter 6's loss function curves, like a real valley, so its steepness is different at every point: steep on the outer walls, and nearly flat at the very bottom. This is exactly why we always speak of the derivative AT one particular point, never one single steepness for a whole curve. ML Studio's Gradient Descent Lab, from Chapter 6's activity, lets you watch this changing steepness directly: press Step near the top of the loss bowl and the line jumps a long way; press Step near the bottom and it barely moves, because the ground has gone nearly flat.

WORD TO KNOW

Gradient — A short list of derivatives, one for each adjustable number in a model, such as each coefficient and the intercept. Each entry says how the loss would change if that one number were nudged, while every other number stayed fixed.

Gradient descent takes its name from exactly this idea. At every step, it finds the gradient, which always points toward higher loss, uphill, then steps a small distance in the opposite direction, downhill, because downhill is where a smaller loss lives. "Descent" simply means moving downward.

WORKED EXAMPLE — READ CHAPTER 6'S FIRST ROUND AS A GRADIENT

Chapter 6 trained a tiny model on 3 homes (1, 2, and 3 air conditioners, with daily costs of \$3, \$5, and \$7), starting from $w = 0$ and $b = 0$. Its first round found residuals of 3, 5, and 7, then computed $\Sigma(\text{residual}) = 15$ and $\Sigma(x \times \text{residual}) = 34$. State the two numbers that describe the gradient's direction at this starting point, and say which way gradient descent moved w and b .

1. The number attached to the intercept, b , comes from $\Sigma(\text{residual}) = 15$.
2. The number attached to the coefficient, w , comes from $\Sigma(x \times \text{residual}) = 34$.
3. Both 15 and 34 are positive, so the loss decreases by increasing both w and b from their starting values of 0.
4. Gradient descent therefore added a positive amount to both w and b , exactly as Chapter 6 found: w moved from 0 to 1.36, and b moved from 0 to 0.6.

Answer: The gradient's two directions, one for b and one for w , were both positive at the starting point, so gradient descent increased both numbers. This is the same first round Chapter 6 already worked out by hand; this appendix simply gives its two key sums, 15 and 34, their proper name: the two entries of the loss function's gradient at that point.

These two sums come directly from the mathematics of the gradient of the least-squares loss. A full calculus derivation multiplies each one by 2, but that constant 2 simply gets absorbed into the learning rate, so Chapter 6's simpler recipe and the true gradient always point in the exact same direction, uphill, before gradient descent flips the sign to step downhill instead.

A.7 Logarithms

Chapter 7's log loss used a function called the natural logarithm without fully explaining what a logarithm actually computes. This closing section fills that gap, using whole numbers you can check by hand, then reconnects to Chapter 7's own values.

WORD TO KNOW

Logarithm — The power you must raise a fixed base number to, in order to reach a chosen result. A logarithm reverses an exponent, the same way subtraction reverses addition, or division reverses multiplication.

EQUATION — BASE-10 LOGARITHMS OF SMALL POWERS OF TEN

$$\begin{array}{l} 10^2 = 100 \Rightarrow \log(100) = 2, \quad 10^3 = 1000 \Rightarrow \\ \log(1000) = 3 \end{array}$$

In words: The base-10 logarithm of a number answers one question: 10 raised to what power gives this number? One hundred needs a power of 2, and one thousand needs a power of 3, so those are their logarithms.

log(100) ten raised to what power gives one hundred (count)

log(1000) ten raised to what power gives one thousand (count)

Note: Not every number is a clean power of ten. A calculator finds the logarithm of any positive number the same way, such as $\log(250) \approx 2.398$, a value between 2 and 3 because 250 sits between 100 and 1000.

WORKED EXAMPLE — FIND LOG(1000) BY REVERSING A POWER OF TEN

Find $\log(1000)$, the base-10 logarithm of 1000.

1. Ask the reverse-exponent question: 10 raised to what power gives 1000?
2. Try small whole-number powers of 10 one at a time: 10 raised to the power of 1 is just 10 itself. $10^2 = 100$.
3. Try the next power: $10^3 = 1000$. This one matches.

Answer: $\log(1000) = 3$. Ten multiplied by itself three times gives exactly 1000, so the base-10 logarithm of 1000 is 3.

WORKED EXAMPLE — SHOW THAT LOGARITHMS TURN MULTIPLICATION INTO ADDITION

Using $\log(100) = 2$ and $\log(10) = 1$, find $\log(100 \times 10)$ and compare it with $\log(100) + \log(10)$.

1. Multiply first: $100 \times 10 = 1000$.
2. Find $\log(1000)$ by asking what power of 10 gives 1000: $10^3 = 1000$, so $\log(1000) = 3$.

3. Separately, add the two known logarithms: $\log(100) + \log(10) = 2 + 1 = 3$.

Answer: Both routes give exactly 3: $\log(100 \times 10) = \log(100) + \log(10) = 3$. Logarithms turn a multiplication problem into an easier addition problem, one reason they appear so often in probability formulas, including Chapter 7's log loss, which adds log-probabilities together instead of multiplying plain probabilities.

Chapter 7's log loss uses a close cousin of this idea, the natural logarithm, written $\ln$, which asks the same reversing question using a different base number, $e \approx 2.71828$, instead of 10. Chapter 7 never asked you to compute a natural logarithm by hand, and this appendix will not either; a calculator, or a line of Python code, finds $\ln$ of any positive number directly, exactly the way Chapter 7's worked examples already did.

One more property of logarithms explains why log loss can grow so large. As a number gets closer and closer to 0, its logarithm grows more and more negative, without any limit, because there is no power you can raise 10 (or e) to that gives exactly 0. This is exactly why Chapter 7's log loss can grow extremely large when a confident prediction turns out to be completely wrong: the predicted probability for the true answer sits close to 0, its logarithm plunges toward a very large negative number, and the minus sign in the log loss formula flips that plunge into a very large penalty instead.

CHAPTER SUMMARY

- A scalar is one number; a vector is an ordered list of numbers, usually one example's features; a matrix is a rectangular table of numbers, with a shape written as (rows, columns).
- The mean averages a list of numbers; variance averages each value's squared distance from the mean; standard deviation is variance's square root, back in the data's original unit.
- A dot product multiplies two vectors' matching entries and adds the products; a linear regression prediction is a dot product between coefficients and features, plus an intercept.
- A norm measures a vector's straight-line length; the distance between two points is the norm of their difference, the same idea Chapters 1 and 8 already used, one square root away.
- A probability is a number between 0 and 1 that estimates how likely an event is; it can be estimated from past data, and Chapter 4's precision is a conditional probability in disguise.
- A derivative measures the slope of a curve at one exact point; a straight line's derivative is simply its coefficient, the same value everywhere on the line.
- A gradient collects one derivative per adjustable number in a model; gradient descent steps opposite the gradient's direction, downhill, toward a smaller loss.

- A logarithm reverses an exponent; base-10 logarithms of round numbers are easy to check by hand, and Chapter 7's log loss uses the natural logarithm, the same idea with a different base.
- None of the mathematics in this appendix is new. Every idea already did real work in an earlier chapter; this appendix only slowed each one down and gave it a proper name.

REFERENCE

APPENDIX B

Python, NumPy, and scikit-learn Field Guide

A pocket reference for the code patterns used throughout this book

This appendix collects the short pieces of Python and NumPy code used across the book into one field guide. Like Appendix A, you do not need to read it start to finish. Return to one short section whenever a chapter's code moves faster than you would like, and work through its own small example, line by line, at your own pace. Every section here assumes no earlier programming experience at all. Each new piece of vocabulary is explained in plain words before any code appears, and every code block repeats numbers you can already check by hand.

B.1 Variables, Lists, and Arrays

Before any machine learning code makes sense, a few basic building blocks of the Python programming language need names. A variable is simply a labeled box that stores one value, so that value can be used again later by name, instead of being retyped. Python creates a variable with a single equals sign: `age = 21` stores the number 21 under the name `age`. A value can be a whole number, a decimal number, or a short piece of text inside quotation marks, called a string.

A list is Python's basic way to store several values together, in order, inside square brackets, separated by commas. `values = [2, 4, 6]` stores three numbers together under one name, `values`. A list can hold numbers, text, or even other lists, and every small dataset in this book started life as exactly this kind of plain list, before NumPy turned it into something faster and more powerful.

WORD TO KNOW

NumPy array — A table of numbers, built by the NumPy library, that is faster than a plain Python list at exactly the arithmetic machine learning needs most: adding, multiplying, and averaging many numbers at once. Every feature matrix and label list in this book, once inside code, becomes a NumPy array.

NumPy is short for "Numerical Python." Nearly every code example in this book starts with the same line, `import numpy as np`, which loads NumPy's tools and gives them the short nickname `np`, so later lines can call them quickly.

WORKED EXAMPLE — FIND THE MEAN OF A SMALL LIST BY HAND

A list stores three numbers: 2, 4, and 6. Find the mean of this list by hand.

1. Add the three numbers: $2 + 4 + 6 = 12$.
2. Divide by the count of numbers, 3: $12 \div 3 = 4$.

Answer: The mean is 4. The code block below stores this exact same list and asks NumPy to compute its mean automatically, so you can check that the computer agrees with your own pencil-and-paper answer.

Python · NumPy

 runnable in the app

```
import numpy as np

values = [2, 4, 6]
x = np.array(values)

print(x.mean())
print(x.shape)
```

The first line loads NumPy under the nickname np. The second line stores the plain Python list [2, 4, 6] under the name values. The third line hands that list to np.array(...), which turns it into a NumPy array, stored under the name x. The word print simply displays a value on the screen, so you can see a result while a program runs. Running x.mean() prints 4.0, matching the hand calculation above, and x.shape prints (3,), meaning this array holds 3 entries arranged in a single row, exactly the vector shape Appendix A.1 described.

Plain Python lists and NumPy arrays behave differently in one important way, and this difference surprises many beginners. Multiplying a plain list by 2 repeats the whole list twice. Multiplying a NumPy array by 2 instead multiplies every single entry by 2.

Python · NumPy

 runnable in the app

```
plain_list = [2, 4, 6]
print(plain_list * 2)

import numpy as np
array_version = np.array(plain_list)
print(array_version * 2)
```

print(plain_list * 2) prints [2, 4, 6, 2, 4, 6], the plain list repeated twice, one copy after the other. print(array_version * 2) prints [4 8 12] instead, because NumPy multiplies every entry by 2. This one difference is the main reason NumPy exists: it lets you write short, simple arithmetic that applies to an entire dataset at once, instead of one number at a time.

One more pattern is worth knowing before this appendix moves on: the for loop. A for loop repeats the same block of code once for each item in a list, or once

for each number in a range of numbers. Chapter 6's gradient descent code used exactly this pattern to repeat the same update steps for several rounds in a row.

Python · NumPy

 runnable in the app

```
w, b = 0, 0

for round_number in range(1, 4):
    # the real update happens here, using that round's residuals
    print("Round", round_number, "-> w:", w, " b:", b)
```

`range(1, 4)` produces the whole numbers 1, 2, and 3, stopping just before 4. The indented block underneath for repeats once for each of those three numbers, with `round_number` taking each value in turn. A line starting with the symbol `#` is a comment: Python ignores it completely, and it exists only to leave a short note for a human reader. Running this loop prints Round 1, Round 2, and Round 3, one line at a time, exactly the pattern Chapter 6 used to print the coefficient and intercept after each round of gradient descent.

B.2 Rows, Columns, and Shapes

Appendix A.1 arranged four melons into a matrix with shape (4, 2): four rows, one per melon, and two columns, one per feature. In code, this exact table becomes a two-dimensional NumPy array, and its shape tells you the same two numbers instantly, without counting by hand.

Following the convention Chapter 6's code already used, this book stores a whole feature table under the capital letter `X`, and its matching column of labels or targets under the lowercase letter `y`. `X.shape` returns (number of rows, number of columns). `y.shape` usually returns just one number, because `y` is a single column, not a full table.

WORKED EXAMPLE — PREDICT THE SHAPE OF A SMALL FEATURE MATRIX AND LABEL ARRAY

A dataset has 6 examples and 3 features per example. Its features are stored in `X` and its labels are stored in `y`. State the shape of `X` and the shape of `y`.

1. `X` has one row per example, so `X` has 6 rows.
2. `X` has one column per feature, so `X` has 3 columns.
3. Write `X`'s shape as (rows, columns): (6, 3).
4. `y` stores one label per example and nothing else, so `y` is a single list of length 6, written as (6,).

Answer: `X.shape` is (6, 3), and `y.shape` is (6,). Notice `y`'s shape has only one number, with a trailing comma and no second number, because `y` is a single column of labels, not a full table.

Python · NumPy

 runnable in the app

```
import numpy as np

X = np.array([[4, 8], [6, 10], [2, 2], [2, 4]])
y = np.array(["ripe", "ripe", "unripe", "unripe"])

print(X.shape)
print(y.shape)
print(X[2, 1])
print(X[:, 0])
```

X stores the same four melons Appendix A.1 used, one row per melon and one column per feature, in the same order: weight, then sound score. `print(X.shape)` prints (4, 2), and `print(y.shape)` prints (4,), confirming X is a full table while y is a single column. Python and NumPy count positions starting from 0, not from 1, so `X[2, 1]` means row index 2, column index 1: the third row and the second column, counting from the top and the left. This prints 2, the sound score of the first unripe melon, the same entry Appendix A.1 found by hand using the math convention of counting from 1 instead. The last line, `X[:, 0]`, uses a colon to mean "every row," so it pulls out the whole first column, weight, for all four melons at once: [4 6 2 2].

A shape mismatch is one of the most common early bugs in any machine learning script. If X has 500 rows but y has only 480, scikit-learn refuses to fit a model and raises an error, which is actually a small act of kindness: it is far better to see a clear error immediately than to silently train a model on rows that do not really match their labels. Checking `print(X.shape)` and `print(y.shape)` right after loading a dataset, and confirming the row counts agree, takes one second and can prevent a confusing afternoon later.

WORKED EXAMPLE — RESHAPE A FLAT LIST OF SIX NUMBERS INTO A MATRIX

A flat list holds six numbers, in order: 1, 2, 3, 4, 5, 6. Reshape them into a matrix with 2 rows and 3 columns, filling row by row. State the resulting matrix and its shape.

1. Fill row 1 with the first three numbers, in order: 1, 2, 3.
2. Fill row 2 with the next three numbers, in order: 4, 5, 6.
3. Count the rows, 2, and the columns, 3, to write the shape: (2, 3).

Answer: The matrix is `[[1, 2, 3], [4, 5, 6]]`, with shape (2, 3). Reshaping this way only works when the total count of numbers, 6, exactly matches rows × columns, $2 \times 3 = 6$.

Python · NumPy

 runnable in the app

```
import numpy as np

flat = np.arange(1, 7)
grid = flat.reshape(2, 3)

print(grid)
print(grid.shape)
```

`np.arange(1, 7)` creates the numbers 1 through 6, stopping just before 7. `.reshape(2, 3)` arranges them into 2 rows and 3 columns, filling row by row, exactly like the hand calculation above. `print(grid)` shows `[[1 2 3] [4 5 6]]`, and `print(grid.shape)` confirms `(2, 3)`. Reshaping only works when rows times columns exactly equals the original count of numbers; trying `.reshape(2, 4)` on these same six numbers would raise an error, because $2 \times 4 = 8$ does not equal 6.

B.3 The Common scikit-learn Pattern

Nearly every scikit-learn tool used across this book, whether it is a preprocessing step from Chapter 3 or a model from Chapters 6 through 11, follows the exact same small set of methods. Learning this one pattern once means every new tool feels immediately familiar, even on the first day you meet it.

EQUATION — THE CORE SCIKIT-LEARN METHODS

first fit, then transform or predict

In words: Almost every scikit-learn tool studies the training data with one method, fit, and then does its real work afterward with a second method, such as transform or predict.

fit(X, y) learn from training data; a model needs both X and y, a preprocessing step usually needs only X (*returns the fitted tool itself*)

transform(X) apply an already-learned preprocessing step to new data (*a changed version of X*)

fit_transform(X) fit a preprocessing step and transform the same data, in one call (*a changed version of X*)

predict(X) return a predicted class or number for each row of X (*one prediction per row*)

predict_proba(X) return the predicted probability of each class, for models that support it (*one set of probabilities per row*)

score(X, y) return one quick, default performance number (*a single number*)

Note: Always call fit only on the training set, never on the validation or test set. Chapter 3's pipeline lesson, and Appendix B.5's checklist, explain exactly why this order matters.

fit is always the learning step: it studies the training data and stores whatever the tool needs to remember, a mean and a standard deviation for a scaler, or a full set of coefficients for a linear regression model. transform and predict are always the using step, applied afterward, often to brand-new data the tool has never seen before. A preprocessing tool, such as Chapter 3's standardization, uses transform, because its job is to change the data's scale. A model, such as Chapter 6's linear regression, uses predict, because its job is to output an answer.

predict_proba works only for models that can output a probability, such as Chapter 7's logistic regression. Asking a plain linear regression model for predict_proba raises an error, because a regression model was never trained to produce probabilities in the first place. score(X, y) is a fast, one-line sanity check, but Chapter 4 already showed that a single default score, such as plain accuracy, can hide an important problem. A real project almost always reports one or two metrics chosen deliberately, the way Chapter 4 taught, rather than relying on score alone.

WORKED EXAMPLE — PREDICT MELON Q'S CLASS WITH 1-NEAREST-NEIGHBOR, BY HAND

Using the four melons from Chapter 1 — ripe melons at (4, 8) and (6, 10); unripe melons at (2, 2) and (2, 4) — and melon Q at (5, 7), find melon Q's single nearest neighbor ($k = 1$) with Chapter 8's k-Nearest Neighbors method, by comparing squared distances to all four melons. State the predicted class.

1. Squared distance to ripe melon (4, 8): $(5 - 4)^2 + (7 - 8)^2 = 1^2 + (-1)^2 = 1 + 1 = 2$.
2. Squared distance to ripe melon (6, 10): $(5 - 6)^2 + (7 - 10)^2 = (-1)^2 + (-3)^2 = 1 + 9 = 10$.
3. Squared distance to unripe melon (2, 2): $(5 - 2)^2 + (7 - 2)^2 = 3^2 + 5^2 = 9 + 25 = 34$.
4. Squared distance to unripe melon (2, 4): $(5 - 2)^2 + (7 - 4)^2 = 3^2 + 3^2 = 9 + 9 = 18$.
5. Compare all four squared distances: 2, 10, 34, and 18. The smallest is 2, to the ripe melon at (4, 8).

Answer: Melon Q's single nearest neighbor is the ripe melon at (4, 8), so a 1-Nearest-Neighbor model predicts Ripe. This matches Chapter 1's nearest-centroid answer for the same melon, even though the two methods work differently: nearest-centroid compares against two averaged melons, while 1-Nearest-Neighbor compares against all four real melons individually.

Python · NumPy

 runnable in the app

```

from sklearn.neighbors import KNeighborsClassifier

X = [[4, 8], [6, 10], [2, 2], [2, 4]]
y = ["ripe", "ripe", "unripe", "unripe"]

model = KNeighborsClassifier(n_neighbors=1)
model.fit(X, y)
print(model.predict([[5, 7]]))

```

`KNeighborsClassifier(n_neighbors=1)` creates a 1-Nearest-Neighbor model, matching $k = 1$ from the hand calculation above. `model.fit(X, y)` studies the four labeled melons and simply remembers them; this kind of model does not need to calculate anything else in advance. `model.predict([[5, 7]])` then asks for a prediction on melon Q; notice the double square brackets, because `predict` always expects a full table of examples, even a table with only one row. Running this prints `['ripe']`, matching the hand calculation exactly.

Python · NumPy

 runnable in the app

```

from sklearn.preprocessing import StandardScaler
import numpy as np

sizes = np.array([[50], [70], [90], [110], [130]])
scaler = StandardScaler()
scaled = scaler.fit_transform(sizes)
print(scaled.round(2))

```

`sizes` stores Chapter 6's five home sizes as a column, shape (5, 1): five rows, one column. `scaler.fit_transform(sizes)` does two jobs in one call: `fit` studies this column and remembers its mean, 90, and its standard deviation, about 28.28, using exactly Appendix A.2's formulas; `transform` then subtracts the mean and divides by the standard deviation for every value. Running this prints roughly `-1.41, -0.71, 0, 0.71, and 1.41`: five numbers centered on 0, with the original middle value, 90, landing exactly on 0, because 90 is the mean of the five sizes.

This same small set of methods extends naturally to more advanced tools later in the book. Chapter 17's grid search, for example, still uses `fit` and `predict`, exactly like any other model; it simply tries many settings internally during `fit`, then keeps the best one automatically. Learning the `fit/predict` pattern once, here in this appendix, means a more advanced tool met later never feels like a brand new language, only a familiar pattern wearing a new name.

B.4 Reproducibility

Several methods in this book rely on randomness on purpose. Chapter 2's train/test split shuffles rows before dividing them. Chapter 10's bootstrap sampling draws random rows with replacement. Chapter 12's k-means starts from randomly placed centroids. Randomness is not a flaw in any of these methods; it is often exactly what makes them work well. But randomness also creates a real problem: running the exact same code twice can give two different results, which makes debugging, and fairly comparing two models, much harder than it needs to be.

WORD TO KNOW

Random seed — A starting number, chosen once, that makes a program's "random" choices happen in the exact same order every time the code runs. Using the same random seed twice always produces the exact same result; a different seed produces a different, but equally valid, result.

scikit-learn's tools almost all accept a setting called `random_state` for exactly this purpose. Setting `random_state` to any fixed whole number, such as 42, 0, or 7, fixes every random choice that tool makes, for as long as that same number keeps being used.

Python · NumPy

 runnable in the app

```
from sklearn.model_selection import train_test_split
import numpy as np

X = np.arange(10).reshape(10, 1)
y = np.arange(10)

train_a, test_a = train_test_split(X, test_size=0.3, random_state=42)
train_b, test_b = train_test_split(X, test_size=0.3, random_state=42)
print(np.array_equal(train_a, train_b))
```

`np.arange(10)` creates the whole numbers 0 through 9, and `.reshape(10, 1)` arranges them into a column, a matrix with shape (10, 1), matching Appendix A.1's shape idea. `train_test_split` randomly divides these 10 rows into a 70% training group and a 30% test group, using `test_size=0.3`. Because both calls use the exact same `random_state`, 42, they split the rows the exact same way both times. `print(np.array_equal(train_a, train_b))` prints True, confirming the two training groups are identical, row for row.

Without a fixed seed, running the exact same grid search from Chapter 17 twice could occasionally point to two different "best" settings, not because either choice was wrong, but because two different random train/validation splits happened to favor slightly different settings. Fixing the seed removes this one source of confusion, so any difference you see later must come from an actual change you made, not from luck.

NumPy also has its own, separate way to fix randomness, `np.random.seed(42)`, which controls any randomness that comes directly from NumPy itself, such as shuffling a plain array. scikit-learn's `random_state` is the setting used far more often in this book's code, because nearly every scikit-learn tool accepts it directly as one of its own settings, without needing a separate line beforehand. Many programmers pick 42 out of habit rather than any special mathematical meaning; any fixed whole number works exactly as well, as long as it stays the same across the runs you want to compare.

A random seed is a tool for exact repeatability, not a promise that a result is trustworthy. Two people can each fix a seed, get a perfectly reproducible number, and still be looking at a lucky or unlucky split. Chapter 4's cross-validation, which deliberately reruns training across several different splits and averages the results, remains the better way to check whether a score is genuinely stable, not merely repeatable. A careful project record also writes down the package versions used, the version of the dataset, the exact split rule, and the list of features, so the whole project, not just one random seed, can be reproduced later.

B.5 A Safe Checklist for Every Code Example

Every code block in this book follows the same handful of safety habits, gathered from across every chapter you have already read. None of these steps takes long, but skipping even one of them, especially fitting a preprocessing step before splitting the data, is exactly how a project ends up reporting a score that looks excellent on a laptop and then quietly disappoints once real, new data arrives, the same honest warning Chapter 19 gives about a model's health after it goes live.

Before trusting any code example in this book, or any code you write yourself, run through one short checklist. Confirm the unit of one row and the true target, exactly as Chapter 2 recommends, so every column's meaning is clear before any code runs. Check the shape of `X` and `y` first, the way Appendix B.2 showed, since a silent mismatch between the number of rows in each is one of the most common early bugs. Split the data into training and test sets, or training, validation, and test sets, before fitting any preprocessing step, never after, so that no information from the test set quietly leaks into training, exactly the **data leakage** trap Chapter 2 warned about. Fit preprocessing and the model together inside one pipeline, the habit Chapter 3 built, so the same safe order runs automatically every single time. Compare every new model against a simple baseline, the way Chapter 2 insisted, so a fancy model's score always means something. Use the validation set for every choice along the way, and open the test set only once, at the very end, exactly as Chapters 2 and 4 taught. Report the final metric in plain, understandable units, a dollar amount or a percentage, not just a bare, unexplained number. Finally, save the complete fitted pipeline, not only the bare model, and write down its known limitations, so that anyone using it later, including a future version of yourself, knows exactly what it was trained to do, and where it might struggle.

Python · NumPy

 runnable in the app

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", Ridge(alpha=0.2)),
])
pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)

```

Pipeline([...]) chains two steps together under one single name, pipe: a scaling step named "scale," and a ridge regression model named "model," reusing Chapter 6's own ridge settings. pipe.fit(X_train, y_train) runs both steps in the safe order automatically: it fits the scaler on X_train only, then fits the model on the scaled result. pipe.predict(X_test) then scales X_test using that exact same mean and standard deviation already learned from X_train, before asking the model for a prediction. Wrapping a project in a pipeline this way makes it very hard to accidentally fit a preprocessing step on data it should never have seen, exactly the safety this checklist is built around.

WORKED EXAMPLE — APPLY THE SAFE-CODE CHECKLIST TO A NEW PROJECT

A team wants to predict a bus's arrival delay, in minutes, from its route length, using 500 past trips. They plan a 60% training, 20% validation, and 20% test split. Walk through the safe checklist for this project, including the size of each split.

1. Name the unit and the target: one row is one past bus trip, and the target is arrival delay, in minutes.
2. Find the training set size: 60% of 500 is $0.6 \times 500 = 300$ trips.
3. Find the validation set size: 20% of 500 is $0.2 \times 500 = 100$ trips.
4. Find the test set size: 20% of 500 is $0.2 \times 500 = 100$ trips.
5. Check the three sizes add back to the total: $300 + 100 + 100 = 500$, which matches.
6. Confirm the safe order: split first into these three groups, then fit any preprocessing step and the model only on the 300 training trips, inside one pipeline, and compare against a baseline such as always predicting the mean delay.

Answer: The project needs 300 training trips, 100 validation trips, and 100 test trips, and every preprocessing step must be fit only on the 300 training trips, never on the validation or test trips. Working through the checklist and the split sizes together, before writing a single line of

code, is exactly the habit this whole book has been building, chapter after chapter.

Notice that this checklist never once mentions which model to choose. That is intentional, and it echoes Chapter 1's very first lesson: a safe order of operations protects a project no matter which model eventually gets used inside it. A k-Nearest Neighbors model, a random forest, and a small neural network all fit inside this exact same checklist, unchanged.

CHAPTER SUMMARY

- A variable stores one value under a name; a list stores several values in order; a NumPy array is a fast, table-shaped upgrade of a list, built for the arithmetic machine learning needs most.
- Multiplying a plain Python list repeats its contents; multiplying a NumPy array multiplies every entry, one reason this book converts every list of numbers into an array before doing real arithmetic.
- A feature table is stored as a capital X, with shape (rows, columns); a label column is stored as a lowercase y, with shape (rows,); check both shapes before fitting anything.
- Python and NumPy count positions starting from 0, not from 1, so a matrix's third row and second column, in ordinary counting, is written `X[2, 1]` in code.
- Almost every scikit-learn tool follows the same small set of methods: `fit` learns from training data, `transform` or `predict` then apply what was learned, and `predict_proba` returns probabilities when a model supports them.
- A random seed, set with scikit-learn's `random_state` or NumPy's `np.random.seed`, makes a program's random choices exactly repeatable, which matters for debugging and for fair comparisons between models.
- A repeatable result is not automatically a stable one; cross-validation, not a fixed seed alone, is the honest way to check whether a score would hold up across different data.
- A Pipeline chains preprocessing and a model together under one name, so `fit` and `predict` always run in the same safe order, without relying on memory alone to get that order right.
- Before trusting any code example, confirm the unit and the target, check the shapes of X and y, split before any preprocessing step, use a pipeline, and compare against a baseline.
- Every code example in this book repeats numbers already checked by hand first; running the code is a way to confirm a hand calculation, never a replacement for understanding it.

REFERENCE

Glossary

Every important term used in this book, in one place

This glossary gives a concise definition for every important term used in this book, so you can look up a word quickly any time you forget it.

Accuracy — The fraction of predictions a model gets exactly right, usually written as a percentage. Accuracy is one common performance measure, though later chapters show it is not always the right one.

Activation function — A function applied to a neuron's combined score, z , that bends the result instead of leaving it a plain straight-line calculation. Without an activation function, stacking layers adds no power at all.

Algorithm — A clear, exact set of steps that solves a problem or finishes a task. A learning algorithm is the exact set of steps a computer follows to search for a useful pattern inside data.

Anomaly — An example that looks noticeably different from most of the other examples in a dataset. An anomaly might point to a real problem, such as a broken sensor or a fraudulent charge, or it might simply be a rare, honest example. Finding an anomaly is only the first step; deciding why it is unusual takes a closer, human look.

Anomaly score — A number that measures how unusual one example looks, compared with normal data. A higher anomaly score means an example sits farther from typical behavior. This chapter builds anomaly scores from distance, though other methods, briefly named later in this chapter, build them in other ways.

Backpropagation — A method that works backward through a network, from the final loss toward the first layer, calculating exactly how much each coefficient contributed to the current mistake. Backpropagation does not change any coefficient itself; it only calculates the gradient that **gradient descent** then uses to make the actual update.

Bagging — Short for "bootstrap aggregating": training many copies of a model, each on its own bootstrap sample, then combining every copy's prediction by voting or averaging. Bagging mainly reduces variance, because the different mistakes made by differently-trained models tend to cancel out when combined.

Baseline — A simple method used as a minimum comparison for a new model. A useful model should outperform the baseline on the chosen performance measure.

Bias — A model's tendency to make the same kind of mistake, in the same direction, again and again, because its assumptions are too simple for the real pattern. High bias shows up as underfitting.

Bias-variance trade-off — The pattern where lowering a model's bias, usually by giving it more capacity, tends to raise its variance, and lowering its variance tends to raise its bias. Because of this trade-off, the best model is usually not the simplest one and not the most flexible one, but somewhere in between.

Boosting — Building an ensemble by training models one at a time, in sequence, where each new model focuses on correcting the mistakes the current ensemble is still making. The models' predictions are then added together, usually after scaling each new model's contribution down with a **learning rate**.

Bootstrap sample — A new dataset built by drawing rows from an original dataset at random, with replacement, until the new dataset is the same size as the original. Because rows can be drawn more than once, a bootstrap sample usually repeats some rows and leaves other rows out completely.

Calibration — How well a model's predicted probabilities match real-world frequencies. A model is well calibrated if, among all the times it predicts a 70% chance of something, that thing genuinely happens about 70% of the time.

Centroid — The average position of a group of points. To find a centroid, average each feature separately across every point in the group.

Churn — A customer ending their relationship with a business, such as canceling a subscription, closing an account, or simply not renewing. In a churn project, the label usually marks whether a customer churns within a fixed future window, such as the next 30 days.

Class imbalance — A situation where one class in a classification dataset has far more examples than another class. The common class is called the majority class, and the rare class is called the minority class.

Class weight — A number attached to a class before training that multiplies how much a mistake on that class costs inside the training algorithm's loss. A high weight on the minority class pushes the algorithm to work harder to avoid missing it.

Classification — A learning task where the model predicts a category, chosen from a fixed, small list of possible answers. "Spam or not spam" and "sweet or not sweet" are both classification tasks.

Clustering — A learning task where the model groups similar examples together, even though nobody gave the groups a name in advance. The computer finds the groups; a person usually studies each group afterward and decides what it seems to represent.

Coefficient — The number a feature is multiplied by inside a linear regression equation. A coefficient of 0.7 means the prediction rises by 0.7 for every extra unit of that feature. A coefficient is sometimes called a weight, because it controls how heavily that feature counts toward the prediction.

Confusion matrix — A small table that sorts every prediction a classifier makes into one of four groups, by comparing what the model predicted

against what was actually true. The four groups are usually shortened to four letters. table: TP (true positive) | the true class was positive | the model predicted positive too, so the prediction was correct FN (false negative) | the true class was positive | the model predicted negative, so the prediction was wrong — a miss FP (false positive) | the true class was negative | the model predicted positive, so the prediction was wrong — a false alarm TN (true negative) | the true class was negative | the model predicted negative too, so the prediction was correct.

Cross-validation — A method that repeats training and validation across several different splits of the training data, then combines the resulting scores into one more reliable estimate. The test set stays completely untouched during every round.

Data drift — A change, over time, in the data a deployed model sees, compared with the data it was trained on. Drift can show up in the input features themselves, or in how those features relate to the true answer, and either kind can make old predictions less trustworthy.

Data leakage — Information entering training that would not actually be available at the real moment of prediction. Leakage makes a model's test score look better than the model will truly perform in real use.

Dataset — An organized collection of examples used for training or testing a model. In a table, one row is usually one example, and one column is usually one measured property.

Decision boundary — The line, or surface, that separates the region where a classifier predicts one category from the region where it predicts another. For logistic regression, the decision boundary sits exactly where the predicted probability equals 0.5.

Decision tree — A model that predicts a label by asking a sequence of simple yes-or-no questions about an example's features, one question at a time, until it reaches a final answer. Each question narrows down the possibilities, exactly like a technician narrowing down a diagnosis.

Deep learning — Using a neural network with many hidden layers stacked one after another, so the network builds increasingly complex learned features on its own, layer by layer. Deep learning powers most modern image and language systems, but training it well needs far more data, computing power, and specialized technique than a short introduction can responsibly teach.

Deployment — Making a trained model available so it produces real predictions that people or other systems actually use, instead of sitting inside a notebook or script that only its author runs by hand.

Derivative — A number that says how much one output changes for a tiny change in one input, measured at one exact point. Another name for a derivative is the slope, or the steepness, of a curve at that exact point.

Dimensionality reduction — Reducing the number of features used to describe each example, while trying to keep as much of the data's real, useful

information as possible. Each original feature is often called a dimension, so cutting their number down is called reducing the dimensionality.

Dot product — A way to combine two vectors of the same length into a single number: multiply each pair of matching entries together, then add all of the products. Written with a small dot between the two vectors, such as $w \cdot x$, said "w dot x."

Elbow method — A way to choose a sensible k by plotting inertia against several values of k and looking for the point where adding another group stops helping very much. The plotted line usually drops sharply at first, then bends and flattens out, forming a shape like a bent arm, with the "elbow" marking a good choice of k .

Encoding — Turning a category into numbers that a model can use. Different encoding methods are needed for unordered categories and for ordered, ordinal categories.

Ensemble — A model built by combining the predictions of many smaller models instead of relying on just one. An ensemble usually predicts by letting its members vote, for a category, or by averaging their guesses, for a number.

Euclidean distance — The straight-line distance between two points, found by squaring the difference in each feature, adding the squared differences together, and taking the square root of that sum. With two features, this is exactly the same idea as measuring the diagonal side of a right triangle.

Explained variance ratio — The share of a dataset's total variance that one principal component accounts for, found by dividing that component's own variance by the sum of every component's variance. A ratio close to 1, or 100%, means that component captures nearly all of the data's spread.

F1 score — A single score that combines precision and recall, using a kind of average that only stays high when both precision and recall are high. F1 drops sharply if either precision or recall is low, even if the other one is excellent.

Fairness — The requirement that a model's benefits, errors, and decisions do not systematically disadvantage one group of people more than another, in ways that matter for the situation. What counts as fair depends on context, and usually needs more than one kind of check.

Feature — One measured property of an example, used as an input to a model. Weight, sound, age, and price can all be features.

Feature engineering — Creating new features from a dataset's existing columns, using domain knowledge or simple arithmetic, so a model can find a useful pattern more easily. Good feature engineering can help a simple model perform as well as, or better than, a complex model working from raw features alone.

Feature importance — A score given to each feature that estimates how much that feature helped a tree-based model make good splits, usually based on how much impurity that feature's splits removed, added up across the whole ensemble. A higher score means the model leaned on that feature more.

Feature selection — Choosing a smaller, more useful subset of a dataset's features to actually give a model, instead of using every feature that happens to be available. Good feature selection can reduce overfitting risk and make a model both faster to train and easier to understand.

Generalization — A model's ability to perform well on new examples it has never seen before, not only on the examples it trained on. Generalization, not training performance, is the true goal of every machine learning project.

Gini impurity — The most common way to measure impurity in a decision tree. For each label present in a node, square the fraction of examples carrying that label, add these squares together, then subtract the total from 1.

Gradient — A short list of derivatives, one for each adjustable number in a model, such as each coefficient and the intercept. Each entry says how the loss would change if that one number were nudged, while every other number stayed fixed.

Gradient descent — A method that finds good values for a model's coefficients and intercept by taking many small steps, each step moving a little further in the direction that makes the loss function smaller. "Descent" means moving downward, exactly like walking down a hill.

Grid search — A tuning method that tries every possible combination of hyperparameter values from a chosen list for each hyperparameter, training and validating one model for every combination.

Hidden layer — A layer of neurons sitting between a network's input features and its final output, so called because its output values are not directly observed in the data, only computed along the way. A network can have one hidden layer or many; this chapter mostly uses one.

Hyperparameter — A setting chosen before training begins, rather than a value the learning algorithm discovers from data. Changing a hyperparameter changes how a model learns, including how much freedom it has to bend and twist.

Impurity — A number that measures how mixed together the labels are inside one node. A node with only one label present has zero impurity. A node with an even mix of labels has high impurity.

Inertia — A clustering's total squared distance from every example to the centroid it is assigned to, added together across the whole dataset. A lower inertia means the examples sit closer, on average, to their own group's centroid.

Interaction feature — A new feature built by combining two or more existing features together, usually by multiplying them, so a model can see their combined effect directly, instead of only ever seeing each feature's separate effect.

Intercept — The fixed number added at the end of a linear regression equation, after every coefficient has already been multiplied by its feature. The intercept is the model's prediction when every feature equals zero.

Interpretability — How well a person can understand why a model made a particular prediction, and what patterns it relies on in general. A highly interpretable model is easy for a person to check, question, and explain to someone else.

k-means — A clustering method that sorts examples into k groups by repeating two steps: assign every example to its nearest centroid, then move each centroid to the average position of the examples now assigned to it. The letter k is chosen ahead of time and simply means "however many groups we are looking for."

k-Nearest Neighbors — A method that predicts a label for a new example by finding the k training examples most similar to it, then letting those k examples vote on the answer. The method is usually written kNN for short. The letter k simply stands for however many neighbors get a vote.

Label — The known, correct answer attached to one training example. A label can be a category, such as "sweet," or a number, such as a price.

Lasso regression — Linear regression with an L1 penalty added to the loss: λ times the sum of the absolute value of every coefficient. Lasso can shrink a weak coefficient all the way to exactly zero, which removes that feature from the model entirely.

Learning curve — A plot of training performance and validation performance as the size of the training set grows. Reading the shape of the two lines together helps diagnose whether a model is underfitting, overfitting, or generalizing well.

Learning rate — A small number, chosen before training begins, that controls how big each gradient descent step is. A learning rate that is too large can overshoot the best line and never settle down; a learning rate that is too small wastes time taking tiny, timid steps.

Least squares — The most common loss function for linear regression. It squares every residual, so that positive and negative residuals never cancel each other out, then adds all the squares together. The best line, by least squares, is whichever line makes this sum as small as possible.

Linear regression — A method that predicts a number by multiplying each feature by its own number, adding the results together, and then adding one more fixed number at the end. With a single feature, this is exactly a straight line drawn through the data.

Log loss — A loss function for probabilities, also called cross-entropy loss, that heavily punishes a confident prediction that turns out to be wrong. A perfect, fully confident, correct prediction scores a log loss of 0; a confident, wrong prediction can score a very large loss.

Logarithm — The power you must raise a fixed base number to, in order to reach a chosen result. A logarithm reverses an exponent, the same way subtraction reverses addition, or division reverses multiplication.

Logistic regression — A method that predicts the probability of a yes-or-no answer. It starts with the same recipe as linear regression, a coefficient for

each feature plus an intercept, but then squeezes that score through the sigmoid function, so the result always lands between 0 and 1.

Loss function — A formula that turns a model's mistakes into a single number, where a smaller number always means a better fit to the data. Training a model almost always means searching for the coefficients and the intercept that make the loss function as small as possible.

Machine learning — A way to build computer programs that find useful rules from data, instead of a person writing every rule by hand. The computer studies many examples and works out a pattern on its own.

Matrix — A rectangular table of numbers, arranged in rows and columns. A matrix often stores a whole dataset: one row per example, and one column per feature.

Mean absolute error (MAE) — The average size of a regression model's errors, after ignoring whether each one was too high or too low. MAE stays in the same real-world unit as the target itself.

Mean squared error (MSE) — The average of a regression model's squared errors. Squaring each error removes its sign automatically, and it also makes large errors count for much more than small ones.

Metric — A number used to measure how well a model is performing. Different metrics pay attention to different kinds of mistakes, so choosing the right metric for a project matters as much as choosing the right model.

Missing value — A feature value that was not recorded, and is simply blank or empty in the table. A missing value is not automatically zero, and it is not automatically an error; it just means the true value is unknown.

Model — The rule that a learning algorithm produces after studying data. A model takes a new input and returns an output, such as a prediction.

Model capacity — How wide a range of patterns a model is able to represent, from simple to wildly complicated. A low-capacity model can only represent simple patterns, such as a straight line. A high-capacity model can represent complicated, wiggly patterns.

Monitoring — Regularly checking a deployed model's incoming data, its predictions, and its real-world outcomes, so that problems are caught quickly instead of discovered by accident, much later.

Neuron — A single computing unit inside a neural network that takes in several numbers, multiplies each by its own coefficient, adds them together with one extra bias number, and passes the result through an activation function to produce one output number. A neuron is really just a small logistic-regression-style calculation, reused as a building block.

Norm — A single number that measures the length of a vector: the straight-line distance from the all-zero point to the point the vector describes. The most common norm, used throughout this book, squares every entry, adds the squares, then takes the square root.

NumPy array — A table of numbers, built by the NumPy library, that is faster than a plain Python list at exactly the arithmetic machine learning needs most: adding, multiplying, and averaging many numbers at once. Every feature matrix and label list in this book, once inside code, becomes a NumPy array.

One-hot encoding — A way to encode an unordered category by creating one new 0-or-1 column for every possible category value. Exactly one of those new columns is set to 1 for each row, and every other new column is set to 0.

Out-of-bag (OOB) — For one tree in a random forest, the training rows that its own bootstrap sample happened to leave out. Because that tree never trained on its own out-of-bag rows, predicting on them shows how the tree handles data it has genuinely never seen.

Outlier — A value that sits far away from most of the other values in its column. An outlier might be a genuine, unusual example, or it might be a mistake, such as an extra digit typed by accident. Only careful checking can tell the two apart.

Overfitting — Learning the noise and small accidents of the training data so well that the model performs worse on new data it has not seen before. An overfit model has often memorized its training data instead of learning a general rule.

Oversampling — Adding more copies, or new similar examples, of the minority class to a training set, so it makes up a larger share of the data the model learns from.

Permutation importance — How much a trained model's validation score drops after one feature's values are randomly shuffled, breaking that feature's link to the outcome while leaving every other feature untouched. A bigger drop means the model depended on that feature more.

Pipeline — An ordered chain of preprocessing steps and a model, bundled together into one single object. A pipeline always applies its steps in the exact same order, using rules learned only once, from the training data.

Precision — Among all the examples a model predicted as positive, the fraction that were genuinely positive. Precision answers the question, "when this model raises an alarm, how often is it right?".

Precision-recall curve — A plot showing precision and recall together at every possible threshold, not just one. The area underneath this curve, often shortened to PR-AUC, gives one summary score for how well a model balances precision and recall across every threshold at once.

Prediction — The output a model gives for one new example. A prediction can be a category, such as "sweet," or a number, such as a weight in kilograms.

Preprocessing — All the steps that prepare raw data for a model, done before training begins. Preprocessing can mean cleaning mistakes, filling in missing values, turning categories into numbers, or putting numbers on a shared scale.

Principal component — One of the new directions PCA finds, ranked by how much of the data's spread it captures. The first principal component is the single direction that captures the most spread possible; each later principal component captures the most spread still remaining, once the earlier ones are accounted for.

Principal Component Analysis (PCA) — A dimensionality-reduction method that finds new directions through a dataset, ranked by how much of the data's spread each direction captures, so that a small number of these new directions can stand in for a much larger set of original features. The method is usually called PCA for short.

Protected attribute — A personal characteristic, such as gender, age, ethnicity, religion, or disability status, that law or an organization's own policy says must not be the reason someone receives a worse outcome. Protected attributes vary somewhat by country and by situation.

Proxy variable — A feature that is not itself a protected attribute, but is so closely related to one that it carries almost the same information. A model can rebuild a pattern connected to a protected attribute through a proxy, even after the protected attribute itself is removed from the data.

Pruning — Cutting back branches from an already-grown decision tree, removing splits that do not truly improve performance on a validation set. Pruning trades away a small amount of training accuracy in exchange for a tree that generalizes better to new examples, much like a gardener trimming a real tree's overgrown branches to keep it healthy.

Random forest — An ensemble of many decision trees, each trained on its own bootstrap sample, where each split is also chosen from only a random subset of the features. A random forest predicts by letting all of its trees vote, for classification, or by averaging their predictions, for regression.

Random search — A tuning method that tries a fixed number of randomly chosen hyperparameter combinations, instead of every possible combination, and keeps the best one found.

Random seed — A starting number, chosen once, that makes a program's "random" choices happen in the exact same order every time the code runs. Using the same random seed twice always produces the exact same result; a different seed produces a different, but equally valid, result.

Recall — Among all the examples that were genuinely positive, the fraction the model actually found. Recall answers the question, "of all the true cases out there, how many did this model catch?".

Regression — A learning task where the model predicts a number that can take almost any value on a scale. Tomorrow's temperature and a house's selling price are both regression targets.

Regularization — Adding a penalty, inside training itself, that discourages a model from becoming more complex than the data truly justifies. A model can still bend where the data strongly supports bending, but it must "earn" every bend.

Reinforcement learning — A third kind of learning, less common in this book, where a computer program called an agent takes actions inside an environment and learns from rewards and penalties, instead of from labeled examples.

ReLU — Short for "rectified linear unit," an activation function that keeps a positive number exactly as it is, and replaces any zero or negative number with 0. ReLU is simple to compute and is one of the most common activation functions used in hidden layers today.

Residual — The true value minus the predicted value for one example: $\text{residual} = y - \hat{y}$. A positive residual means the model predicted too low. A negative residual means the model predicted too high. A residual of zero means the prediction was exactly right.

Ridge regression — Linear regression with an L2 penalty added to the loss: λ times the sum of every coefficient squared. Ridge shrinks every coefficient toward zero, shrinking large coefficients the most, but it almost never pushes a coefficient all the way to exactly zero.

Rollback — Switching a live system back to its last known-good model or version, quickly, when monitoring shows the current version is behaving badly, rather than leaving the faulty version live while investigating.

Root mean squared error (RMSE) — The square root of MSE. Taking the square root undoes the squaring step, bringing the error back into the target's original, real-world unit.

Root Node and Leaf Node — The root node is the single starting point of a decision tree, where the very first question is asked, using the whole training set. A leaf node is an ending point that asks no further questions and simply gives a prediction, reached only after answering every question along the path from the root.

Scalar — A single number, on its own, with no other numbers attached to it. A melon's weight, 4.2 kilograms, is a scalar. A model's accuracy, 85%, is also a scalar.

Sigmoid function — A function that squeezes any real number into a value between 0 and 1, tracing a smooth S-shaped curve. Logistic regression uses it to turn a plain score into a probability.

SMOTE — Short for Synthetic Minority Oversampling Technique. A resampling method that creates new, synthetic minority-class examples by picking a point partway between a real minority example and one of its nearest neighbors of the same class, instead of copying an existing example exactly.

Standardization — Rescaling a numerical feature by subtracting its training mean and dividing by its training standard deviation, so the feature ends up centered around 0, with most values falling between about -2 and 2 .

Supervised learning — Learning from examples that already have a correct label attached. Classification and regression are both supervised learning, because every training example already carries its own answer.

Target — The answer a model is trained to predict for each unit of observation. In classification the target is a category; in regression the target is a number.

Test set — A group of examples kept completely separate from training, then used once to check how well the finished model performs on data it has never seen before.

Threshold — A cutoff value that turns a score into a decision. Anything at or beyond a threshold is treated one way; anything short of it is treated another way. This chapter uses a threshold on an anomaly score, but the same idea returns for very different kinds of scores later in this book.

Time-based split — A way of splitting data by date instead of by random chance, so that every training example happens before every validation or test example. A time-based split protects a project from a model accidentally learning from the future.

Training — The process of using examples to fit, or adjust, a model. During training, the model changes its internal rule so that it matches the training examples more closely.

Training-serving skew — A mismatch between how a feature is computed during training and how that same feature is computed for real, live predictions. The deployed model ends up receiving different information than it learned from, even though nothing about the real world has actually changed.

Underfitting — Failing to learn enough of the real pattern in the data, so the model performs poorly even on the training data itself. An underfit model is usually too simple for the problem it is trying to solve.

Undersampling — Removing examples from the majority class in a training set, so it makes up a smaller share of the data the model learns from.

Unit of observation — The real-world thing that one row of a dataset represents. A unit of observation might be one person, one house, one email, one machine, or, as in this chapter's clinic example, one patient visit.

Unsupervised learning — Learning from examples that have no label at all. Clustering is unsupervised learning, because the model only ever sees the features, never a correct answer.

Validation set — A group of examples, separate from both training and test data, used to compare candidate models and choose settings before ever opening the test set. A model's settings may be adjusted based on validation results, but the test set stays completely untouched until the very end.

Variance — A model's tendency to change a great deal when it is trained on a different sample of the same kind of data. High variance shows up as overfitting.

Vector — An ordered list of numbers. A vector often describes one example, one number per feature, in a fixed, agreed order. The melon (4, 8) is a vector: weight 4 kilograms, sound score 8, always in that order.

Workflow — An ordered set of steps used to complete a project in a safe, repeatable way. Following the same workflow every time helps a whole team avoid easy-to-make mistakes.

z-score — How many standard deviations a value sits above or below the mean of normal data. A z-score of 0 means exactly average. A positive z-score means above average, and a negative z-score means below average, and the size of the number says how far.